

Università degli Studi di Napoli
Federico II

A.A. 2026-2027

Ingegneria del Software

Indice

0 Prefazione 9

I Ingegneria del Software e processi software 10

1 Introduzione all'Ingegneria del Software 12
Il corso in breve 12
Il progetto come gioco di ruolo 12
Testi consigliati 13
Perché «ingegneria»? 13
Un po' di storia 14
Gli inizi: l'hardware è il protagonista 14
Anni '60: la crisi del software 14
L'approccio ingenuo 14
La dimensione conta 15
Programmi e prodotti software 16
Tipi di prodotti software 17
Lo sviluppo professionale 18
Cos'è l'Ingegneria del Software 18
Perché è difficile 19
Progetti che sono andati male 19
Quando i problemi non sono solo economici 19
Il lavoro dell'ingegnere del software 20

2 I processi software 22
Cos'è un processo software 22
Le quattro attività fondamentali 23
Software Specification (specifica) 23
Software Implementation (sviluppo) 23
Software Validation (validazione) 23
Software Evolution (evoluzione) 23
Il ciclo di vita del software 25
Quanto costa il software 25
Hardware contro software 25
Dove vanno i soldi: i costi per fase 26
I modelli di processo 27

3 Il modello a cascata 28
Origini e caratteristiche 28
Perché «a cascata»? 28
Le sei fasi in sintesi 29
Fase 1: Requirements Engineering 30
Cosa, non come 30
Requisiti funzionali e non funzionali 30
Tecniche per raccogliere e specificare i requisiti 31
Fase 2: System Design 32

Fase 3: Software e UI Design 33
Alto livello e basso livello a confronto 33
Fase 4: Implementation 34
Fase 5: Testing 35
Le attività di testing 36
Fase 6: Operation e Maintenance 37
Il problema del ritorno all'indietro 39
Pregi, difetti e quando usarlo 40
Il problema dei team «a compartimenti stagni» 40
Quando ha senso usarlo 41

II La qualità del software 42

4 La qualità del software 44
Cos'è la qualità di un prodotto 44
Cos'è la qualità del software? 44
Esempio 1: efficiente, ma comprensibile? 44
Esempio 2 e 3: sicuro? facile da usare? 45
La lezione: tanti punti di vista 45
I modelli di qualità ISO/IEC 46
Il modello di qualità del prodotto 47
Functional Suitability 47
Reliability 47
Performance Efficiency 49
Usability 49
Security 50
Compatibility 51
Maintainability 52
Flexibility 53
Safety 53
Il quadro completo 54
Qualità, modularità e lavoro di squadra 55
Il modello di qualità in uso 55
Usability (in uso) 56
Safety (in uso) 56
Flexibility (in uso) 56
Il legame tra i due modelli 57
Qualità e compromessi 57

III Ingegneria dei requisiti 59

5 Requisiti: elicitazione e analisi 61
Cos'è un requisito 61
Due livelli: requisiti utente e requisiti di sistema 62
Requisiti funzionali e non funzionali 63
Come si scrivono le due famiglie 63
Tipi di requisiti non funzionali 63
Requisiti di dominio 64
Le proprietà di un buon requisito 65
I demoni dell'ambiguità 65
Testabilità 66
Il processo di Requirements Engineering 66
Un processo iterativo, non lineare 67
Elicitazione e analisi dei requisiti 68
Chi sono gli stakeholder 68
Perché è difficile 69

Il processo di elicitazione e analisi 69
Le tecniche di elicitazione 70
Interviste con gli stakeholder 70
Etnografia 71
Personas 71
Storie e scenari 72
Mock-up a bassa fedeltà 73
La specifica dei requisiti 74
Quattro approcci alla specifica 75
Specifica in linguaggio naturale 75
Specifica strutturata 75
Come si scrive una frase di requisito 76
Un modello per i requisiti funzionali 77
I documenti dei requisiti 77
IEEE 830-1998 77
ISO/IEC/IEEE 29148:2018 78

6 Diagrammi dei casi d'uso 79
Casi d'uso e attori 79
Gli attori 80
I casi d'uso 80
Le relazioni di un diagramma dei casi d'uso 81
Associazione 81
Attori secondari e cardinalità 81
Generalizzazione tra attori 81
La relazione «include» 82
La relazione «extend» 82
Generalizzazione tra casi d'uso 83
Riepilogo delle relazioni 84
Leggere un diagramma dei casi d'uso 85
Errori tipici 85
Errore I: casi d'uso che non sono casi d'uso 85
Errore II: più attori sullo stesso caso d'uso 86
Errore III: generalizzazioni inutili 86
Errore IV: «include» per le relazioni temporali 86
Errore IV bis: diagrammi troppo complessi 87
Specificare i casi d'uso 87
I formati 87
Il template di Cockburn 87
Main scenario ed estensioni 88
Un esempio completo: il bancomat 89
I mock-up 89
Il caso d'uso fully-dressed 89
Le estensioni 90

IV Modellazione con UML 92

7 UML e il diagramma delle classi 94
Modellare il dominio del problema 94
Che cos'è UML 94
Come si usa UML 95
Regole prescrittive e descrittive 95
I diagrammi di UML 2 95
Il diagramma delle classi 96
Le classi 96
Gli attributi 96
Le operazioni 97
Le associazioni 98
Il verso di navigazione 98
La molteplicità delle associazioni 99

Associazioni molti a molti 100
Associazioni uno a uno 101
Controesempio: associazioni uno a uno inutili 102
Le enumerazioni 102
Classi associative 103
Associazioni riflessive 103
Aggregazione e composizione 103
Generalizzazione 105
Responsabilità e note 106
Le responsabilità 106
Le note 107
Dominio del problema e dominio della soluzione 107
Costruire un modello di dominio 107
1. Identificare le classi 107
2. Identificare associazioni e attributi 108
3. Identificare generalizzazioni e interfacce 109
4. Assegnare le responsabilità alle classi 109
Errori comuni 110
Due esempi completi 110
Complementi: dipendenze e interfacce 113
La dipendenza 113
Le interfacce 113
La notazione lollipop 114

8 Diagrammi dei package e di sequenza 116
Il diagramma dei package 116
I diagrammi di interazione 119
Gli elementi 120
Creazione e distruzione degli oggetti 120
Quando si usa 121
Cicli e condizioni 121
Esempio: dal codice al diagramma di sequenza 122

9 Altri diagrammi UML 125
Il diagramma dei componenti 125
Interfacce e porte 126
Sottosistemi 127
Il diagramma di deployment 128
Il diagramma di stato (statechart) 130
Regioni, vertici e transizioni 130
Gli stati 130
Le transizioni 130
Tre esempi 131
Dal diagramma al codice 132
Attività interne 135
Stati compositi e regioni parallele 136
Diagrammi di stato e interfacce grafiche 137
Lo pseudostato history 137
Il diagramma di attività 138
Attività e flussi 138
Decisioni: branch e merge 138
Concorrenza: fork e join 139
Le swimlane 141
Macro-attività 142
I segnali 143

V Progettazione 145

10 System Design e buone pratiche di progettazione 147

Dove siamo nel processo 147
Cambia l'interlocutore 148
Le attività della progettazione di sistema 148
Gli obiettivi di progetto 148
I criteri di progetto 149
I compromessi di progetto 149
Criteri generali: progettare per il cambiamento 150
La scomposizione in sottosistemi 150
Perché scomporre conviene 150
Modellare i sottosistemi 150
Quanti sottosistemi? 151
L'indipendenza funzionale 151
Coesione 152
Il principio di singola responsabilità 152
I vantaggi della coesione 154
Tipi e livelli di coesione 154
Un controesempio: la God Class 156
Accoppiamento 156
Le forme di accoppiamento 157
Pensare per interfacce 157
Un esempio completo: il ragazzo dei giornali 159
Perché è sbagliato 159
La versione migliorata 160
Perché è meglio 160
La Legge di Demetra 161
Com'è fatta una violazione 162
Eliminare il codice di navigazione 162
Progettazione guidata dalle responsabilità 163
Le schede CRC 163

11 Architetture software 165

Che cos'è un'architettura software 165
1. L'architettura è un'astrazione 165
2. Ogni sistema ha un'architettura 165
3. Un disegno a scatole e frecce non basta 166
L'architettura nel System Design 166
Identificare i sottosistemi 166
Divide et impera 166
Due dimensioni: sottosistemi e livelli 167
L'architettura a livelli 167
Livelli e partizioni 168
Le principali architetture 169
Client/server 169
Repository 170
A livelli 171
Three-tier 172
Multi-tier 174
Orientata ai servizi (SOA) 174
Confronto 174

12 Architetture software moderne 176

Che cos'è, di nuovo, un'architettura 176
Perché l'architettura conta 176
Le proprietà che dipendono dall'architettura 177
Scalabilità 177
Scalare orizzontalmente, verticalmente, diagonalmente 178
Il bilanciamento del carico 178
Disponibilità 179
Quantificare la disponibilità: i «nove» 180
Come si ottiene un'alta disponibilità: il fail-over 180
Osservabilità e monitoraggio 181
Dati e consistenza 182
Le proprietà ACID 182

Replica master-slave e master-master 182
Federazione e sharding 183
Denormalizzazione 183
I database NoSQL e la consistenza finale 183
Elaborazione asincrona 184
Code di messaggi e message broker 185
Un altro scenario: lo stesso utente su più dispositivi 186
Microservizi 187
Perché mai dovrei farlo? 187
Le motivazioni: consegnare spesso e bene 187
Scalare un monolite e scalare dei microservizi 188
Conviene davvero? 188
Progettare i microservizi 188
La comunicazione fra microservizi 189
La gestione dei dati 190
È una questione di compromessi 191
Un caso reale: un'applicazione a microservizi 191
Il progetto 191
L'architettura 191
Gli anti-pattern individuati 193
Architettura e organizzazione dei team 195
Come si organizza un team grande 195
Che fare con la legge di Conway 195

VI Infrastruttura e servizi 197

13 Virtualizzazione e cloud computing 199
Perché la virtualizzazione 199
L'evoluzione delle CPU 199
Il problema dei multi-core 199
Il data center tradizionale 200
Le conseguenze 200
Le macchine virtuali 201
L'idea 201
Le proprietà di una macchina virtuale 201
La virtualizzazione in esercizio 202
Un server o un'infrastruttura 202
Come funziona l'alta disponibilità 203
La migrazione a caldo 203
I vantaggi della virtualizzazione 203
Il consolidamento 203
Semplificazione e costi 204
Il cloud computing 205
Scalabilità ed elasticità 205
Il provisioning dinamico 206
I modelli di servizio 207
Infrastructure as a Service (IaaS) 208
Platform as a Service (PaaS) 208
Software as a Service (SaaS) 209
I modelli di deployment 210
Cloud pubblico 210
Cloud privato 210
Cloud ibrido 210
L'ecosistema del cloud 211

14 Il paradigma REST 212
Il contesto architetturale 212
Frontend e backend 212
Application Programming Interface 212

REST 213
Un ripasso di HTTP 213
I verbi e i codici di stato 213
I fondamenti di REST 214
Lo stato e le sue rappresentazioni 214
Progettare un'API REST 215
Le risorse e i loro URI 215
Le risorse annidate 217
Filtrare e ordinare 218
Proteggere un'API REST 219
L'autenticazione basata su token 219
JSON Web Token (JWT) 220
La firma e le funzioni di hash crittografiche 221
Documentare un'API: OpenAPI 222

VII OBJECT DESIGN 224

15 | DESIGN PATTERN 226
Dove siamo nel processo 226
Dalle architetture ai design pattern 226
Che cos'è un design pattern 227
Un po' di storia 227
Le quattro parti di un pattern 227
A che cosa servono 227
Come leggere i disegni della memoria 228
I pattern della Gang of Four 229
Pattern creazionali 230
Singleton 230
Builder 234
Pattern strutturali 241
Adapter 241
Composite 246
Decorator 251
Pattern comportamentali 255
Observer 255
State 260
Strategy 264
Pattern moderni: la Dependency Injection 265
Conclusioni 269

Prefazione

Estratto

Il seguente documento rappresenta una raccolta di appunti, riorganizzata e migliorata nelle vesti grafiche, del corso di Ingegneria del Software del professor Porfirio Tramontana, per il corso di laurea in Informatica A.A. 2026-2027, della Federico II di Napoli.

L'obiettivo del corso è comprendere l'Ingegneria del Software come disciplina: i suoi processi, le sue fasi e i documenti che produce. Si imparano le tecniche di modellazione usate per progettare, sviluppare e mantenere il software, i metodi per scrivere codice leggibile e manutenibile, i fondamenti dell'interazione uomo-macchina applicati alla progettazione di interfacce e le sfide del lavoro in team su progetti software reali.

Si tiene a specificare che questo documento non ha lo scopo di **sostituire libri di testo**, note del professore o lezioni frontali, nonostante siano tutte fonti utilizzate per la stesura, ma quello di integrare al materiale più complesso una rilettura degli argomenti da un punto di vista meno tecnico e quindi più lento nel soffermarsi nei passaggi critici.

Si ringraziano **Valentino Bocchetti** e **Pasquale Miranda** ideatori e gestori del progetto *Unina Docs*,

Si ringraziano inoltre tutti i revisori che hanno contribuito al documento con consigli e proposte di modifica, invitando a dare feedback in caso di errori.

Si ringraziano in fine i lettori nella speranza che questo lavoro sia stato utile.

Redazione a cura di **Fabio Rosiello**

Parte I

Ingegneria del Software e processi software

Introduzione all'Ingegneria del Software

Il corso in breve

Prima di entrare nel vivo, conviene avere chiaro **cosa** si studierà e **come** si viene valutati, perché il modo in cui è organizzato l'esame rispecchia molto da vicino i contenuti del corso.

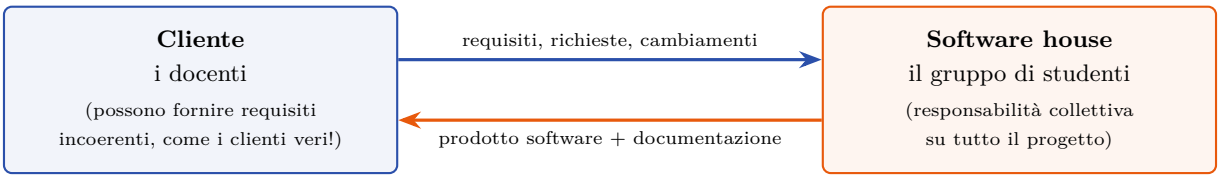
Finora (Programmazione 1, Programmazione a Oggetti, …) abbiamo imparato a scrivere *programmi*. In questo corso si fa un salto di scala: si impara a produrre *software* in modo professionale, cioè a lavorare su sistemi grandi, in gruppo, con clienti veri e con vincoli di tempo e di budget. In inglese si parla di **programming in the large**, in contrapposizione alla programmazione «in piccolo» a cui siamo abituati.

Crediti e lezioni	9 CFU, circa 36 lezioni tra teoria ed esercitazioni in aula.
Prerequisiti	Buona capacità di programmazione, progettazione e programmazione a oggetti, basi di UML (diagrammi delle classi e di sequenza).
Esame scritto	Domande ed esercizi numerici su tutto il programma. Può essere sostituito da **due prove intercorso** (una a metà corso e una a fine corso): serve la sufficienza in entrambe e il voto è la media.
Progetto di gruppo	Gruppi da 2 o 3 persone (più si è, più funzionalità vanno implementate). Tema unico con varianti per gruppo.
Voto finale	Scritto e progetto hanno **lo stesso peso**. La partecipazione attiva alle attività pratiche viene considerata positivamente.
Tempistiche	Scritto e discussione del progetto sono indipendenti e si possono sostenere in qualsiasi ordine. Chi consegna il progetto presto ha meno funzionalità da implementare.

Tabella 1.1: Organizzazione del corso e modalità d'esame.

Il progetto come gioco di ruolo

Il progetto non è un semplice «esercizio di programmazione», ma una **simulazione realistica** di come si lavora in azienda:



Il gruppo deve replicare **l'intero processo software** (che vedremo nel Capitolo 3): ingegneria dei requisiti, progettazione di sistema, progettazione degli oggetti e dell'interfaccia, implementazione e testing. Quindi non si consegna (solo) codice, ma anche tutti i documenti che un'azienda produrrebbe.

La discussione del progetto avviene in tre momenti:

1. **Presentazione tecnica** con slide (circa 10 minuti) per convincere i «clienti» della qualità del prodotto e delle scelte fatte;

2. **Demo dal vivo** dell'applicazione su un portatile del gruppo;
3. **Discussione** delle scelte di progettazione e implementazione.

Plagio e contenuti generati

Tutti gli elaborati consegnati vengono analizzati con strumenti automatici di rilevamento del plagio e dei contenuti generati da IA. Progetti troppo simili vengono invalidati e ai gruppi coinvolti viene assegnata una nuova traccia.

Testi consigliati

Le slide **non sostituiscono** lo studio sui libri. Per argomento, i testi suggeriti sono:

Argomento	Testo
Parti generali	I. Sommerville, *Software Engineering*, Pearson; R. S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw Hill
Progettazione a oggetti	C. Larman, *Applicare UML e i Pattern*, Pearson
Codice	R. Martin, *Clean Code*, Apogeo
UML	P. Stevens, R. Pooley, *Usare UML*, Addison Wesley
Approfondimenti	P. Ammann, J. Offutt, *Introduction to Software Testing*, Cambridge University Press; E. Gamma et al., *Design Patterns*, Addison Wesley
Usabilità e HCI	B. Shneiderman et al., *Designing the User Interface*, Pearson; J. Nielsen, *Usability Engineering*, Morgan Kaufmann

Lettura consigliata: *The Humble Programmer* di Edsger W. Dijkstra (1972), il discorso per il premio Turing in cui Dijkstra riflette su come la crescita della potenza dei calcolatori abbia reso la programmazione un problema *più* difficile, non più facile.

Perché «ingegneria»?

Partiamo da un'analogia. Osserviamo questi due ponti:



Passerella in legno
1 artigiano, 10 giorni, circa 5.000 €.
Un approccio **non strutturato** è sufficiente.



Viadotto San Giorgio (Genova)
1.000 persone, 330 aziende coinvolte.
Serve un approccio **sistematico e strutturato**.

Figura 1.1: Stessa funzione (attraversare), complessità incomparabili.

Entrambi sono «ponti», ma nessuno costruirebbe un viadotto autostradale come si costruisce una passerella nel bosco: servono progettisti, calcoli, normative, collaudi, coordinamento tra centinaia di persone. Con il software succede esattamente la stessa cosa: un programmino da 200 righe si può scrivere «di getto», un sistema bancario da milioni di righe no.

L'idea chiave

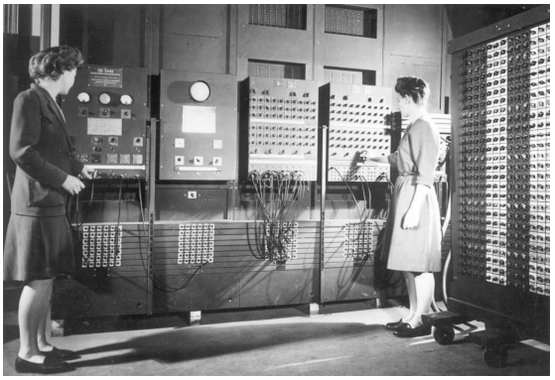
L'**ingegneria** entra in gioco quando la complessità del problema supera la capacità di un singolo individuo di tenerlo sotto controllo. A quel punto l'improvvisazione non basta più: servono **metodi, processi e strumenti**.

Un po' di storia

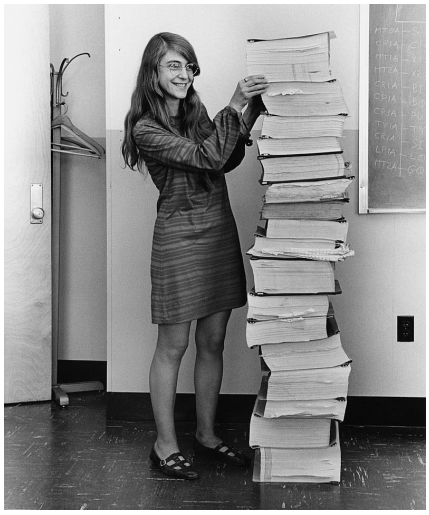
Gli inizi: l'hardware è il protagonista

Nel 1946 viene presentato l'**ENIAC** (*Electronic Numerical Integrator and Computer*), uno dei primi calcolatori elettronici programmabili. In quegli anni:

- i calcolatori erano macchine enormi, costosissime e sbalorditive;
- la macchina era molto più «spettacolare» (e costosa) di qualche foglio di codice;
- la programmazione era considerata un'attività **secondaria** rispetto all'ingegneria dell'hardware;
- i programmi venivano sviluppati per lo più **individualmente**, spesso da chi poi li usava.



Jean Bartik (a sinistra) e Frances Spence al lavoro sull'ENIAC.



Margaret Hamilton nel 1969, accanto al codice scritto dal suo team per l'Apollo Guidance Computer.

Anni '60: la crisi del software

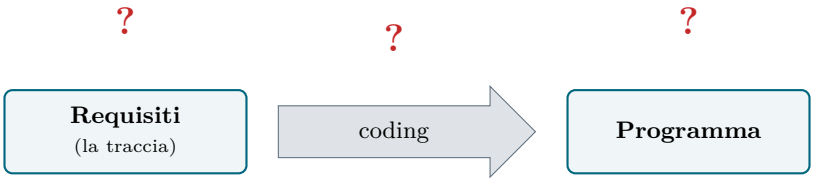
Negli anni '60 i sistemi diventano molto più complessi (si pensi ai programmi spaziali della NASA). Gli approcci individuali **non scalano**: quello che funziona per un programma piccolo crolla su sistemi grandi.
Nasce così la cosiddetta **crisi del software**: i sistemi erano

- **inaffidabili** (pieni di errori);
- **fuori budget** (costavano molto più del previsto);
- **in ritardo** rispetto ai tempi di consegna pianificati.

Diventa evidente il bisogno di una nuova disciplina che studi **metodologie per la produzione del software**. Il termine *software engineering* si diffonde in quegli anni: fu usato da Margaret Hamilton, che guidava il team del software di bordo delle missioni Apollo, e diede il titolo a una famosa conferenza NATO del 1968 dedicata proprio alla crisi del software.

L'approccio ingenuo

L'approccio con cui abbiamo lavorato finora nei corsi di programmazione è più o meno questo:



Leggiamo la traccia, scriviamo il codice, otteniamo il programma. Funziona benissimo per un esercizio d'esame, ma appena il problema diventa reale emergono subito delle domande a cui questo schema non sa rispondere:

Aspetto	Domande senza risposta
Requisiti	Da dove arrivano? Sono corretti? Sono completi? Il cliente sa davvero cosa vuole?
Organizzazione	Come strutturo il programma? In quali moduli e classi lo divido?
Lavoro in team	Come divido il lavoro tra più programmatori? Come ci coordiniamo?
Correttezza	I requisiti sono davvero soddisfatti? Come faccio a sapere che il programma è corretto?
Cambiamento	Cosa succede quando i requisiti cambiano a metà strada (e cambieranno)?

L'Ingegneria del Software esiste proprio per dare una risposta **sistematica** a ciascuna di queste domande.

La dimensione conta

Qual è il programma più grande che abbiamo mai scritto? E prima ancora: **come si misura la «dimensione» di un software?** Una misura semplice (anche se imperfetta) sono le **LOC** (*Lines Of Code*), cioè le righe di codice, di solito contate escludendo commenti e righe vuote.

Un progetto medio del corso di Programmazione a Oggetti dello scorso anno contava circa **3.000 righe** di codice non di commento (incluse quelle generate automaticamente) e circa **35 classi**. Sembra tanto lavoro, vero? Confrontiamolo con alcuni sistemi reali:

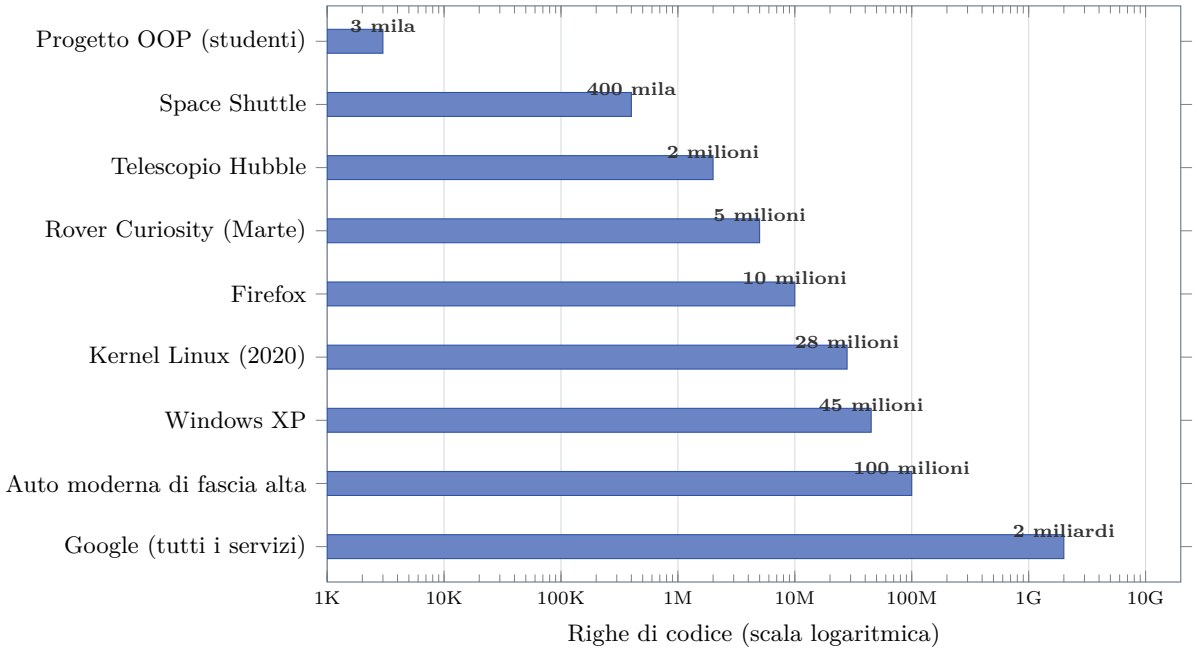


Figura 1.2: Ordini di grandezza del software reale (stime diffuse pubblicamente, arrotondate).

Attenzione alla scala

L'asse orizzontale è **logaritmico**: ogni tacca vale 10 volte la precedente. Il progetto di OOP è quindi più piccolo di un fattore 100 rispetto al software dello Space Shuttle, e di quasi un milione di volte rispetto all'insieme dei servizi Google. A queste dimensioni nessuna persona può conoscere tutto il codice: **servono metodo e organizzazione**.

Programmi e prodotti software

Una distinzione fondamentale è quella tra **programma** e **prodotto software**.

Programma

Software sviluppato generalmente da **una sola persona**, che spesso ne è anche l'utente principale. Non richiede un processo formale né documentazione e in genere non è commercializzabile.

Prodotto software

Software sviluppato da **team** e usato da **altre persone**. È un prodotto industriale, sviluppato e testato secondo standard, che comprende molto più del solo codice sorgente o dell'eseguibile e che deve **evolvere nel tempo**.

	Programma	Prodotto software
Chi lo sviluppa	Una persona	Uno o più team
Chi lo usa	Lo sviluppatore stesso	Altre persone (clienti, utenti finali)
Processo	Nessun processo formale	Processo strutturato e documentato
Costi	Bassi	Significativamente più alti
Cosa comprende	Codice (ed eseguibile)	Codice, documentazione, manuali utente, manuali di installazione e configurazione, suite di test automatici
Ciclo di vita	«Usa e getta», one-shot	Evolve nel tempo (versioni, aggiornamenti, correzioni)
Esempio	Uno script per rinominare le foto delle vacanze	Un'app di home banking

Tabella 1.2: Programma e prodotto software a confronto.

Esempio 1.1 — Lo stesso compito: programma o prodotto

Rinominare le foto delle vacanze, come programma per sé, sono poche righe scritte in dieci minuti:

Rinomina.java

```java
public class Rinomina {
  public static void main(String[] args) {
    File[] foto = new File("/home/anna/Vacanze2026").listFiles();
    for (int i = 0; i < foto.length; i++) {
      foto[i].renameTo(new File(foto[i].getParent(), "sardegna_" + (i + 1) + ".jpg"));
    }
  }
}
```

Funziona per Anna, che sa che la cartella esiste, che contiene solo foto e che l'ordine non le interessa. Lo stesso compito, come **prodotto** per altre persone, deve fare i conti con tutto il resto:

Rinominatore.java

```java
/**
 * Prepara la rinomina delle foto di una cartella in ordine di data: prefisso_001.jpg, prefisso_002.jpg...
 * Non tocca nessun file: restituisce il piano, che l'utente vede in anteprima prima di confermarlo.
 * @throws IllegalArgumentException se la cartella non esiste
 */
public List<Rinomina> pianifica(Path cartella, String prefisso) throws IOException {
  if (!Files.isDirectory(cartella)) {
    throw new IllegalArgumentException("Cartella inesistente: " + cartella);
  }
  try (Stream<Path> file = Files.list(cartella)) {
    List<Path> foto = file.filter(Rinominatore::eUnaFoto)              // salta video, testi e file nascosti
                          .sorted(Comparator.comparing(Rinominatore::dataDiScatto))
                          .toList();
    List<Rinomina> piano = new ArrayList<>();
    for (int i = 0; i < foto.size(); i++) {
      String nome = String.format("%s_%03d.jpg", prefisso, i + 1);
      piano.add(new Rinomina(foto.get(i), cartella.resolve(nome)));
    }
    return piano;
  }
}
```

E il codice è solo una parte: al prodotto servono anche i test automatici (cartella vuota, nomi già usati, mille foto), un'interfaccia, un installatore, il manuale, le traduzioni, e qualcuno che lo aggiorni quando esce un nuovo formato di immagine.

Tipi di prodotti software

I prodotti software si dividono in due grandi famiglie. La differenza chiave è **chi definisce i requisiti**.

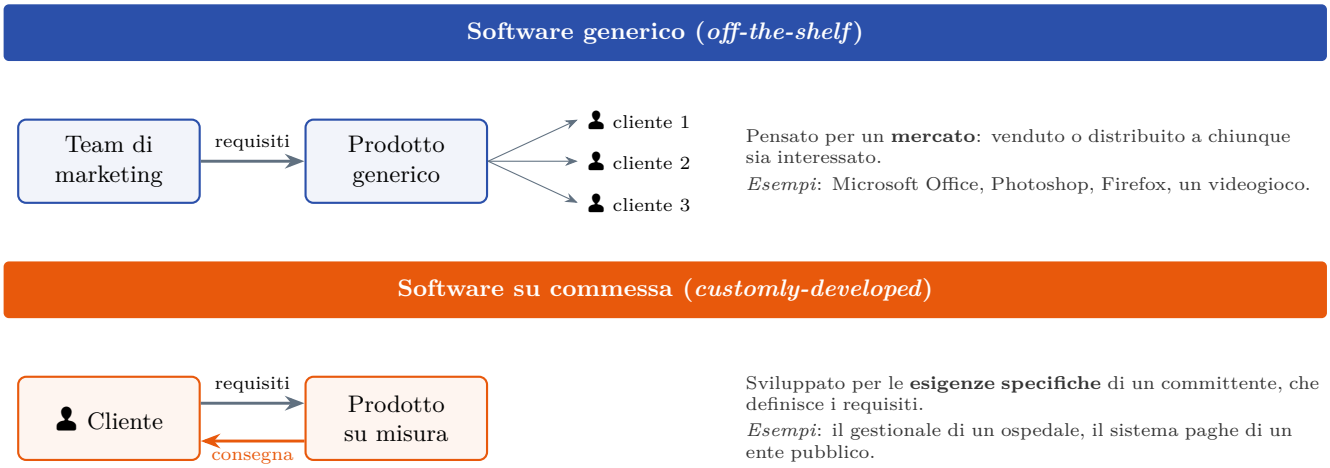


Figura 1.3: La differenza chiave: chi definisce i requisiti (il marketing o il cliente).

Lo sviluppo professionale

Lo sviluppo professionale del software non è (solo) un'**arte** artigianale né (solo) una **scienza**: è un **processo industriale**.

- Il software viene sviluppato da **aziende**, organizzate in **team** con molti membri.
- Si lavora con **vincoli di tempo e di costo** e con **requisiti di qualità** stringenti.
- Come per ogni processo industriale, sono state sviluppate **metodologie** che guidano progettazione, sviluppo e verifica. Queste metodologie non nascono dal nulla: **formalizzano l'esperienza** accumulata in decenni di progetti (riusciti e falliti).

Essere il miglior programmatore non basta

Da professionisti dell'informatica queste metodologie vanno **conosciute**. Un ottimo programmatore che non sa lavorare dentro un processo, in un team, con un cliente, produrrà comunque software di scarsa qualità.

Cos'è l'Ingegneria del Software

Esistono diverse definizioni, che guardano la disciplina da punti di vista leggermente diversi.

Ingegneria del Software (Sommerville)

È la **disciplina ingegneristica** che si occupa di **tutti gli aspetti** della produzione di software professionale.

Analizziamo le due parti in grassetto:

- **Disciplina ingegneristica**: gli ingegneri *fanno funzionare le cose*. Applicano teorie, metodi e strumenti **dove sono appropriati**, per risolvere problemi rispettando i vincoli organizzativi ed economici. Non si cerca la soluzione «perfetta» in astratto, ma quella migliore *date le risorse disponibili*.
- **Tutti gli aspetti della produzione**: non solo la programmazione! Rientrano anche la **gestione del progetto** (pianificazione, persone, costi) e lo sviluppo di strumenti, metodi e teorie a supporto dello sviluppo.

Ingegneria del Software (IEEE)

È l'applicazione di un approccio **sistematico, disciplinato e quantificabile** allo sviluppo, all'esercizio e alla manutenzione del software; in altre parole, l'applicazione dell'ingegneria al software.

Ingegneria del Software (Bruegge e Dutoit)

È lo stato dell'arte dello sviluppo di **software di qualità**, **nei tempi** e **nei costi** previsti.

Cosa hanno in comune le definizioni

Tutte e tre sottolineano che non basta «far funzionare il codice»: conta **come** ci si arriva (in modo sistematico), **cosa** si ottiene (qualità) e **a quali condizioni** (tempo e budget). Inoltre il lavoro non finisce con la consegna: esercizio e manutenzione fanno parte della disciplina.

Perché è difficile

Produrre software di alta qualità rispettando budget e vincoli organizzativi **non è facile**. I motivi principali sono tre.

1. È astratto e intangibile

Non è vincolato dalle proprietà dei materiali o dalle leggi della fisica. Per questo può diventare **estremamente complesso** molto in fretta, senza che nessuno «veda» il problema.

2. È ovunque ed è diverso

Infrastrutture nazionali, elettrodomestici, industria, finanza, intrattenimento… Ogni dominio ha esigenze molto diverse dagli altri.

3. Non esiste un metodo universale

Nessuna metodologia va bene per ogni tipo di software, azienda e situazione. E nessuna tecnologia è una **pallottola d'argento** che risolve tutto.

No Silver Bullet

L'espressione «pallottola d'argento» viene dal celebre articolo *No Silver Bullet* di Fred Brooks (1986): come nelle leggende solo una pallottola d'argento uccide il lupo mannaro, così si sperava in una tecnologia capace di eliminare da sola la complessità del software. Brooks sosteneva che **non esiste**: la complessità è in buona parte *essenziale*, cioè intrinseca al problema, e nessuno strumento può farla sparire.

Progetti che sono andati male

Ancora oggi i progetti software falliscono o incontrano problemi seri: sforano il budget, sforano i tempi di rilascio, consegnano un prodotto che non rispetta i requisiti di qualità… o che non funziona affatto.

Progetto	Cosa è successo	Costo
Healthcare.gov (USA)	Portale sanitario commissionato dal governo federale. Nella prima settimana solo l'1% degli utenti riuscì a iscriversi.	**1,5 mld $** (15 volte i 93,7 mln previsti)
Queensland Health (Australia)	Nuovo sistema paghe per 80.000 dipendenti di 16 dipartimenti. Avviato nel 2007 con rilascio previsto nel 2008: rilasciato nel 2010, funzionava a malapena, sistemato solo nel 2018.	**~850 mln $** (200 volte i 4,2 mln previsti)
Cyberpunk 2077	Videogioco annunciato nel 2012, uscita prevista ad aprile 2020, rinviata più volte fino a dicembre 2020. Al rilascio era pieno di bug e ingiocabile su alcune console.	**~400 mln $**
Per confronto: Viadotto San Giorgio	1,1 km di lunghezza, 45 m di altezza, 4 corsie + 2 di emergenza. Costruito in circa **un anno**.	**202 mln €**

Tabella 1.3: Alcuni progetti software problematici, confrontati con un'opera di ingegneria civile.

Il confronto è impietoso: un viadotto autostradale è costato meno di molti sistemi informativi falliti. L'ingegneria civile ha secoli di esperienza e metodi consolidati; l'ingegneria del software è molto più giovane.

Quando i problemi non sono solo economici

A volte le conseguenze di un software difettoso vanno ben oltre il budget:

- **London Ambulance Service (1992)**: il nuovo sistema computerizzato di smistamento delle ambulanze andò in crisi poco dopo l'avvio, causando enormi ritardi nei soccorsi; ai disservizi furono attribuiti numerosi decessi.
- **Ariane 5, volo 501 (1996)**: il razzo europeo si autodistrusse circa 40 secondi dopo il lancio. Un modulo software riusato dall'Ariane 4 convertiva un numero in virgola mobile a 64 bit in un intero a 16 bit: con le traiettorie più veloci del nuovo razzo il valore non ci stava più (*overflow*).
- **Boeing 737 MAX (2018–2019)**: due incidenti con 346 vittime complessive, legati al sistema automatico di controllo

MCAS, che si basava sui dati di un singolo sensore.

Esempio 1.2 — L'overflow dell'Ariane 5, in Java

Il software di riferimento inerziale convertiva la velocità orizzontale da un numero in virgola mobile a 64 bit a un intero a 16 bit, che arriva al massimo a 32 767. Sull'Ariane 4 bastava; l'Ariane 5 era più veloce.

Conversione.java

```java
double velocitaOrizzontale = 40000.0;        // con l'Ariane 4 restava sotto 32767
short bh = (short) velocitaOrizzontale;      // da 64 bit in virgola mobile a 16 bit interi
System.out.println(bh);                      // stampa -25536
```

In Java la conversione «riesce» e produce silenziosamente un numero assurdo. Il software dell'Ariane era scritto in Ada, dove la stessa conversione solleva un'eccezione: nessuno la gestiva, il calcolatore si spense e quello di riserva, con lo stesso software, si era già spento allo stesso modo. Quel valore, per di più, serviva solo prima del decollo. Un controllo esplicito avrebbe trasformato la catastrofe in un caso gestito:

Conversione.java

```java
if (velocitaOrizzontale > Short.MAX_VALUE || velocitaOrizzontale < Short.MIN_VALUE) {
  bh = saturaOSegnala(velocitaOrizzontale);  // decisione presa in progetto, non lasciata al caso
}
```

Il messaggio

Il software controlla ospedali, aerei, centrali, banche. Un difetto di **processo** (requisiti sbagliati, test insufficienti, riuso senza verifica, cattiva comunicazione) può costare vite umane. Per questo servono metodi ingegneristici.

Il lavoro dell'ingegnere del software

Riassumendo, un ingegnere del software deve:

- adottare un approccio **sistematico e organizzato** alla produzione del software;
- usare **strumenti, tecniche e metodologie appropriati**, scelti in base al problema da risolvere, ai vincoli di sviluppo e organizzativi e alle risorse disponibili;

per produrre **software di alta qualità**…



In sintesi

- L'Ingegneria del Software nasce dalla **crisi del software** degli anni '60: gli approcci individuali non scalano.
- Un **prodotto software** non è un programma: è sviluppato da team, per altri, include documentazione e test, ed evolve.
- Il software è **difficile** da ingegnerizzare: intangibile, ovunque, senza metodi universali.
- L'obiettivo è produrre **qualità** rispettando **tempi** e **costi**, gestendo il **cambiamento**.

I PROCESSI SOFTWARE

COS'È UN PROCESSO SOFTWARE

Nel capitolo precedente abbiamo visto che serve un approccio **sistematico** alla produzione del software. Il primo strumento per ottenerlo è il concetto di *processo*.

Processo software

Un **processo software** è un insieme di **attività correlate** che porta alla produzione di un sistema software.

Un'analogia utile è la costruzione di una casa: non si comincia posando mattoni a caso, ma si segue una sequenza di attività (capire le esigenze di chi ci abiterà, fare il progetto, ottenere i permessi, costruire, collaudare, fare manutenzione). Il processo software è l'equivalente per il software: stabilisce **quali attività** svolgere, **in che ordine**, **chi** le svolge e **cosa** producono.

Non esiste il processo perfetto

Non esiste un processo universale che funzioni sempre. Esistono (e si usano) **molti processi diversi**, e ognuno va **adattato alla situazione**: al tipo di software, all'azienda, al cliente, al team.

Per capire perché, basta confrontare due progetti molto diversi:

	Software di un pacemaker	App di una startup
Conseguenze di un errore	Possono essere letali	Utenti insoddisfatti, perdita di clienti
Stabilità dei requisiti	Molto stabili, fissati da normative	Cambiano di settimana in settimana
Documentazione	Estesa e obbligatoria (certificazioni)	Ridotta al minimo indispensabile
Priorità	Sicurezza e correttezza	Velocità di rilascio e feedback
Processo adatto	Rigoroso, pianificato, molto controllato	Leggero, iterativo, flessibile

Tabella 2.1: Contesti diversi richiedono processi diversi.

Le quattro attività fondamentali

Per quanto i processi possano essere diversi tra loro, **tutti** includono, in qualche forma, quattro attività fondamentali.

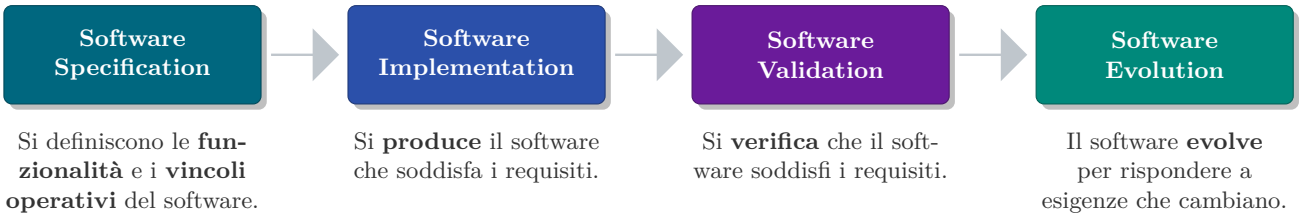


Figura 2.1: Le quattro attività fondamentali presenti in ogni processo software.

Software Specification (specifica)

È l'attività in cui si stabilisce **cosa** il sistema deve fare e **sotto quali vincoli** deve operare. Si parla con il cliente e con gli utenti, si analizza il dominio del problema e si mettono per iscritto i requisiti.

Esempio 2.1 — Specifica di un sistema per la prenotazione degli esami

Funzionalità: lo studente può vedere gli appelli disponibili, prenotarsi e cancellare la prenotazione.
Vincoli operativi: il sistema deve gestire 5.000 prenotazioni contemporanee nel giorno di apertura degli appelli e deve essere accessibile anche da smartphone.

Software Implementation (sviluppo)

È l'attività in cui si **produce** il software che soddisfa la specifica. Attenzione: nonostante il nome, non si tratta solo di scrivere codice. L'implementazione in senso ampio comprende la **progettazione** (come organizzare il sistema) e la **programmazione** vera e propria. In alcuni testi questa attività è infatti chiamata *Software Development*.

Software Validation (validazione)

Il software prodotto va **controllato**: bisogna assicurarsi che faccia quello che la specifica richiede e che risponda alle aspettative del cliente. Questo avviene principalmente tramite revisioni e **test**.

Software Evolution (evoluzione)

Il software **non è mai «finito»**. Una volta in uso deve cambiare per rispondere a esigenze che cambiano. Alcuni esempi tipici:

- esce una **nuova versione del sistema operativo** e l'applicazione va adattata;
- viene scoperto un **bug** sfuggito ai test;
- entra in vigore una **nuova legge** che cambia le procedure (es. un nuovo calcolo delle tasse);
- il cliente chiede **nuove funzionalità**.

Esempio 2.2 — Le quattro attività su una sola funzionalità

Specifica: «lo studente può prenotarsi a un appello aperto, ma non due volte allo stesso appello».

Implementazione: la specifica diventa un metodo.

ServizioPrenotazioni.java

```java
public Prenotazione prenota(Studente studente, Appello appello) {
  if (!appello.isAperto()) {
    throw new IllegalStateException("Appello chiuso");
  }
  if (appello.haPrenotato(studente)) {
    throw new IllegalStateException("Già prenotato");
  }
  return appello.aggiungi(studente);
}
```

Validazione: un test controlla che il codice faccia ciò che la specifica chiede.

ServizioPrenotazioniTest.java

```java
@Test
void nonCiSiPuoPrenotareDueVolte() {
  servizio.prenota(anna, appelloDiGennaio);
  assertThrows(IllegalStateException.class, () -> servizio.prenota(anna, appelloDiGennaio));
}
```

Evoluzione: un anno dopo il regolamento cambia, e ogni studente può avere al massimo tre prenotazioni attive. La specifica si aggiorna, il metodo guadagna un controllo e i test un caso in più:

ServizioPrenotazioni.java

```java
if (studente.prenotazioniAttive() >= 3) {
  throw new IllegalStateException("Hai già tre prenotazioni attive");
}
```

Attività complesse, fatte da persone

- Queste attività sono **complesse** e ciascuna comprende molte **sotto-attività** (le vedremo nel modello a cascata).
- Come ogni processo creativo, i processi software si basano su **persone** che prendono **decisioni** ed esprimono **giudizi**: nessun processo può essere eseguito «meccanicamente».
- Oltre alle quattro attività principali, un processo include tipicamente **attività di supporto**, come la **pianificazione** del progetto e il suo **monitoraggio** (stiamo rispettando tempi e costi?).

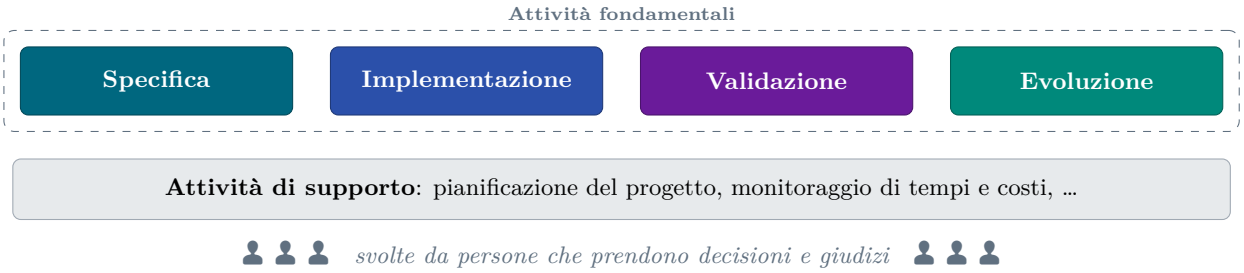


Figura 2.2: Le attività di supporto accompagnano per tutta la durata le attività fondamentali.

Il ciclo di vita del software

Le quattro attività fondamentali, ripetute nel tempo, descrivono il **ciclo di vita** del software. Si parla di «ciclo» perché l'evoluzione riporta continuamente a nuove specifiche: ogni modifica richiede di capire cosa cambiare, implementarlo, validarlo e rilasciarlo.

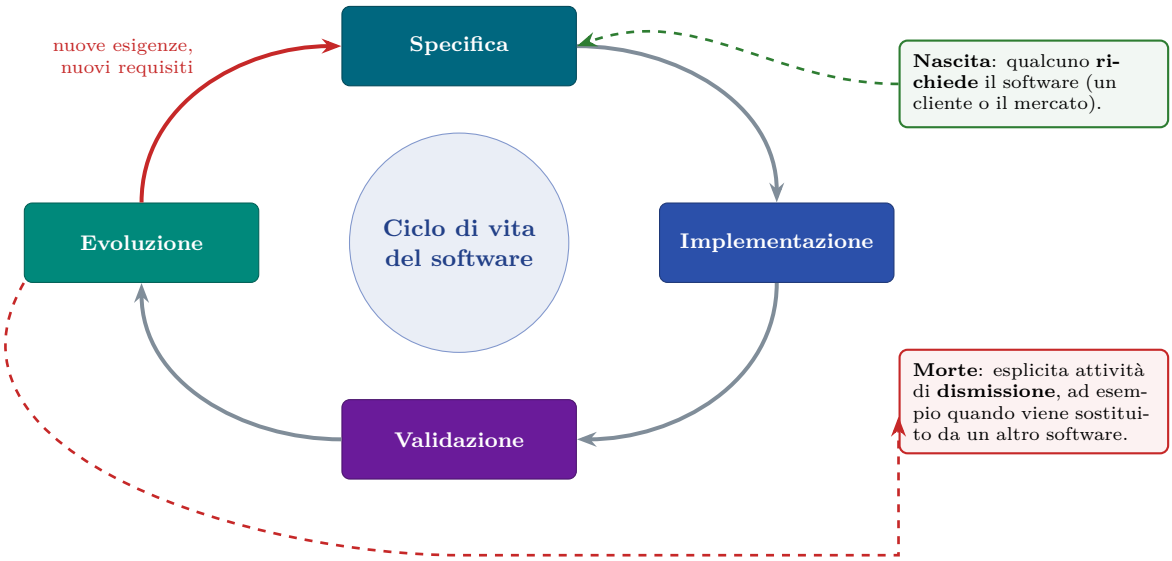


Figura 2.3: Il ciclo di vita: dalla richiesta iniziale fino alla dismissione.

Nascita e morte del software

La vita di un software **inizia** quando qualcuno ne fa richiesta e **termina** solo con un'esplicita **attività di dismissione** (ad esempio quando va sostituito con un altro sistema). Tra questi due estremi il software attraversa ripetutamente le attività di specifica, implementazione, validazione ed evoluzione.

Un aspetto spesso sottovalutato è **quanto** dura questa vita. Lo sviluppo iniziale può durare mesi, ma l'esercizio e la manutenzione possono durare **anni o decenni**: molte banche e pubbliche amministrazioni usano ancora sistemi (detti *legacy*) scritti in COBOL decenni fa.

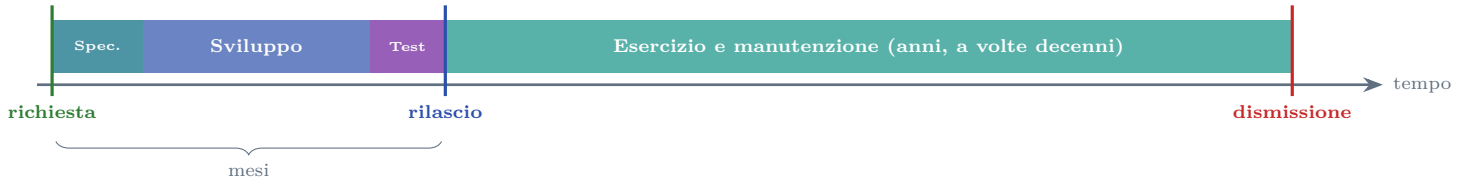


Figura 2.4: Durata qualitativa delle fasi della vita di un software: la manutenzione è di gran lunga la più lunga.

Quanto costa il software

Hardware contro software

Agli albori dell'informatica la parte costosa di un sistema era l'**hardware**. Con il passare dei decenni il costo dell'hardware è crollato, mentre il software è diventato sempre più grande e complesso. Oggi la situazione è ribaltata: la quota maggiore del costo di un sistema è il **software**.

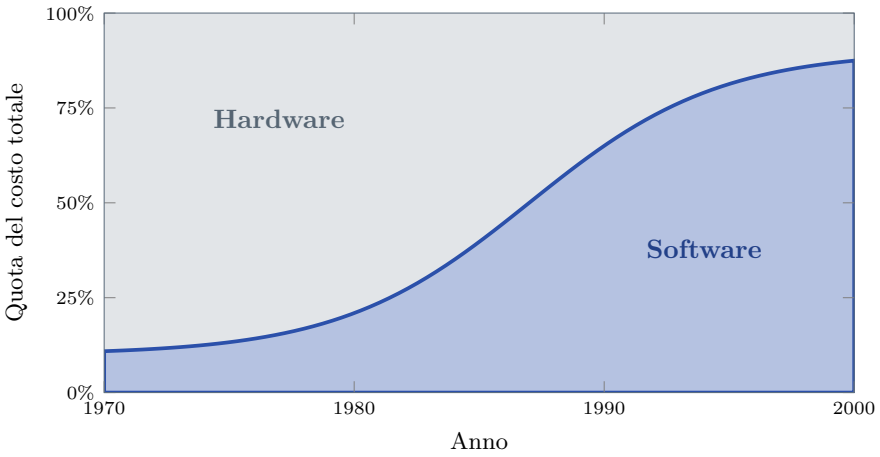


Figura 2.5: Andamento qualitativo dei costi relativi di hardware e software (da D. Bell, *Software Engineering for Students*).

Dove vanno i soldi: i costi per fase

Ancora più interessante è vedere **come** si distribuiscono i costi tra le varie fasi della vita di un software.

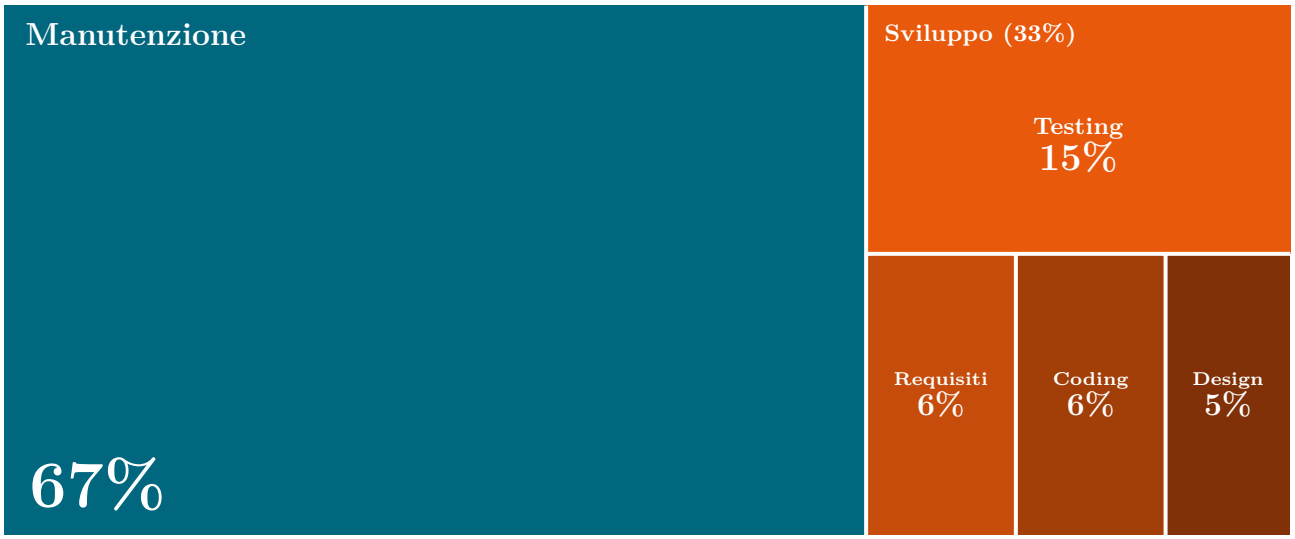


Figura 2.6: Costi relativi delle fasi di un processo software: l'area di ogni rettangolo è proporzionale al costo (da D. Bell).

Questo grafico contiene alcune delle lezioni più importanti dell'intero corso:

1. **La manutenzione costa circa il doppio di tutto lo sviluppo messo insieme.** Il grosso della spesa arriva *dopo* il rilascio.
2. **Scrivere codice è una frazione piccola del totale** (circa il 6%). Chi pensa che «fare software» significhi «programmare» sta guardando solo una piccola parte del problema.
3. **Il testing costa più della programmazione**: verificare che il software funzioni è un lavoro enorme.

La conseguenza pratica: progettare per il cambiamento

Se due terzi del costo sono manutenzione, allora conviene investire nelle fasi iniziali (requisiti e progettazione) per ottenere un software **facile da modificare**: modulare, leggibile, ben documentato e ben testato. Ogni ora risparmiata «scrivendo codice di fretta» si paga con gli interessi negli anni di manutenzione.

Prendere i numeri con le pinze

Le percentuali esatte variano molto da progetto a progetto e da studio a studio. Quello che conta è l'**ordine di grandezza**: la manutenzione domina, la scrittura del codice è una parte minoritaria.

I MODELLI DI PROCESSO

Un processo software reale è pieno di dettagli: persone, documenti, riunioni, strumenti, eccezioni. Per ragionarci sopra servono delle rappresentazioni semplificate.

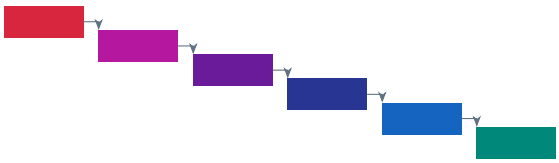
Modello di processo software (SDLC)

Un **Software Process Model**, detto anche **Software Development Life Cycle (SDLC)**, è una **rappresentazione semplificata** di un processo software. Un modello può concentrarsi su una particolare **prospettiva** (ad esempio la sequenza delle attività, i ruoli coinvolti o i documenti prodotti).

Il rapporto tra modello e processo è lo stesso che c'è tra una **cartina** e il **territorio**: la cartina non riporta ogni albero, ma è proprio questa semplificazione a renderla utile. E come esistono cartine diverse per scopi diversi (stradale, topografica, dei sentieri), esistono modelli di processo diversi che mettono in luce aspetti diversi.

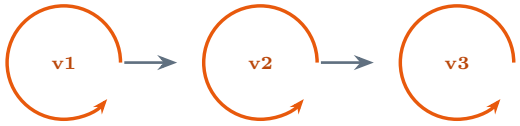
Nel corso vedremo alcuni modelli di processo molto generali:

Modelli guidati dal piano (*plan-driven*)



Tutto viene pianificato all'inizio; le fasi si susseguono in ordine. **Esempio**: il modello a cascata (*Waterfall*) — *prossimo capitolo.*

Modelli incrementali e agili



Il software cresce per piccoli incrementi, ripetendo le attività in cicli brevi. **Esempio**: modelli incrementali, metodologie agili — *più avanti nel corso.*

Figura 2.7: Due grandi famiglie di modelli di processo.

In sintesi

- Un **processo software** è un insieme di attività correlate che porta alla produzione di software. Non ne esiste uno universale.
- Ogni processo comprende **specifica, implementazione, validazione ed evoluzione**, più attività di supporto come pianificazione e monitoraggio.
- Queste attività formano il **ciclo di vita**: dalla richiesta alla dismissione, con la manutenzione come fase più lunga.
- La **manutenzione** è la voce di costo principale (circa 2/3): bisogna progettare pensando al cambiamento.
- Un **modello di processo (SDLC)** è una rappresentazione semplificata di un processo. Il primo che studiamo è il modello a cascata.

Il modello a cascata

Origini e caratteristiche

Il primo modello di processo che studiamo è anche il più semplice e il più «storico»: il **modello a cascata** (*Waterfall model*). Deriva dai modelli di processo usati nell'ingegneria dei grandi **sistemi militari** ed è stato descritto per il software da Winston W. Royce nel 1970.

Le sue caratteristiche fondamentali sono quattro:

1. il processo è composto da una serie di **fasi sequenziali**: ne finisce una e ne inizia un'altra;
2. è un processo **guidato dal piano** (*plan-driven*): tutte le attività vengono pianificate all'inizio e l'avanzamento si misura rispetto al piano;
3. il risultato di ogni fase è un **documento** che deve essere **approvato** formalmente (*signed-off*);
4. la fase successiva **non può iniziare** finché il risultato della fase precedente non è completo.

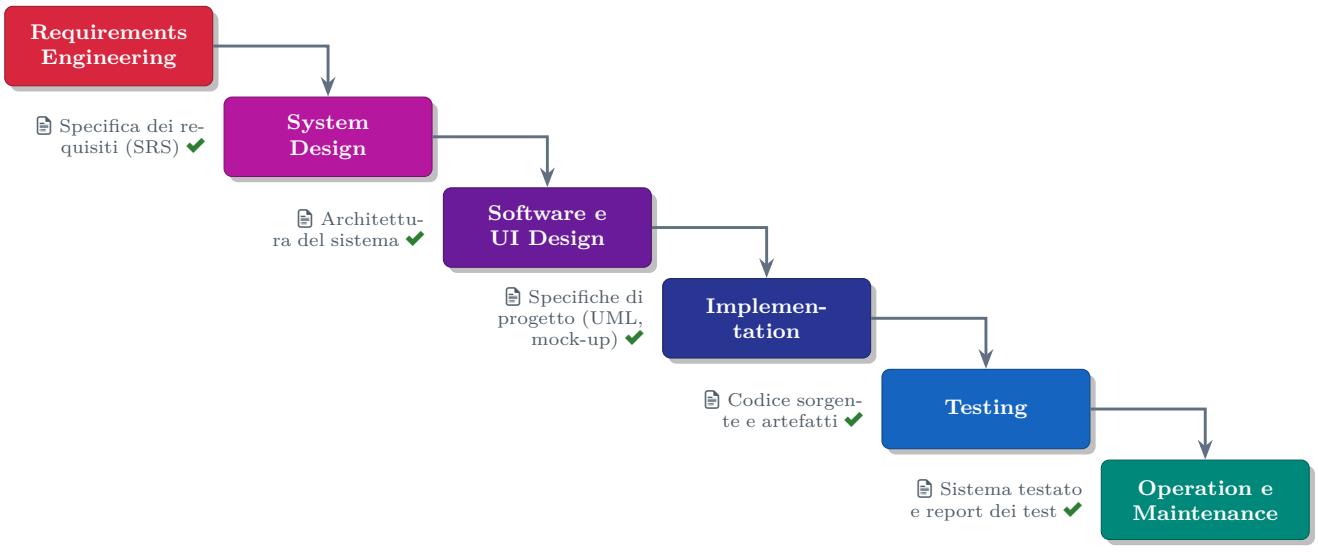


Figura 3.1: Il modello a cascata. A sinistra di ogni fase, il documento (approvato) prodotto dalla fase precedente.

Perché «a cascata»?

Il nome deriva dalla forma del diagramma: ogni fase «riversa» il proprio risultato nella successiva, proprio come l'acqua di una cascata scende da un gradino all'altro. E come l'acqua, il flusso è pensato per andare **in una sola direzione**: verso il basso.

Nome e difetto sono due cose diverse

Il nome viene dalla forma del diagramma, non dal fatto che «bisogna tornare indietro». Il ritorno all'indietro (che vedremo più avanti) è invece la **conseguenza** più critica di questa struttura: poiché il modello non prevede di risalire la cascata, correggere un errore scoperto tardi costa moltissimo.

Curiosità: Royce non era un fan della cascata pura

Nel suo articolo del 1970 Royce presentava il modello sequenziale puro come **rischioso** e proponeva già dei correttivi: tornare alle fasi precedenti, costruire un prototipo prima del sistema definitivo, coinvolgere il cliente durante il processo. Ironia della sorte, è passata alla storia soprattutto la versione rigida che lui stesso criticava.

LE SEI FASI IN SINTESI

Prima di approfondire le singole fasi, ecco una visione d'insieme.

Fase	*Domanda guida*	Cosa si fa	Output principale
Requirements Engineering	*Cosa deve fare il sistema?*	Raccolta e specifica dei requisiti insieme a clienti e utenti	Documento di specifica dei requisiti (SRS)
System Design	*Com'è organizzato il sistema, a grandi linee?*	Architettura: moduli, responsabilità, allocazione su hardware	Documento di architettura
Software e UI Design	*Come si realizza ogni modulo?*	Classi, design pattern, prototipi dell'interfaccia	Specifiche di progetto (UML, wireframe)
Implementation	*Come lo scrivo bene?*	Traduzione del progetto in codice «pulito»	Codice sorgente e artefatti
Testing	*Funziona? È ciò che serviva?*	Verifica e validazione: ispezioni e test	Report dei test, sistema pronto
Operation e Maintenance	*Come lo mantengo in vita?*	Installazione, uso reale, correzioni e adattamenti	Nuove versioni del sistema

Tabella 3.1: Panoramica delle fasi del modello a cascata.

Le sei fasi non sono altro che un modo più dettagliato di organizzare le **quattro attività fondamentali** viste nel capitolo precedente:

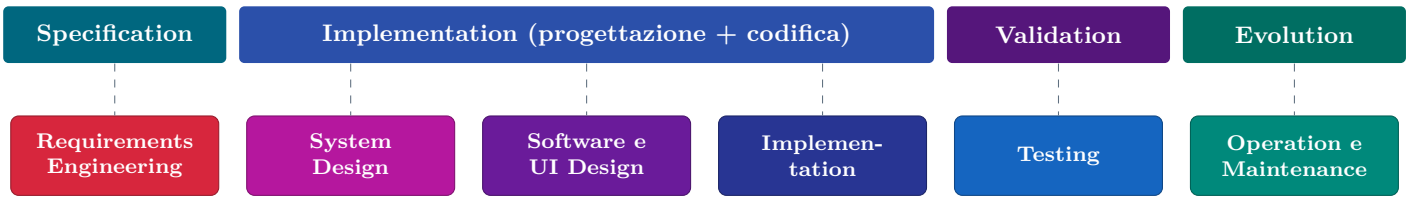
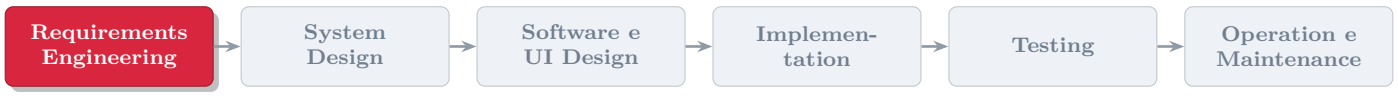


Figura 3.2: Corrispondenza tra le quattro attività fondamentali e le sei fasi del modello a cascata.

Fase 1: Requirements Engineering



Ingegneria dei requisiti

Fase in cui si cerca di capire **cosa** il software da realizzare (*software-to-be*) deve fare, **non come** implementarlo. Richiede un'analisi attenta dei bisogni degli utenti e del dominio del problema e coinvolge clienti, utenti finali e ingegneri del software. L'output principale è il **documento di specifica dei requisiti** (*Software Requirements Specification*, SRS).

È una fase di **modellazione**: non si costruisce ancora la soluzione, ma un **modello di ciò che il sistema dovrà fare**, sia in termini di **funzionalità** sia in termini di **qualità**. Per questo:

- **non si scrive codice**: si producono documenti e diagrammi;
- **non si fanno scelte tecnologiche** (linguaggi, database, framework) e non si parla di hardware, a meno che non si tratti di un **vincolo** imposto dal cliente (ad esempio: «il sistema deve girare sui server già presenti in azienda»);
- si guarda il sistema **dall'esterno**, come una scatola nera: cosa vede e cosa fa l'utente, come interagisce con il sistema, non come il sistema è fatto dentro.

Questa fase è così importante da meritare un capitolo a parte: la riprenderemo in dettaglio nel Capitolo 5.

Cosa, non come

La distinzione tra *cosa* e *come* è la più importante di questa fase. Vediamola su un sistema per una biblioteca:

Affermazione	È un requisito?
L'utente deve poter cercare un libro per titolo, autore o ISBN.	✔ Sì (cosa)
Il risultato della ricerca deve comparire entro 2 secondi.	✔ Sì (qualità)
La ricerca usa un indice sulla tabella `LIBRI` di un database MySQL.	✖ No (come)
La classe `RicercaService` espone il metodo `cerca(String)`.	✖ No (come)
Il sistema deve funzionare sui computer Windows già presenti in biblioteca.	✔ Sì (vincolo)

Tabella 3.2: Requisiti (cosa) e decisioni di progetto (come).

Requisiti funzionali e non funzionali

Il modello del sistema descrive sia le funzionalità sia le qualità. Da qui la distinzione classica:

Requisiti funzionali

Descrivono **cosa** il sistema deve fare: servizi offerti, reazioni agli input, comportamento in situazioni specifiche.

Es.: «Il bibliotecario può registrare la restituzione di un libro.»

Es.: «L'utente riceve un'e-mail tre giorni prima della scadenza del prestito.»

Requisiti non funzionali (di qualità)

Descrivono **quanto bene** il sistema deve farlo e con quali vincoli: prestazioni, sicurezza, usabilità, affidabilità...

Es.: «Il catalogo deve essere disponibile 24 ore su 24.»

Es.: «Solo il personale autenticato può modificare i prestiti.»

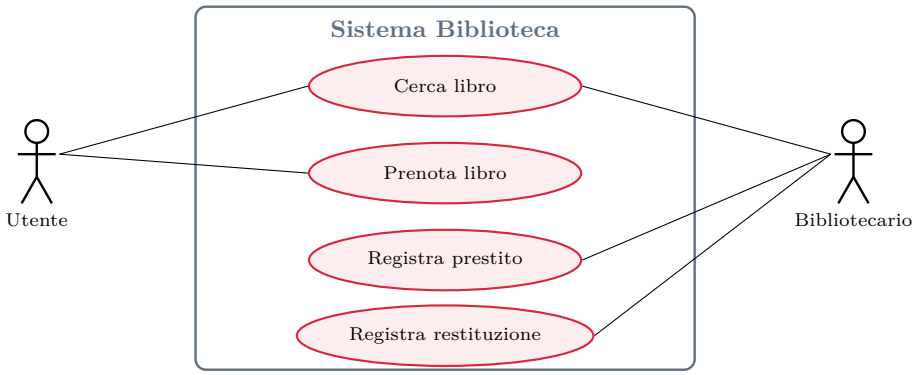
I requisiti non funzionali si collegano direttamente ai **modelli di qualità** che studieremo nel Capitolo 4.

Raccolta (*elicitation*)	
Interviste	Colloqui con gli *stakeholder* (clienti, utenti, esperti del dominio) per capire bisogni e problemi.
Personas	Utenti «tipo» immaginari ma realistici (es. «Anna, 68 anni, poco pratica di tecnologia») usati per non perdere di vista chi userà davvero il sistema.
Storie e scenari	Racconti concreti di come un utente usa il sistema per raggiungere un obiettivo.
Specifica	
Casi d'uso	Descrivono le interazioni tra gli attori (utenti o altri sistemi) e il sistema per raggiungere un obiettivo.
Linguaggio naturale	Requisiti scritti in frasi, possibilmente brevi, precise e verificabili.
Modelli di dominio	Diagrammi dei concetti del mondo reale e delle loro relazioni (es. Libro, Prestito, Utente).
Mock-up	Bozze dell'interfaccia che aiutano cliente e sviluppatori a capirsi.

Stakeholder

Qualsiasi persona o organizzazione che ha un **interesse** nel sistema o ne è **influenzata**: il cliente che paga, gli utenti finali, il personale che lo manterrà, ma anche, ad esempio, un ente regolatore.

Esempio 3.1 — Un diagramma dei casi d'uso per la biblioteca

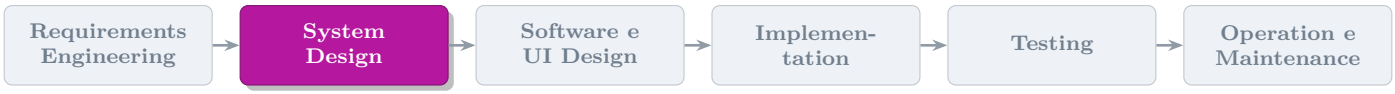


Gli **attori** (omini) sono all'esterno del sistema; i **casi d'uso** (ellissi) sono gli obiettivi che gli attori raggiungono usando il sistema. Il diagramma dice *cosa* si può fare, senza dire nulla su *come* sarà realizzato.

Il cliente non sa sempre cosa vuole

I requisiti raccolti possono essere **incompleti**, **ambigui** o persino **contraddittori** (stakeholder diversi vogliono cose diverse). Non a caso, nel progetto d'esame i docenti si comportano come clienti tipici e possono fornire requisiti incoerenti: individuarli e chiarirli fa parte del lavoro.

Fase 2: System Design



Una volta chiaro *cosa* fare, si passa al *come*. L'obiettivo delle due fasi di progettazione è definire una **struttura adeguata** (architettura) per il software. Si lavora su **due livelli**: prima quello di sistema (alto livello), poi quello del singolo modulo (basso livello).

System Design (progettazione di sistema)

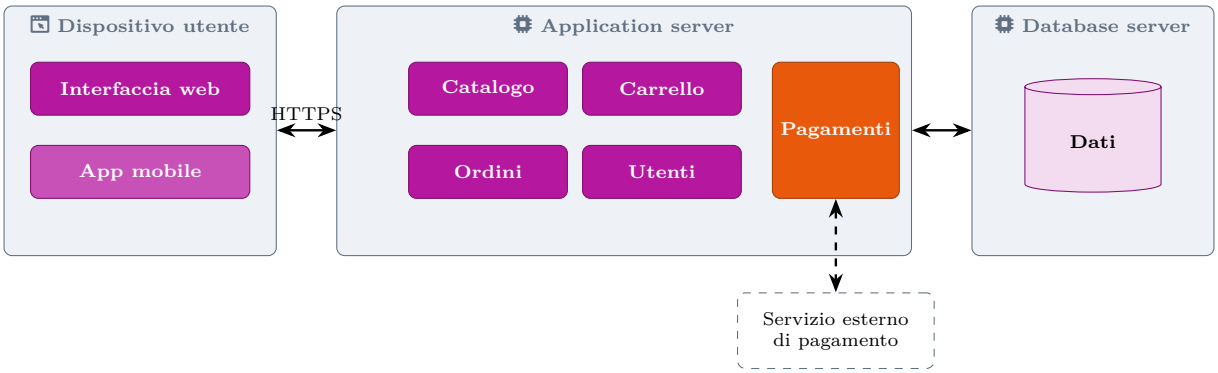
Definisce l'**architettura complessiva** del sistema, cioè la sua vista «di massima»:

- la **decomposizione** in moduli e componenti (sottosistemi);
- l'**allocazione** dei requisiti/funzionalità ai moduli e dei moduli alle risorse hardware;
- le **relazioni e collaborazioni** tra i moduli.

Un'analogia: è come la **pianta di una casa** disegnata dall'architetto. Stabilisce quante stanze ci sono, a cosa servono e come sono collegate, ma non sceglie ancora il modello delle maniglie.

In questa fase si ricorre spesso a **pattern architetturali**, cioè soluzioni collaudate per organizzare un sistema: ad esempio *client-server*, architettura *a livelli* (presentazione, logica, dati) o *Model-View-Controller*.

Esempio 3.2 — Architettura di massima di un e-commerce

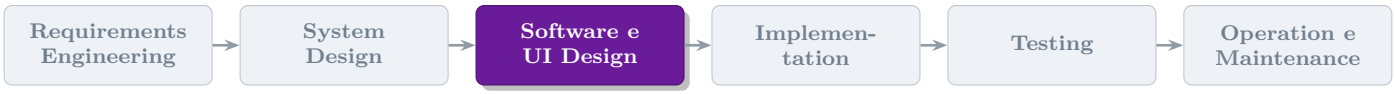


Il sistema è scomposto in **moduli** con responsabilità precise (catalogo, carrello, ordini, utenti, pagamenti), i moduli sono **allocati** a nodi hardware diversi e sono definite le **collaborazioni** (chi comunica con chi).

Perché progettare in modo modulare

Un sistema ben scomposto in moduli indipendenti è molto più **flessibile**: se domani cambia il fornitore dei pagamenti, si interviene solo sul modulo *Pagamenti* senza toccare il resto. Ricordando che la manutenzione vale circa due terzi dei costi, la modularità decisa in questa fase è un investimento che ripaga per anni.

Fase 3: Software e UI Design



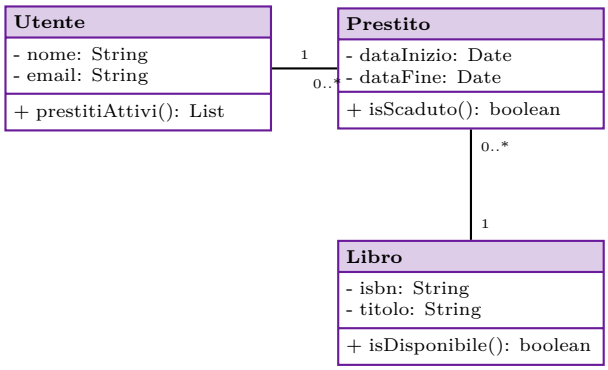
Software e UI Design (progettazione di dettaglio)

Scende nel **dettaglio di come implementare ciascun modulo**. Comprende la progettazione dell'architettura software di basso livello (quali **classi e oggetti** servono per realizzare ogni sottosistema?) e la **prototipazione dell'interfaccia utente**.

Le principali attività e tecniche sono:

- definire gli **oggetti** (classi, attributi, metodi, relazioni) necessari a realizzare ogni sottosistema;
- applicare i **design pattern**, cioè soluzioni riusabili a problemi di progettazione ricorrenti;
- applicare l'**ingegneria dell'usabilità** per progettare interfacce facili da usare;
- produrre **wireframe ad alta fedeltà**, cioè prototipi dell'interfaccia molto vicini al risultato finale.

L'output è un insieme di **specifiche di progetto**, spesso formalizzate con linguaggi di modellazione come **UML**.



(a) Diagramma delle classi UML



(b) Wireframe di una schermata

Figura 3.3: Due tipici prodotti della progettazione di dettaglio.

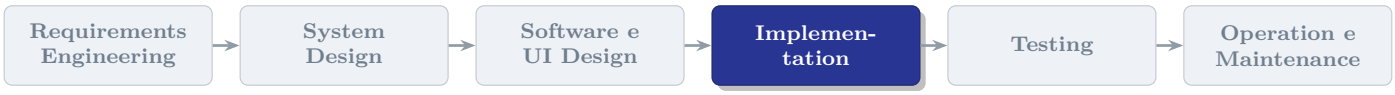
Alto livello e basso livello a confronto

	System Design	Software e UI Design
Livello	Alto: il sistema nel suo insieme	Basso: il singolo modulo
Elementi	Sottosistemi, componenti, nodi hardware	Classi, oggetti, metodi, schermate
Strumenti	Pattern architetturali	Design pattern, usabilità, wireframe
Analogia	La pianta della casa	Il progetto esecutivo di ogni stanza (impianti, arredi)
Domanda	Quali pezzi e come comunicano?	Come è fatto dentro ogni pezzo?

Scelte tecnologiche e design pattern: due precisazioni

- **Il linguaggio**: le scelte tecnologiche (linguaggio, framework, database) si concretizzano durante la progettazione, quando servono a definire la soluzione, a meno che non siano già un vincolo imposto nei requisiti. Il criterio non è solo «il linguaggio migliore in assoluto»: contano l'adeguatezza al problema, le librerie disponibili e le **competenze del team**.
- **I design pattern** sono in larga parte **indipendenti dal linguaggio**: descrivono un'idea di soluzione (ad esempio «notifica automaticamente gli interessati quando un oggetto cambia stato»). È la loro *implementazione* che dipende dalle caratteristiche del linguaggio scelto.

Fase 4: Implementation



Implementazione

Fase in cui la specifica di progetto viene **tradotta** nel linguaggio di programmazione e nelle tecnologie scelte. Ogni sottosistema viene implementato, producendo il codice sorgente e gli altri artefatti necessari.

Non una traduzione qualsiasi, però: deve essere una traduzione di **alta qualità**. Il codice risultante deve essere:

Leggibile (*readable*)	Un altro sviluppatore (o noi tra sei mesi) lo capisce senza fatica?
Manutenibile (*maintainable*)	È facile correggerlo o modificarlo senza rompere altro?
Riusabile (*reusable*)	Parti del codice possono essere usate in altri contesti?
Estendibile (*extendable*)	Si possono aggiungere funzionalità senza stravolgere l'esistente?
Testabile (*testable*)	È facile scrivere test automatici che ne verifichino il comportamento?

…o, in una sola parola, **clean** (pulito). Oltre al codice, in questa fase si usano **framework** (strutture già pronte che forniscono le fondamenta dell'applicazione) e **ORM** (*Object-Relational Mapping*: librerie che traducono automaticamente gli oggetti del programma in righe delle tabelle di un database relazionale e viceversa).

Esempio 3.3 — Stesso comportamento, qualità molto diversa

Entrambe le versioni calcolano il totale di un carrello applicando gli sconti. La prima «funziona», ma è illeggibile:

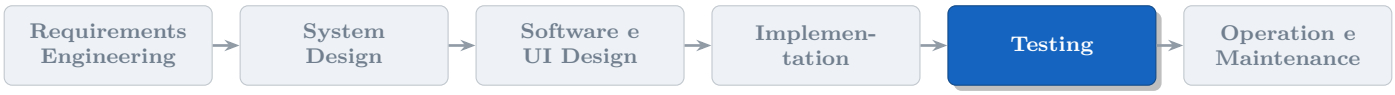
```java
public double c(List<double[]> l) {
    double t = 0;
    for (double[] x : l) t += x[0] * x[1] * (1 - x[2]);
    return t;
}
```

Cosa sono `x[0]`, `x[1]` e `x[2]`? Senza leggere il resto del programma è impossibile saperlo. La seconda versione dice esattamente cosa fa:

```java
public double calcolaTotaleCarrello(List<RigaCarrello> righe) {
    double totale = 0;
    for (RigaCarrello riga : righe) {
        totale += riga.getPrezzoUnitario()
                * riga.getQuantita()
                * (1 - riga.getSconto());
    }
    return totale;
}
```

Nomi espressivi e tipi dedicati rendono il codice leggibile, più facile da modificare (es. aggiungere l'IVA) e da testare.

Fase 5: Testing



Abbiamo un'implementazione. Ma questa implementazione **soddisfa davvero i bisogni degli utenti**? Per rispondere si svolgono le attività di **verifica e validazione** (V&V). Sono due concetti diversi e spesso confusi.

Verifica (Verification)

Il sistema è **conforme alla sua specifica**? In altre parole: *abbiamo costruito il prodotto nel modo giusto?* Si fa tipicamente con il **testing del programma**: si esegue il software in un ambiente controllato e si controlla che il suo comportamento sia corretto rispetto alle specifiche.

Validazione (Validation)

Il sistema **soddisfa le aspettative del cliente**? In altre parole: *abbiamo costruito il prodotto giusto?* Si fa tipicamente definendo ed eseguendo **test di accettazione**, spesso insieme al cliente.

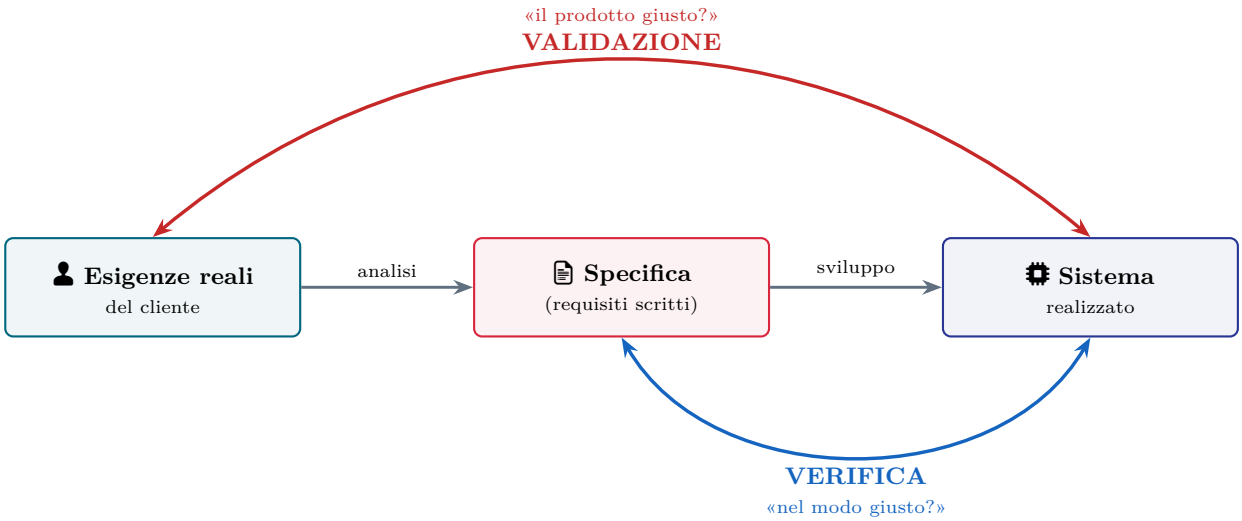


Figura 3.4: La verifica confronta il sistema con la specifica; la validazione lo confronta con le esigenze reali.

Esempio 3.4 — Verificato ma non validato

La specifica dice: «il sistema deve esportare il report mensile delle vendite in PDF». Gli sviluppatori realizzano un'esportazione PDF perfetta: tutti i test passano, quindi il sistema è **verificato**. Alla consegna, però, il cliente scopre che il suo ufficio contabilità ha bisogno di *rielaborare* quei dati in un foglio di calcolo: un PDF non gli serve a nulla. Il sistema **non è validato**. L'errore non è nel codice, ma nei requisiti.

Le attività di testing

- **Ispezioni del codice** (*code inspection*): il codice viene letto ed esaminato da altre persone *senza eseguirlo*, alla ricerca di errori e cattive pratiche.
- **Test funzionale**: si esegue il software e si confrontano i risultati con quelli attesi. Si svolge a più livelli: *unità*, *integrazione*, *sistema*.
- **Test di usabilità**: si osservano utenti reali mentre usano il sistema per scoprire difficoltà dell'interfaccia.

I livelli del test funzionale procedono dal piccolo al grande: si controlla che **metodo per metodo** e **classe per classe** il codice faccia quello che deve, poi che i **moduli** funzionino insieme e infine che l'**intero sistema** rispetti i requisiti.

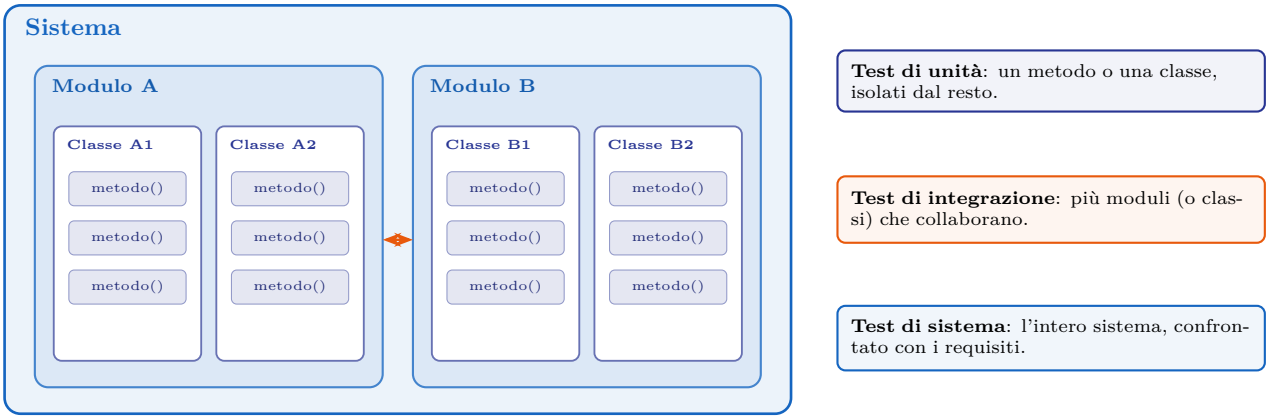


Figura 3.5: I livelli del test funzionale, dal più piccolo al più grande.

Esempio 3.5 — Un test di unità con JUnit

Il test di unità della `calcolaTotaleCarrello` vista nella fase di implementazione: si prepara un carrello noto, si chiama il metodo e si confronta il risultato con quello atteso, calcolato a mano. JUnit esegue il test in un istante, e lo si può ripetere a ogni modifica.

CarrelloTest.java

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

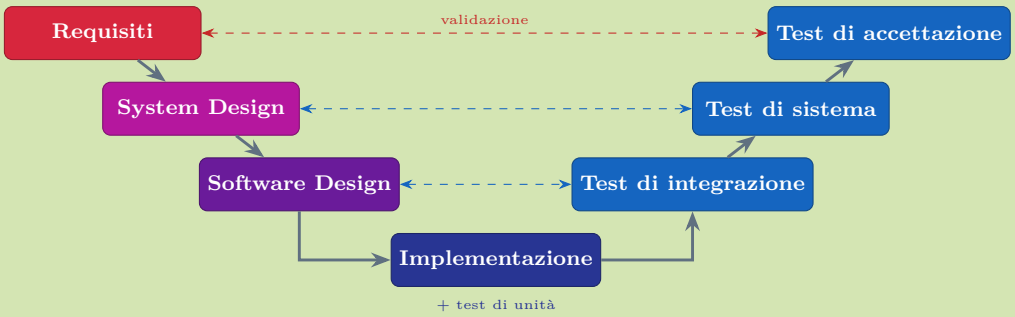
class CarrelloTest {
  @Test
  void applicaLoScontoRigaPerRiga() {
    List<RigaCarrello> righe = List.of(
        new RigaCarrello(10.00, 2, 0.0),     // 2 pezzi da 10 euro, senza sconto: 20
        new RigaCarrello(50.00, 1, 0.2));    // 1 pezzo da 50 euro, sconto del 20%: 40

    double totale = new Carrello().calcolaTotaleCarrello(righe);

    assertEquals(60.00, totale, 0.001);      // il terzo argomento e' la tolleranza sui double
  }
}
```

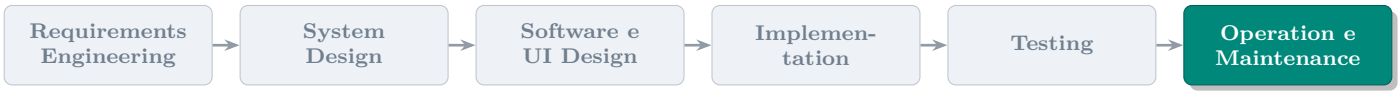
Per approfondire: il modello a V

Una variante molto diffusa del modello a cascata, il **modello a V**, rende esplicito che ogni fase di progettazione ha una fase di test «gemella» che ne verifica il risultato:



Il testing verrà ripreso durante il corso ed è l'argomento centrale del corso magistrale di *Software Testing*.

Fase 6: Operation e Maintenance



Esercizio (Operation)

Il sistema viene **distribuito e installato** presso i clienti e messo in **uso reale**.

Manutenzione (Maintenance)

Prima o poi il software **dovrà cambiare**. La manutenzione comprende tutte le modifiche apportate dopo il rilascio.

I motivi per cui il software cambia sono essenzialmente tre:

- cambiano le **esigenze del cliente** (nuove funzionalità, processi aziendali diversi);

- cambia il **contesto d'uso** (ad esempio entra in vigore una nuova legge che richiede procedure diverse, oppure esce un nuovo sistema operativo);
- emergono **bug** sfuggiti alla fase di verifica e validazione.

Questi motivi corrispondono alla classificazione tradizionale dei tipi di manutenzione:

Tipo	Scopo	Esempio
Correttiva	Correggere errori non scoperti nelle fasi precedenti	Il calcolo degli interessi sbaglia l'arrotondamento negli anni bisestili
Adattiva	Adattare il software a un ambiente che cambia	Aggiornare l'app per la nuova versione di Android; recepire una nuova normativa fiscale
Perfettiva	Migliorare o aggiungere funzionalità su richiesta	Il cliente chiede di poter pagare anche con lo smartphone
Preventiva	Rendere il software più facile da mantenere in futuro	Ristrutturare (*refactoring*) un modulo diventato troppo complicato

Tabella 3.3: I tipi di manutenzione.

Esempio 3.6 — Manutenzione correttiva, in pratica

Un cliente segnala che il 29 febbraio 1900 risulta una data valida. Il difetto è nella regola degli anni bisestili:

Calendario.java

```java
// prima: divisibile per 4 non basta
static boolean bisestile(int anno) {
  return anno % 4 == 0;
}

// dopo la correzione: i secoli sono bisestili solo se divisibili per 400
static boolean bisestile(int anno) {
  return (anno % 4 == 0 && anno % 100 != 0) || anno % 400 == 0;
}
```

Insieme alla correzione si aggiunge un **test di regressione**, che riproduce il caso segnalato e impedisce al difetto di tornare con una modifica futura:

CalendarioTest.java

```java
@Test
void ilMilleNovecentoNonEraBisestile() {
  assertFalse(Calendario.bisestile(1900));
  assertTrue(Calendario.bisestile(2000));
  assertTrue(Calendario.bisestile(2024));
}
```

Un problema specifico della cascata

Nel modello a cascata la manutenzione è l'ultima fase, ma una modifica importante (ad esempio un nuovo requisito) non si può «fare e basta»: richiede di **ripercorrere le fasi** a partire da quella interessata, aggiornando e riapprovando requisiti, progetto, codice e test. È qui che la rigidità del modello si fa sentire di più. L'evoluzione del software è approfondita nel corso magistrale di *Software Project Management and Evolution*.

Il problema del ritorno all'indietro

Un approccio rigido e «a senso unico» (*feed-forward*) ha senso nell'**ingegneria dell'hardware**, dove i costi di produzione sono altissimi: una volta prodotti migliaia di chip, o costruito un ponte, non si torna indietro. Per questo è essenziale aver completato e approvato ogni fase prima di passare alla successiva.

Nello sviluppo software, invece, le fasi **si sovrappongono e si scambiano informazioni** di continuo:

- durante il **System Design** si scoprono problemi nei **requisiti** (ambigui, incompleti, impossibili da realizzare);
- durante l'**implementazione** si scoprono problemi nel **progetto**;
- e soprattutto i **requisiti possono cambiare** in qualsiasi momento.

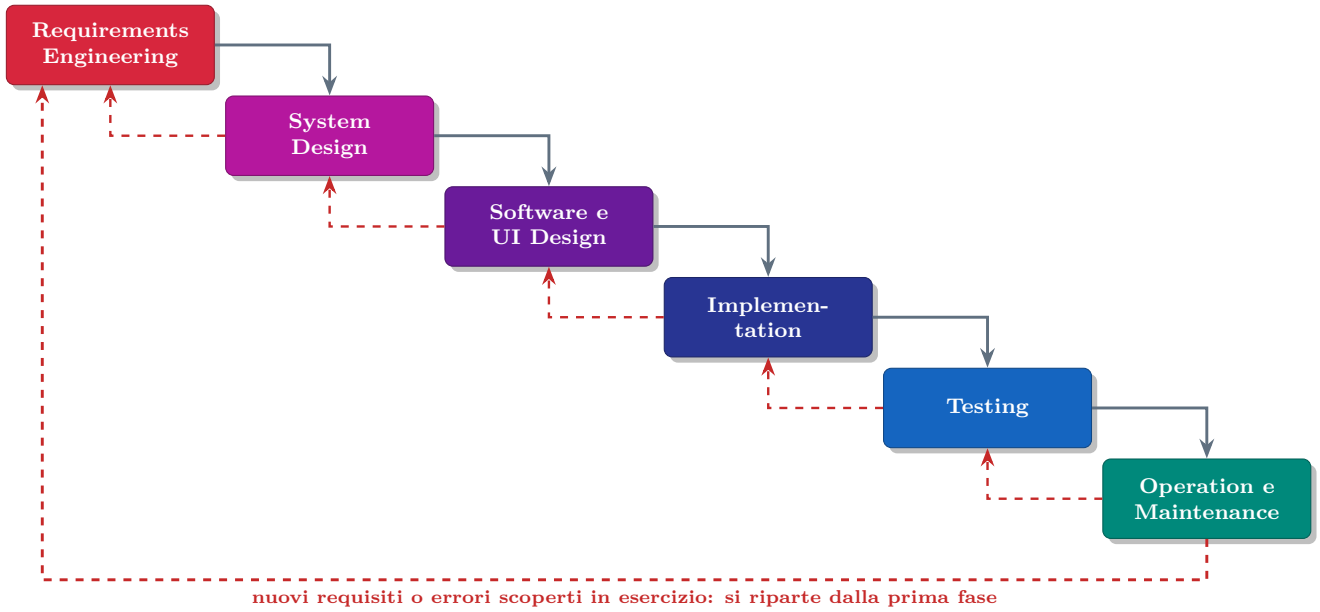


Figura 3.6: Nella realtà ogni fase può rimandare a quelle precedenti (frecce tratteggiate).

Supponiamo di aver terminato il software e di accorgerci che una parte della specifica era sbagliata, oppure che il cliente vuole una nuova funzionalità. Nel modello a cascata bisogna **risalire** fino alla fase dei requisiti e poi ridiscendere tutte le fasi: aggiornare la specifica, rivedere l'architettura e il progetto, modificare il codice, ripetere i test, e riapprovare ogni documento.

Il risultato è che **più tardi si scopre un errore, più costa correggerlo**: un requisito sbagliato corretto durante l'analisi richiede di modificare un paragrafo; lo stesso errore scoperto dopo il rilascio può richiedere di rifare progetto, codice e test. Questo andamento è stato quantificato da Barry Boehm: prendendo come unità il costo di correzione in fase di requisiti, l'ordine di grandezza cresce in modo esplosivo.

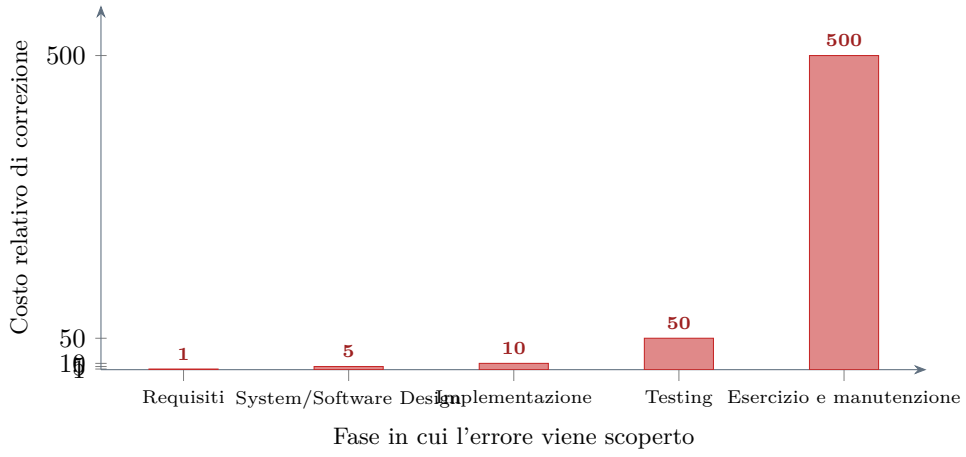


Figura 3.7: Costo relativo per correggere un errore, in funzione della fase in cui viene scoperto (B. W. Boehm, *Software Engineering Economics*).

La criticità principale

Il difetto più grave del modello a cascata è la sua **scarsa flessibilità**: è costruito sull'ipotesi che i requisiti siano stabili e noti fin dall'inizio, ipotesi che nel software è raramente vera.

Pregi, difetti e quando usarlo

✔ Pregi	✖ Difetti
Semplice da capire e da spiegare	Rigido: gestisce male i cambiamenti dei requisiti
Facile da pianificare e da gestire: fasi, scadenze e responsabilità sono chiare	Il cliente vede il software funzionante solo alla fine
Produce molta documentazione, utile per certificazioni e manutenzione	Gli errori nei requisiti emergono tardi, quando costano di più
Adatto a contratti in cui tutto va definito a priori	Molto tempo speso a produrre e approvare documenti
Il progresso è misurabile (fase completata = documento approvato)	I team di fasi diverse comunicano solo tramite documenti

Tabella 3.4: Pregi e difetti del modello a cascata.

Il problema dei team «a compartimenti stagni»

Il modello a cascata si sposa bene con un'organizzazione in cui esiste un **gruppo specializzato per ogni fase**: analisti, architetti, sviluppatori, tester. Sembra efficiente, ma introduce un problema serio: ogni gruppo riceve un documento dal gruppo precedente, lo interpreta e lo «passa oltre il muro» al successivo. Come nel gioco del **telefono senza fili**, a ogni passaggio si perde o si distorce un po' di informazione.



Figura 3.8: Passaggi di consegne tra team specializzati: i «muri» tra le fasi filtrano l'informazione.

Per un'azienda di dimensioni medio-piccole, con requisiti che evolvono e la necessità di mostrare presto risultati al cliente, questa organizzazione è spesso inadatta: si preferiscono modelli **incrementali** o **agili**, con team che seguono il prodotto dall'inizio alla fine e ricevono feedback frequenti dal cliente.

Quando ha senso usarlo

✔ **Adatto quando...**

- i requisiti sono **chiari, stabili** e ben compresi;
- il sistema è **critico per la sicurezza** e richiede documentazione e certificazioni;
- il progetto è grande e coinvolge **più aziende** che lavorano su parti diverse;
- il software è legato a **hardware** sviluppato in parallelo.

✖ **Poco adatto quando...**

- i requisiti sono **incerti** o destinati a cambiare;
- il cliente vuole vedere presto **risultati concreti**;
- il prodotto è **innovativo** e va scoperto strada facendo;
- il team è piccolo e deve reagire in fretta al mercato.

In sintesi

- Il **modello a cascata** è un processo **plan-driven** a fasi sequenziali: ogni fase produce un documento approvato e la successiva non inizia prima.
- Le fasi sono: **Requirements Engineering** (cosa, non come), **System Design** (architettura di massima), **Software e UI Design** (dettaglio di moduli e interfaccia), **Implementation** (codice pulito), **Testing** (verifica e validazione), **Operation e Maintenance**.
- **Verifica**: «abbiamo costruito il prodotto nel modo giusto?». **Validazione**: «abbiamo costruito il prodotto giusto?».
- Il limite principale è la **rigidità**: tornare indietro è costoso, e il costo di correzione cresce di ordini di grandezza fase dopo fase.
- È adatto a requisiti stabili e sistemi critici; per contesti dinamici si preferiscono modelli incrementali e agili.

Parte II

La qualità del software

La qualità del software

Nei capitoli precedenti la parola *qualità* è comparsa continuamente: l'Ingegneria del Software vuole produrre «software di alta qualità, nei tempi e nei costi previsti». Ma cosa significa, esattamente, che un software è di qualità?

Sapere rispondere a questa domanda serve da due punti di vista opposti:

- come **produttori**, per capire se ciò che stiamo costruendo è davvero di qualità (e per dimostrarlo al cliente);
- come **committenti o finanziatori**, per capire se il software che stiamo pagando vale quello che costa.

Il problema è che la qualità è **difficile da quantificare**: non esiste un unico numero che la riassuma. Per questo servono più definizioni, organizzate in modelli condivisi.

Cos'è la qualità di un prodotto

La qualità di un prodotto è un concetto **complesso**. Proviamo a pensarci fuori dal software: cosa rende «di qualità» questi oggetti?

Automobile	Orologio	Componenti meccanici
Si guasta raramente? Consuma poco carburante? Costa poco mantenerla?	È molto preciso? Resiste ad acqua e polvere? Resiste ai graffi?	Hanno tolleranze ridotte? Rispettano le specifiche? Resistono all'usura?

Già da questi esempi emergono due cose:

- la qualità dipende dal **tipo di prodotto**: la precisione conta per un orologio, i consumi per un'auto;
- la qualità dipende da **chi la giudica**: per il guidatore conta il consumo, per chi la ripara la facilità di manutenzione.

Cos'è la qualità del software?

Con il software è ancora più difficile. Vediamo tre esempi famosi.

Esempio 1: efficiente, ma comprensibile?

Questa è l'implementazione della radice quadrata inversa ($1/\sqrt{x}$) contenuta nel videogioco *Quake III Arena*, attribuita a John Carmack. Il calcolo di $1/\sqrt{x}$ è usatissimo in computer grafica (ad esempio per calcolare angoli di incidenza e riflessione della luce).

```c
float Q_rsqrt(float number) {
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
    i  = * ( long * ) &y;
    i  = 0x5f3759df - ( i >> 1 );
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );

    return y;
}
```

All'epoca questa approssimazione era circa **4 volte più veloce** di `(float)(1.0/sqrt(x))` e molti la considerano un esempio di programmazione geniale. Ma sapresti dire come funziona?

Come funziona (in breve) e perché è un problema

Il codice legge i bit del `float` come se fossero un intero, li combina con la «costante magica» `0x5f3759df` per ottenere una prima stima del risultato, riconverte i bit in `float` e infine raffina la stima con un passo del metodo di Newton. Il risultato è velocissimo, ma:

- è quasi **incomprensibile** senza una spiegazione matematica dettagliata (la costante non ha nome né commento);
- è **fragile**: presuppone che `long` occupi 32 bit come il `float`; su molti sistemi a 64 bit `long` occupa 64 bit e il codice non funziona più correttamente;
- è difficile da **modificare** e da **testare** in modo affidabile.

Quindi: essere efficienti è sinonimo di alta qualità? **Dipende da quale qualità ci interessa.**

Esempio 2 e 3: sicuro? facile da usare?

OpenBSD

Molti lo considerano un sistema operativo di altissima qualità: è molto stabile e difficile da violare. Secondo il progetto stesso, nell'installazione predefinita sono state scoperte solo **due** vulnerabilità sfruttabili da remoto in tutta la sua storia.

Essere molto sicuri contro gli attacchi significa essere di alta qualità?

Vim

Molti lo considerano il miglior editor di testo in assoluto: gli utenti esperti lavorano a velocità impressionanti. Altri lo trovano troppo difficile da imparare (è celebre la domanda «come si esce da Vim?») e quindi di bassa qualità.

La qualità dipende da chi lo usa?

La lezione: tanti punti di vista

Non esiste **una sola** qualità, ma molti approcci e punti di vista diversi. Ogni *stakeholder* guarda aspetti differenti:



Figura 4.1: La qualità del software vista da stakeholder diversi.

Serve quindi un **modello condiviso** che metta ordine tra tutti questi aspetti. È ciò che forniscono gli standard internazionali.

I MODELLI DI QUALITÀ ISO/IEC

La famiglia di standard **ISO/IEC 25000** (detta *SQuaRE, Systems and software Quality Requirements and Evaluation*) si occupa della qualità del software. Lo standard **ISO/IEC 25002** descrive i **modelli di qualità** del software, che sono due:

Product Quality Model

Modello di qualità del prodotto

Composto da **9 caratteristiche**, suddivise in sotto-caratteristiche, che riguardano le **proprietà del prodotto**. Fornisce un riferimento per **specificare, misurare e valutare** la qualità dei prodotti.

Punto di vista: il software in sé.

influenza

Quality-in-use Model

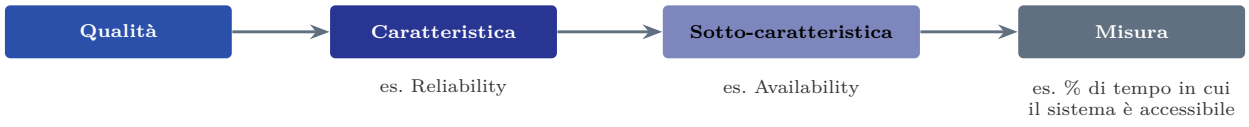
Modello di qualità in uso

Composto da **3 caratteristiche**, suddivise in sotto-caratteristiche, che descrivono l'effetto del prodotto sugli **stakeholder** quando viene **usato in uno specifico contesto d'uso**.

Punto di vista: il software mentre qualcuno lo usa.

Figura 4.2: I due modelli di qualità: il primo guarda il prodotto, il secondo il prodotto in uso.

Entrambi i modelli sono **gerarchici**: la qualità si scompone in **caratteristiche**, ogni caratteristica in **sotto-caratteristiche**, e queste ultime sono abbastanza concrete da poter essere **misurate**.



Nota sui nomi

Il modello del prodotto è descritto in dettaglio nello standard **ISO/IEC 25010**. Nella sua revisione più recente alcune caratteristiche hanno nomi diversi da quelli usati a lezione: ad esempio *Usability* è chiamata *Interaction Capability*. In questa dispensa manteniamo i nomi usati nelle slide del corso (in inglese, come nello standard).

Il modello di qualità del prodotto

Il modello di qualità del prodotto è composto da nove caratteristiche:

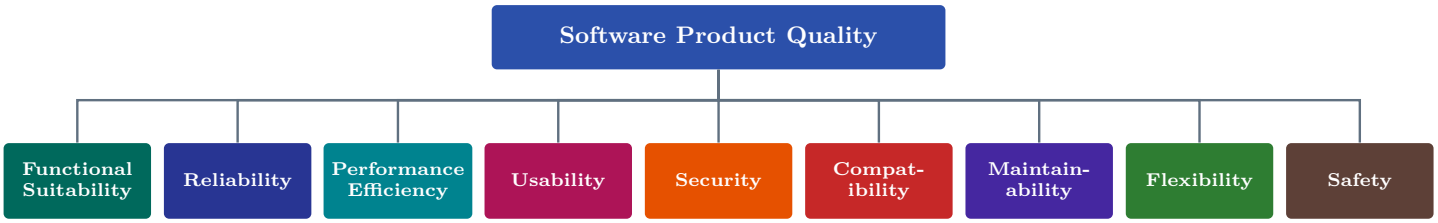


Figura 4.3: Le nove caratteristiche del modello di qualità del prodotto.

Vediamole una per una. Per ciascuna trovi una **domanda guida** che ne riassume il senso e una tabella delle sotto-caratteristiche con un esempio concreto.

Functional Suitability

Idoneità funzionale

Grado in cui un componente o un sistema fornisce funzioni che soddisfano le esigenze **dichiarate e implicite** quando viene usato nelle condizioni specificate.

Domanda guida: *Fa quello che serve, e lo fa bene?*

Sotto-caratteristica	Significato	Esempio (app di home banking)
Functional completeness *completezza*	Le funzioni coprono **tutti** i compiti e gli obiettivi specificati.	L'app permette davvero di fare bonifici, vedere il saldo, pagare bollettini, come richiesto.
Functional correctness *correttezza*	Il prodotto fornisce risultati **accurati**.	Il saldo mostrato è esatto al centesimo.
Functional appropriateness *appropriatezza*	Le funzioni **facilitano** il raggiungimento dei compiti.	Per un bonifico a un contatto frequente bastano pochi passaggi, non dieci schermate.

Reliability

Affidabilità

Grado in cui il sistema svolge le sue funzioni nelle **condizioni specificate** per un **periodo di tempo specificato**.

Domanda guida: *Funziona senza interruzioni, anche quando qualcosa va storto?*

Sotto-caratteristica	Significato	Esempio
Faultlessness *assenza di guasti*	Svolge le funzioni **senza guasti** durante il normale funzionamento.	L'app di messaggistica non si chiude inaspettatamente durante l'uso quotidiano.
Availability *disponibilità*	È **operativo e accessibile** quando serve.	Il sito delle prenotazioni è raggiungibile anche alle 8:00 del giorno di apertura degli appelli.
Fault tolerance *tolleranza ai guasti*	Funziona come previsto **nonostante** guasti hardware o software.	Se si rompe un disco del server, i dati restano accessibili grazie a una copia su un altro disco.
Recoverability *recuperabilità*	Dopo un'interruzione o un guasto, riesce a **ripristinarsi**.	Dopo un crash, l'editor riapre il documento con le modifiche salvate automaticamente.

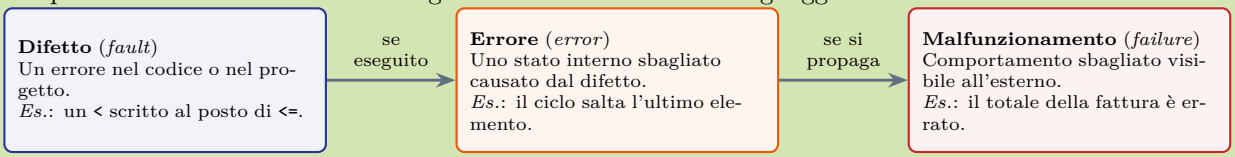
La disponibilità si esprime spesso come **percentuale di tempo** in cui il sistema è accessibile. Le percentuali sembrano tutte altissime, ma la differenza in termini di interruzioni è enorme:

Disponibilità	Interruzione massima in un anno	In pratica…
99%	≈ 3,65 giorni	Accettabile per un sito personale
99,9%	≈ 8 ore e 46 minuti	Tipico di molti servizi web
99,99%	≈ 52 minuti	Servizi importanti (e-commerce, banche)
99,999%	≈ 5 minuti	Sistemi critici (telecomunicazioni, emergenze)

Tabella 4.1: Disponibilità e tempo di inattività annuo corrispondente (un anno = 8.760 ore).

Guasto, errore, malfunzionamento

Quando si parla di affidabilità è utile distinguere tre concetti che nel linguaggio comune si confondono:



Un difetto non sempre produce un malfunzionamento (magari quel codice non viene mai eseguito). La *fault tolerance* consiste proprio nell'impedire che un difetto o un guasto diventi un malfunzionamento visibile.

Esempio 4.1 — Dal difetto al malfunzionamento, e come tollerarlo

Il metodo somma le righe di una fattura. Il difetto è un `<` al posto di `<=` (o, che è lo stesso, un `- 1` di troppo):

Fattura.java

```java
/** Somma gli importi delle righe di una fattura. */
public static double totale(double[] importi) {
  double totale = 0;
  for (int i = 0; i < importi.length - 1; i++) {   // difetto: l'ultima riga resta fuori
    totale += importi[i];
  }
  return totale;
}
```

Il difetto c'è sempre, ma diventa un **errore** solo quando il ciclo gira (lo stato interno `totale` è sbagliato) e un **malfunzionamento** solo quando quel totale arriva all'utente: `totale({10, 20, 5})` restituisce 30 invece di 35, e una fattura con una sola riga da 99,90 euro risulta di 0 euro.

La tolleranza ai guasti, invece, si progetta: qui un servizio esterno che può non rispondere viene interrogato fino a tre volte e, se non risponde, si ripiega su un valore in cache invece di propagare il guasto all'utente.

ServizioQuotazioni.java

```java
public Quotazione leggiQuotazione(String titolo) {
  for (int tentativo = 1; tentativo <= 3; tentativo++) {
    try {
      return borsa.quotazione(titolo);              // servizio remoto, puo' fallire
    } catch (IOException e) {
      log.warn("tentativo " + tentativo + " fallito: " + e.getMessage());
    }
  }
  return cache.ultimaQuotazioneNota(titolo);        // degrado controllato, non un crash
}
```

Performance Efficiency

Efficienza prestazionale

Grado in cui un prodotto svolge le sue funzioni entro i **vincoli di risorse** specificati ed è efficiente nell'uso delle risorse (CPU, memoria, spazio di archiviazione, energia, …).

Domanda guida: *Risponde in fretta, senza sprecare risorse?*

Sotto-caratteristica	Significato	Esempio
Time behaviour *comportamento temporale*	Tempi di risposta e *throughput* (lavoro svolto per unità di tempo) rispettano i requisiti.	La ricerca di un prodotto restituisce i risultati in meno di un secondo.
Resource utilization *uso delle risorse*	Quantità e tipo di risorse usate rispettano i requisiti.	L'app di navigazione non scarica la batteria dello smartphone in un'ora.
Capacity *capacità*	I **limiti massimi** di un parametro del sistema rispettano i requisiti.	Il sito di vendita dei biglietti regge 100.000 utenti connessi quando apre la vendita di un concerto.

Esempio 4.2 — Stesso risultato, comportamento temporale molto diverso

Ogni ordine del giorno va confrontato con l'elenco degli utenti bloccati. Le due versioni sono **funzionalmente identiche**: rifiutano esattamente gli stessi ordini.

ControlloOrdini.java

```java
// Versione 1: un milione di email in una lista
List<String> bloccati = caricaUtentiBloccati();
for (Ordine ordine : ordiniDiOggi) {                // centomila ordini
  if (bloccati.contains(ordine.email())) {          // scorre la lista: fino a un milione di confronti
    rifiuta(ordine);
  }
}

// Versione 2: le stesse email in un insieme basato su hash
Set<String> bloccati = new HashSet<>(caricaUtentiBloccati());
for (Ordine ordine : ordiniDiOggi) {
  if (bloccati.contains(ordine.email())) {          // un solo accesso, in media
    rifiuta(ordine);
  }
}
```

Nel caso peggiore la prima versione fa $10^5 \times 10^6 = 10^{11}$ confronti, la seconda circa 10^5 accessi: la differenza fra minuti e millisecondi. La correttezza funzionale non dice nulla sull'efficienza: è una caratteristica a sé.

Usability

Usabilità

Grado in cui un prodotto o sistema può essere **usato** dai suoi utenti, cioè quanto è facile interagire con esso.

Domanda guida: *Gli utenti riescono a usarlo senza fatica e senza sbagliare?*

Sotto-caratteristica	Significato	Esempio
Recognizability *riconoscibilità*	Gli utenti capiscono se il sistema è **adatto alle loro esigenze**.	La pagina iniziale spiega chiaramente a cosa serve il servizio.
Learnability *apprendibilità*	Le funzioni si imparano a usare **in un tempo specificato**.	Un nuovo cassiere impara a usare il gestionale del negozio in una mattina.
Operability *operabilità*	Il sistema è facile da **far funzionare e controllare**.	Le azioni più comuni sono raggiungibili con un clic.
User error protection *protezione dagli errori*	Il sistema **previene gli errori** dell'utente.	Prima di eliminare un file chiede conferma; per le date propone un calendario invece di un campo di testo libero.
Inclusivity *inclusività*	Può essere usato da persone con **background diversi** (età, abilità, culture, lingue, generi, situazioni economiche…).	L'app è disponibile in più lingue, funziona con i lettori di schermo e resta usabile su smartphone economici.

L'usabilità avrà un ruolo centrale in una parte importante del corso (Modulo B). Può fare la differenza tra **successo e fallimento** di un prodotto e, in alcuni contesti (dispositivi medici, cabine di pilotaggio, sale di controllo), perfino tra **la vita e la morte**.

Prova tu: User Inyerface

Il sito `https://userinyerface.com` è un «gioco» che simula un modulo di registrazione progettato apposta per essere **il più frustrante possibile**: pulsanti ingannevoli, caselle pre-selezionate al contrario, testi di aiuto che ostacolano. Provare a completarlo è il modo migliore per capire, per contrasto, cosa significa *usabilità*.

SECURITY

Sicurezza (protezione)

Grado in cui un sistema si **difende dagli attacchi** di soggetti malintenzionati e protegge informazioni e dati, applicando adeguati meccanismi di **autorizzazione**.

Domanda guida: *I dati sono protetti da chi non dovrebbe accedervi o modificarli?*

Sotto-caratteristica	Significato	Esempio (fascicolo sanitario / banca)
Confidentiality *riservatezza*	I dati sono accessibili **solo a chi è autorizzato**.	Solo il paziente e i suoi medici possono leggere il referto.
Integrity *integrità*	Stato e dati sono protetti da **modifiche o cancellazioni non autorizzate**.	Nessuno, tranne il medico che l'ha redatto, può alterare il referto.
Non-repudiation *non ripudio*	Si può **dimostrare** che un'azione è avvenuta, così che non possa essere negata in seguito.	Un bonifico firmato digitalmente non può essere disconosciuto da chi l'ha disposto.
Accountability *responsabilità*	Le azioni di un'entità sono **tracciabili** in modo univoco fino a essa.	Un registro (*log*) memorizza quale operatore ha consultato quale cartella clinica.
Authenticity *autenticità*	Si può **provare** che l'identità di un soggetto o di una risorsa è quella dichiarata.	L'accesso richiede credenziali più un codice inviato allo smartphone (autenticazione a due fattori).

Esempio 4.3 — Una falla di riservatezza in una riga

L'applicazione cerca i referti di un paziente costruendo la query SQL concatenando il codice fiscale scritto dall'utente:

ArchivioReferti.java

```java
// VULNERABILE: il testo dell'utente diventa parte del comando SQL
String sql = "SELECT * FROM referti WHERE paziente = '" + codiceFiscale + "'";
ResultSet referti = connessione.createStatement().executeQuery(sql);
```

Se qualcuno scrive come codice fiscale `' OR '1'='1`, la condizione diventa sempre vera e la query restituisce **i referti di tutti**: la riservatezza è persa (è la *SQL injection*). La versione corretta passa il valore come parametro, che il database tratta sempre come un dato e mai come codice:

ArchivioReferti.java

```java
PreparedStatement query = connessione.prepareStatement("SELECT * FROM referti WHERE paziente = ?");
query.setString(1, codiceFiscale);
ResultSet referti = query.executeQuery();
log.info("referti di " + codiceFiscale + " consultati da " + operatore);   // accountability
```

COMPATIBILITY *Compatibilità*

Grado in cui un sistema può **scambiare informazioni** con altri prodotti, sistemi o componenti, e/o svolgere le proprie funzioni **condividendo lo stesso ambiente** e le stesse risorse con altri sistemi.

Domanda guida: *Convive e dialoga con gli altri sistemi?*

Sotto-caratteristica	Significato	Esempio
Co-existence *coesistenza*	Funziona in modo efficiente condividendo ambiente e risorse con altri prodotti, **senza danneggiarli**.	L'antivirus non rallenta al punto da rendere inutilizzabili le altre applicazioni.
Interoperability *interoperabilità*	Può **scambiare informazioni** con altri sistemi e **usare** quelle ricevute.	Gli eventi dell'app universitaria si esportano in un formato standard e compaiono nel calendario dello smartphone.

Esempio 4.4 — Interoperabilità attraverso un formato standard

L'app universitaria non deve conoscere i calendari di Google, Apple o Microsoft: le basta esportare gli esami nel formato standard iCalendar (RFC 5545), che tutti sanno leggere.

esame.ics

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//Universita//App Esami//IT
BEGIN:VEVENT
UID:esame-igsw-20270115@app-esami.example.it
DTSTAMP:20261020T100000Z
DTSTART:20270115T090000Z
DTEND:20270115T120000Z
SUMMARY:Esame di Ingegneria del Software
LOCATION:Aula T1
END:VEVENT
END:VCALENDAR
```

MAINTAINABILITY *Manutenibilità*

Grado di efficacia ed efficienza con cui un prodotto può essere **modificato** per migliorarlo, correggerlo o adattarlo a cambiamenti dell'ambiente e dei requisiti.

Domanda guida: *Si può modificare facilmente, senza rompere nulla?*

Sotto-caratteristica	Significato	Esempio
Modularity *modularità*	Il software è composto da componenti distinti: una modifica a uno ha **impatto minimo** sugli altri.	Cambiare il fornitore dei pagamenti richiede di modificare solo il modulo *Pagamenti*.
Reusability *riusabilità*	Un modulo può essere usato come risorsa in **più sistemi**.	Il modulo di autenticazione è usato sia dal sito sia dall'app mobile.
Modifiability *modificabilità*	Può essere modificato senza introdurre difetti o degradare la qualità.	Aggiungere l'IVA al calcolo del carrello richiede di toccare un solo metodo.
Testability *testabilità*	Si possono stabilire criteri di test ed **eseguire test** per verificarli.	Ogni classe può essere testata in isolamento con test automatici.

Il collegamento con i costi

Ricordando che la manutenzione rappresenta circa **due terzi** del costo di un software, la manutenibilità è una delle caratteristiche con il maggiore impatto economico. È esattamente ciò che si ottiene con una buona progettazione modulare (System Design) e con codice pulito (Implementation).

Esempio 4.5 — Modularità e testabilità in codice

«Cambiare il fornitore dei pagamenti richiede di modificare solo il modulo Pagamenti» si ottiene facendo dipendere la cassa da un'**interfaccia**, non da un fornitore.

Cassa.java

```java
public interface GatewayPagamenti {
  Ricevuta addebita(String carta, BigDecimal importo);
}

public class Cassa {
  private final GatewayPagamenti gateway;     // un'interfaccia: Stripe, PayPal, un finto per i test...

  public Cassa(GatewayPagamenti gateway) {
    this.gateway = gateway;
  }

  public Ricevuta paga(Carrello carrello, String carta) {
    return gateway.addebita(carta, carrello.totale());
  }
}
```

La stessa scelta rende la cassa **testabile** in isolamento: nel test le si passa un gateway finto che non addebita nulla e ricorda l'ultima richiesta.

CassaTest.java

```java
@Test
void addebitaIlTotaleDelCarrello() {
  GatewayFinto finto = new GatewayFinto();
  Cassa cassa = new Cassa(finto);

  cassa.paga(carrelloDa("30.00"), "4111111111111111");

  assertEquals(new BigDecimal("30.00"), finto.ultimoImporto());
}
```

FLEXIBILITY *Flessibilità*

Grado in cui un prodotto può essere **adattato** a cambiamenti nei requisiti, nei contesti d'uso o nell'ambiente operativo.

Domanda guida: *Si adatta a nuovi ambienti, carichi e contesti?*

Sotto-caratteristica	Significato	Esempio
Adaptability *adattabilità*	Può essere adattato o trasferito a **hardware, software o ambienti** diversi.	L'applicazione desktop funziona su Windows, macOS e Linux.
Scalability *scalabilità*	Gestisce carichi di lavoro **crescenti o decrescenti**, adattando la propria capacità.	Durante il Black Friday l'e-commerce aggiunge server automaticamente, e li rilascia a fine giornata.
Installability *installabilità*	Può essere **installato e disinstallato** con successo, in modo efficace ed efficiente.	L'installazione richiede pochi clic e la disinstallazione non lascia file sparsi nel sistema.
Replaceability *sostituibilità*	Può **sostituire** un altro prodotto con lo stesso scopo nello stesso ambiente.	Il nuovo gestionale importa i dati del vecchio, così l'azienda può passare da uno all'altro.

Esempio 4.6 — Adattabilità: lo stesso percorso su tre sistemi operativi

Configurazione.java

```java
// Funziona solo su Windows: unita' C:, cartella Users, separatore "\"
File config = new File("C:\\Users\\" + utente + "\\app\\config.txt");

// Funziona ovunque: la cartella dell'utente e il separatore li sceglie la piattaforma
Path config = Path.of(System.getProperty("user.home"), "app", "config.txt");
```

Il primo programma è corretto su Windows e inutilizzabile su macOS e Linux; il secondo gira identico sui tre.

SAFETY *Sicurezza (incolumità)*

Grado in cui un prodotto **evita** di trovarsi in uno stato in cui la vita umana, la salute, i beni o l'ambiente sono messi in pericolo.

Domanda guida: *Può fare del male a persone, beni o ambiente?*

Sotto-caratteristica	Significato	Esempio
Fail safe *sicurezza in caso di guasto*	In caso di guasto si porta **automaticamente** in una modalità operativa sicura.	Se il treno perde il segnale di controllo, frena automaticamente.
Risk identification *identificazione dei rischi*	Riconosce una sequenza di eventi o operazioni che può portare a un **rischio inaccettabile**.	Il microinfusore di insulina rileva che la dose richiesta è anomala rispetto allo storico.
Hazard warning *avviso di pericolo*	Avvisa dei rischi **in tempo utile** per poter reagire.	L'auto segnala il rischio di collisione con qualche secondo di anticipo.

Esempio 4.7 — Fail safe: nel dubbio, lo stato sicuro

Il software di bordo di un treno riceve un segnale di controllo dalla linea. Se il segnale manca da più di due secondi non sa più se la via è libera, e l'unica scelta sicura è frenare.

ControlloMarcia.java

```java
public class ControlloMarcia {
  private static final long TIMEOUT_MS = 2000;
  private volatile long ultimoSegnale = System.currentTimeMillis();

  public void segnaleRicevuto() {            // chiamato a ogni segnale dalla linea
    ultimoSegnale = System.currentTimeMillis();
  }

  public void ciclo() {                      // eseguito ogni 100 ms
    if (System.currentTimeMillis() - ultimoSegnale > TIMEOUT_MS) {
      freni.emergenza();                     // segnale perso: si va nello stato sicuro
    }
  }
}
```

Security e Safety: in italiano sono entrambe «sicurezza»

In italiano usiamo una sola parola per due concetti ben distinti, e confonderli è un errore frequente:

SECURITY: proteggere il sistema dal mondo



SAFETY: proteggere il mondo dal sistema



Security riguarda attacchi *intenzionali* contro il sistema e i suoi dati (furto di password, accessi non autorizzati). **Safety** riguarda i *danni* che il sistema può causare a persone, beni o ambiente, anche senza alcun attaccante (un difetto nel software di frenata di un'auto).

Il quadro completo

Functional Suitability
- Functional Completeness
- Functional Correctness
- Functional Appropriateness

Reliability
- Faultlessness
- Availability
- Fault Tolerance
- Recoverability

Performance Efficiency
- Time Behaviour
- Resource Utilization
- Capacity

Usability
- Recognizability
- Learnability
- Operability
- User Error Protection
- Inclusivity

Security
- Confidentiality
- Integrity
- Non-repudiation
- Accountability
- Authenticity

Compatibility
- Co-existence
- Interoperability

Maintainability
- Modularity
- Reusability
- Modifiability
- Testability

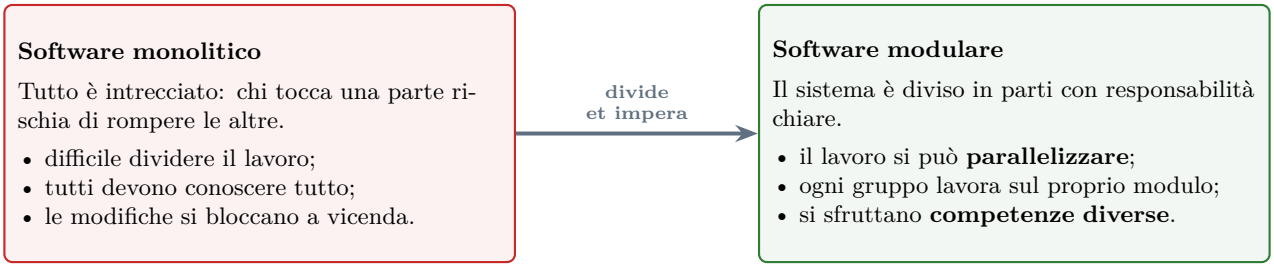
Flexibility
- Adaptability
- Scalability
- Installability
- Replaceability

Safety
- Fail Safe
- Risk Identification
- Hazard Warning

Qualità, modularità e lavoro di squadra

C'è un aspetto della qualità che si vede solo quando si passa dal «programma» al «prodotto software»: un software di qualità deve essere **flessibile**, e per essere flessibile deve essere **modulare**. È il vecchio principio del **divide et impera**: dividere un problema grande in parti più piccole e gestibili.

La modularità non serve solo alla manutenzione: è ciò che rende possibile il **lavoro in gruppo**.



Esempio 4.8 — Venti persone su un progetto

Immaginiamo un progetto da realizzare con venti sviluppatori. Se il software è un unico blocco indistinto, venti persone si ostacolano a vicenda: modificano gli stessi file, si sovrappongono, passano il tempo a coordinarsi.

Se invece il sistema è ben diviso in moduli, il lavoro si può distribuire e svolgere **in parallelo**: chi si occupa dell'interfaccia grafica lavora sulle schermate, chi è esperto di basi di dati lavora sulla persistenza, chi conosce il dominio implementa la logica applicativa. Ognuno lavora sul pezzo più adatto alle proprie competenze, e i moduli si integrano attraverso interfacce concordate.

Il legame con le caratteristiche ISO

Questa proprietà non è un'opinione, ma è codificata nel modello di qualità: è la sotto-caratteristica **Modularity** (dentro *Maintainability*), che a sua volta abilita **Reusability**, **Modifiability** e **Testability**. Ed è decisa molto presto, nella fase di **System Design**.

Il modello di qualità in uso

Il secondo modello cambia prospettiva: non guarda il software «sul banco di prova», ma il software **mentre viene usato**.

Qualità in uso

Grado in cui un prodotto o sistema può essere usato da **specifici utenti** per soddisfare le loro esigenze e raggiungere **specifici obiettivi** con **efficacia**, **efficienza**, **assenza di rischi** e **soddisfazione**, in **specifici contesti d'uso**.



Figura 4.4: Il modello di qualità in uso: 3 caratteristiche e le loro sotto-caratteristiche.

Usability (in uso)

Misura quanto gli utenti riescono a raggiungere i propri obiettivi usando il sistema in modo efficiente e soddisfacente.

- **Effectiveness** (efficacia): quanto bene gli utenti **completano** i compiti che volevano svolgere.
- **Efficiency** (efficienza): quante **risorse** (tempo, fatica) servono per completarli.
- **Satisfaction** (soddisfazione): il **comfort** e l'esperienza positiva dell'utente durante l'uso.

Usability compare due volte: perché?

- Nel **modello del prodotto**, l'usabilità è misurata sulle **caratteristiche del prodotto** (l'interfaccia protegge dagli errori? è coerente? è accessibile?).
- Nel **modello in uso**, l'usabilità è misurata dal **punto di vista degli utenti** che lo usano davvero (sono riusciti a fare quello che volevano? in quanto tempo? erano soddisfatti?).

Esempio: un navigatore può avere pulsanti grandi e menu ben organizzati (usabilità del prodotto), ma se un automobilista di notte, sotto la pioggia, non riesce a impostare la destinazione, l'usabilità in uso in quel contesto è scarsa.

Safety (in uso)

Valuta la capacità del sistema di **prevenire danni** a persone, ambiente e interessi commerciali durante l'uso.

- **Commercial damage**: quanto il sistema evita perdite economiche o danni all'attività (es. un errore nei prezzi dell'e-commerce).
- **Operator health and safety**: quanto protegge **chi lo usa** da rischi per la salute o la sicurezza.
- **Public health and safety**: quanto evita rischi o danni per il **pubblico** in generale.
- **Environmental harm**: quanto evita o riduce gli impatti negativi sull'**ambiente**.

Flexibility (in uso)

Indica la capacità del sistema di adattarsi e operare efficacemente in **contesti o ambienti diversi**.

- **Context conformity**: capacità di adattarsi ai requisiti e ai vincoli specifici di contesti diversi.
- **Context extensibility**: potenzialità di estendersi o adattarsi a contesti nuovi o in cambiamento senza modifiche significative.
- **Accessibility**: quanto efficacemente il sistema può essere usato da **tutte le persone**, incluse quelle con disabilità, in ambienti diversi.

Il legame tra i due modelli

I due modelli non sono indipendenti: le proprietà del prodotto **influenzano** la qualità percepita durante l'uso, ma quest'ultima dipende anche dal **contesto** (chi usa il sistema, per fare cosa, in quale ambiente).

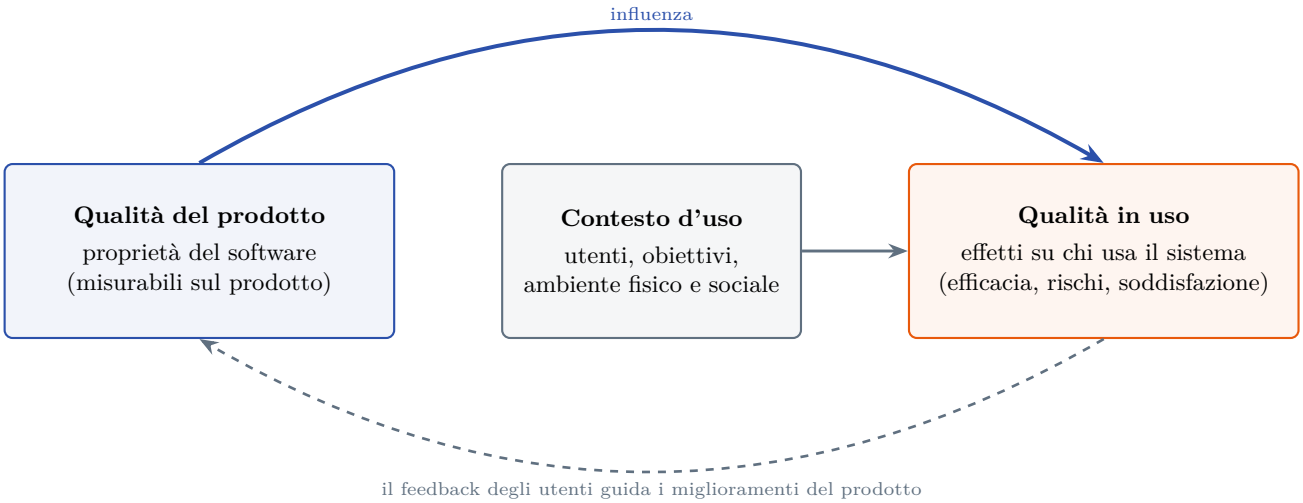


Figura 4.5: Qualità del prodotto e qualità in uso: la seconda dipende dalla prima e dal contesto.

QUALITÀ E COMPROMESSI

Un'ultima osservazione fondamentale: le caratteristiche di qualità **non si possono massimizzare tutte insieme**. Spesso migliorarne una ne peggiora un'altra.

Caratteristiche in conflitto			Perché
Security	↔	**Usability**	Password complesse e autenticazione a due fattori proteggono meglio, ma rendono l'accesso più scomodo.
Performance	↔	**Maintainability**	Il codice di Quake III è velocissimo ma quasi impossibile da capire e modificare.
Safety	↔	**Reliability** (availability)	Un treno che frena a ogni anomalia è sicuro, ma il servizio si interrompe spesso.
Flexibility	↔	**Performance**	Un'applicazione che gira su ogni piattaforma rinuncia spesso alle ottimizzazioni specifiche di ciascuna.

Tabella 4.2: Alcuni compromessi tipici tra caratteristiche di qualità.

Il ruolo dell'ingegnere

Per questo i requisiti di qualità vanno **esplicitati e ordinati per priorità** già in fase di Requirements Engineering: per il software di un pacemaker la *safety* viene prima di tutto, per un videogioco contano soprattutto prestazioni e soddisfazione. Trovare il giusto equilibrio, dati il problema e i vincoli, è esattamente il lavoro dell'ingegnere del software.

In sintesi

- La qualità del software **non è una sola**: dipende dal prodotto e dal punto di vista degli stakeholder, ed è difficile da quantificare.
- Gli standard **ISO/IEC 25000** (SQuaRE) definiscono due modelli gerarchici: **qualità del prodotto** e **qualità in uso**.
- Il modello del prodotto ha **9 caratteristiche**: Functional Suitability, Reliability, Performance Efficiency, Usability, Security, Compatibility, Maintainability, Flexibility, Safety.
- Il modello in uso ha **3 caratteristiche**: Usability (efficacia, efficienza, soddisfazione), Safety e Flexibility.
- **Security** protegge il sistema dagli attacchi; **Safety** protegge persone e ambiente dal sistema.
- La **modularità** (divide et impera) è ciò che rende il software flessibile e permette di dividere il lavoro in un team.
- Le caratteristiche sono spesso in **conflitto**: vanno prioritizzate in base al contesto.

Parte III

Ingegneria dei requisiti

Requisiti: elicitazione e analisi

Nel Capitolo 3 abbiamo incontrato la prima fase del modello a cascata, la **Requirements Engineering**, e abbiamo visto la regola d'oro: in questa fase si decide *cosa* il sistema deve fare, non *come* lo farà. Ora entriamo nel dettaglio: come si scoprono i requisiti, come si scrivono e come si evita che siano ambigui o inutilizzabili.

> *La parte più difficile nella costruzione di un sistema software è decidere con precisione cosa costruire. Nessun'altra parte del lavoro concettuale è così difficile, nessun'altra danneggia tanto il sistema finale se fatta male, nessun'altra è più difficile da correggere in seguito.*
>
> — **Fred P. Brooks** (parafrasi)

Requirements Engineering (RE)

Sotto-area dell'Ingegneria del Software che si occupa del processo di **definizione dei requisiti** per il software da realizzare. Il suo scopo è fornire agli ingegneri **metodi, tecniche e strumenti** per capire e documentare che cosa un sistema software deve fare.

Cos'è un requisito

Requisito

Un **requisito** è la descrizione di ciò che il sistema deve fare: i **servizi** che deve fornire ai suoi utenti e i **vincoli operativi** entro cui deve funzionare.

Qualche esempio, riferito a una piattaforma per la gestione dei corsi universitari:

Requisito	Tipo
Il sistema deve permettere agli studenti di cercare i corsi per parole chiave.	servizio
Il sistema deve permettere agli studenti di iscriversi a un corso.	servizio
Le interfacce utente devono essere realizzate come app native Android e iOS.	vincolo
Il sistema deve gestire almeno 10.000 studenti collegati contemporaneamente.	vincolo
Il sistema deve autenticare gli utenti anche tramite il sensore di impronte digitali del dispositivo.	servizio + vincolo

Due livelli: requisiti utente e requisiti di sistema

Nell'industria il termine «requisito» viene usato in modo non uniforme: a volte indica una descrizione generale e astratta, altre volte una definizione molto dettagliata. Per questo si distinguono due **livelli**.

Requisiti utente (User Requirements)

Descrizione **astratta e di alto livello** di un servizio che il sistema deve fornire o di un vincolo che deve rispettare. Il punto di vista è quello dell'**utente finale**, quindi sono scritti in modo comprensibile a chi non è tecnico.

Requisiti di sistema (System Requirements)

Definizione **dettagliata e formale** di una funzione del sistema. Il punto di vista è quello del **sistema da costruire**, quindi sono precisi e non lasciano spazio a interpretazioni.

Requisito utente	Requisiti di sistema corrispondenti
1. Il sistema deve generare un report mensile delle iscrizioni ai corsi.	**1.1** I report delle iscrizioni devono essere generati l'ultimo giorno lavorativo di ogni mese. **1.2** Deve essere generato un report per ogni corso, che mostri il numero totale di iscrizioni del mese corrente e l'età media degli studenti iscritti. **1.3** L'accesso ai report deve essere riservato ai responsabili dei corsi. **1.4** …

Tabella 5.1: Un requisito utente si espande in molti requisiti di sistema.

Perché mantenere due livelli invece di uno solo? Perché servono a due scopi diversi, entrambi di natura **contrattuale**.

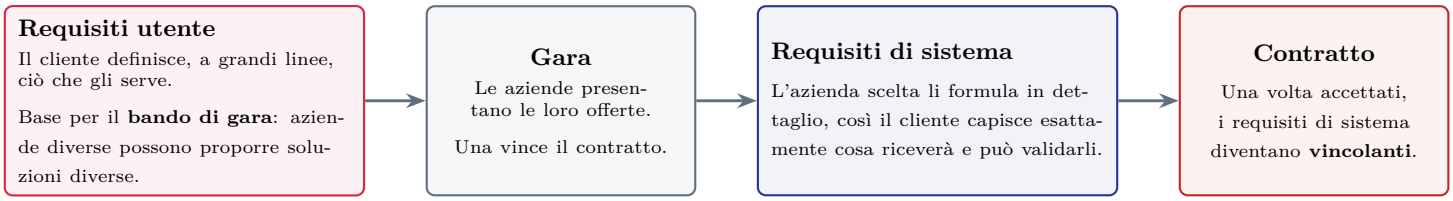


Figura 5.1: I due livelli di requisiti hanno una doppia funzione contrattuale.

Perché i requisiti valgono tanto

Un errore nei requisiti è l'errore più costoso che si possa fare. Come abbiamo visto nel Capitolo 3 con la curva di Boehm, un difetto individuato in fase di requisiti costa **1**; lo stesso difetto scoperto quando il sistema è già in esercizio può costare **500 volte tanto**. In più, se i requisiti di sistema sono già nel contratto, ogni modifica diventa anche una trattativa legale ed economica.

Requisiti funzionali e non funzionali

La classificazione più usata divide i requisiti in due famiglie.

Requisiti funzionali

Descrivono i **servizi** che il sistema deve fornire, come deve **reagire a particolari input** e come deve comportarsi in determinate situazioni.

Es.: «Un utente deve potersi iscrivere a un corso.»

Es.: «Un utente deve poter annullare la propria iscrizione a un corso.»

Requisiti non funzionali

Sono **vincoli** sui servizi o sulle funzioni offerte: vincoli di tempo, vincoli sul processo di sviluppo, vincoli imposti da standard. Spesso riguardano il sistema **nel suo insieme**, non la singola funzione.

Es.: «Il sistema deve essere conforme al GDPR.»

Es.: «Il sistema deve gestire fino a 10.000 utenti contemporanei.»

La distinzione non è sempre netta

Consideriamo il requisito: «solo gli utenti autorizzati devono poter accedere al sistema». Sembra non funzionale (è un vincolo di sicurezza sull'intero sistema), ma se lo sviluppiamo in dettaglio genera requisiti chiaramente **funzionali**: «l'utente deve poter effettuare il login e autenticarsi», «il sistema deve mostrare un messaggio di errore dopo tre tentativi falliti»…
Morale: i requisiti **non sono indipendenti**: un requisito spesso ne genera o ne vincola altri.

Come si scrivono le due famiglie

- I requisiti **funzionali utente** si scrivono in **linguaggio naturale**, in modo che siano comprensibili anche a persone non tecniche (utenti, dirigenti).
- I requisiti **funzionali di sistema** devono essere **dettagliati**: descrivono con precisione input, output ed eccezioni, così che gli sviluppatori sappiano esattamente cosa implementare.

I requisiti non funzionali, invece, non riguardano un servizio specifico: vincolano **come** il sistema deve essere realizzato (ad esempio «deve essere sviluppato in Java») o specificano proprietà globali (tempi di risposta, uso della memoria, …).

I requisiti non funzionali sono spesso i più critici

Se un requisito funzionale è implementato male, gli utenti spesso trovano un modo per aggirare il problema. Se invece **non** è soddisfatto un requisito non funzionale, il sistema può diventare **inutilizzabile**: un sistema non conforme al GDPR non può proprio essere messo in esercizio in Europa, per quanto bene funzioni.

Tipi di requisiti non funzionali

I requisiti non funzionali possono nascere da tre sorgenti diverse: dal **prodotto**, dall'**organizzazione** (del cliente o di chi sviluppa) o da **fonti esterne**.

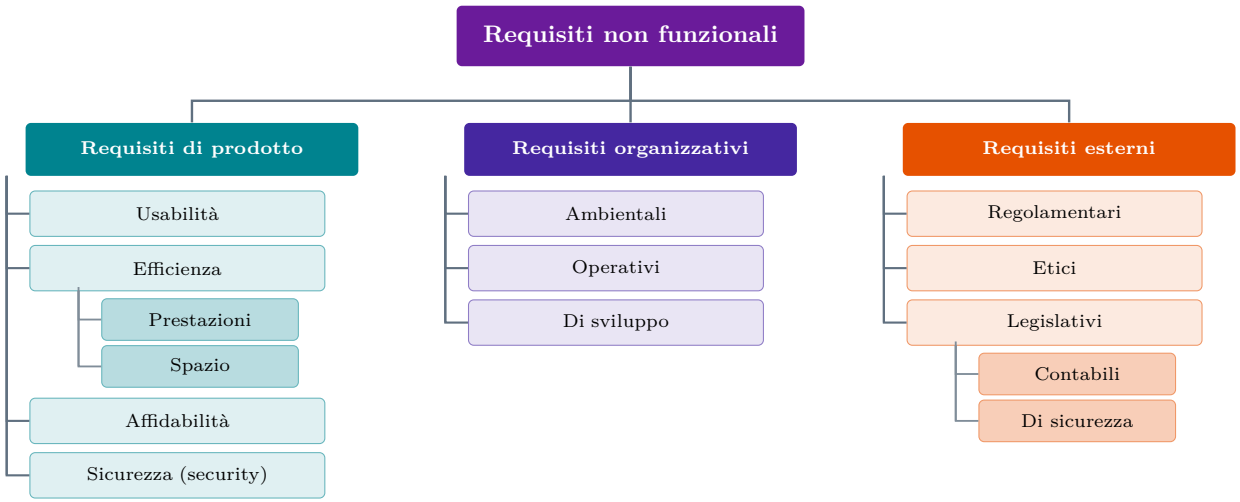


Figura 5.2: Tassonomia dei requisiti non funzionali in base alla loro origine.

Origine	Cosa riguardano	Esempi
Di prodotto	Caratteristiche richieste al prodotto stesso	Velocità del sistema, memoria occupata, usabilità, tasso di guasti accettabile
Organizzativi	Politiche e procedure dell'organizzazione del cliente o dello sviluppatore	Processi operativi con cui il sistema sarà usato, linguaggio di programmazione obbligatorio, ambiente operativo da rispettare
Esterni	Fattori esterni al sistema e al processo di sviluppo	Approvazione da parte di un ente regolatore, obblighi di legge, requisiti etici per l'accettabilità da parte di utenti e pubblico

Requisiti di dominio

Requisiti di dominio

Requisiti che derivano dallo **specifico settore o contesto industriale** in cui il software viene usato. Possono essere sia funzionali sia non funzionali.

Esempio 5.1 — Un requisito di dominio per un dispositivo medico

«La sicurezza del sistema deve essere garantita secondo lo standard IEC 60601-1: *Medical Electrical Equipment — Part 1: General Requirements for Basic Safety and Essential Performance.*»

Questo requisito vincola sia la **progettazione** del sistema sia il **processo di sviluppo**: un software perfettamente funzionante ma non sviluppato secondo lo standard **non potrebbe essere usato** in pratica.

Misurare i requisiti non funzionali

Per essere utile, un requisito non funzionale deve essere associato a una **metrica**:

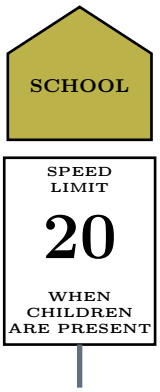
Proprietà	Metriche tipiche
Prestazioni	Operazioni elaborate al secondo; tempo di risposta; tempo di aggiornamento dello schermo
Dimensione	Megabyte occupati
Facilità d'uso	Ore di formazione necessarie; tasso di errore degli utenti; richieste di assistenza
Affidabilità	Tempo medio tra due guasti; percentuale di disponibilità
Robustezza	Tempo di ripristino dopo un guasto; probabilità di perdita di dati

LE PROPRIETÀ DI UN BUON REQUISITO

Proprietà	Significato
Chiaro	Facile da capire, soprattutto per i requisiti utente.
Non ambiguo	Una sola interpretazione possibile: l'ambiguità porta a **contenziosi** con il cliente.
Completo	Non deve mancare nulla di ciò che serve.
Coerente	I requisiti non devono contraddirsi tra loro.
Testabile	Dato un requisito, deve essere possibile stabilire **senza dubbi** se il sistema lo soddisfa.
Tracciabile	Deve essere collegabile «in avanti e all'indietro» alle altre parti della documentazione: progetto, codice, test.

I demoni dell'ambiguità

Il linguaggio naturale è comodo ma insidioso. Consideriamo un cartello stradale che tutti capiamo al volo:



Per un essere umano l'intento è chiaro: proteggere i bambini in una zona scolastica. Ma proviamo a spiegarlo a un computer:

- perché il cartello dice «children» (plurale) e non «child»? La regola vale solo se c'è **più di un** bambino?
- chi è considerato un «bambino»? Un ragazzo di 17 anni lo è?
- cosa significa essere «presenti»? Dentro la scuola? Sul marciapiede? Visibili dalla strada?
- la regola vale solo quando la scuola è aperta?

Figura 5.3: Un cartello chiarissimo per una persona, pieno di ambiguità per una macchina (H. Robinson, *The demons of ambiguity*).

Esempio 5.2 — Le domande nascoste in un requisito

Requisito: *«L'applicazione deve permettere agli utenti di avviare e arrestare un servizio su una macchina remota.»*

Sembra chiaro, ma:

- cosa succede se si avvia un servizio **già in esecuzione**? E se si arresta un servizio **non in esecuzione**?
- se la macchina remota si **riavvia**, il servizio deve ripartire automaticamente?
- se cade la **connessione**: l'applicazione mostra un messaggio d'errore? Riprova? Dopo quanto tempo? Quanti tentativi al massimo?

Ognuna di queste domande, se non trova risposta nei requisiti, diventerà una discussione (o un contenzioso) con il cliente.

Testabilità

Deve essere possibile stabilire **senza ambiguità** se il sistema soddisfa un requisito: altrimenti gli sviluppatori sosterranno che il requisito è soddisfatto e il cliente sosterrà il contrario. In pratica:

- i requisiti **funzionali di sistema** devono essere il più dettagliati possibile e non lasciare spazio a interpretazioni;
- i requisiti **non funzionali** devono includere, quando possibile, **indicatori quantitativi**.

✖ Poco testabile	✔ Testabile
Il sistema deve essere affidabile.	Il sistema deve avere una disponibilità mensile del 99,99%.
Il sistema deve essere facile da usare.	Dopo al più due ore di formazione, gli utenti devono saper usare tutte le funzioni del sistema; dopo la formazione, il numero medio di errori non deve superare i due per ora di utilizzo.
Il sistema deve essere veloce e reattivo.	Il tempo medio di risposta del sistema non deve superare i 100 ms.

Tabella 5.2: Trasformare requisiti vaghi in requisiti verificabili.

Esempio 5.3 — Un requisito testabile è un test che si può scrivere

«Il tempo medio di risposta del sistema non deve superare i 100 ms» si traduce direttamente in un test automatico:

PrestazioniTest.java

```java
@Test
void tempoMedioDiRispostaEntroCentoMillisecondi() {
  int richieste = 1000;
  long inizio = System.nanoTime();
  for (int i = 0; i < richieste; i++) {
    catalogo.cerca("pantaloni");
  }
  double mediaMs = (System.nanoTime() - inizio) / 1_000_000.0 / richieste;

  assertTrue(mediaMs <= 100, "tempo medio: " + mediaMs + " ms");
}
```

Per «il sistema deve essere veloce e reattivo», invece, nessuno saprebbe che cosa scrivere nell'`assertTrue`.

Il processo di Requirements Engineering

Il processo di RE si articola in tre attività principali.

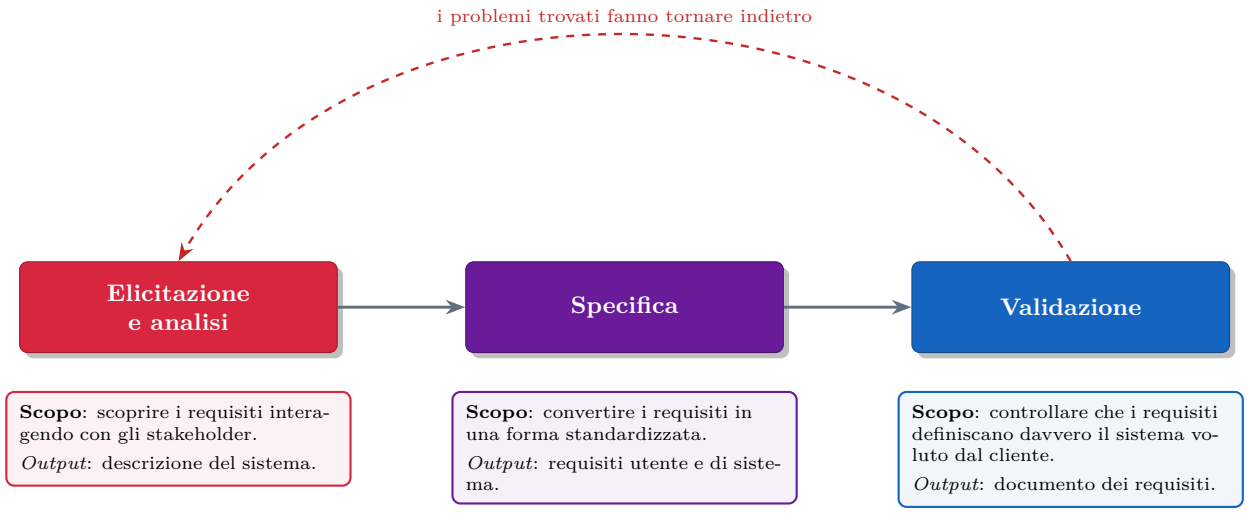


Figura 5.4: Le tre attività del processo di Requirements Engineering.

Un processo iterativo, non lineare

Nella pratica queste attività non si susseguono una sola volta: si **intrecciano** in un processo iterativo che lavora a **livelli di granularità** diversi.

1. Nella prima iterazione ci si concentra sui **requisiti di business**: perché il progetto è necessario? Chi ne beneficia? Quando e dove verrà usato?
2. Poi si definiscono i **requisiti utente**.
3. Infine si definiscono i **requisiti di sistema**.

Ogni iterazione comprende elicitazione e analisi, specifica e validazione.

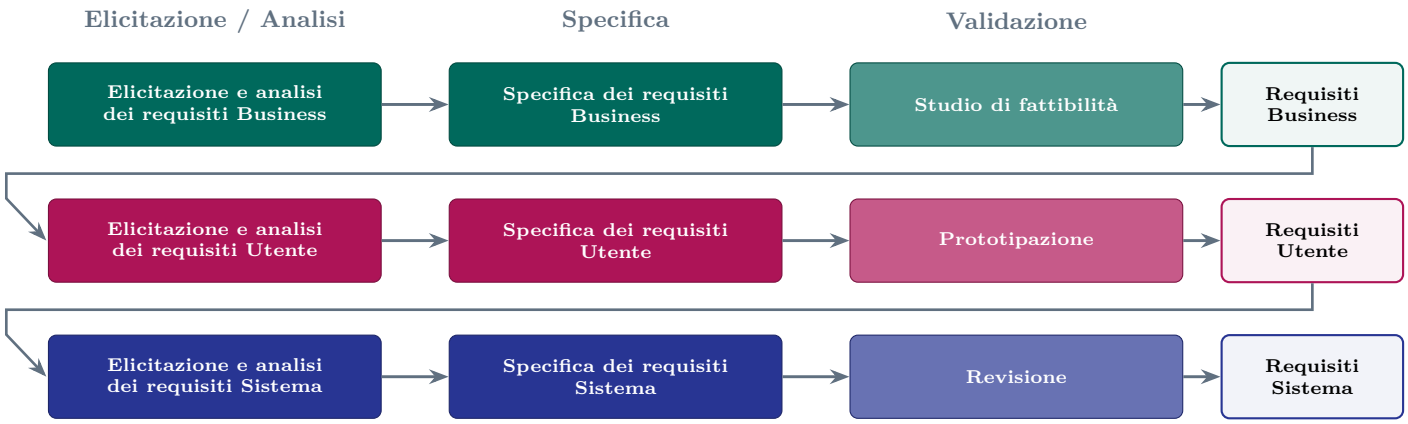


Figura 5.5: Il processo iterativo: ogni livello di requisiti attraversa le tre attività e alimenta il livello successivo.

Elicitazione e analisi dei requisiti

Elicitazione dei requisiti

Attività il cui obiettivo è **scoprire che cosa i clienti vogliono e di cosa hanno bisogno**, interagendo con gli stakeholder. È considerata la parte **più critica** dell'intera ingegneria dei requisiti.

Detta così sembra banale; in realtà è la parte più difficile del mestiere, per un motivo semplice: le persone non sanno sempre cosa vogliono, e quasi mai lo sanno spiegare in termini utilizzabili da chi deve costruire il software.

Chi sono gli stakeholder

Stakeholder

Le **persone o i gruppi** che sono in qualche modo interessati o influenzati dal progetto software.

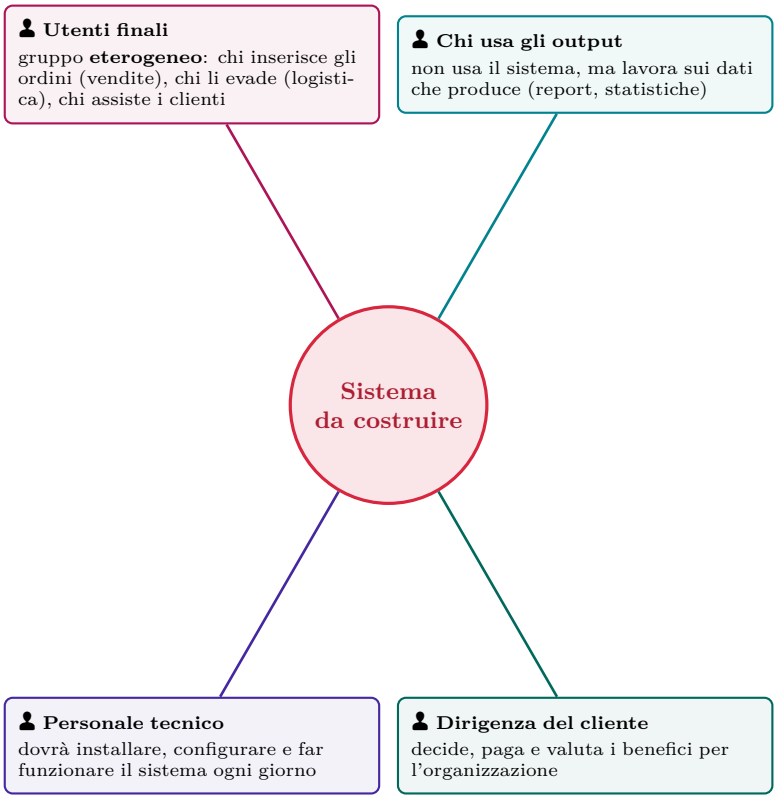


Figura 5.6: Gli stakeholder tipici di un sistema di gestione degli ordini.

Perché è difficile

Non sanno cosa vogliono
Gli stakeholder hanno idee solo generali. Non sanno cosa è tecnicamente fattibile e possono fare richieste irrealistiche.

Parlano il loro gergo
Sono esperti del **loro** dominio ed esprimono i requisiti con i loro termini, dando per scontate conoscenze implicite.

Dicono cose diverse
Stakeholder diversi esprimono requisiti diversi e in modi diversi: bisogna trovare i punti in comune e risolvere i conflitti.

Ci sono fattori politici
Un dirigente può chiedere una funzionalità perché gli fa aumentare la propria influenza nell'organizzazione, non perché serva.

Le priorità sono in conflitto
Chi sente che le proprie esigenze non sono state considerate può arrivare a **ostacolare** il processo.

I requisiti cambiano
Durante l'elicitazione emergono nuovi requisiti, magari da stakeholder che all'inizio non erano stati considerati.

Figura 5.7: Le sei principali difficoltà dell'elicitazione dei requisiti.

Il processo di elicitazione e analisi

L'attività si svolge come un **ciclo** che si ripete finché i requisiti non sono sufficientemente stabili.

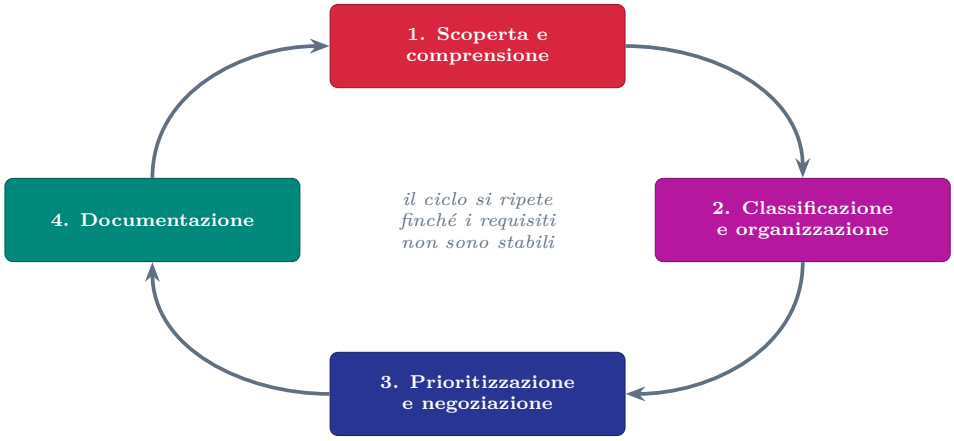


Figura 5.8: Il ciclo di elicitazione e analisi dei requisiti.

1. Scoperta e comprensione	Si interagisce con gli stakeholder per **scoprire** i requisiti.
2. Classificazione e organizzazione	I requisiti correlati vengono **raggruppati** e organizzati in categorie coerenti.
3. Prioritizzazione e negoziazione	Si risolvono i **conflitti** che nascono dalle esigenze contrastanti dei diversi stakeholder.
4. Documentazione	I requisiti trovati vengono **documentati**, come base per l'iterazione successiva.

Le tecniche di elicitazione

Non esiste una tecnica unica: si combinano approcci diversi.



Interviste con gli stakeholder

Le interviste, formali o informali, sono lo strumento fondamentale della RE: il team chiede agli stakeholder come lavorano, quale sistema usano oggi e cosa si aspettano dal sistema da sviluppare. I requisiti si ricavano dalle risposte.

Intervista chiusa	Lo stakeholder risponde a una **lista di domande predefinita**.
Intervista aperta	Non c'è un'agenda prestabilita: si lascia parlare l'intervistato.
In pratica	Si usa una **combinazione** delle due. Le interviste completamente aperte funzionano raramente: conviene partire da un insieme di domande preparate, che di solito portano ad altri temi da discutere in modo meno strutturato.

✔ Il lato buono

Alle persone in genere piace parlare del proprio lavoro e partecipano alle interviste con un atteggiamento positivo.

✖ Le difficoltà

- usano **gergo** tecnico e danno per scontati molti requisiti;
- conoscono bene il **proprio** lavoro, ma hanno una visione parziale o distorta di quello dei colleghi di altri reparti;
- possono essere **reticenti** su vincoli organizzativi, per via di equilibri di potere interni.

Rispettare il tempo degli stakeholder

Gli stakeholder hanno il loro lavoro da fare: le interviste vanno preparate bene, per non sprecare il loro tempo (e il nostro) con riunioni inutili.

Esempio 5.4 — Il gergo: una storia vera

Durante un tirocinio presso la direzione finanziaria di un grande Comune, un tributo minore veniva gestito quasi interamente con fogli di calcolo. Alla domanda «mi racconta come funziona la gestione di questo tributo?», la risposta è stata:

> *«Certo! È molto semplice! Un operatore sul campo mi invia la **lettura** per un dato anno per un certo contribuente. Quindi inserisco la lettura in una maschera che calcola l'**imponibile**. Successivamente faccio la stampa delle **ingiunzioni di pagamento**, che vengono poi emesse. Una volta notificata l'ingiunzione, il contribuente ha 60 giorni per pagare. Se non paga entro il termine, devo procedere all'**iscrizione al ruolo coattivo**: inserisco i dati in una maschera che genera il **Tracciato 290** da trasmettere all'ente di riscossione.»*

«È molto semplice» — ma per capire davvero quella frase servono settimane di studio del dominio. Questo è esattamente il problema del gergo: per l'intervistato è ovvio, per l'ingegnere del software è una lingua straniera. Il lavoro consiste nel farsi spiegare ogni termine, finché la descrizione non diventa un insieme di requisiti precisi.

Etnografia

Etnografia

Tecnica **osservativa** in cui l'ingegnere dei requisiti si **immerge nell'ambiente di lavoro** degli utenti finali: osserva il lavoro quotidiano e prende nota dei compiti realmente svolti e delle persone coinvolte.

Il software non vive isolato: è usato in un ambiente **sociale e organizzativo** che può generare o vincolare requisiti. E, molto spesso, il modo in cui si lavora davvero è **diverso** dal processo formale che si dovrebbe seguire.

L'etnografia è particolarmente efficace per scoprire:

- requisiti che derivano dal modo in cui le persone **lavorano realmente**, invece che dal modo in cui le procedure aziendali dicono che dovrebbero lavorare;
- requisiti che derivano dalla **cooperazione** e dalla consapevolezza delle attività altrui.

Esempio 5.5 — Un requisito che nessuno avrebbe raccontato

Osservando l'ufficio, si nota che prima di inviare un questionario di soddisfazione al cliente, un addetto del servizio clienti telefona ogni volta a un collega della logistica per sapere se l'ordine è stato davvero consegnato.

Nessuno lo avrebbe mai detto in un'intervista («si fa così e basta»), ma da lì nasce un requisito preciso: *«Gli addetti al servizio clienti devono poter verificare se un cliente ha già ricevuto il proprio ordine.»*

Personas

Persona

Un **archetipo ipotetico** di utente reale. Le *personas* non sono persone vere, ma le rappresentano: sono immaginarie, eppure definite con **rigore e precisione**, e non sono inventate di sana pianta, ma **scoperte** come sottoprodotto del processo di elicitazione.

Conoscere l'utente è fondamentale sia per scoprire i requisiti sia (lo vedremo più avanti) per progettare interfacce efficaci. A volte gli utenti li conosciamo bene (ad esempio se sviluppiamo uno strumento per altri ingegneri del software), ma in generale il software è fatto **per altre persone**, con esigenze diverse dalle nostre: la nostra esperienza personale non basta a immaginare come lavorano e cosa vogliono.

Le personas servono anche quando **non abbiamo stakeholder da intervistare**: ad esempio quando sviluppiamo un prodotto generico per il grande pubblico, o quando parte degli utenti finali è il pubblico generico.

Non esiste un formato standard; di solito una persona contiene:

Personalizzazione	**Lavoro**	**Istruzione**	**Obiettivi**	**Frustrazioni**
nome, età, breve biografia	che lavoro fa?	studi ed esperienza	cosa si aspetta dal software?	quali problemi il software può risolverle?

Esempio 5.6 — Una persona per un gestionale di studio dentistico

Supponiamo di raccogliere i requisiti per un software di gestione degli appuntamenti di uno studio dentistico. Intervistando il personale abbiamo già ricavato alcuni requisiti:

- i pazienti devono potersi registrare e autenticare nel sistema;
- i pazienti devono poter prenotare un appuntamento in uno slot libero;
- i medici devono poter vedere gli appuntamenti della settimana corrente;
- i receptionist devono poter cancellare gli appuntamenti;
- i pazienti devono ricevere un promemoria via e-mail il giorno prima dell'appuntamento;
- i pazienti devono ricevere un'e-mail se il loro appuntamento viene annullato.

Costruiamo ora una persona:



Concetta (Tina) Aiello
Età: 68 — vedova
Pensionata (ex maestra elementare)
assertiva, impaziente

Bio. Tina è un'insegnante di scuola primaria in pensione, con oltre 40 anni di esperienza. È dolce e paziente con i bambini, ma diventa impaziente quando si trova fuori dalla sua zona di comfort. Ama la sua routine quotidiana e non le piace cambiarla. La tecnologia non le piace: l'anno scorso i figli le hanno regalato il primo smartphone, che usa solo per videochiamare le nipoti. È diffidente verso internet, che percepisce come un posto pericoloso.

Interessi. Lavoro a maglia; comunità della parrocchia; tempo con le nipoti.

Obiettivi. Prenotare i controlli periodici della protesi dentale; ricevere un promemoria per non dimenticare l'appuntamento; spostare un appuntamento se coincide con un altro impegno.

Cosa abbiamo scoperto. Tina non prenoterà mai da sola tramite l'app: telefonerà allo studio. E un promemoria via e-mail non lo leggerà. Nascono quindi **nuovi requisiti** che nessuno aveva pensato di dirci:

- i receptionist devono poter creare appuntamenti per i pazienti che telefonano allo studio;
- il giorno prima dell'appuntamento, i receptionist devono ricevere una notifica che ricordi loro di **telefonare** ai pazienti che hanno prenotato per telefono.

Storie e scenari

Storia (story)

Testo **narrativo** che presenta una descrizione ad alto livello dell'uso del sistema. È efficace per comunicare gli obiettivi «di insieme» in modo comprensibile a chiunque.

Storia: prenotare un appuntamento con l'app

Pamela ha bisogno di prenotare una visita odontoiatrica perché il dente del giudizio ha iniziato a farle male. Accede al sistema e cerca gli slot disponibili per oggi o per domani. Il sistema mostra l'elenco degli slot liberi. Pamela ne sceglie uno e conferma la prenotazione. Riceve un'e-mail di conferma con i dettagli dell'appuntamento.

Scenario

Descrizione (tipicamente **strutturata**) di un esempio di interazione tra utente e sistema. Raffina una storia aggiungendo informazioni precise: stato iniziale, flusso normale degli eventi, cosa può andare storto e come si gestisce, altre attività in corso nello stesso momento, stato del sistema alla fine.

Scenario: prenotazione di un appuntamento	
Assunzioni iniziali	L'utente ha un account sul sistema di gestione degli appuntamenti.
Flusso normale	L'utente effettua il login e cerca gli slot disponibili per la data odierna o per il giorno successivo. Il sistema mostra l'elenco degli slot liberi nelle date scelte. L'utente seleziona uno degli slot proposti. Il sistema invia un'e-mail di conferma all'utente, al receptionist e al medico.
Cosa può andare storto	• Non c'è nessuno slot disponibile nelle date considerate: il sistema mostra un messaggio di errore e chiede di scegliere date diverse. • Lo slot scelto non è più disponibile al momento della conferma, perché un altro utente lo ha prenotato: il sistema mostra un messaggio di errore e torna alla schermata degli slot disponibili.
Stato finale del sistema	Il sistema ha memorizzato il nuovo appuntamento.

Tabella 5.3: Uno scenario strutturato che raffina la storia precedente.

Esempio 5.7 — Dallo scenario al test di accettazione

Ogni riga «cosa può andare storto» di uno scenario è un caso da verificare. Il secondo, lo slot preso da un altro utente mentre Pamela sceglie, diventa un test con la stessa struttura dello scenario:

PrenotazioneTest.java

```
@Test
void loSlotNonEPiuDisponibileAlMomentoDellaConferma() {
  // assunzioni iniziali: Pamela e Bruno hanno un account, lo slot delle 10:00 e' libero
  Slot dieci = agenda.slotLiberi(oggi).get(0);

  // un altro utente conferma per primo
  agenda.prenota(bruno, dieci);

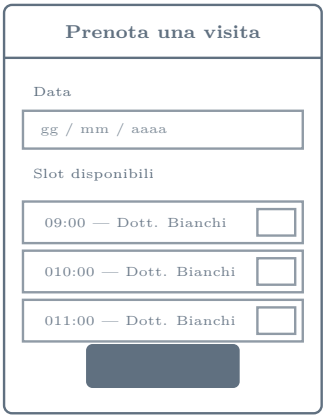
  // flusso: Pamela conferma lo stesso slot
  Esito esito = agenda.prenota(pamela, dieci);

  // cosa puo' andare storto: errore, e lo slot non e' piu' fra quelli liberi
  assertEquals(Esito.SLOT_NON_DISPONIBILE, esito);
  assertFalse(agenda.slotLiberi(oggi).contains(dieci));
}
```

Mock-up a bassa fedeltà

Mock-up a bassa fedeltà (wireframe)

Schizzi semplificati delle interfacce utente del sistema. Sono un modo rapido ed economico per visualizzare la struttura e i flussi principali: l'attenzione è sulla **funzionalità e sul flusso**, non sull'estetica.



Perché servono

- **Facilitano la comprensione**: aiutano gli stakeholder a cogliere le funzionalità di base e sono un punto di partenza per la discussione.
- **Fanno scoprire nuovi requisiti**: è molto più facile ragionare su un'interfaccia concreta che su frasi astratte.
- **Coinvolgono e permettono di iterare**: si ottiene feedback rapido senza investire in grafica.
- **Sono una base per lo sviluppo**: da qui si passa ai prototipi ad alta fedeltà e poi all'implementazione.

Figura 5.9: Un wireframe non deve essere bello: deve far capire struttura e flusso.

Strumenti

Carta e matita: rapidi e accessibili per i primi schizzi, ma diventano disordinati per sistemi complessi.
Strumenti digitali: software commerciali come *Balsamiq* o *Figma* permettono di creare e condividere wireframe elettronicamente; esistono anche alternative open source come *mydraft.cc*, *draw.io* o *Pencil Project*.

LA SPECIFICA DEI REQUISITI

Specifica dei requisiti

Il processo di **mettere per iscritto** i requisiti utente e di sistema in un **documento dei requisiti**. I requisiti utente devono essere comprensibili a utenti e clienti senza background tecnico; i requisiti di sistema sono più dettagliati e possono contenere informazioni tecniche.

Poiché il documento dei requisiti può diventare **parte del contratto** di sviluppo, è importante che sia il più completo e dettagliato possibile.

Requisiti e progetto sono inseparabili

In linea di principio i requisiti dicono *cosa* il sistema deve fare e il progetto dice *come* lo fa. In pratica i due aspetti si intrecciano:

- i requisiti possono essere **organizzati** seguendo un'architettura di sistema di alto livello;
- il sistema può dover **interoperare** con altri sistemi, e questo genera requisiti di progetto;
- l'uso di una specifica architettura per soddisfare requisiti non funzionali può essere esso stesso un requisito di dominio.

Quattro approcci alla specifica



Figura 5.10: Dal testo libero alla matematica: quattro modi di specificare i requisiti.

Approcci diversi si adattano a domini diversi:

Contesto	Approccio tipico
Sistemi critici per la sicurezza	Specifiche **formali** e linguaggio naturale strutturato
Una semplice app per le liste di cose da fare	Linguaggio naturale non strutturato
Un sistema informativo di media o grande dimensione	Notazioni **semi-formali** e modelli: un buon compromesso

Specifica in linguaggio naturale

I requisiti sono scritti come frasi in linguaggio naturale, eventualmente accompagnate da diagrammi e tabelle. Si usa perché è **espressivo, intuitivo e universale**: i requisiti risultano comprensibili a utenti e clienti, e permette di esprimere sia requisiti funzionali sia non funzionali.

✔ Linee guida

- definire un **formato standard** e usarlo per tutti i requisiti, ad esempio:
  <parte del sistema> shall <requisito> (<motivazione>);
- usare il linguaggio in modo **coerente**: *shall* per i requisiti obbligatori, *should* per quelli desiderabili;
- **evidenziare** graficamente le parti chiave del requisito;
- evitare il **gergo informatico**;
- includere sempre la **motivazione** (*rationale*) del requisito.

✖ Problemi tipici

- **Mancanza di chiarezza**: è difficile essere precisi senza rendere il documento pesante da leggere;
- **confusione tra requisiti**: requisiti funzionali e non funzionali tendono a mescolarsi;
- **amalgama di requisiti**: più requisiti diversi finiscono espressi insieme in un'unica frase.

Esempio 5.8 — Un requisito con la sua motivazione

*«Il report mensile complessivo delle iscrizioni **shall** includere statistiche sul genere degli studenti iscritti (necessarie per compilare il report sulla parità di genere).»*

La motivazione tra parentesi è preziosa: se un domani quell'obbligo dovesse cadere, sapremo che quel requisito può essere rimosso.

Specifica strutturata

Specifica strutturata

Approccio in cui la **libertà** di chi scrive i requisiti viene limitata e si impone un **modo standardizzato** di scriverli (un modulo con campi prestabiliti).

Funziona bene per alcuni tipi di requisiti (ad esempio nei sistemi di controllo embedded), mentre risulta troppo rigida per i requisiti dei sistemi gestionali.

Microinfusore di insulina / Software di controllo / SRS / 3.3.2	
Funzione	Calcolo della dose di insulina con livello di zucchero nella norma.
Descrizione	Calcola la dose di insulina da somministrare quando il livello di zucchero misurato è compreso tra 3 e 7 unità.
Input	Lettura corrente dello zucchero (r_2); le due letture precedenti (r_0 e r_1).
Sorgente	Lettura corrente dal sensore; le altre letture dalla memoria.
Output	`CompDose`: la dose di insulina da somministrare.
Azione	`CompDose` è zero se il livello è stabile o in calo, oppure se sta crescendo ma con velocità di crescita decrescente. Se il livello cresce e la velocità di crescita aumenta, `CompDose` si calcola dividendo per 4 la differenza tra il livello corrente e quello precedente e arrotondando. Se il risultato arrotondato è zero, `CompDose` è posta alla dose minima somministrabile.
Richiede	Le due letture precedenti, per poter calcolare la velocità di variazione.
Pre-condizione	Il serbatoio contiene almeno la dose singola massima ammessa.
Post-condizione	In memoria, r_0 è sostituito da r_1 e poi r_1 è sostituito da r_2.
Effetti collaterali	Nessuno.

Tabella 5.4: Esempio di specifica strutturata (da I. Sommerville).

Come si scrive una frase di requisito

Lo standard ISO/IEC/IEEE 29148 detta regole molto precise sul linguaggio da usare:

shall	Introduce un **requisito obbligatorio e vincolante**. È la forma da usare per i requisiti veri e propri.
will	Introduce **dichiarazioni di fatto**, di intenzione o di contesto: non è vincolante.
should	Introduce **preferenze o obiettivi** desiderabili ma non obbligatori: non sono requisiti.
may	Introduce **suggerimenti o concessioni**: non è vincolante.
must	**Da evitare**: rischia di essere interpretato erroneamente come requisito.
Altre regole di scrittura	
Forma positiva	Usare affermazioni positive ed evitare requisiti negativi come «*shall not*».
Forma attiva	Usare la forma attiva, evitando il passivo («*it is required that…*»).
Verbi precisi	Evitare espressioni come «*shall be able to*»: appesantiscono senza aggiungere precisione.

Termini vaghi da evitare

Le parole vaghe generano requisiti impossibili da verificare o interpretabili in più modi. In particolare:

- **superlativi**: «il migliore», «il massimo»;
- **linguaggio soggettivo**: «intuitivo», «facile da usare», «conveniente»;
- **pronomi vaghi**: «esso», «questo», «quello»;
- **avverbi e aggettivi ambigui**: «quasi sempre», «significativo», «minimo»; e congiunzioni ambigue come «e/o»;
- **termini aperti e non verificabili**: «fornire supporto», «incluso ma non limitato a», «come minimo»;
- **frasi comparative**: «migliore di», «di qualità superiore»;
- **scappatoie**: «se possibile», «se appropriato», «ove applicabile»;
- **termini assoluti**: «tutti», «sempre», «mai», «ogni» (difficilissimi da verificare);
- **riferimenti incompleti**: citare un documento senza data, versione o sezione applicabile.

Un modello per i requisiti funzionali

Un modo pratico per scrivere requisiti funzionali chiari è seguire un **pattern** fisso:



Forma completa	*Quando viene ricevuto il segnale x* [Condizione], *il sistema* [Soggetto] *deve impostare* [Azione] *il bit di ricezione del segnale x* [Oggetto] *entro 2 secondi* [Vincolo].
Variante	*Con stato del mare pari a 1* [Condizione], *il sistema radar* [Soggetto] *deve rilevare* [Azione] *i bersagli* [Oggetto] *fino a 100 miglia nautiche di distanza* [Vincolo].
Senza condizione	*Il sistema di fatturazione* [Soggetto] *deve mostrare le fatture dei clienti in sospeso* [Azione] *in ordine crescente di data di scadenza* [Vincolo].

Tabella 5.5: Lo stesso schema applicato a tre requisiti diversi.

Esempio 5.9 — Dal requisito al codice, parte per parte

Il terzo requisito della tabella, «il sistema di fatturazione deve mostrare le fatture dei clienti in sospeso in ordine crescente di data di scadenza», si ritrova pezzo per pezzo nel metodo che lo realizza. È questa corrispondenza che permette di **tracciare** un requisito fino al codice e ai test.

ServizioFatturazione.java

```java
public List<Fattura> fattureInSospeso(Cliente cliente) {
  return fatture.stream()
      .filter(f -> f.cliente().equals(cliente))        // le fatture dei clienti...
      .filter(f -> !f.pagata())                        // ...in sospeso [Oggetto]
      .sorted(Comparator.comparing(Fattura::scadenza)) // in ordine crescente di scadenza [Vincolo]
      .toList();
}
```

I DOCUMENTI DEI REQUISITI

Esistono standard che definiscono la **struttura** del documento dei requisiti.

IEEE 830-1998

Definisce una struttura generica da adattare a ogni specifico sistema:

1. Introduzione	Scopo del documento; ambito del prodotto; definizioni, acronimi e abbreviazioni; riferimenti; panoramica dell'organizzazione del documento.
2. Descrizione generale	Prospettiva del prodotto (in quale contesto sarà usato? interagirà con sistemi esistenti?); funzioni del prodotto; caratteristiche degli utenti; vincoli generali (leggi, regolamenti); assunzioni e dipendenze (hardware o software esistenti).
3. Requisiti specifici	Requisiti funzionali (per ciascuno: introduzione, input, elaborazione, output); requisiti di interfaccia esterna (utente, hardware, software, comunicazioni); requisiti di prestazione; vincoli di progetto (conformità a standard, limiti hardware); attributi del sistema software (sicurezza, manutenibilità); altri requisiti (database, operatività, adattamento al sito).
4. Appendici	Materiale di supporto.
5. Indice analitico	

Tabella 5.6: Struttura di un documento dei requisiti secondo lo standard IEEE 830-1998.

Nel 2011 è stato pubblicato un nuovo standard, aggiornato nel 2018: *Systems and software engineering — Life cycle processes requirements engineering*. Non definisce un unico documento, ma **quattro documenti** corrispondenti ai diversi livelli visti all'inizio del capitolo.

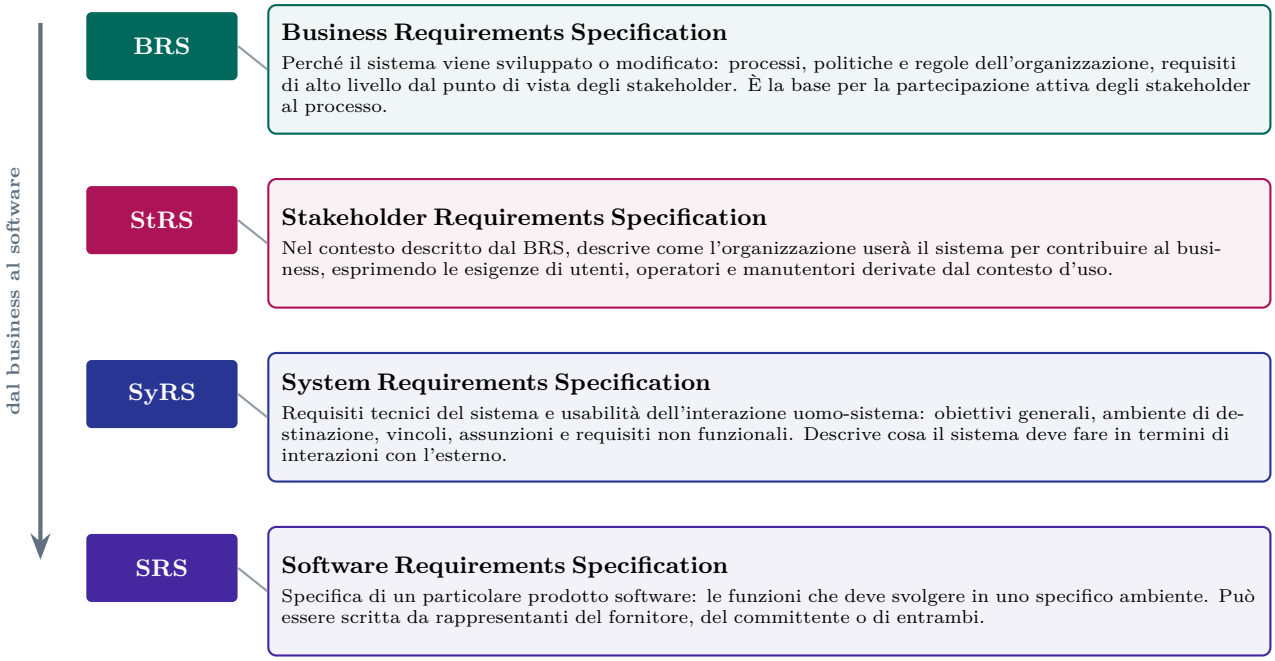


Figura 5.11: I quattro documenti previsti dallo standard ISO/IEC/IEEE 29148:2018.

Cosa deve specificare la parte «Functions»

Lo standard entra molto nel dettaglio. Per le funzioni del software richiede, ad esempio, di definire: i controlli di validità sugli input; l'esatta sequenza delle operazioni; le risposte a situazioni anomale (overflow, problemi di comunicazione, guasti hardware, gestione e recupero degli errori); l'effetto dei parametri; la relazione tra output e input (sequenze di input/output e formule di conversione).

In sintesi

- Un **requisito** descrive un servizio che il sistema deve offrire o un vincolo che deve rispettare. Si distinguono **requisiti utente** (astratti, per il cliente) e **requisiti di sistema** (dettagliati, vincolanti in contratto).
- I requisiti si dividono in **funzionali** (cosa fa il sistema) e **non funzionali** (vincoli sul sistema nel suo insieme: di prodotto, organizzativi, esterni). I requisiti **di dominio** derivano dal settore d'uso.
- Un buon requisito è **chiaro, non ambiguo, completo, coerente, testabile e tracciabile**; i requisiti non funzionali vanno espressi con **indicatori quantitativi**.
- Il processo di RE comprende **elicitazione e analisi**, **specifica** e **validazione**, ripetute in modo **iterativo** sui livelli business, utente e sistema.
- Le tecniche di elicitazione principali sono **interviste**, **etnografia**, **personas**, **storie e scenari**, **mock-up a bassa fedeltà**.
- La specifica può essere in linguaggio naturale, in linguaggio naturale **strutturato**, con notazioni **semi-formali** (UML) o **formali**. Lo standard ISO/IEC/IEEE 29148 regola persino l'uso di *shall*, *should*, *will* e *may*.

Diagrammi dei casi d'uso

Nel Capitolo 5 abbiamo visto come si **scoprono** i requisiti (interviste, personas, storie e scenari, mock-up) e i quattro approcci per **specificarli**. Uno di questi approcci usa notazioni **semi-formali**: in questo capitolo vediamo la più diffusa per i requisiti funzionali, i **casi d'uso** (*use cases*) di UML, con il loro diagramma e la loro descrizione testuale.

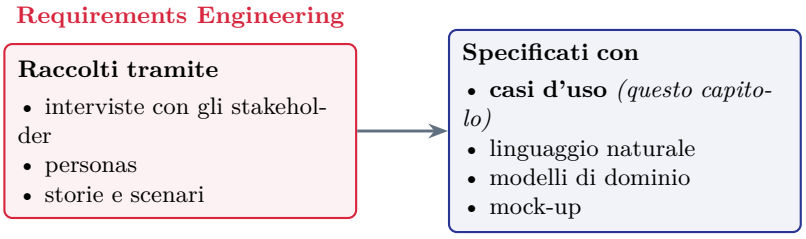


Figura 6.1: Dove si collocano i casi d'uso nella prima fase del ciclo di vita.

Casi d'uso e attori

Casi d'uso (Use Cases)

Modo di descrivere le **interazioni tra gli utenti e un sistema** usando un **modello grafico** e un **testo in linguaggio naturale strutturato**. Sono una parte fondamentale di UML (*Unified Modelling Language*) e servono a rappresentare l'insieme dei **requisiti funzionali** di un sistema.

Un modello dei casi d'uso individua due cose:

- gli **attori**, cioè le **categorie di utenti** del sistema (non necessariamente esseri umani);
- i **casi d'uso**, cioè i **tipi di interazione** (le funzionalità) che il sistema offre.

Le informazioni aggiuntive su ogni interazione si danno con **descrizioni testuali** (strutturate) oppure con altri modelli semi-formali, ad esempio i *Sequence Diagram* o gli *Statechart* di UML.

A cosa servono davvero

I casi d'uso sono prima di tutto uno strumento di **comunicazione con il cliente** per concordare le funzionalità del sistema, quindi devono essere **il più semplici possibile**. Dicono **cosa** il sistema deve fare, non **come** sarà implementato: il punto di vista è quello degli utenti, per cui il sistema è una **scatola nera**. Di solito si documentano con un diagramma di alto livello, lo **Use Case Diagram (UCD)**.

Gli attori

Attore

Entità **esterna** che interagisce con il sistema in sviluppo (*System Under Development*, **SUD**). Può essere una classe di utenti umani, un altro sistema, oppure l'ambiente fisico. Si disegna con un **omino stilizzato** e ha un **nome univoco**.



Un attore è un ruolo, non una persona

Un attore non corrisponde a una singola entità, ma a una **classe di utenti che hanno lo stesso ruolo**. La stessa persona può avere **ruoli diversi** nello stesso sistema: l'amministratore di un sito web può anche visitarlo come utente non autenticato.
Gli attori sono inoltre **più grossolani delle personas** del Capitolo 5: a un solo attore possono corrispondere più personas.

Euristiche per trovare gli attori. Per individuare gli attori conviene chiedersi:

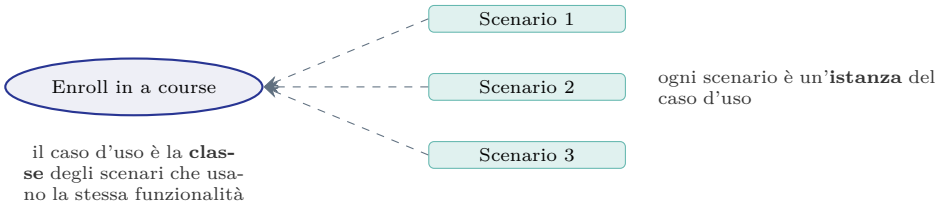
1. quali gruppi di utenti sono **supportati** dal SUD nel loro lavoro?
2. quali gruppi di utenti eseguono le **funzionalità principali** del SUD?
3. quali gruppi di utenti eseguono le **funzioni secondarie**, come l'amministrazione?
4. il SUD interagisce con **sistemi o software esterni**? Ciascuno di essi sarà un attore.

I casi d'uso

Caso d'uso

Funzionalità offerta dal sistema che fornisce un **beneficio o un'utilità** a uno o più attori. Si disegna come un'**ellisse con un nome** e modella un **requisito funzionale**.

Un caso d'uso non descrive una sola sequenza di azioni: **astrae molti scenari possibili** per la stessa funzionalità. Ci possono essere, ad esempio, più modi di iscriversi a un corso.



In altre parole il rapporto tra caso d'uso e scenario è lo stesso che c'è tra **classe e oggetto**: lo scenario è un'istanza concreta, il caso d'uso è la classe di tutti gli scenari con lo stesso obiettivo.

Le relazioni di un diagramma dei casi d'uso

Oltre ad attori e casi d'uso, un UCD contiene diversi tipi di **relazioni** tra di essi.

Associazione

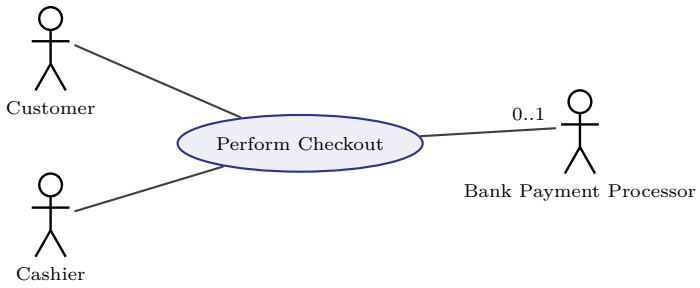
Associazione

Un'**associazione** tra un attore e un caso d'uso indica che l'attore **può eseguire** quel caso d'uso. Si disegna come una **linea** che collega l'attore all'ellisse.

Attori secondari e cardinalità

Un caso d'uso può essere associato a **più attori**. Il significato è che quegli attori devono **collaborare** in qualche modo per eseguire il caso d'uso. In UML 2.5 le associazioni possono avere una **cardinalità**.

Esempio 6.1 — La cassa di un negozio



- **Customer** e **Cashier** devono cooperare per eseguire il checkout.
- **Opzionalmente** (cardinalità `0..1`) può essere coinvolto anche un **Bank Payment Processor**, un sistema esterno: ad esempio quando il cliente paga con carta di credito o di debito.

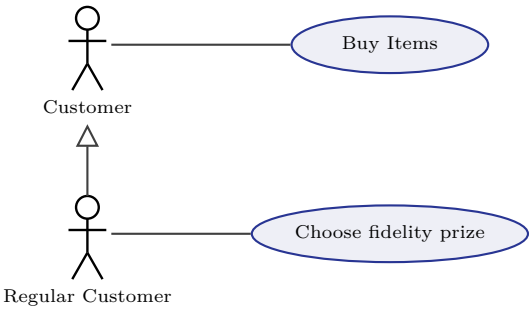
Il diagramma non dice *come* collaborano

Gli UCD di UML **non hanno strumenti** per specificare in che modo i diversi attori partecipano a un caso d'uso. Le modalità di interazione e le responsabilità di ciascuno si specificano nelle **descrizioni testuali** che vedremo più avanti in questo capitolo.

Generalizzazione tra attori

Generalizzazione tra attori

Si usa quando un attore è un **sottotipo** di un altro. Notazione e significato sono gli stessi dei diagrammi delle classi UML: una **freccia con la punta vuota** che va dall'attore specializzato a quello generale. L'attore specializzato può eseguire **tutti** i casi d'uso che può eseguire il genitore.

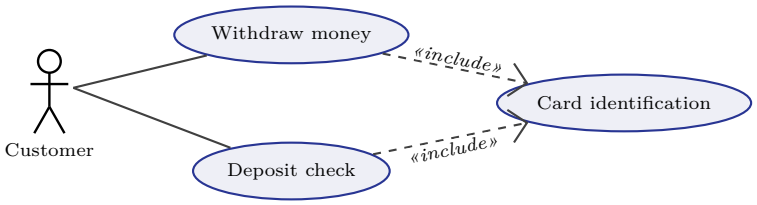


Regular Customer può scegliere il premio fedeltà **e** anche comprare articoli, perché eredita i casi d'uso di Customer.

La relazione «include»

Relazione «include»

Si usa quando due o più casi d'uso hanno una **parte di comportamento in comune**. La parte comune viene estratta in un caso d'uso separato, che viene **incluso** da tutti i casi d'uso base che la condividono. Si disegna come una **freccia tratteggiata** con lo stereotipo «include», che va **dal caso d'uso base a quello incluso**.



Poiché «include» serve soprattutto a **riusare** parti comuni, quello che resta nel caso d'uso base di solito **non è completo da solo**: ha senso solo insieme alle parti incluse. Lo dice anche il verso della freccia: il caso d'uso base **dipende** da quello incluso, non viceversa (prelevare senza identificare la carta non ha senso).

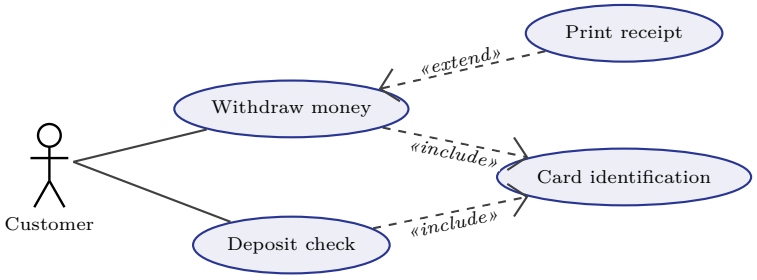
La relazione «include» è utile per:

- **decomporre** un'interazione complessa in interazioni più piccole e gestibili;
- **fattorizzare** sequenze di passi comuni a più casi d'uso.

La relazione «extend»

Relazione «extend»

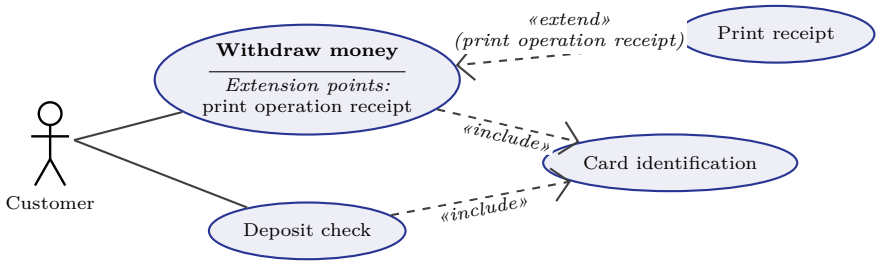
Si usa quando c'è un **comportamento aggiuntivo** che **può** essere aggiunto, eventualmente **sotto condizione**, al comportamento di uno o più casi d'uso. Si disegna come una **freccia tratteggiata** con lo stereotipo «extend», che va **dal caso d'uso che estende a quello esteso**.



Qui la dipendenza è **rovesciata** rispetto a «include»:

- il caso d'uso **esteso** (Withdraw money) è definito **indipendentemente** da quello che lo estende e ha senso anche da solo;
- il caso d'uso che **estende** (Print receipt) definisce un comportamento che, di solito, **non ha senso da solo** (stampare la ricevuta di cosa?).

Extension points. Gli *extension point* indicano il **punto preciso** del caso d'uso base in cui può essere inserito il comportamento di un caso d'uso di estensione. Sono utili quando un caso d'uso può essere esteso in **più punti**. Si elencano dentro l'ellisse, sotto il nome, e l'etichetta della freccia «extend» dice quale punto si sta usando.



Esempio 6.2 — «include» ed «extend» quando si arriva al codice

Un caso d'uso descrive un'interazione, non un metodo: non c'è una traduzione meccanica in codice. Quando però si arriva all'implementazione, le due relazioni lasciano tracce riconoscibili, con lo stesso verso delle dipendenze.

Bancomat.java

```java
public class Bancomat {
  private final List<EstensionePrelievo> estensioni = new ArrayList<>();   // chi estende si registra qui

  public void preleva(Carta carta, int importo) {
    identificaCarta(carta);                     // «include»: sempre, e il prelievo dipende da lei
    erogaContanti(importo);
    for (EstensionePrelievo e : estensioni) {   // extension point "print operation receipt"
      e.dopoOperazione(carta, importo);         // «extend»: se c'e', quando serve
    }
  }

  public void versaAssegno(Carta carta, Assegno assegno) {
    identificaCarta(carta);                     // la stessa parte comune, riusata
    acquisisciAssegno(assegno);
  }
}
```

StampaRicevuta.java

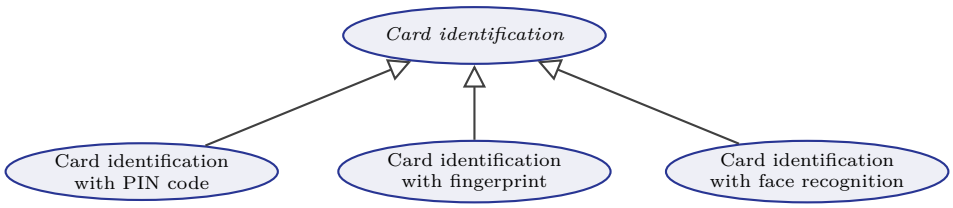
```java
public class StampaRicevuta implements EstensionePrelievo {
  @Override
  public void dopoOperazione(Carta carta, int importo) {
    if (cliente.vuoleLaRicevuta()) {            // la condizione dell'estensione
      stampante.stampa("Prelievo di " + importo + " euro");
    }
  }
}
```

`Bancomat` chiama `identificaCarta` e non potrebbe farne a meno; non conosce invece `StampaRicevuta`, che dipende da lui (implementa la sua interfaccia) e da sola non avrebbe senso.

Generalizzazione tra casi d'uso

Generalizzazione tra casi d'uso

Si usa quando un caso d'uso è una **specializzazione** di un altro. La notazione è quella standard di UML per la specializzazione (freccia con punta vuota verso il caso d'uso generale).



Generalizzazione o «extend»?

A differenza di «extend», nella generalizzazione **non ci sono punti precisi** in cui i casi d'uso specializzati si discostano dal genitore: le specializzazioni possono essere anche **molto diverse** dal caso d'uso generale. Con «extend» il caso base resta lo stesso e gli si **aggiunge** qualcosa in un punto preciso; con la generalizzazione il caso specializzato **sostituisce** il genitore.

Riepilogo delle relazioni

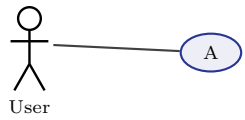
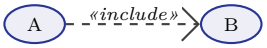
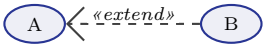
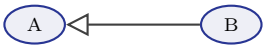
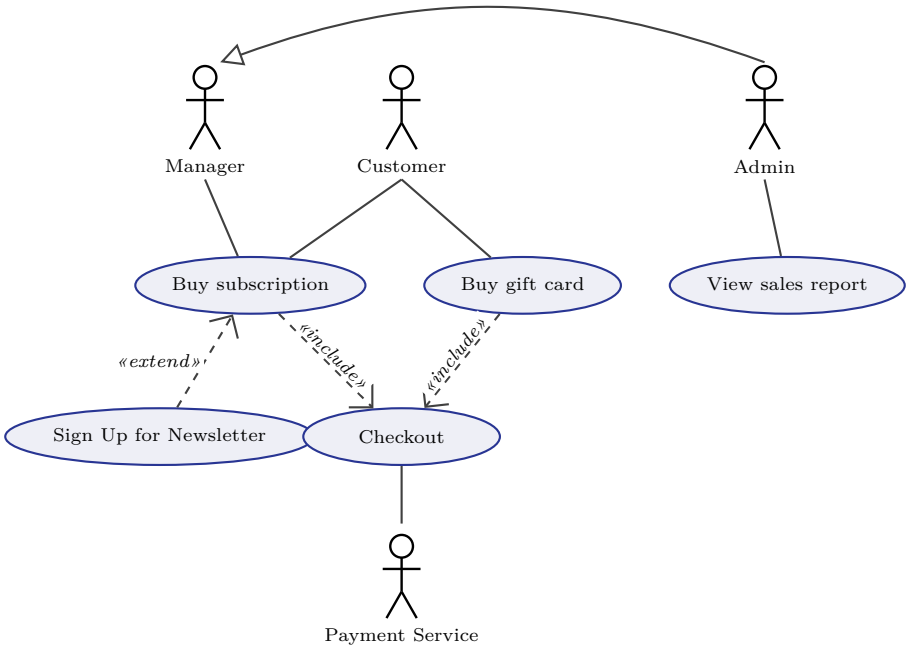
Relazione	Notazione	Significato
Associazione		L'attore è associato al caso d'uso: *User può eseguire A*.
Include		Il comportamento comune viene fattorizzato: *A include i passi definiti in B*.
Extend		Individua variazioni (eventualmente opzionali): *A può includere la sequenza di passi di B*.
Generalizza-zione		Vale per attori e casi d'uso: *B eredita il comportamento di A*, e *A può essere sostituito da un'istanza di B*.

Tabella 6.1: Le quattro relazioni di un UCD. Attenzione al verso delle frecce: «include» va dal base all'incluso, «extend» dall'estensione al caso esteso.

Leggere un diagramma dei casi d'uso

Esempio 6.3 — Abbonamenti e buoni regalo



Il diagramma si legge così:

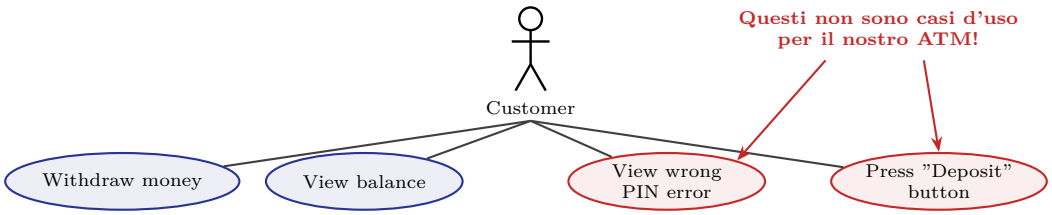
- i **Customer** possono usare il sistema per comprare un **buono regalo** o un **abbonamento**;
- entrambi i casi d'uso **includono** un comportamento comune di **Checkout** (la sequenza di passi del pagamento), che richiede la collaborazione di un servizio esterno, il **Payment Service**;
- per comprare un abbonamento serve anche la collaborazione di un **Manager** (o di un Admin): ad esempio perché deve approvare l'acquisto;
- **opzionalmente**, mentre compra un abbonamento, il cliente può **iscriversi alla newsletter** («extend»);
- gli **Admin** sono gli unici che possono vedere i **report delle vendite**; poiché Admin specializza Manager, possono anche fare tutto ciò che fa un Manager.

Errori tipici

Errore I: casi d'uso che non sono casi d'uso

Un caso d'uso deve portare un **beneficio** all'attore, aiutarlo a completare il suo lavoro o a raggiungere un obiettivo. Per questo, di solito:

- il nome di un caso d'uso contiene un **verbo**;
- il nome di un attore è un **sostantivo**.



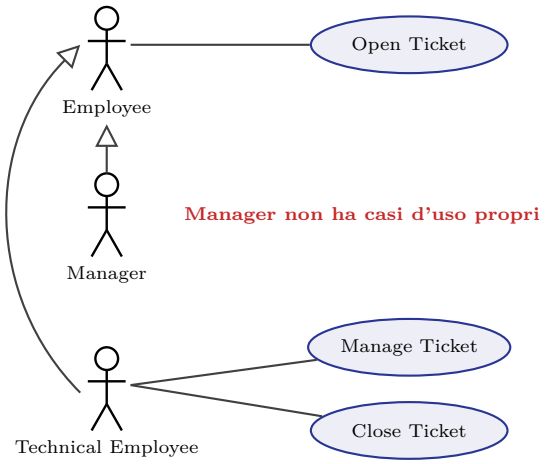
Vedere un messaggio di errore o premere un pulsante non sono obiettivi del cliente: sono **passi** (o eccezioni) all'interno di un caso d'uso, e troveranno posto nella sua descrizione testuale.

Errore II: più attori sullo stesso caso d'uso

Due attori sulla stessa ellisse non sono un «oppure»

Se due attori sono associati allo stesso caso d'uso (con cardinalità non nulla), significa che **entrambi** sono coinvolti, e devono **collaborare**, in **ogni istanza** (scenario) di quel caso d'uso. Nell'esempio della cassa, Customer e Cashier partecipano insieme a ogni checkout.
Non significa che ciascuno dei due può eseguire il caso d'uso per conto proprio! Per dire «lo possono fare entrambi» si usa invece una generalizzazione tra attori.

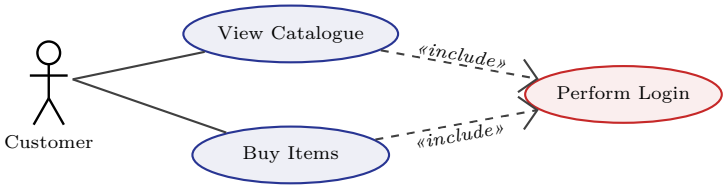
Errore III: generalizzazioni inutili



Ogni attore specializzato deve avere i suoi casi d'uso

Un attore specializzato può già eseguire tutti i casi d'uso dei suoi antenati. Se **non ha casi d'uso propri** (come Manager qui sopra), allora o la generalizzazione è **inutile**, oppure **mancano** dei casi d'uso nel diagramma.

Errore IV: «include» per le relazioni temporali



«include» non vuol dire «prima»

Questo diagramma dice che il Customer deve eseguire **tutti i passi** di Perform Login **ogni volta** che vuole comprare articoli o consultare il catalogo! Il fatto che l'utente debba essere autenticato è una **relazione temporale**, e va espresso con una **precondizione** nella descrizione testuale del caso d'uso («il cliente ha effettuato il login»).

Errore IV bis: diagrammi troppo complessi

Un diagramma dei casi d'uso non deve diventare complesso e disordinato:

- le relazioni tra casi d'uso e le generalizzazioni tra attori vanno usate **con moderazione**;
- la modellazione deve restare a un livello di **astrazione sufficientemente alto**;
- bisogna essere **ordinati**: ad esempio evitare le linee che si intersecano.

Un diagramma complesso è il segno di un cattivo analista

Un UCD pieno di ellissi, frecce «include»/«extend» incrociate e gerarchie di attori non comunica niente al cliente, che è proprio il suo destinatario. I dettagli vanno nelle descrizioni testuali, non nel diagramma.

SPECIFICARE I CASI D'USO

Lo UCD dà una panoramica **di altissimo livello** dei requisiti funzionali: non è abbastanza dettagliato per stabilire i **requisiti di sistema**. Per **ogni** caso d'uso del diagramma serve quindi una **specifica dettagliata**, che descriva ogni aspetto dell'interazione dal punto di vista dell'attore, compresi tutti gli scenari e le variazioni possibili.

Come per gli scenari del Capitolo 5, la descrizione testuale di un caso d'uso comprende in genere:

1. cosa il sistema e gli utenti si **aspettano** quando il caso d'uso inizia;
2. il **flusso normale** degli eventi (**main scenario**);
3. cosa può causare **errori** e come si gestiscono i problemi che ne derivano;
4. lo **stato del sistema** al termine del caso d'uso.

I formati

I casi d'uso si possono scrivere con diversi livelli di formalità.

Formato	Descrizione	Quando si usa
Brief	Riassunto conciso di un paragrafo, di solito del solo scenario principale di successo.	Nelle **prime fasi** della specifica, per farsi un'idea veloce di argomento e perimetro.
Casual	Paragrafi informali: più paragrafi che coprono i vari scenari.	Come il formato brief: nelle **prime fasi**.
Fully-dressed	Tutti i passi e tutte le variazioni sono scritti in dettaglio, con sezioni di supporto come precondizioni e garanzie di successo.	**Più avanti**, come **base per il contratto** e per specificare in dettaglio il comportamento del sistema.

Il template di Cockburn

Per le descrizioni fully-dressed sono stati proposti molti formati; noi usiamo un template basato su quello di **Alistair Cockburn**. I nomi dei campi restano in inglese.

USE CASE #X	**Nome del caso d'uso**			
Goal in Context	Obiettivo del caso d'uso			
Preconditions	Tutte le condizioni che devono valere per poter iniziare il caso d'uso			
Success End Condition	Stato del sistema se il caso d'uso ha successo			
Failed End Condition	Stato del sistema se il caso d'uso fallisce			
Primary Actor	Attore principale del caso d'uso			
Trigger	Azione dell'attore principale che avvia il caso d'uso			
Main Scenario	**Step**	**Actor 1**	**Actor n**	**System**
	1	Azione di avvio (trigger)		
	2			Risposta
	...	Azione 2	...	...
	n			Azione finale
Extension #1 (breve descrizione)	**Step**	**Actor 1**	**Actor n**	**System**
	x <condizione>	...	...	...
	...			Azione finale (eventualmente torna a un passo del main scenario)
Extension #n	...			
Open Issues	Aspetti ancora da chiarire. **Alla consegna del documento deve essere vuoto.**			

Tabella 6.2: Il template fully-dressed di Cockburn.

Main scenario ed estensioni

Main scenario

Sequenza di azioni che avviene quando nel caso d'uso **tutto va come previsto**.

Ma un caso d'uso si può eseguire in **modi diversi** (l'utente può autenticarsi con il PIN oppure con l'impronta digitale) e in qualunque punto può verificarsi un **errore**. Quando si definisce il comportamento funzionale del sistema bisogna descrivere anche queste sequenze alternative.

Extension

Sequenza di azioni **alternativa** al main scenario, che parte da un passo preciso quando si verifica una **condizione**. Termina il caso d'uso oppure **torna** a un passo del main scenario.

Dove sta davvero il lavoro

Di solito c'è **molto più testo nelle estensioni** che nel main scenario: il percorso «felice» è uno solo, i modi in cui qualcosa può andare storto o diversamente sono tanti.

Un esempio completo: il bancomat

Vogliamo descrivere in formato fully-dressed il caso d'uso **Withdraw money** dello UCD del bancomat (ATM) visto sopra.

I mock-up

Prima di scrivere i passi è utile disegnare dei **mock-up** delle schermate (Capitolo 5): i passi del caso d'uso potranno farvi riferimento per nome (M1, M2, …).

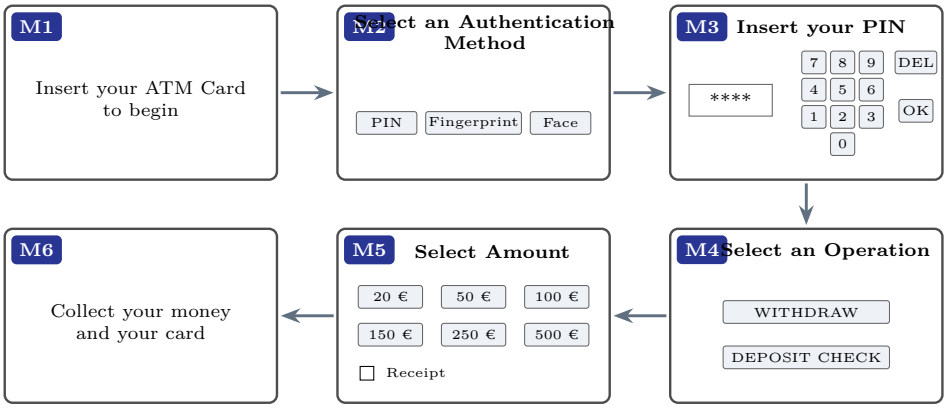


Figura 6.2: I mock-up M1–M6 del prelievo al bancomat, nell'ordine del main scenario.

Il caso d'uso fully-dressed

USE CASE #1	**Withdraw money**		
Goal in Context	Un cliente vuole prelevare denaro dal bancomat.		
Preconditions	Il cliente ha un conto presso la banca e possiede una carta bancomat. Ha 500 € sul conto.		
Success End Condition	Il sistema registra l'operazione di prelievo ed eroga il denaro richiesto.		
Failed End Condition	Non viene effettuata alcuna transazione.		
Primary Actor	Customer		
Trigger	Il cliente si avvicina al bancomat e tocca lo schermo per attivarlo.		
Main Scenario	**Step**	**Customer**	**System**
	1	Tocca lo schermo	
	2		Mostra M1
	3	Inserisce la carta	
	4		Mostra M2
	5	Tocca il pulsante «PIN»	
	6		Mostra M3
	7	Inserisce il PIN	
	8		Mostra M4
	9	Tocca il pulsante «Withdraw»	
	10		Mostra M5
	11	Tocca il pulsante «50 €»	
	12		Eroga il denaro, espelle la carta, mostra M6

Si noti la struttura a **ping-pong**: ogni azione del cliente è seguita da una risposta del sistema, e ogni risposta del sistema fa riferimento a un mock-up preciso.

Ora ci si chiede sistematicamente **cosa può andare storto** e **cosa può andare diversamente**.

Cosa può andare storto?

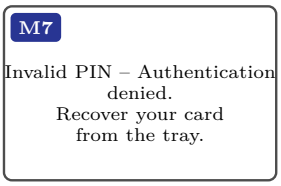
- il PIN non è corretto
- il cliente non ha abbastanza soldi sul conto
- il bancomat non ha abbastanza contanti
- la carta è segnalata come rubata
- la carta è illeggibile
- …

Cosa può andare diversamente?

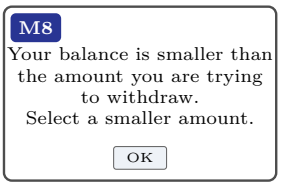
- il cliente si autentica con l'impronta digitale o con il riconoscimento facciale
- il cliente chiede la ricevuta stampata

Ognuno di questi scenari va descritto con un'estensione. Il passo di un'estensione si numera con il **numero del passo del main scenario** da cui devia, seguito da una **lettera** che identifica l'estensione (7a, 8a, … per la prima; 11b, 12b, … per la seconda). Il primo passo riporta tra parentesi angolari la **condizione** che attiva l'estensione.

Extension #1 (il cliente inserisce un PIN non valido)	**Step**	**Customer**	**System**
	7a <PIN errato>	Inserisce il PIN	
	8a		Mostra M7 e termina il caso d'uso



Extension #2 (il cliente non ha abbastanza soldi)	**Step**	**Customer**	**System**
	11b	Tocca il pulsante «500 €»	
	12b		Mostra M8
	13b	Clicca «OK»	
	14b		Torna al passo 10 del main scenario



Estensioni che terminano, estensioni che rientrano

Le due estensioni mostrano i due modi in cui può finire una deviazione: l'Extension #1 **termina** il caso d'uso (si arriva alla *Failed End Condition*: nessuna transazione), l'Extension #2 invece **rientra** nel main scenario al passo 10, dove il cliente può scegliere un importo più piccolo.

Precondizioni ed estensioni devono essere coerenti

Nella slide la precondizione dice che il cliente ha 500 € sul conto, ma l'Extension #2 scatta proprio quando il cliente prova a prelevare 500 € e il saldo è insufficiente. Le due cose non possono valere insieme: in un caso d'uso reale la precondizione andrebbe riformulata (ad esempio «il cliente ha almeno 50 € sul conto», quanto serve al main scenario). È un buon esempio del tipo di incoerenza da cercare in fase di **validazione**.

Per approfondire

Oltre al libro originale di Cockburn esistono molte linee guida per scrivere casi d'uso efficaci, ad esempio B. Wei, L. Deng, Y. Wang, *A Practical Style Guide and Templates Repository for Writing Effective Use Cases*, SERA 2022 (doi:10.1007/978-3-031-09145-2_5). L'articolo propone anche dei template di casi d'uso generici (`https://github.com/Washingtonwei/use-case-style-guide`) e un esempio completo, il *SuperFrog Scheduler*.

In sintesi

- I **casi d'uso** (UML) descrivono i requisiti funzionali con un diagramma (UCD) e descrizioni testuali: dicono **cosa** fa il sistema, visto come scatola nera.
- Gli **attori** sono ruoli esterni al SUD (utenti, sistemi, ambiente fisico); i **casi d'uso** sono funzionalità che portano un beneficio agli attori e astraggono molti scenari.
- Relazioni: **associazione** (con eventuale cardinalità), **generalizzazione** di attori e casi d'uso, **«include»** (parte comune, sempre eseguita, dal base all'incluso) e **«extend»** (comportamento opzionale o condizionale, dall'estensione al caso esteso, eventualmente in un *extension point*).
- Errori tipici: casi d'uso senza verbo o che sono semplici passi, più attori sullo stesso caso d'uso letti come «oppure», attori specializzati senza casi d'uso propri, «include» per il login, diagrammi troppo complessi.
- Ogni caso d'uso va specificato (formati **brief**, **casual**, **fully-dressed**); il template di **Cockburn** prevede goal, precondizioni, condizioni di fine, attore principale, trigger, **main scenario**, **estensioni** e open issues (vuoti alla consegna).

Parte IV

Modellazione con UML

UML E IL DIAGRAMMA DELLE CLASSI

Nel Capitolo 6 abbiamo usato il primo diagramma UML, quello dei casi d'uso, per descrivere i requisiti funzionali. In questo capitolo vediamo che cos'è UML in generale e poi il suo diagramma più diffuso, il **diagramma delle classi**: prima la notazione, poi il **procedimento** per costruire un modello partendo dal testo dei requisiti.

MODELLARE IL DOMINIO DEL PROBLEMA

Modello di dominio

Modello che descrive il **dominio del problema**. Si chiama anche **modello concettuale** o, talvolta, **modello di business**.

Problema o soluzione?

Non bisogna confondere i **modelli di dominio**, che descrivono **il problema**, con i **modelli di progetto**, che descrivono **la soluzione**. Il primo dice quali concetti esistono nel mondo reale (studenti, esami, voti); il secondo dice come saranno realizzati nel software (classi Java, tabelle del database, …). Torneremo su questa distinzione alla fine del capitolo.

Si distinguono due tipi di modelli:

- i modelli **statici**, che descrivono gli **elementi** del dominio del problema e le **relazioni** tra loro;
- i modelli **dinamici**, che descrivono i **comportamenti** riconoscibili nel dominio del problema.

Per descrivere questi modelli si usano **linguaggi di modellazione**. In passato ogni metodologia usava i propri, nelle varie fasi del processo di sviluppo (ad esempio i diagrammi di flusso o i *Data Flow Diagram*). Negli ultimi anni si è affermato UML come linguaggio **unificato**, usato in tutte le attività di modellazione e in molte altre attività del ciclo di vita del software.

CHE COS'È UML

UML (Unified Modelling Language)

Linguaggio **grafico standard** nato per modellare software **orientato agli oggetti**. Una specifica UML definisce in pratica un **metamodello**: l'insieme delle regole con cui si possono costruire modelli UML.

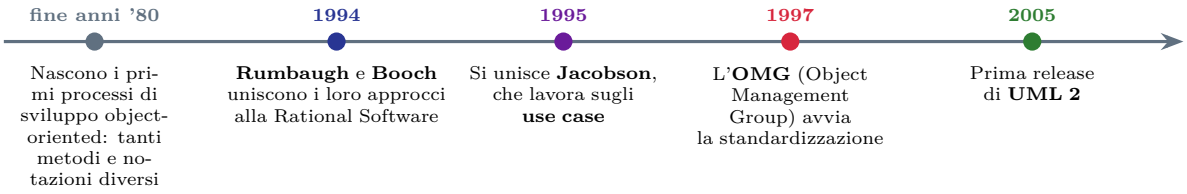


Figura 7.1: Le tappe principali della nascita di UML. La storia di tutte le specifiche è su `http://www.omg.org/spec/UML/`.

Come si usa UML

Uso	Descrizione
Come abbozzo	Per tracciare un **modello di massima** del sistema da realizzare.
Come progetto	Per realizzare un **modello completo** della soluzione architetturale del sistema.
Come linguaggio di programmazione	Per modellare in maniera completa e precisa il sistema software. L'approccio **MDA** (*Model Driven Architecture*) esplora la possibilità di usare UML come linguaggio di programmazione. Strumenti come **PlantUML** permettono di descrivere i diagrammi UML in forma testuale, come codice.

Regole prescrittive e descrittive

UML ha due tipi di regole:

- le regole **prescrittive**, stabilite da organismi di standardizzazione, che fissano con precisione lessico, sintassi e semantica;
- le regole **descrittive**, che servono solo a **comunicare** il significato dei diagrammi, anche con estensioni libere ma intuitive delle regole di base.

Poiché lo scopo principale di UML è descrivere efficacemente situazioni reali, oggi prevalgono le regole **descrittive**.

Regola delle informazioni soppresse

L'**assenza** di un'informazione in un diagramma UML **non** significa, in generale, che quell'informazione non esista o sia nulla. Significa solo che è un aspetto non ancora trattato nella fase in cui il diagramma è stato tracciato, oppure che non lo si vuole mostrare in quel diagramma (per inserirlo in altri diagrammi).

I diagrammi di UML 2

UML 2 ha **13 tipi di diagrammi**, divisi in tre categorie.

Comportamentali
Modellano il comportamento del sistema.
Es. casi d'uso, attività, macchine a stati.

Di interazione
Modellano le interazioni fra le entità del sistema.
Es. **sequence diagram**, communication diagram.

Strutturali
Modellano l'organizzazione del sistema.
Es. **class diagram**, package diagram, deployment diagram.

Figura 7.2: Le tre categorie di diagrammi di UML 2. In grassetto i due più usati.

In questo corso ci si concentra sui due diagrammi più utilizzati: il **diagramma delle classi** (questo capitolo) e il **diagramma di sequenza** (Capitolo 8).

Strumenti e riferimenti

Visual Paradigm (`https://www.visual-paradigm.com/`) è un ottimo strumento per disegnare diagrammi UML, disponibile anche in una *Community Edition* gratuita. Occasionalmente si usa anche il plug-in *Diagrams* di IntelliJ IDEA per generare codice e diagrammi l'uno dall'altro.
Il testo di riferimento è Martin Fowler, *UML Distilled*: capitolo 1 (introduzione a UML), capitoli 3 e 5 (diagramma delle classi), capitolo 4 (diagramma di sequenza), capitolo 7 (diagramma dei package).

Il diagramma delle classi

Il diagramma delle classi (*class diagram*) è il più diffuso diagramma di UML. È un diagramma **statico**, che si può usare:

- per la modellazione **concettuale** del dominio di un problema;
- per modellare le **specifiche** richieste a un sistema;
- per modellare l'**implementazione** (object-oriented) di un sistema software.

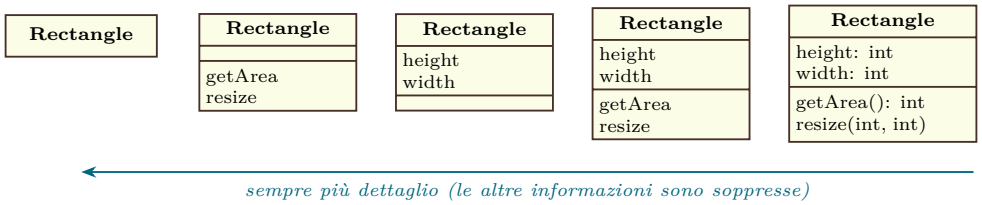
I suoi concetti fondamentali sono estensioni di quelli della programmazione orientata agli oggetti. Nel seguito lo presentiamo nella versione più completa, quella usata per modellare l'implementazione.

Elemento	Che cosa rappresenta
Classi	I tipi di dati presenti nel sistema
Associazioni	I collegamenti tra istanze di classi
Attributi	I dati semplici presenti nelle classi e nelle loro istanze
Operazioni	Le funzioni svolte dalle classi e dalle loro istanze
Generalizzazioni	Raggruppano le classi in gerarchie di ereditarietà

Tabella 7.1: Gli elementi principali di un diagramma delle classi.

Le classi

Una classe si rappresenta con un **rettangolo** con il nome della classe; il concetto di classe è lo stesso della programmazione a oggetti. Il rettangolo può avere altri due scomparti, uno per gli **attributi** e uno per le **operazioni**, che si possono mostrare con un livello di dettaglio crescente.



Tutte e cinque rappresentano la stessa classe: per la regola delle informazioni soppresse, il primo rettangolo non dice che `Rectangle` non ha attributi, dice solo che in quel diagramma non li mostriamo. La firma completa di un'operazione è:

```
nomeOperazione(nomeParametro: TipoParametro, …): TipoRitornato
```

Gli attributi

Sintassi di un attributo

```
visibilità nome molteplicità: tipo = default {proprietà}
```

- Il **nome** è l'unica parte obbligatoria.
- Il **tipo** può essere un tipo primitivo (`int`, `double`, `char`, …) oppure il nome di una classe dello stesso diagramma (ma in quel caso, forse, andrebbe rappresentato con un'associazione).
- **default** è il valore iniziale dell'attributo.
- **{proprietà}** indica caratteristiche aggiuntive, ad esempio `{readOnly}` (sola lettura).

La **visibilità** ha tre livelli, che valgono sia per gli attributi sia per le operazioni:

Simbolo	Visibilità	Chi può usare l'elemento	In Java
`+`	**Pubblica**	Tutte le classi	`public`
`#`	**Protetta**	Solo le classi derivate dalla classe originale	`protected`
`-`	**Privata**	Solo la classe originale	`private`

La **molteplicità** indica **quanti** valori contiene l'attributo: così si possono rappresentare array e liste. Il valore predefinito è `1`.

Molteplicità	Significato
`1`	Uno e uno solo (predefinita)
`0..1`	Al più uno
`*`	Un numero imprecisato, eventualmente nessuno (equivale a `0..*`)
`1..*`	Almeno uno

Gli elementi di una molteplicità sono considerati un **insieme**: se sono ordinati si aggiunge `{ordered}`, se possono esserci duplicati si aggiunge `{nonunique}`.

Esempio 7.1 — Un attributo completo

```
- name [1]: String = "Untitled" {readOnly}
```

Un attributo **privato** di nome `name`, con un solo valore, di tipo `String`, che vale inizialmente `"Untitled"` e non può essere modificato.

Le operazioni

Sintassi di un'operazione

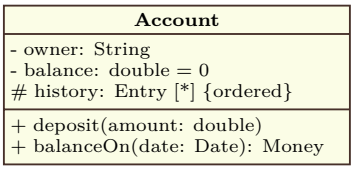
```
visibilità nome (lista parametri): tipo-ritornato {proprietà}
```

Visibilità e nome seguono le stesse regole degli attributi. Ogni parametro della lista ha la forma

```
direzione nome: tipo = valore-di-default
```

dove la **direzione** è `in` (input, il valore predefinito), `out` (output) o `inout` (entrambi). Il **tipo ritornato** dovrebbe essere un tipo appartenente a una classe standard.

Esempio 7.2 — Dalla classe UML al codice Java




```java
public class Account {
    private String owner;
    private double balance = 0;
    protected List<Entry> history;  // [*] {ordered}

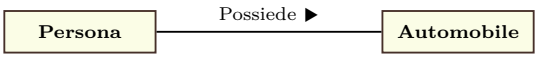
    public void deposit(double amount) { ... }
    public Money balanceOn(Date date) { ... }
}
```

Ogni simbolo di visibilità diventa il modificatore Java corrispondente, e un attributo con molteplicità `[*]` diventa una collezione; `{ordered}` suggerisce una `List` invece di un `Set`.

LE ASSOCIAZIONI

Associazione

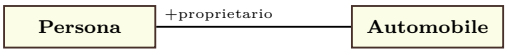
Un'**associazione** rappresenta una relazione (fisica o concettuale) tra classi. Si disegna come una **linea** tra le due classi, con un **nome**; un piccolo triangolo accanto al nome indica il **verso di lettura**.



Il triangolo dice che l'associazione si legge «la **Persona** possiede l'**Automobile**», e non «l'Automobile possiede la Persona». Una possibile implementazione:

```java
class Persona {
    Automobile automobilePosseduta;
}
class Automobile {
}
```

In alternativa al nome si può indicare il **ruolo** che una delle due classi ha nell'associazione, scritto vicino al suo estremo.



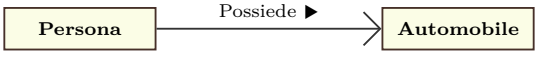

```java
class Persona {
}
class Automobile {
    Persona proprietario;
}
```

Qui la Persona ha il ruolo di **proprietario** dell'Automobile: il nome del ruolo diventa il nome dell'attributo.

Il verso di navigazione

Verso di navigazione

Indica in quale direzione è possibile **reperire le informazioni**: da quale classe si può arrivare all'altra. Si disegna con una **freccia** sulla linea dell'associazione. È un'informazione utile soprattutto nel progetto di dettaglio: essendo una scelta di progetto, di solito **non** compare nei diagrammi concettuali.



- Nota una persona, è possibile sapere quali automobili possiede (se ne possiede).
- Nota un'automobile, **non** è possibile risalire al suo possessore.
- Il diagramma non dice nulla, invece, su **quante** automobili ha una persona né su quanti proprietari ha un'automobile: non sappiamo se sono informazioni non note o semplicemente soppresse.

Esempio 7.3 — Implementazione corretta ed errata

Corretta

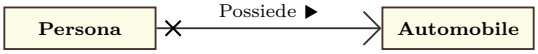
```
class Persona {
    Automobile automobilePosseduta;
}
class Automobile {
}
```

Errata

```
class Persona {
}
class Automobile {
    Persona possessore;
}
```

Nella versione errata, data una persona non c'è modo di risalire alla sua automobile: il codice permette la navigazione nel verso **opposto** a quello richiesto dal diagramma.

Più di recente è stato introdotto anche un simbolo per il verso in cui un'associazione **non** può essere navigata: una **croce** vicino all'estremo.



Data la persona si conosce l'automobile; data l'automobile **non** si può conoscere la persona.

Verso di lettura e verso di navigazione

Sono due cose diverse. Il **triangolino** accanto al nome (▶) dice solo come si **legge** il nome dell'associazione; la **freccia** sulla linea dice in che direzione il software può **navigare** da un oggetto all'altro. Un diagramma concettuale ha spesso il primo ma quasi mai il secondo.

La molteplicità delle associazioni

Molteplicità di un'associazione

Indica il **numero di istanze** di ciascuna classe che possono partecipare a un'istanza dell'associazione. Si scrive vicino all'estremo della classe a cui si riferisce.



- Ogni **Persona** possiede **almeno una** Automobile (`1..*`, scritto dal lato di Automobile). Evidentemente le persone senza automobile non fanno parte del problema.
- Ogni **Automobile** è posseduta da **una e una sola** Persona (`1`, dal lato di Persona). Evidentemente non interessa tenere traccia dei proprietari precedenti (auto di seconda o terza mano).

È un'associazione **uno a molti**: una Persona, molte Automobili.

Da che lato si legge la molteplicità

La molteplicità scritta vicino a una classe dice quante istanze di **quella** classe sono collegate a **una** istanza della classe **dall'altra parte**. Il `1..*` vicino ad Automobile si legge: «*una* Persona possiede *da 1 a molte* Automobili».

Esempio 7.4 — Implementare una molteplicità uno a molti

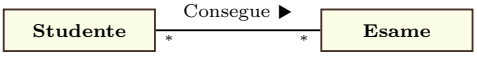
```java
class Persona {
    ArrayList<Automobile> automobiliPossedute = new ArrayList<>();

    // la persona diventa proprietaria di un'automobile esistente
    Persona(Automobile a) {
        automobiliPossedute.add(a);
        a.proprietario = this;
    }
    // la persona diventa proprietaria di un'automobile nuova
    Persona() {
        automobiliPossedute.add(new Automobile(this));
    }
}
class Automobile {
    Persona proprietario;
    Automobile(Persona p) { proprietario = p; }
}
```

I **costruttori** fanno rispettare le molteplicità: non può esistere una Persona senza almeno un'Automobile, né un'Automobile senza proprietario. La slide usa due nomi diversi per la lista (`automobiliPossedute` nella dichiarazione, `automobilePosseduta` nei costruttori) e non la inizializza: così com'è, il codice non compila e, corretto il nome, darebbe una `NullPointerException`. Qui la lista è creata insieme all'oggetto.

Associazioni molti a molti

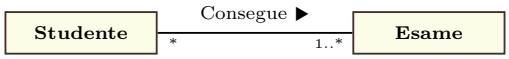
- Uno studente può superare un numero non limitato di esami.
- Un esame può essere superato da un numero non limitato di studenti.
- Possono esserci studenti che non hanno superato esami.
- Possono esserci esami che (ancora) nessuno ha superato.




```java
class Studente { ArrayList<Esame> esamiConseguiti = new ArrayList<>(); }
class Esame    { ArrayList<Studente> studentiEsaminati = new ArrayList<>(); }
```

Esempio 7.5 — Uno studente esiste solo dal primo esame

Se uno studente va considerato solo dal momento in cui supera il primo esame, la molteplicità dal lato di Esame diventa `1..*`.

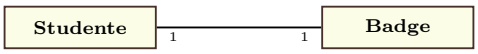



```java
class Studente {
    ArrayList<Esame> esamiConseguiti = new ArrayList<>();
    Studente(Esame e) {             // non si crea uno studente senza un esame
        esamiConseguiti.add(e);
        e.aggiungiStudente(this);   // aggiorna anche l'altro lato
    }
}
class Esame {
    private ArrayList<Studente> studentiEsaminati = new ArrayList<>();
    public void aggiungiStudente(Studente s) { studentiEsaminati.add(s); }
}
```

Come osserva la slide, se la lista di Esame è privata serve un metodo pubblico per aggiungere uno studente. Anche qui le liste vanno **inizializzate**: nella versione della slide non lo sono, e creare uno studente darebbe una `NullPointerException` (verificato eseguendo il codice).

Associazioni uno a uno

Ogni studente ha **uno e un solo** badge, e ogni badge identifica **uno e un solo** studente. Con questo modello non si può rappresentare lo storico dei badge (ad esempio quelli smarriti e sostituiti).



La soluzione «simmetrica» non funziona

La slide propone una soluzione simmetrica, in cui ciascun costruttore crea l'altro oggetto:

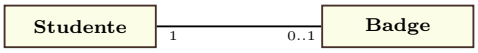
```java
class Studente {
    Badge badge;
    Studente() { badge = new Badge(); badge.studente = this; }
}
class Badge {
    Studente studente;
    Badge() { studente = new Studente(); studente.badge = this; }
}
```

Ma `new Studente()` chiama `new Badge()`, che chiama `new Studente()`, che chiama `new Badge()`… all'infinito: il programma termina con uno `StackOverflowError` (verificato eseguendo il codice). La simmetria va **rotta**: un solo lato crea l'altro e gli passa se stesso.

```java
class Studente {
    Badge badge;
    Studente() { badge = new Badge(this); }  // lo studente crea il suo badge
}
class Badge {
    Studente studente;
    Badge(Studente s) { studente = s; }      // il badge riceve lo studente
}
```

Esempio 7.6 — Studenti temporaneamente senza badge

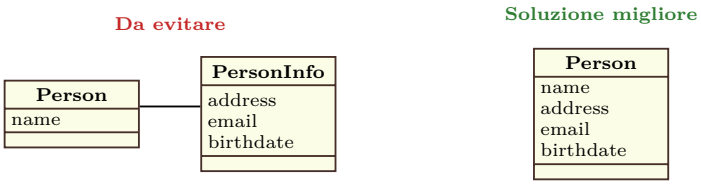
Se uno studente può restare, magari temporaneamente, senza badge, la molteplicità dal lato di Badge diventa `0..1`.




```java
class Studente {
    Badge badge;          // puo' restare null
}
class Badge {
    Studente studente;
    Badge(Studente s) { studente = s; s.badge = this; }
}
```

Lo studente può esistere senza badge, quindi il suo costruttore non crea nulla; il badge invece richiede sempre uno studente **già esistente**, e non deve crearne uno nuovo.

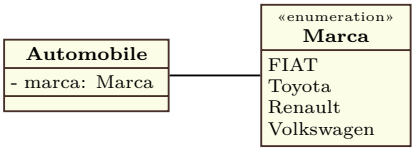
Controesempio: associazioni uno a uno inutili



Se due classi sono legate da un'associazione `1`–`1` e una non ha senso senza l'altra, spesso sono in realtà **una sola classe** spezzata in due senza motivo.

Le enumerazioni

Le **enumerazioni** indicano un elenco di possibili valori di un dato, come in Java. In UML si disegnano come una classe con lo stereotipo **«enumeration»**. Si possono collegare alle classi con una linea di associazione (di solito senza ruoli né molteplicità) e usare come tipo degli attributi.

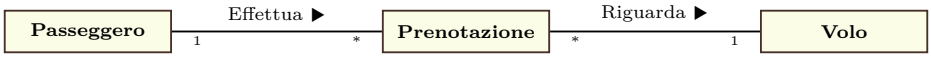



```java
enum Marca { FIAT, TOYOTA, RENAULT,
    VOLKSWAGEN }
```

Esempio 7.7 — Passeggeri, prenotazioni e voli

Dai requisiti:

- una prenotazione si riferisce **sempre a un solo** passeggero: non esistono prenotazioni senza passeggero (quindi, prima di creare una prenotazione, il passeggero deve esistere) né prenotazioni per più passeggeri;
- un passeggero può avere **zero, una o più** prenotazioni;
- una prenotazione si riferisce a **un** volo, e un volo può avere **più** passeggeri prenotati.

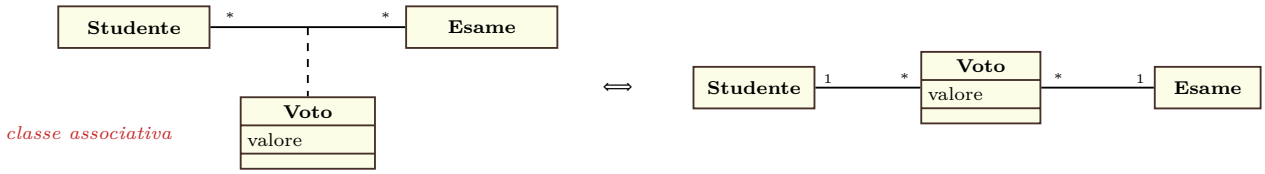


Ogni frase dei requisiti è diventata una molteplicità. Il diagramma è ricavato dal testo della slide, che non lo riporta.

Classi associative

Classe associativa

Classe collegata con una linea **tratteggiata** a un'associazione. Serve quando un attributo riguarda **due** classi collegate ma non si può attribuire a **nessuna** delle due, tipicamente nelle associazioni molti a molti. Come ogni classe, può avere attributi, metodi e altre associazioni.



Le due soluzioni sono considerate equivalenti: nella seconda la classe associativa diventa una classe normale, e la molteplicità *–* si spezza in due associazioni uno a molti. In entrambi i casi, prima di creare un voto devono esistere lo studente e l'esame a cui si riferisce:

```
Studente s = new Studente();
Esame e = new Esame();
Voto v = new Voto(s, e, 30);
```

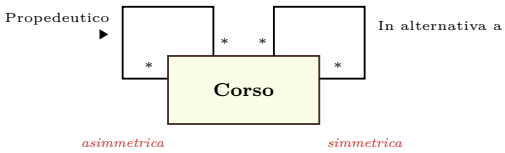
Una sottile differenza

Con la classe associativa, per ogni coppia (studente, esame) può esistere **al più un** voto, perché le istanze di un'associazione sono coppie distinte. Nella seconda soluzione nulla vieta due voti per lo stesso studente e lo stesso esame (ad esempio se l'esame si può ripetere). Se questo è ciò che serve, la seconda soluzione è quella giusta.

Associazioni riflessive

Un'associazione **riflessiva** collega una classe con se stessa. Un corso può essere **propedeutico** ad altri corsi (associazione **asimmetrica**: se A è propedeutico a B, B non è propedeutico ad A) o **in alternativa** ad altri corsi (associazione **simmetrica**).

```
class Corso {
    ArrayList<Corso> corsiPropedeutici;
    ArrayList<Corso> corsiAlternativi;
}
```



AGGREGAZIONE E COMPOSIZIONE

Aggregazione

Associazione speciale che rappresenta una relazione **tutto–parti**. Il lato del «tutto», detto **aggregato**, ha un **rombo vuoto**. La molteplicità dal lato del tutto, quando è sottintesa, vale `0..1`: una parte appartiene al più a un aggregato.



Un'associazione diventa un'aggregazione quando:

- si può dire che le parti sono «**parte di**» un insieme, e che l'aggregato è «**composto da**» parti;
- chi possiede o controlla l'aggregato possiede o controlla anche le sue parti.

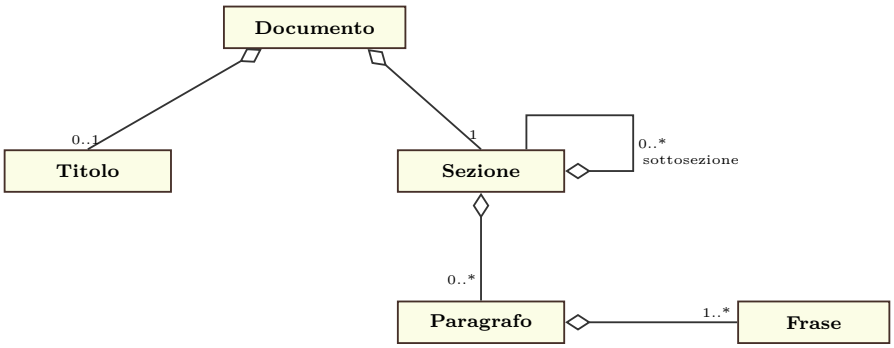
Le aggregazioni sono importanti per l'espressività del modello, ma spesso si implementano esattamente come le associazioni con la stessa molteplicità.

Esempio 7.8 — Implementare un'aggregazione

```java
class Veicolo {
    ArrayList<Ruota> ruote = new ArrayList<>();  // obbligatorio: il veicolo aggrega ruote
}
class Ruota {
    Veicolo v = null;   // consigliato, ma puo' restare null
}
```

L'attributo `ruote` in Veicolo è **obbligatorio**, perché esprime il fatto che un veicolo aggrega ruote. L'attributo `v` in Ruota è **consigliato**, perché permette di navigare la relazione anche dalla ruota; ma può valere `null`, perché potrebbe trattarsi di una ruota di scorta che non è montata su nessun veicolo.

Esempio 7.9 — Una gerarchia di aggregazioni: il documento

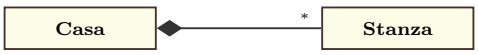


Un documento ha al più un titolo e almeno una sezione; una sezione può contenere sottosezioni (aggregazione riflessiva) e paragrafi; un paragrafo è composto da una o più frasi. Qui le molteplicità scritte dal lato delle parti dicono quante parti ha ogni aggregato. (Nella slide la molteplicità di Sezione è `1`: un documento avrebbe una sola sezione di primo livello, che contiene tutte le altre.)

Composizione

Forma **forte** di aggregazione, con il **rombo pieno**: se l'aggregato viene distrutto, vengono distrutte anche le sue parti, che **non esistono senza il tutto**. La molteplicità dal lato del tutto, quando è sottintesa, vale `1`.

Esempio 7.10 — La casa e le stanze



In questo dominio ipotizziamo che non abbia senso parlare di una stanza finché non è legata alla casa in cui si trova.

```java
class Casa {
    ArrayList<Stanza> stanze = new ArrayList<>();
}
class Stanza {
    Casa c;
    Stanza(Casa casa) { c = casa; }   // una Stanza nasce sempre dentro una Casa
}
```

Rispetto all'aggregazione, ora l'attributo `c` in Stanza è **obbligatorio** e viene inizializzato dal costruttore: non può esistere una Stanza che non sia associata a una Casa.

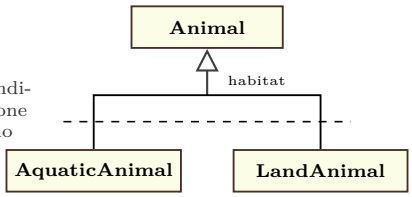
Relazione	Simbolo	Significato	Molteplicità sottintesa del tutto
Associazione	——	Relazione generica tra classi	—
Aggregazione	◇——	Tutto–parti: le parti possono esistere da sole	`0..1`
Composizione	◆——	Tutto–parti forte: le parti muoiono con il tutto	`1`

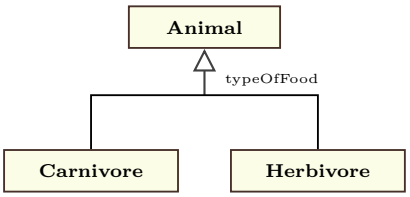
Tabella 7.2: Associazione, aggregazione e composizione a confronto. Il rombo sta sempre dal lato del tutto.

Generalizzazione

I concetti di generalizzazione e specializzazione di UML sono del tutto analoghi a quelli della programmazione a oggetti. Si disegnano con una **freccia a punta vuota** dalla sottoclasse alla superclasse.

La linea tratteggiata indica che la generalizzazione è **incompleta**: esistono altri tipi di animali.





- Il **discriminatore** (`habitat`, `typeOfFood`) dice secondo quale criterio si specializza. Si potrà implementare come un attributo che ha valori diversi nelle sottoclassi.
- Una generalizzazione **incompleta** (nella notazione standard `{incomplete}`) lascia spazio ad altre sottoclassi non disegnate.
- A livello **concettuale**, la generalizzazione esprime una relazione **is-a** («è un») tra un concetto generale e le sue specializzazioni.
- A livello di **progetto di dettaglio**, diventa una relazione di **ereditarietà** tra classi: le sottoclassi ereditano attributi e metodi `public` e `protected` della superclasse.

Esempio 7.11 — La generalizzazione in Java, e il limite dei due discriminatori

La prima gerarchia diventa ereditarietà; il discriminatore `habitat` diventa un metodo che ogni sottoclasse ridefinisce.

Animal.java

```java
public abstract class Animal {
  private final String nome;

  protected Animal(String nome) {
    this.nome = nome;
  }

  public abstract String habitat();            // il discriminatore
}

public class AquaticAnimal extends Animal {
  public AquaticAnimal(String nome) { super(nome); }

  @Override
  public String habitat() { return "acqua"; }
}

public class LandAnimal extends Animal {
  public LandAnimal(String nome) { super(nome); }

  @Override
  public String habitat() { return "terra"; }
}
```

Il diagramma, però, ha **due** generalizzazioni indipendenti della stessa classe, e un delfino è insieme acquatico e carnivoro. Java ha l'ereditarietà singola: non si può scrivere `class Delfino extends AquaticAnimal, Carnivore`. Il secondo discriminatore diventa allora un **attributo**, come suggerisce la notazione stessa:

Animal.java

```java
public enum Dieta { CARNIVORO, ERBIVORO, ONNIVORO }

public abstract class Animal {
  private final Dieta typeOfFood;              // il secondo discriminatore, come attributo
  // ...
}
```

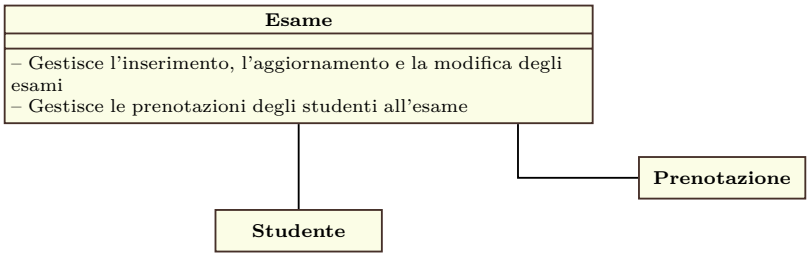
Responsabilità e note

Le responsabilità

Nei diagrammi che descrivono il **dominio del problema** di solito non si mettono metodi, perché sarebbero un elemento della **soluzione**. Al loro posto si possono usare le **responsabilità**.

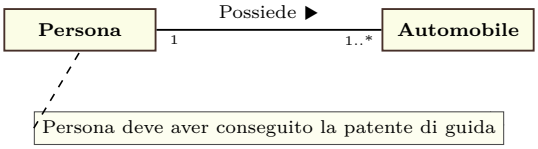
Responsabilità

Insieme di **servizi e compiti** che una classe dovrebbe garantire. Si scrivono nello scomparto delle operazioni, preceduti da `--`. Sono utili per controllare la **completezza** del modello di dominio.



Le note

A qualunque diagramma UML si possono aggiungere informazioni in linguaggio naturale con le **note**: un rettangolo con l'angolo piegato, collegato con una linea tratteggiata all'elemento a cui si riferisce. Una nota è l'equivalente di un **commento** in un linguaggio di programmazione.



Dominio del problema e dominio della soluzione

Modello del dominio del problema	Modello del dominio della soluzione
Tiene conto della **descrizione dei requisiti** del problema da risolvere, **senza** riferimenti a elementi della soluzione che verrà realizzata.	Tiene conto delle **scelte tecnologiche e realizzative** della soluzione.
Unica eccezione: se una parte di soluzione **esistente** va riusata, può far parte del dominio del problema (ad esempio le specifiche di un vecchio software con cui il nuovo deve comunicare).	Può modellare, ad esempio, le classi che realizzeremo in Java, oppure il modello relazionale di un database MySQL.
Niente metodi: al più **responsabilità**. Niente verso di navigazione.	Metodi, visibilità, tipi, verso di navigazione.

COSTRUIRE UN MODELLO DI DOMINIO

Un modello di dominio si costruisce con un **processo di astrazione** in cinque passi.

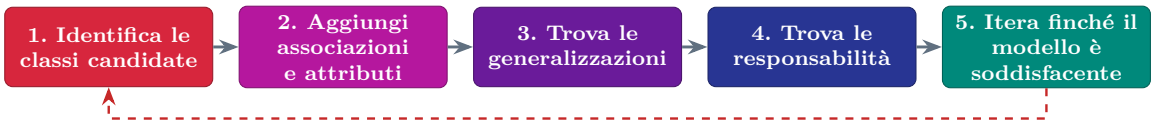


Figura 7.3: Il processo di astrazione per costruire un modello di dominio.

1. Identificare le classi

Quando si sviluppa un modello di dominio si tende a **scoprire** classi che fanno parte del dominio; quando si lavora all'interfaccia utente o all'architettura si tende invece a **inventare** classi, necessarie a risolvere un problema di progetto (e se ne possono inventare anche per il modello di dominio). In ogni caso, a una classe dovrebbe corrispondere un'**entità del dominio del problema**.

Com'è fatta una buona classe di analisi?

- Il suo **nome** ne rispecchia l'intento.
- È un'**astrazione ben definita** che modella un elemento del dominio del problema.
- Ha un insieme **ridotto e ben definito** di responsabilità.
- Ha la massima **coesione interna**: una classe è tanto più coesa quanto più riesce ad assolvere **da sola** alle proprie responsabilità.

Analisi dei nomi

Tecnica semplice per scoprire le classi del dominio:

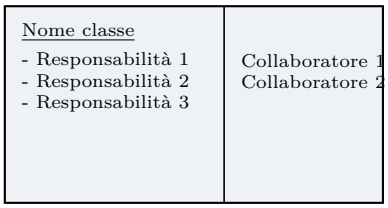
1. si analizza la documentazione di partenza, ad esempio la descrizione dei requisiti;
2. si estraggono i **nomi** e i **predicati nominali** (nomi con i loro aggettivi);
3. si eliminano i nomi che sono **ridondanti** (rappresentano la stessa classe), che rappresentano **istanze** e non classi, che sono **vaghi** o troppo generici, o che corrispondono a classi **non necessarie** al livello considerato.

Attenzione alle classi che rappresentano **tipi di utente** o altri attori: servono davvero?

Le schede CRC

CRC sta per *Class Responsibility Collaboration*.

1. Per ogni classe identificata si scrive il nome su una scheda (*card*).
2. Man mano che si trovano attributi e responsabilità, si elencano sulla scheda, insieme alle classi con cui collabora.
3. Si sistemano le schede su una lavagna per creare il diagramma delle classi.
4. Si disegnano le linee di associazioni e generalizzazioni.



Il modello di una scheda CRC (Larman).

Perché proprio una scheda di carta

La scheda ha uno spazio **limitato**, e questo impone all'analista, almeno psicologicamente, di non creare classi con troppi attributi e metodi. Se la scheda è piena, probabilmente la classe va **divisa** in due o più classi più semplici.

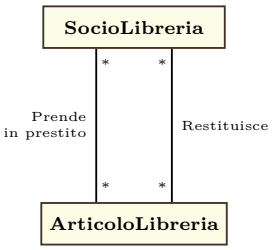
2. Identificare associazioni e attributi

- Si parte dalle classi ritenute **centrali** e importanti, stabilendo i dati ovvi che contengono e le loro relazioni con le altre classi; poi si procede con le classi meno importanti.
- Si evita di aggiungere troppi attributi e associazioni a una classe: un sistema è più semplice se manipola meno informazioni, e le classi devono avere poche dipendenze dalle altre.

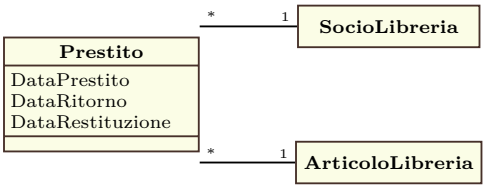
Un'associazione dovrebbe esistere se una classe **possiede**, **controlla**, **è collegata a**, **si riferisce a**, **è parte di**, **ha come parti**, **è membro di** oppure **ha come membri** un'altra classe del modello. Per ogni associazione si specificano le **molteplicità da entrambi i lati** e un **nome chiaro**.

Esempio 7.12 — Azioni scambiate per associazioni

Un errore comune è modellare delle **azioni** come se fossero associazioni.



Errore!



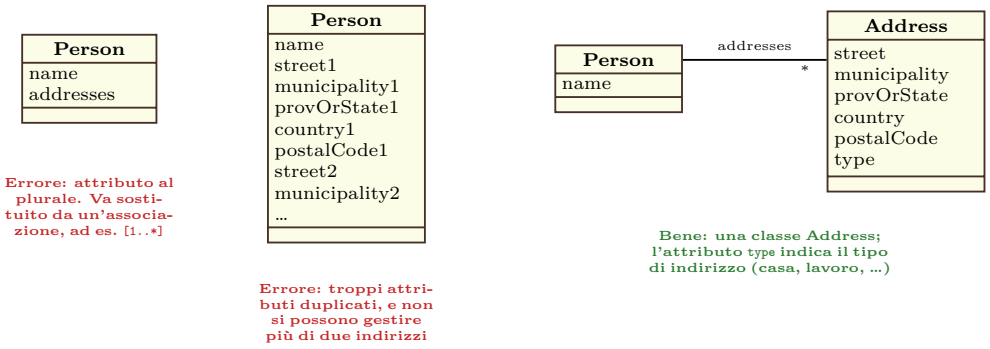
Corretto

«Prendere in prestito» e «restituire» non sono relazioni stabili tra un socio e un articolo: sono **operazioni**. Il concetto che esiste davvero nel dominio è il **Prestito**, con le sue date. L'operazione *presta* crea un Prestito, l'operazione *restituisci* imposta la data di restituzione: entrambe vanno assegnate alla classe Prestito, che nell'implementazione avrà un riferimento a un SocioLibreria e uno a un ArticoloLibreria.

2.a Identificare gli attributi

- Si cercano le informazioni che devono essere conservate per ciascuna classe.
- Nomi scartati come classi possono ora diventare attributi.
- Un attributo dovrebbe in genere contenere **un solo valore** (una stringa, un numero, …).
- Meglio evitare molti attributi duplicati: se un sottoinsieme degli attributi forma un **gruppo coerente**, si crea una classe distinta per quegli attributi.

Esempio 7.13 — Gli indirizzi di una persona



---

3. Identificare generalizzazioni e interfacce

Ci sono due modi per trovare le generalizzazioni:

- **bottom-up**: si raggruppano classi simili creando una nuova superclasse;
- **top-down**: si cercano prima le classi più generali, e poi le si specializza.

4. Assegnare le responsabilità alle classi

Una **responsabilità** è qualcosa che è richiesto al sistema. La responsabilità di **ogni requisito funzionale** va attribuita a una delle classi, anche se il requisito sarà poi svolto dalla collaborazione di più classi.

- Tutte le responsabilità di una classe dovrebbero essere **chiaramente correlate**.
- Se una classe ha **troppe** responsabilità, conviene valutare di dividerla in più classi.
- Se una classe **non ha** responsabilità, potrebbe essere inutile.
- Se una responsabilità non si può attribuire a nessuna classe esistente, va creata una **nuova** classe.
- Per trovare le responsabilità si cercano **verbi** e nomi che descrivono **azioni** nella descrizione del sistema.

Categorie di responsabilità

- leggere e impostare (*get* e *set*) gli attributi
- leggere o salvare dati in una memoria persistente
- aggiungere e cancellare istanze di associazioni
- calcolare risultati numerici
- altro lavoro specifico
- creare e inizializzare nuove istanze
- distruggere istanze
- copiare, convertire, trasformare, trasmettere dati
- navigare e cercare dati di particolari istanze

In un diagramma del dominio (senza elementi di progetto e implementazione) la maggior parte di queste responsabilità **non** si indica. Viceversa, le responsabilità individuate in fase di analisi vanno **sempre** considerate.

Tre errori da non fare

- **I nomi delle classi sono sempre al singolare**: `Studente`, non `Studenti`. Da non confondere con i nomi delle tabelle di un database.
- **Il «Sistema» non è quasi mai una classe**: tutto il modello rappresenta il dominio del problema che verrà risolto dal sistema.
- **Gli «id» non sono quasi mai attributi del dominio**: «id» indica di solito una chiave primaria artificiale di una tabella, senza corrispettivo nel mondo reale. `idStudente` non è un vero attributo (nessuno studente è schedato con un numero qualsiasi), mentre `numeroMatricola` lo è: la matricola viene davvero assegnata allo studente e usata per richiedere servizi, registrare esami, …

DUE ESEMPI COMPLETI

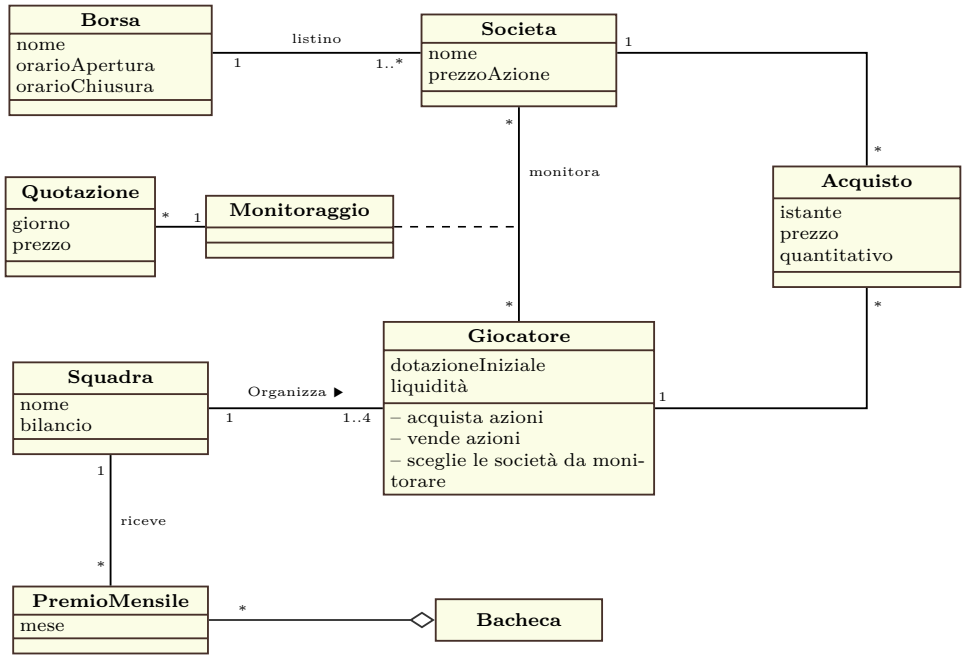
Esempio 7.14 — Un videogioco di borsa: l'analisi del testo

Il testo dei requisiti, con evidenziate le parole che suggeriscono **classi**, *attributi e associazioni* e responsabilità:

- Si vuole progettare un videogioco nel quale il giocatore impersona un agente di borsa, rivivendone le interazioni e basandosi sulle azioni delle società effettivamente quotate alla borsa di New York.
- Ogni **giocatore** ha una *dotazione monetaria iniziale*, pari a 100 milioni di dollari, che può essere investita acquistando delle **azioni**. Possono essere acquistate azioni di ognuna delle **società quotate** nel *listino della* **borsa** di New York.
- Le **azioni** di una **società** *vengono acquistate* al *prezzo* cui sono quotate nell'*istante* dell'**acquisto**. In fase di acquisto, il **giocatore** acquirente deve specificare anche il *quantitativo* di **azioni** che vuole acquistare. Il sistema provvederà a valutare l'effettiva *disponibilità di liquidi* da parte dell'acquirente e, in caso di acquisto possibile, scalerà dalla sua *liquidità* il *denaro necessario* all'**acquisto** delle **azioni**.
- Il **giocatore** potrà anche vendere le **azioni** che *possiede*, al *prezzo* cui sono quotate in quel momento (il sistema provvederà ad aggiornare la *liquidità*).
- Acquisti e vendite di azioni potranno essere effettuati soltanto nel periodo di tempo tra l'*orario di apertura* e l'*orario di chiusura* della **borsa** di New York.
- I **giocatori** possono anche scegliere di monitorare le **azioni** di alcune società. In tal caso il sistema manterrà un valore del *prezzo* delle **azioni** di ognuna delle **società monitorate** per ogni giorno nel quale il **giocatore** vorrà monitorarla.
- I **giocatori** *sono organizzati in* **squadre** di al più 4 elementi. Ogni **squadra** ha un *nome* e il sistema assegnerà un **premio mensile** alla squadra che ha il *bilancio* migliore. Una **bacheca** *manterrà* tutti i **premi mensili** ricevuti.

Applicando l'analisi dei nomi: «giocatori» e «giocatore» sono **ridondanti**; «borsa di New York» è un'**istanza** della classe Borsa; «agente di borsa» è solo il ruolo impersonato dal giocatore; «società quotate» e «società monitorate» sono la stessa classe **Società**, in due associazioni diverse con il giocatore. «Prezzo», «quantitativo», «nome», «bilancio» e gli orari diventano **attributi**.

Esempio 7.15 — Un videogioco di borsa: una possibile soluzione

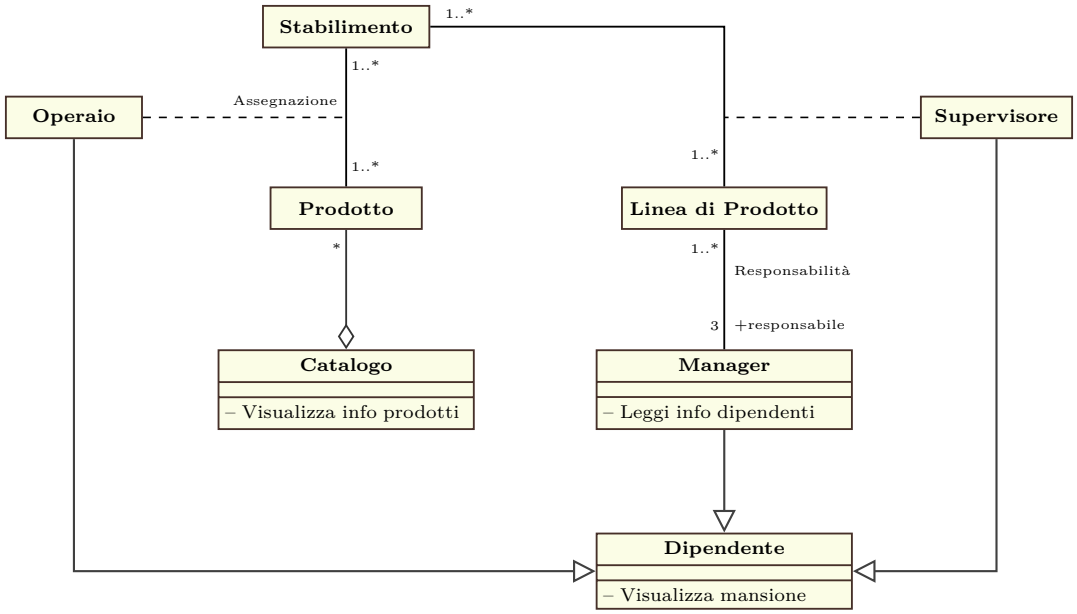


Questa soluzione non è nelle slide: è una proposta ricavata dall'analisi del testo, e altre scelte sono ugualmente valide. Le scelte principali:

- l'**Acquisto** lega un giocatore a una società e memorizza istante, prezzo e quantitativo: è ciò che il testo chiama «le azioni che possiede»;
- il **Monitoraggio** è una classe associativa tra Giocatore e Società (il giocatore sceglie quali società monitorare), e tiene una **Quotazione** per ogni giorno di monitoraggio;
- la dotazione iniziale di 100 milioni è il valore iniziale dell'attributo `liquidità`; il controllo sugli orari di apertura e chiusura è una responsabilità che coinvolge Borsa e Giocatore.

Esempio 7.16 — Un'azienda alimentare

Un'azienda produttrice di prodotti alimentari vuole organizzare un sistema informativo aziendale. Tutti gli utenti dell'applicazione devono poter visualizzare le informazioni sul **catalogo** dei **prodotti**; inoltre i **dipendenti** devono poter accedere alle informazioni sulle loro *mansioni*. I **prodotti** *sono organizzati in* **linee di prodotto** che accomunano prodotti dello stesso tipo (ad esempio pasta e sughi). I prodotti possono essere confezionati in diversi **stabilimenti**. I dipendenti si suddividono in categorie: i **manager**, *responsabili di una o più* linee di produzione (ogni linea è gestita *esattamente da tre* manager, per assicurare una gestione equa); i **supervisori** della produzione, responsabili di tutti i prodotti di una specifica linea in uno specifico stabilimento; gli **operai**, che lavorano su uno specifico prodotto in un determinato stabilimento. L'applicazione deve consentire ai manager di accedere alle informazioni sui dipendenti di cui sono responsabili, e a tutti di accedere alle informazioni sui prodotti.



- **Operaio** e **Supervisore** sono **classi associative**: un operaio è l'assegnazione di un prodotto a uno stabilimento, un supervisore quella di una linea a uno stabilimento.
- L'associazione **Responsabilità** ha molteplicità `3` dal lato di Manager: ogni linea ha esattamente tre manager.
- Il **Catalogo** aggrega i prodotti; Manager, Operaio e Supervisore **specializzano** Dipendente.
- Le responsabilità (`--`) vengono dai verbi del testo: *visualizzare*, *accedere*.

Un'associazione che manca

Il testo dice che i prodotti «sono organizzati in linee di prodotto», ma nel modello della slide non c'è un'associazione tra **Prodotto** e **Linea di Prodotto**. È il tipo di lacuna che si trova rileggendo il testo frase per frase e controllando che ognuna sia rappresentata: andrebbe aggiunta un'associazione con molteplicità `*` dal lato di Prodotto e `1` dal lato di Linea di Prodotto.

COMPLEMENTI: DIPENDENZE E INTERFACCE

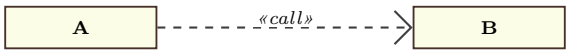
La dipendenza

Dipendenza (dependency)

Relazione tra le istanze di due classi che si realizza a **tempo di esecuzione**. C'è una dipendenza dalla classe A verso la classe B se:

- i metodi di A leggono, scrivono o modificano attributi di oggetti di B;
- i metodi di A invocano metodi di B (caso particolare: A crea oggetti di B, cioè ne invoca il costruttore).

Si disegna con una **freccia tratteggiata** dalla classe dipendente verso quella da cui dipende.



A dipende da B: una modifica a B può richiedere modifiche ad A.

- Le dipendenze si individuano soprattutto nella fase di **progetto di dettaglio**: raramente contribuiscono al modello concettuale.
- Esprimono un **accoppiamento**: la classe da cui parte la dipendenza non può essere riusata senza la classe di destinazione, e una modifica a quest'ultima implica modifiche anche alla classe dipendente.
- Allo stesso modo, la correttezza della classe dipendente è condizionata alla correttezza della classe da cui dipende.

Per distinguere i tipi di dipendenza, UML usa gli **stereotipi**, parole chiave tra doppie parentesi angolari:

Stereotipo	Significato
«call»	chiamata di un metodo
«create»	creazione di un'istanza
«use»	uso di un attributo

Esempio 7.17 — Le tre dipendenze nel codice

`A` non ha attributi di tipo `B` o `C` (non ci sono associazioni), ma dipende da entrambe:

A.java

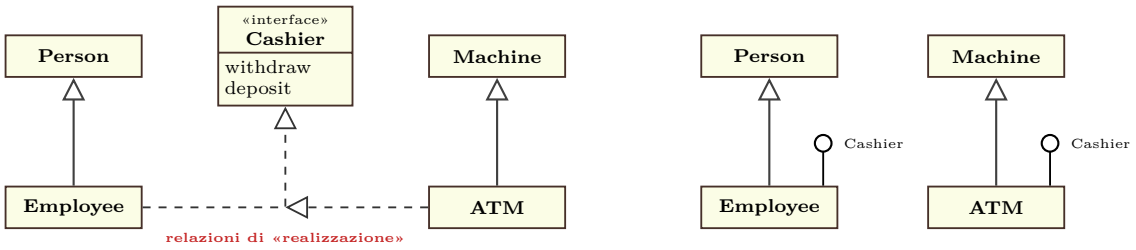
```java
public class A {
  public void esegui(B b) {
    b.calcola();              // «call»: invoca un metodo di B
    int limite = b.soglia;    // «use»: usa un attributo di B
    C registro = new C();     // «create»: crea un'istanza di C
    registro.annota(limite);
  }
}
```

Se `B` rinomina `calcola` o `soglia`, o se il costruttore di `C` cambia, `A` non compila più: è l'accoppiamento che la freccia tratteggiata rende visibile.

Le interfacce

Interfaccia

Descrive una porzione del **comportamento visibile** di un insieme di oggetti. È simile a una classe, ma non ha variabili d'istanza né metodi implementati. Una o più classi possono **realizzarla** (fornirne l'implementazione): la relazione di **realizzazione** si disegna come una generalizzazione ma con la linea tratteggiata.



A sinistra la notazione estesa: Employee e ATM, pur appartenendo a gerarchie diverse, realizzano entrambi l'interfaccia Cashier. A destra la stessa cosa con la notazione abbreviata a **lollipop**.

Esempio 7.18 — Un'interfaccia, due gerarchie

Cashier.java

```java
public interface Cashier {
  void withdraw(int importo);
  void deposit(int importo);
}

public class Employee extends Person implements Cashier {
  public void withdraw(int importo) { /* conta le banconote e le consegna */ }
  public void deposit(int importo)  { /* le riceve e le registra */ }
}

public class ATM extends Machine implements Cashier {
  public void withdraw(int importo) { /* comanda l'erogatore */ }
  public void deposit(int importo)  { /* comanda il lettore di banconote */ }
}
```

Chi usa un `Cashier` non sa, e non deve sapere, se dall'altra parte c'è una persona o una macchina:

Sportello.java

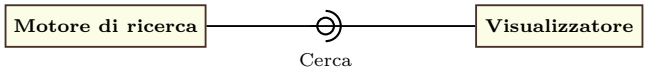
```java
public void prelievo(Cashier cassa, int importo) {
  cassa.withdraw(importo);    // funziona con un Employee e con un ATM
}
```

La notazione lollipop

- La **pallina** (*lollipop*) rappresenta l'interfaccia esposta da una classe (può coincidere con una responsabilità).
- La **linea** dalla classe concreta verso la pallina rappresenta la **realizzazione** dell'interfaccia.
- Il **semicerchio** (*socket*) rappresenta la **richiesta** dell'operazione fornita dall'interfaccia: è una dipendenza.

Esempio 7.19 — Un motore di ricerca

Un motore di ricerca offre il servizio *Cerca* tramite un'interfaccia, e la classe Visualizzatore richiama la ricerca.



Visual Paradigm supporta la notazione lollipop solo nei diagrammi dei componenti (Capitolo 9); nei diagrammi delle classi si usa la notazione equivalente con l'interfaccia esplicita:



Due classi realizzano l'interfaccia Cerca, e il Browser dipende dall'interfaccia, non da una classe concreta: si può cambiare motore di ricerca senza toccare il Browser.

In sintesi

- UML è il linguaggio grafico standard per modellare software a oggetti. Si può usare come abbozzo, come progetto o come linguaggio di programmazione; prevalgono le regole **descrittive** e vale la regola delle **informazioni soppresse**.
- I 13 diagrammi di UML 2 sono comportamentali, di interazione o strutturali; i più usati sono quello delle classi e quello di sequenza.
- Attributi: `visibilità nome molteplicità: tipo = default {proprietà}`; operazioni: `visibilità nome(parametri): tipo {proprietà}`; visibilità `+ # -`.
- Le **associazioni** hanno nome, verso di lettura, ruoli, verso di navigazione e **molteplicità**, che le implementazioni devono rispettare (anche nei costruttori). Esistono classi associative e associazioni riflessive.
- **Aggregazione** (rombo vuoto, tutto–parti) e **composizione** (rombo pieno, le parti muoiono con il tutto); **generalizzazione** (freccia vuota, is-a).
- Un modello di dominio si costruisce iterando: classi (analisi dei nomi, schede CRC), associazioni e attributi, generalizzazioni, responsabilità. Niente metodi, niente «Sistema», niente «id», nomi al singolare.
- Nel progetto di dettaglio compaiono **dipendenze** (freccia tratteggiata, con stereotipi come «call», «create», «use») e **interfacce** realizzate dalle classi (anche in notazione lollipop).

Diagrammi dei package e di sequenza

Il diagramma delle classi del Capitolo 7 descrive gli elementi del sistema uno per uno. Per sistemi reali servono altri due punti di vista: come le classi sono **raggruppate** in moduli (il diagramma dei **package**) e come gli oggetti **collaborano nel tempo** per realizzare una funzionalità (il diagramma di **sequenza**).

Il diagramma dei package

Diagramma dei package

Diagramma diverso da quello delle classi, anche se ne condivide molti aspetti ed elementi. Vi compaiono:

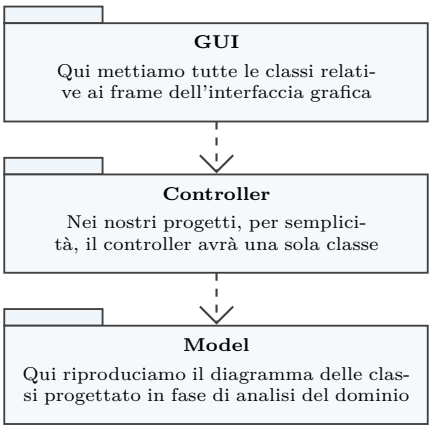
- i **package**, disegnati come cartelle con una linguetta;
- le relazioni di **dipendenza** tra i package;
- le **interfacce** e le relazioni di **realizzazione**.

Talvolta diagramma dei package e diagramma delle classi si fondono in un unico diagramma: dentro ogni package si disegna il diagramma delle classi che gli appartengono.

I diagrammi dei package si usano di solito come diagrammi di **dettaglio**, che mostrano la reale implementazione di un sistema:

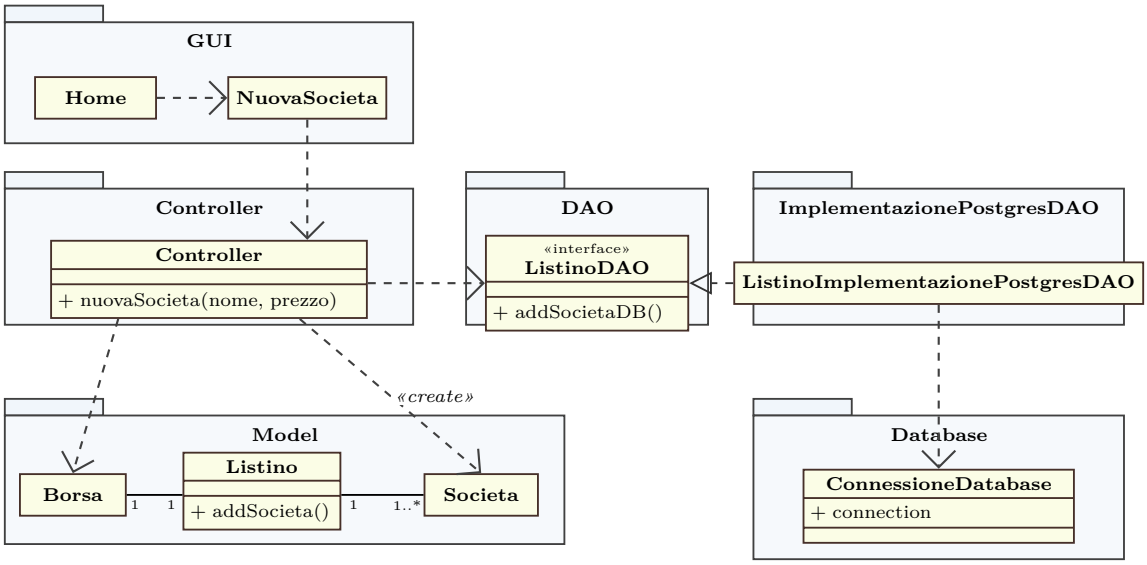
- servono quando si sceglie l'**architettura** del progetto;
- la fase successiva è il disegno dei diagrammi delle classi dell'implementazione, uno per ogni package.

Esempio 8.1 — L'architettura dei progetti del corso



L'interfaccia grafica dipende dal controller, che a sua volta dipende dal modello. Il modello non dipende da nessuno: si può cambiare l'interfaccia senza toccare le classi del dominio.

Esempio 8.2 — Package e classi insieme



- Nel package **GUI** la schermata Home apre la schermata per inserire una nuova società.
- L'unica classe del package **Controller** usa le classi del **Model** (e crea le società, «create»).
- Il Controller non parla direttamente con il database: dipende dall'**interfaccia** `ListinoDAO` (*Data Access Object*), realizzata da una classe specifica per PostgreSQL che usa la connessione al **Database**. Per passare a un altro database basta scrivere un'altra implementazione dell'interfaccia.

Esempio 8.3 — I package nel codice

In Java ogni package UML diventa un `package`, e ogni freccia di dipendenza fra package diventa una serie di `import`: leggendo gli `import` di un file si ricostruisce il diagramma, e un `import` che non dovrebbe esserci (il model che importa la GUI) è una violazione dell'architettura che si vede a colpo d'occhio.

dao/ListinoDAO.java

```java
package dao;

import model.Societa;

public interface ListinoDAO {
  void addSocietaDB(Societa societa);
}
```

controller/Controller.java

```java
package controller;

import dao.ListinoDAO;              // l'interfaccia, non l'implementazione per PostgreSQL
import model.Borsa;
import model.Societa;

public class Controller {
  private final Borsa borsa;
  private final ListinoDAO listinoDAO;

  public Controller(Borsa borsa, ListinoDAO listinoDAO) {
    this.borsa = borsa;
    this.listinoDAO = listinoDAO;
  }

  public void nuovaSocieta(String nome, double prezzo) {
    Societa societa = new Societa(nome, prezzo);   // «create»
    borsa.getListino().addSocieta(societa);
    listinoDAO.addSocietaDB(societa);
  }
}
```

implementazionePostgresDAO/ListinoImplementazionePostgresDAO.java

```java
package implementazionePostgresDAO;

import dao.ListinoDAO;
import database.ConnessioneDatabase;
import model.Societa;

public class ListinoImplementazionePostgresDAO implements ListinoDAO {
  @Override
  public void addSocietaDB(Societa societa) {
    String sql = "INSERT INTO societa (nome, prezzo) VALUES (?, ?)";
    try (PreparedStatement query = ConnessioneDatabase.getInstance().connection.prepareStatement(sql)) {
      query.setString(1, societa.getNome());
      query.setDouble(2, societa.getPrezzo());
      query.executeUpdate();
    } catch (SQLException e) {
      throw new IllegalStateException("Salvataggio non riuscito", e);
    }
  }
}
```

Il Controller non importa nulla da `implementazionePostgresDAO`: chi crea il Controller gli passa un'implementazione di `ListinoDAO`, e passare a un altro database significa scriverne un'altra.

I DIAGRAMMI DI INTERAZIONE

Diagrammi di interazione

Diagrammi che modellano gli aspetti **dinamici** di un sistema software, mettendo in evidenza le **interazioni** tra gli elementi che lo compongono (descritti nei diagrammi strutturali). Appartengono alla famiglia dei diagrammi **comportamentali**.

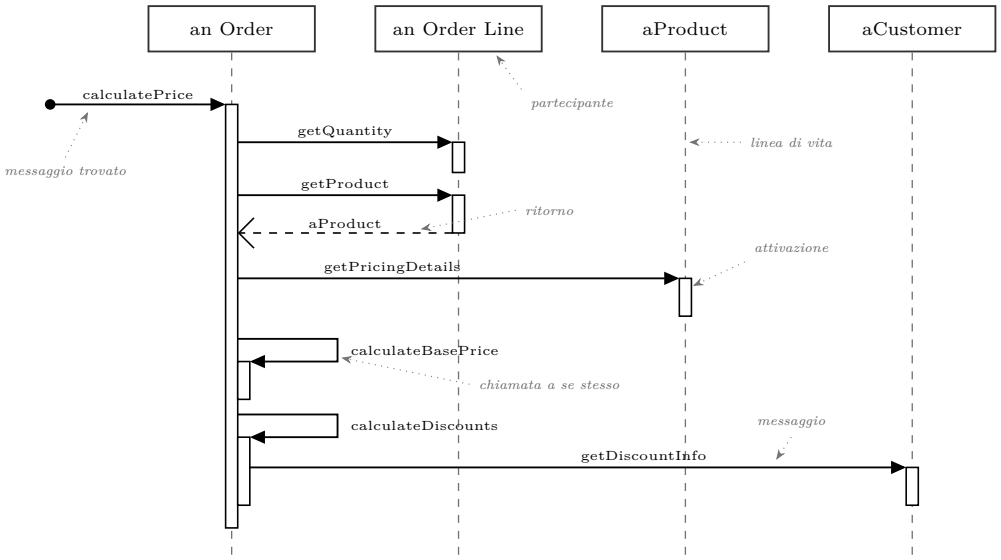
Esistono quattro tipi di diagrammi di interazione:

- i **sequence diagram** (diagrammi di sequenza), di gran lunga i più usati;
- i **communication diagram** (in UML 1 si chiamavano *collaboration diagram*);
- gli **interaction overview diagram**;
- i **timing diagram**.

Diagramma di sequenza

Modella le interazioni tra uno o più **attori** e il sistema software durante l'esecuzione di uno **scenario**. Come prevede il paradigma a oggetti, le interazioni tra attori e sistema, e tra le parti del sistema, sono rappresentate come **messaggi**. Il **tempo scorre dall'alto verso il basso**.

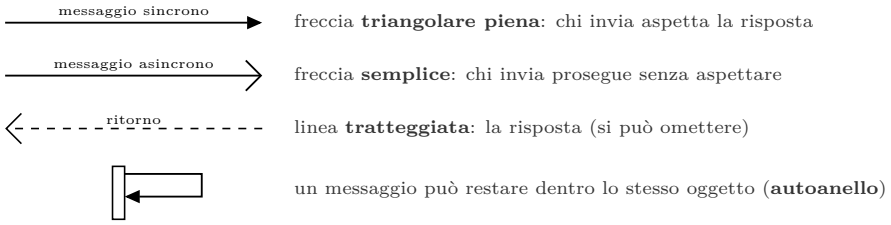
Esempio 8.4 — Il calcolo del prezzo di un ordine



L'ordine riceve la richiesta `calculatePrice` da qualcuno che non compare nel diagramma (**messaggio trovato**, con il pallino). Chiede quantità e prodotto alla riga d'ordine, chiede i dettagli del prezzo al prodotto, calcola il prezzo base e gli sconti chiamando **se stesso**, e per gli sconti interroga il cliente.

Gli elementi

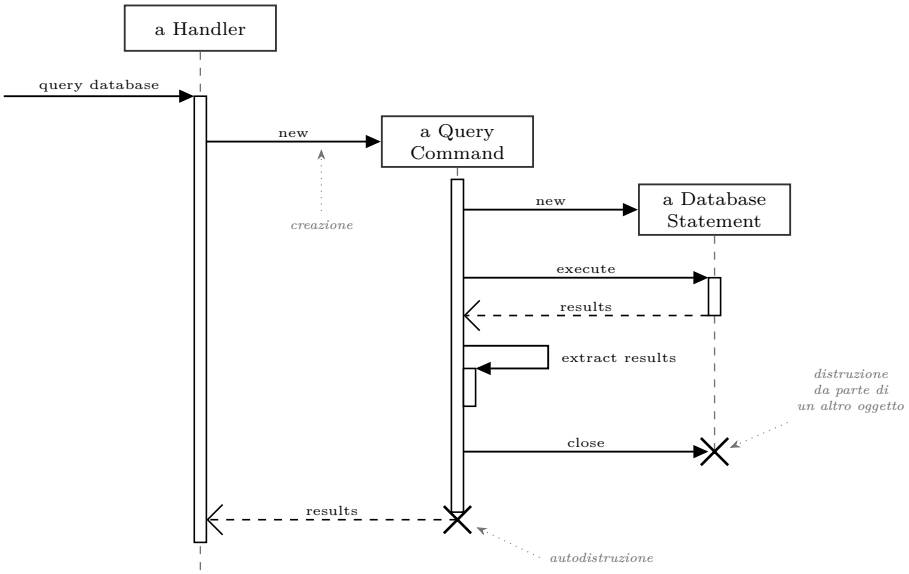
Elemento	Descrizione
Oggetti (istanze di classi)	Rettangoli con il nome della classe e l'identificatore dell'oggetto sottolineati (notazione UML 1, ad esempio `o1:Order`), oppure semplicemente un nome da cui si capisce che è un'istanza della classe (`anOrder`, `aProduct`).
Attori o endpoint	Riportati a sinistra, con frecce verso gli oggetti del sistema. Rappresentano la persona o il metodo che avvia l'elaborazione. Si possono omettere se lo scenario è avviato da un altro scenario.
Messaggi	Frecce da un attore a un oggetto o tra due oggetti. L'ordine dall'alto in basso è l'ordine in cui vengono scambiati.
Linea di vita (*lifeline*)	Linea verticale tratteggiata che parte dal rettangolo dell'oggetto: indica il periodo di vita dell'oggetto, dalla creazione alla distruzione.
Barra di attivazione	Rettangolo sulla linea di vita di un oggetto che riceve un messaggio: rappresenta il tempo necessario per elaborare la richiesta.



Creazione e distruzione degli oggetti

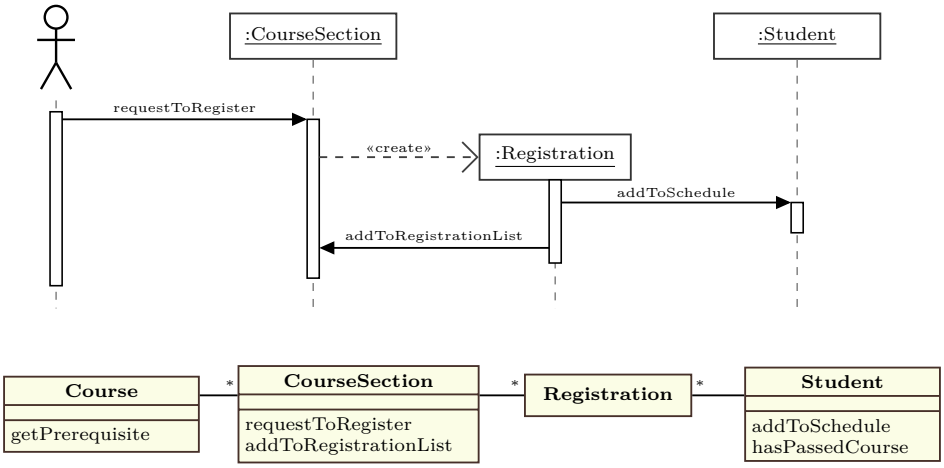
- Le linee di vita degli oggetti che esistono già (ad esempio quelli `static`) partono dalla **cima** del diagramma.
- Una chiamata al **costruttore** crea un oggetto: il rettangolo dell'oggetto si disegna alla **punta** della freccia del messaggio di creazione, più in basso degli altri.
- La **distruzione** di un oggetto si indica con una **croce** che interrompe la sua linea di vita.

Esempio 8.5 — Una query al database



Il gestore crea un comando di query, che a sua volta crea uno statement del database, lo esegue, ne estrae i risultati e lo chiude (lo statement viene distrutto dal comando). Alla fine il comando restituisce i risultati al gestore e si distrugge.

Esempio 8.6 — L'iscrizione a un corso, con il suo diagramma delle classi



Lo studente chiede alla sezione del corso di iscriversi; la sezione **crea** una Registration, che aggiunge il corso al piano di studi dello studente e poi si aggiunge alla lista delle iscrizioni della sezione. Ogni messaggio che un oggetto riceve è un'**operazione** della sua classe nel diagramma delle classi: i due diagrammi devono essere **coerenti**.

Quando si usa

In fase di **progettazione di dettaglio**, i diagrammi di sequenza descrivono la **realizzazione dei metodi**:

- gli oggetti sono istanze delle classi di progetto di dettaglio;
- i messaggi sono le chiamate di metodo sugli oggetti;
- al posto degli elementi architetturali compaiono gli oggetti delle classi che vi danno accesso (ad esempio, invece del «database», l'oggetto JDBC su cui si eseguono le query).

Non per gli algoritmi

Per descrivere **algoritmi**, i diagrammi di sequenza non sono lo strumento più adatto: di solito si preferiscono i diagrammi di attività o di stato (Capitolo 9). Un diagramma di sequenza mostra bene **chi parla con chi**, non la logica di calcolo.

Cicli e condizioni

Cicli e condizioni si indicano con un **riquadro** (*frame*) che racchiude una sottosequenza di messaggi. Nell'angolo in alto a sinistra si indica il costrutto; questa notazione è stata introdotta con UML 2.

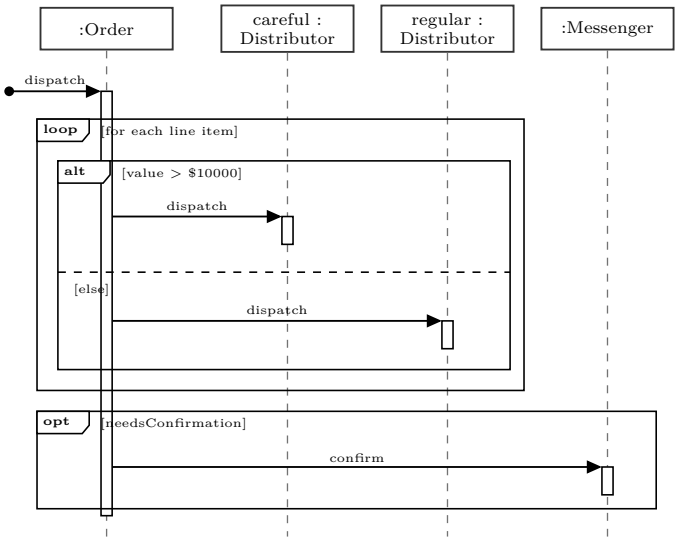
Costrutto	Corrisponde a	Uso
`loop`	`while-do`, `do-while`	Ciclo: la condizione si indica tra parentesi quadre all'inizio o alla fine.
`alt`	`if-then-else`	La condizione si indica in cima; se ci sono rami *else*, una linea tratteggiata separa la zona *then* dalla zona *else*, eventualmente con un'altra condizione accanto a `[else]`.
`opt`	`if-then`	Racchiude una sottosequenza eseguita solo se la condizione in cima è vera.

Esistono anche altri costrutti, ad esempio per il parallelismo (`par`) o le regioni critiche (`critical`).

Esempio 8.7 — La spedizione di un ordine

dispatch (pseudocodice)

```
procedure dispatch
  foreach (lineitem)
    if (product.value > 10K)
      careful.dispatch
    else
      regular.dispatch
    end if
  end for
  if (needsConfirmation)
    messenger.confirm
end procedure
```



Per ogni riga dell'ordine (**loop**), se il valore del prodotto supera i 10.000 dollari la spedizione è affidata al distributore «attento», altrimenti a quello normale (**alt**, con il ramo `[else]` sotto la linea tratteggiata). Alla fine, solo se serve una conferma (**opt**), si avvisa il messaggero.

Riquadri sì, ma con moderazione

Quando l'algoritmo da modellare si fa complesso, è buona norma usare altri tipi di diagramma. Un diagramma di sequenza pieno di riquadri annidati è difficile da leggere quanto il codice che descrive.

ESEMPIO: DAL CODICE AL DIAGRAMMA DI SEQUENZA

Immaginiamo di voler disegnare il diagramma di sequenza di una funzionalità **già implementata**. Sarà inevitabilmente un diagramma di **dettaglio**, in cui compaiono tutte le classi realmente presenti nel codice, comprese quelle dell'interfaccia utente. La funzionalità è **Aggiungi nuova società (nome, valore azione)** del videogioco di borsa del Capitolo 7, con l'architettura GUI–Controller–Model vista sopra.

1. La schermata Home. Servono due input, quindi si crea una nuova finestra e la si apre dalla Home:

```
aggiungiNuovaSocietaButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        AggiungiNuovaSocieta aggiungiNuovaSocieta = new AggiungiNuovaSocieta(controller, frame);
        aggiungiNuovaSocieta.frame.setVisible(true);
        frame.setVisible(false);
    }
});
```

2. Il pulsante OK. Alla pressione bisogna verificare che i dati siano validi (lo si farà in futuro), inserire i dati nel modello tramite il controller e tornare alla schermata precedente (*Annulla* farà solo quest'ultima operazione):

```
OKButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // aggiunge la societa' al listino
        controller.aggiungiSocietaBorsa(nomeSocietaText.getText(),
                Double.parseDouble(valoreAzioneText.getText()));
        frame.setVisible(false);
        frameChiamante.setVisible(true);
        frame.dispose();
    }
});
```

3. Controller e modello. Il controller si limita a inoltrare la richiesta all'oggetto `borsa`, che ha una lista di società inizialmente vuota; `aggiungiSocieta` crea una nuova società e la aggiunge alla lista.

```
// nel Controller
public void aggiungiSocietaBorsa(String text, double parseDouble) {
    borsa.aggiungiSocieta(text, parseDouble);
}

public class Borsa {
    private ArrayList<Societa> societa;
    public Borsa() { societa = new ArrayList<Societa>(); }

    public void aggiungiSocieta(String text, double parseDouble) {
        Societa s = new Societa(text, parseDouble);
        societa.add(s);
    }
}

public class Societa {
    private Double valoreAzione;
    private String nome;
    public Societa(String n, Double valore) { nome = n; valoreAzione = valore; }
}
```

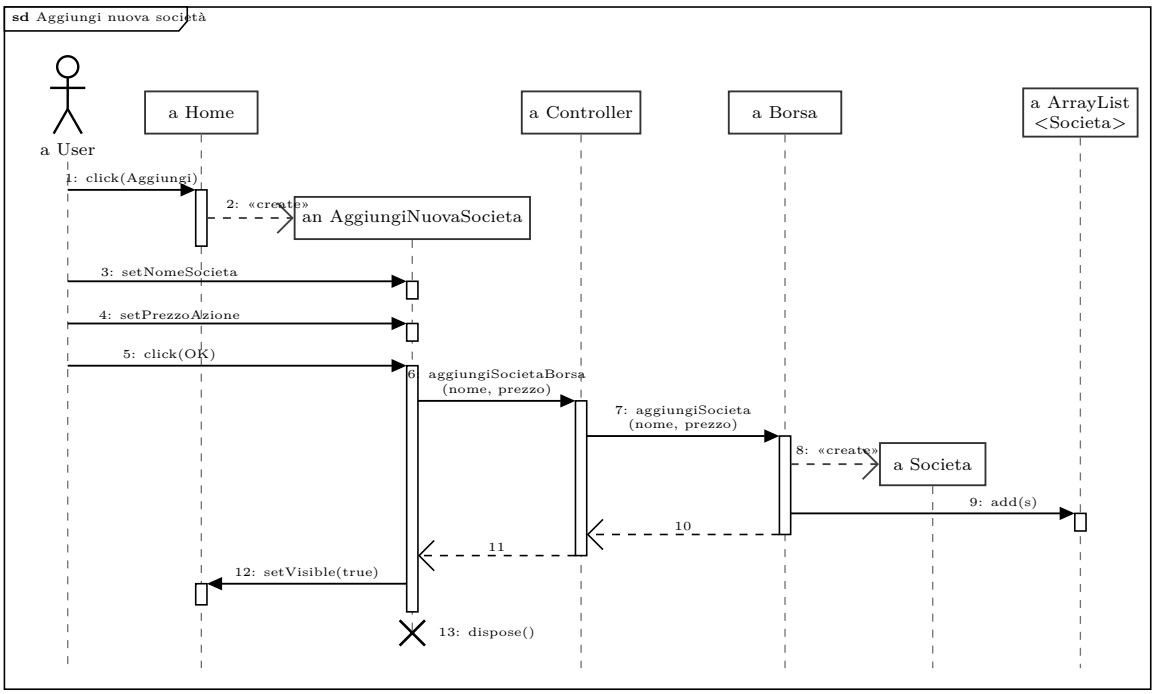


Figura 8.1: Il diagramma di sequenza di «Aggiungi nuova società», ricavato dal codice.

Si notino la **creazione** della finestra (2) e della società (8), disegnate alla punta della freccia; i **ritorni** tratteggiati (10 e 11) quando i metodi terminano; e la **distruzione** della finestra, che si chiude da sola con `dispose()`. L'oggetto `ArrayList` esiste già, perché è stato creato dal costruttore di `Borsa`.

Un nome, tre versioni

Nelle slide lo stesso metodo del controller si chiama `aggiungiSocietaBorsa` nel codice, `aggiungiSocietaListino` nel diagramma di sequenza e `nuovaSocieta` nel diagramma dei package. Qui si usa il nome del codice. Mantenere **coerenti** codice e diagrammi è esattamente il lavoro che, nei progetti reali, gli strumenti di co-generazione (come il plug-in *Diagrams* di IntelliJ) aiutano a fare.

In sintesi

- Il **diagramma dei package** mostra i package (cartelle), le dipendenze tra loro e le interfacce; si usa per scegliere l'architettura, ad esempio GUI–Controller–Model con i DAO per l'accesso ai dati. Può contenere i diagrammi delle classi.
- I **diagrammi di interazione** sono quattro: di sequenza, di comunicazione, interaction overview e timing.
- Il **diagramma di sequenza** mostra i **messaggi** scambiati da attori e oggetti in uno scenario, dall'alto verso il basso: oggetti, linee di vita, barre di attivazione, messaggi sincroni (punta piena), asincroni (punta semplice) e di ritorno (tratteggiati), chiamate a se stessi, creazione e distruzione (croce).
- I riquadri **loop**, **alt** e **opt** (UML 2) rappresentano cicli e condizioni, ma per gli algoritmi complessi sono meglio i diagrammi di attività o di stato.
- Nel progetto di dettaglio i messaggi sono chiamate di metodo, e il diagramma di sequenza deve essere **coerente** con il diagramma delle classi e con il codice.

Altri diagrammi UML

Nei Capitoli 7 e 8 abbiamo visto i diagrammi UML più usati: quello delle classi, quello dei package e quello di sequenza. Completiamo il quadro con altri quattro diagrammi, che guardano il sistema da punti di vista diversi. Nel Capitolo 7 abbiamo diviso i diagrammi UML in tre categorie; poiché i diagrammi di interazione sono un sottogruppo di quelli comportamentali, si possono raggruppare in due grandi famiglie.

Diagrammi statici (strutturali)

Descrivono **com'è fatto** il sistema: le sue parti e come sono collegate fra loro.

- diagramma delle classi (Capitolo 7)
- diagramma dei package (Capitolo 8)
- **diagramma dei componenti**
- **diagramma di deployment**

Diagrammi comportamentali (dinamici)

Descrivono **cosa fa** il sistema nel tempo, in risposta agli eventi.

- diagramma dei casi d'uso (Capitolo 6)
- diagramma di sequenza (Capitolo 8)
- **diagramma di stato** (*statechart*)
- **diagramma di attività**

Figura 9.1: Le due famiglie di diagrammi UML. In grassetto quelli trattati in questo capitolo.

In questo capitolo vediamo quattro diagrammi:

- due **statici**, che descrivono l'architettura: il diagramma dei **componenti** e quello di **deployment**;
- due **comportamentali**: il diagramma di **stato** e quello di **attività**.

Il diagramma dei componenti

Componente

Un **componente** è un contenitore di altri elementi statici di UML, come classi e package. Può contenere a sua volta altri componenti: un componente che ne contiene altri si chiama spesso **sottosistema**.

Che cosa sia concretamente un componente dipende dal contesto:

- spesso è un **elemento fisico**: un file di codice sorgente, un file eseguibile, un database, …;
- in altri casi coincide con un **package**, come nelle librerie Java «impacchettate» in un unico file `.jar`.



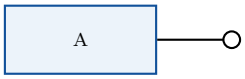
Un componente si disegna come un rettangolo con lo stereotipo «component» e, nell'angolo in alto a destra, l'icona del componente.

Diagramma dei componenti

Mostra un insieme di **componenti** e le loro **relazioni**. Per ogni componente può mostrare esplicitamente le **interfacce** che espone verso l'esterno. È un diagramma **architetturale**: mostra gli oggetti effettivamente realizzati, ma **non** dice nulla sulla loro collocazione fisica (su quale macchina girano).

Interfacce e porte

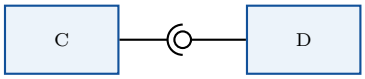
I componenti comunicano attraverso **interfacce**, che si disegnano con due simboli complementari.



Interfaccia fornita (*lollipop*, «lecca-lecca»): A la implementa e la offre agli altri.



Interfaccia richiesta (*socket*, «presa»): B ne ha bisogno per funzionare.

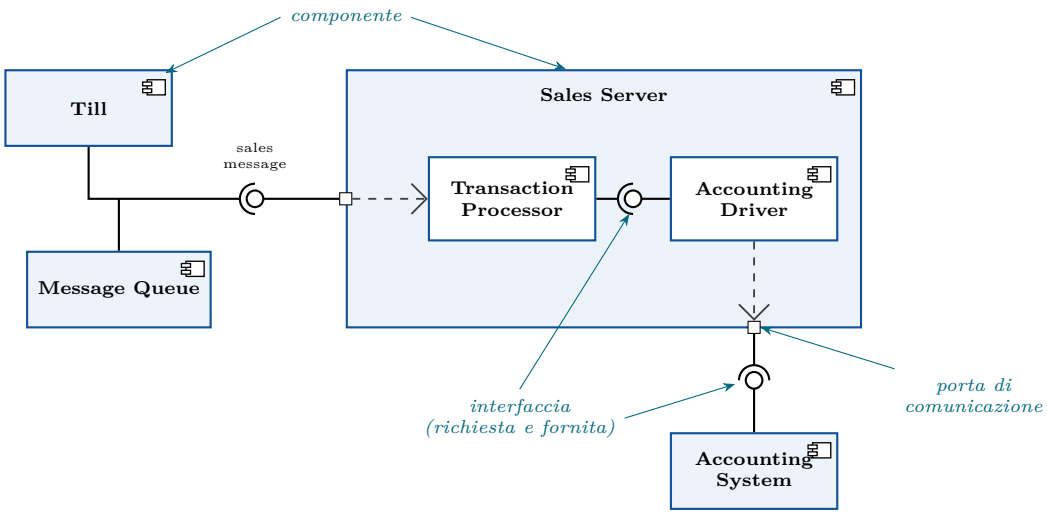


Collegamento: la pallina di D entra nella presa di C. D fornisce ciò che C richiede.

Porta di comunicazione

Piccolo quadrato sul bordo di un componente. Esprime la necessità di **trasferire dati** tra componenti, eventualmente con operazioni di trasformazione, serializzazione o deserializzazione.

Esempio 9.1 — Un server delle vendite



Il **Sales Server** contiene due componenti. Il **Transaction Processor** richiede un'interfaccia che l'**Accounting Driver** fornisce. I messaggi di vendita (*sales message*) arrivano dalla cassa (**Till**) attraverso una coda di messaggi ed entrano nel server da una porta, che li inoltra (freccia tratteggiata) al Transaction Processor. Allo stesso modo l'Accounting Driver comunica con il sistema di contabilità esterno (**Accounting System**) attraverso una porta.

Esempio 9.2 — Interfacce fornite e richieste in Java

La pallina e la presa fra Transaction Processor e Accounting Driver si traducono in un'interfaccia Java: chi la **implementa** la fornisce, chi la riceve nel costruttore la richiede.

Contabilita.java

```java
public interface Contabilita {                              // l'interfaccia fra i due componenti
  void registraVendita(Vendita vendita);
}

public class AccountingDriver implements Contabilita {      // pallina: la fornisce
  @Override
  public void registraVendita(Vendita vendita) {
    sistemaContabile.invia(vendita.comeRecord());            // verso la porta del sistema esterno
  }
}

public class TransactionProcessor {
  private final Contabilita contabilita;                     // presa: la richiede

  public TransactionProcessor(Contabilita contabilita) {
    this.contabilita = contabilita;
  }

  public void elabora(Vendita vendita) {
    // ... controlli, sconti, scontrino ...
    contabilita.registraVendita(vendita);
  }
}
```

Il collegamento pallina-presa avviene quando si assemblano i componenti, ed è l'unico punto che conosce entrambi:

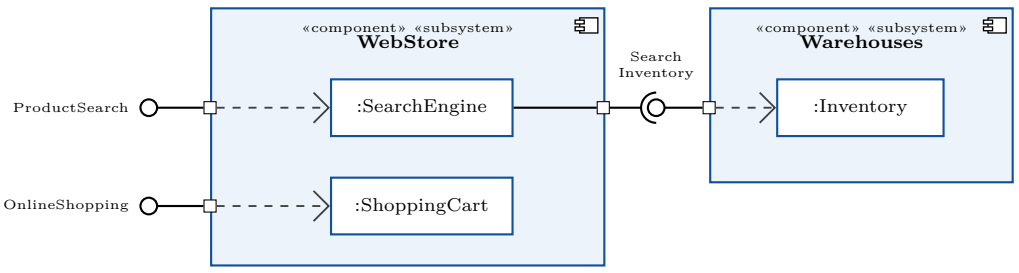
SalesServer.java

```java
TransactionProcessor processore = new TransactionProcessor(new AccountingDriver());
```

Sottosistemi

Un componente che contiene altri componenti è un **sottosistema** (stereotipo «subsystem»). Le porte sul suo bordo collegano le interfacce esterne ai componenti interni che le realizzano: chi usa il sottosistema vede solo le porte, non come è organizzato dentro.

Esempio 9.3 — Un negozio online diviso in sottosistemi (semplificato)



Dall'esterno il **WebStore** offre due interfacce, *ProductSearch* e *OnlineShopping*, e le porte le inoltrano ai componenti interni. Per cercare i prodotti, il SearchEngine usa l'interfaccia *Search Inventory* fornita dal sottosistema **Warehouses**. Nelle slide lo stesso negozio ha anche un sottosistema Accounting (ordini e clienti) e un componente per l'autenticazione.

Il diagramma di deployment

Diagramma di deployment

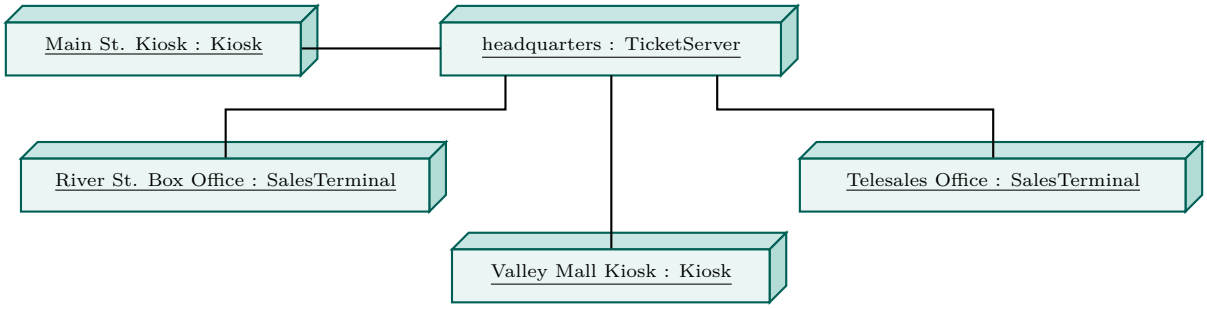
Mostra l'**architettura fisica** del sistema: su quali macchine e dispositivi gira il software e come sono collegati.

I suoi elementi sono:

- il **nodo**, elemento fondamentale: corrisponde a un elemento **fisico** del sistema, come una macchina o un dispositivo. Si disegna come un **parallelepipedo**;
- i **componenti**, che si possono disegnare **dentro** i nodi: così il diagramma di deployment si collega ai diagrammi dei componenti corrispondenti;
- gli **artefatti**, che rappresentano i prodotti dell'elaborazione: un artefatto è qualunque oggetto fisico prodotto dal sistema (un file, un database, …);
- gli **archi**, che corrispondono alle **connessioni fisiche** tra i nodi (spesso etichettati con il protocollo, ad esempio TCP/IP).

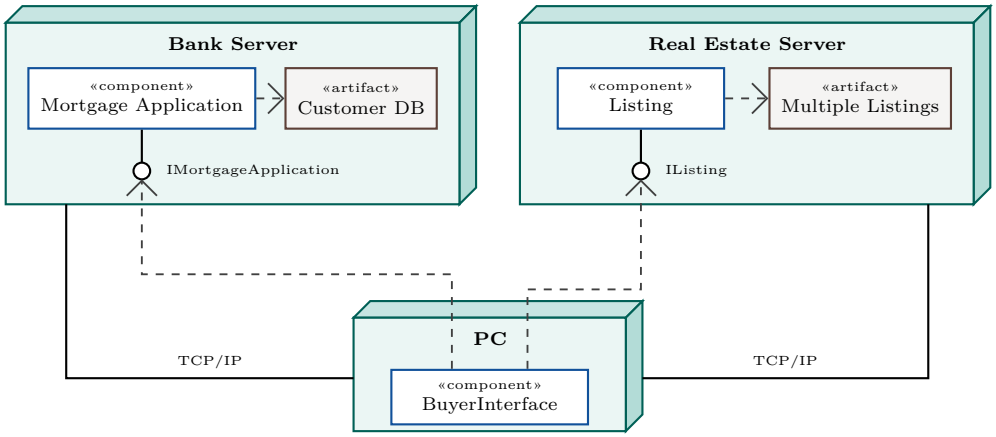
Esempio 9.4 — I nodi di un sistema di vendita di biglietti

Ogni nodo è un'**istanza**: a sinistra dei due punti c'è il nome dell'istanza, a destra il tipo di nodo. Come per gli oggetti UML, il nome è sottolineato.



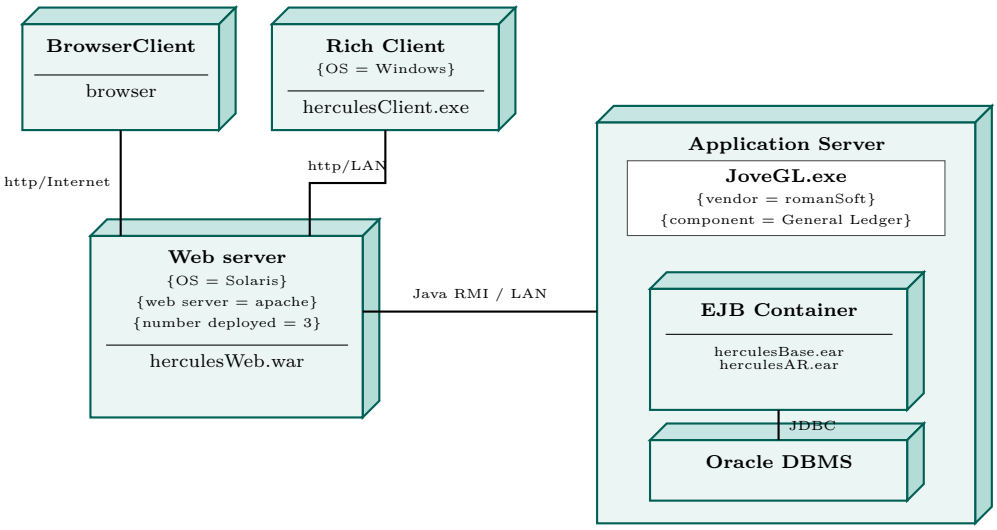
Un server centrale (*headquarters*) è collegato a due chioschi e a due terminali di vendita. Kiosk e SalesTerminal sono tipi di nodo; ogni parallelepipedo è una macchina concreta.

Esempio 9.5 — Nodi che contengono componenti e artefatti



Il PC dell'acquirente ospita la sua interfaccia (*BuyerInterface*), che usa le interfacce offerte dai componenti dei due server, collegati al PC via TCP/IP. Ogni componente dei server lavora su un proprio artefatto: un database dei clienti e un archivio degli annunci immobiliari.

Esempio 9.6 — Valori etichettati e ambienti di esecuzione



Questo esempio introduce due elementi nuovi.

- I **valori etichettati** (*tagged values*), tra parentesi graffe nella forma `{nome = valore}`, aggiungono informazioni su un nodo o un artefatto: il sistema operativo, il software installato, quante copie del nodo esistono (`number deployed = 3`: i web server sono tre).
- Un nodo può contenerne altri. **EJB Container** e **Oracle DBMS** non sono macchine ma **ambienti di esecuzione** (*execution environment*) software, che girano dentro la macchina Application Server e ospitano a loro volta gli artefatti (i file `.ear`).

Le connessioni sono etichettate con il protocollo: i client parlano HTTP con il web server, che a sua volta usa Java RMI per raggiungere l'application server; il container accede al database via JDBC.

Quando si disegnano

I diagrammi di deployment sono diagrammi **architetturali**: si usano nella fase di **progettazione dell'architettura** e in quella di **implementazione**. A volte però servono già in **fase di analisi**, quando il problema stesso impone dei vincoli architetturali. Ad esempio, se il sistema deve ricevere dati da un satellite GPS via rete wireless, questo vincolo fa parte del problema, non della soluzione.

Componenti o deployment?

Il diagramma dei **componenti** dice **che cosa** è stato costruito e come i pezzi si collegano *logicamente* (chi offre quale interfaccia a chi). Il diagramma di **deployment** dice **dove** quei pezzi girano *fisicamente* (su quale macchina, con quale collegamento). Lo stesso componente può comparire in entrambi.

Il diagramma di stato (statechart)

Diagramma di stato

Un **diagramma di stato** (*statechart*) descrive il comportamento di un sistema, di una parte di un sistema o di un singolo oggetto. È un **grafo orientato** in cui i nodi sono gli **stati** e gli archi sono le **transizioni**.

L'idea di fondo è questa:

- in ogni momento il sistema (o l'oggetto) si trova in un certo **stato**;
- trovarsi in uno stato significa **reagire in un modo specifico** agli eventi che si verificano;
- alcuni eventi fanno **cambiare stato** al sistema;
- nel nuovo stato, il sistema reagisce agli eventi **in modo diverso**.

Regioni, vertici e transizioni

Un diagramma di stato UML contiene una **regione** di primo livello. Una regione contiene **vertici** e **transizioni**: i vertici rappresentano gli stati (ne esistono di diversi tipi, con significati diversi), le transizioni sono gli archi orientati tra due vertici.

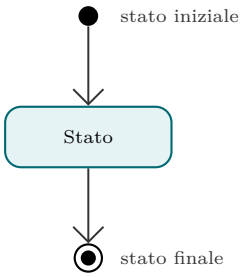
Gli stati

Uno **stato** si disegna come un **rettangolo con gli angoli arrotondati** che contiene il suo nome.

- In ogni momento è attivo **uno e un solo** stato del diagramma (più precisamente, uno per regione: vedremo più avanti le regioni parallele).
- Gli stati sono **stabili**: il sistema resta in uno stato finché un evento non provoca una transizione verso un altro.

Esistono poi due **stati speciali** (pseudostati):

- lo **stato iniziale**, un pallino pieno: non può essere raggiunto (non ha frecce entranti);
- lo **stato finale**, un pallino cerchiato: non può essere lasciato (non ha frecce uscenti).



Le transizioni

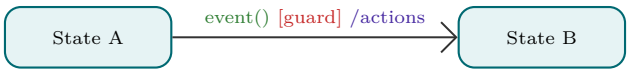
Transizione

Arco orientato che indica un **cambiamento di stato**. Può avere un'etichetta della forma

trigger [guardia] / azioni

- i **trigger** sono gli **eventi** che possono provocare il cambiamento di stato;
- la **guardia** è una **condizione booleana**;
- le **azioni** sono le operazioni da eseguire quando la transizione scatta.

Tutte e tre le parti sono **facoltative**. Sono ammesse anche le **autotransizioni**, che partono da uno stato e vi ritornano.



La semantica. Quando scatta una transizione?

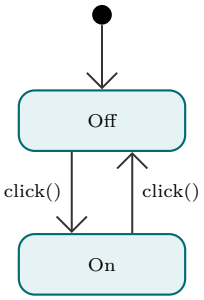
- Una transizione è **attivabile** (*fireable*) quando si sono verificati gli eventi corrispondenti a **tutti** i suoi trigger **e** la guardia vale **vero**.
- Una transizione **spontanea** non ha né trigger né guardia: scatta senza bisogno di eventi.
- Dopo che una transizione è scattata, si eseguono le sue azioni.
- Se sono attivabili **più** transizioni, ne scatta **una sola**, scelta in modo **non deterministico**.

Il non determinismo è quasi sempre un errore

Se da uno stato escono due transizioni con lo stesso evento, le loro guardie dovrebbero essere **mutuamente esclusive** (ad esempio `[credito < 2]` e `[credito >= 2]`). Altrimenti il comportamento del sistema dipende dal caso, e quasi mai è ciò che il progettista voleva.

Tre esempi

Esempio 9.7 — Una lampada con un solo pulsante



Lo stesso evento `click()` ha effetti opposti a seconda dello stato in cui arriva: è esattamente ciò che un diagramma di stato sa esprimere.

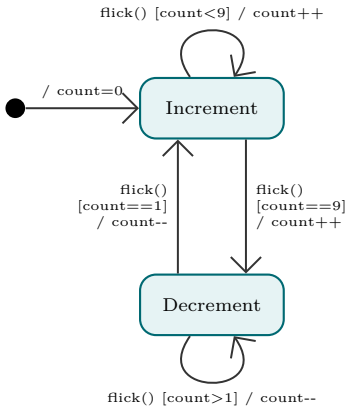
Esempio 9.8 — Un contatore, dal codice Java al diagramma

Il metodo `flick()` fa salire il contatore fino a 10, poi lo fa scendere fino a 0, poi di nuovo salire, e così via.

```java
public class Counter {
  private int count = 0;
  private String mode = "increment";

  public void flick() {
    if (mode.equals("increment"))
      count++;
    else
      count--;

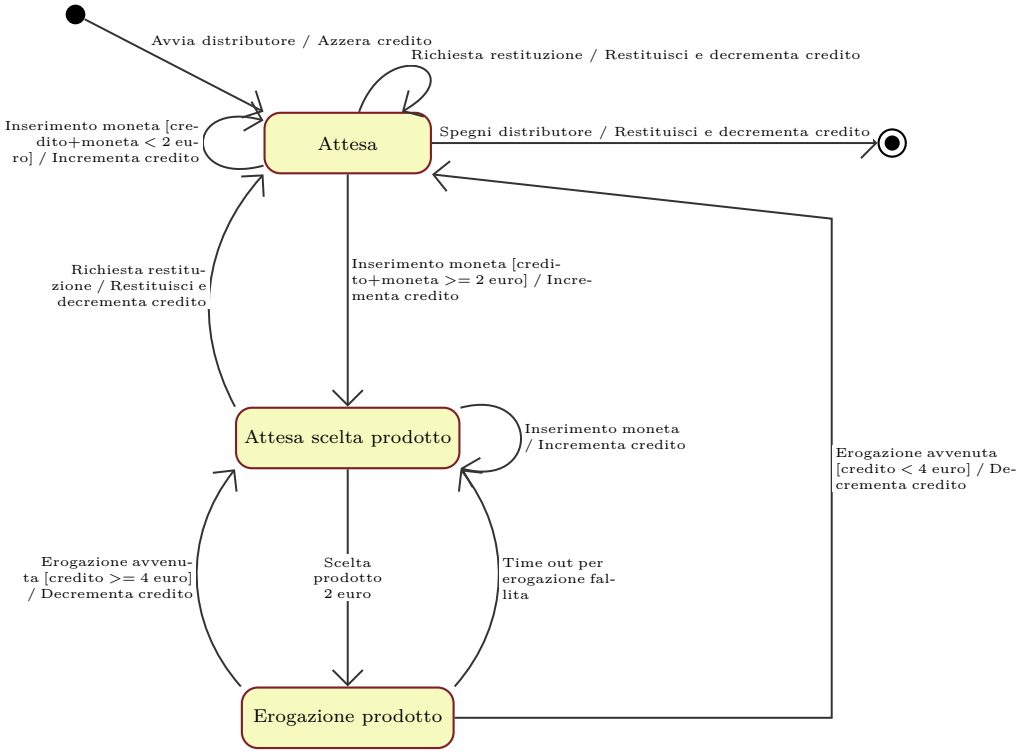
    if (count == 10)
      mode = "decrement";
    else if (count == 0)
      mode = "increment";
  }
}
```



La variabile `mode` è proprio lo **stato**: i suoi due valori diventano i due stati del diagramma. La variabile `count`, invece, non diventa uno stato (servirebbero 11 stati!): compare nelle **guardie** e nelle **azioni**. Si noti che le guardie guardano il valore di `count` **prima** dell'azione: in Increment, con `count==9` lo `flick()` porta il contatore a 10 e fa passare in Decrement.

Esempio 9.9 — Un distributore automatico

Il distributore eroga prodotti che costano 2 euro e accetta diversi tipi di monete.



Leggiamolo seguendo un cliente.

- All'avvio il credito viene azzerato e si va in **Attesa**. Finché il credito resta sotto i 2 euro, ogni moneta lo incrementa e si resta in Attesa; la moneta che porta il credito ad almeno 2 euro fa passare in **Attesa scelta prodotto**. Le due guardie sono complementari, quindi non c'è non determinismo.
- In Attesa scelta prodotto si possono aggiungere altre monete, chiedere la restituzione (e tornare in Attesa) oppure scegliere il prodotto, che fa passare in **Erogazione prodotto**.
- Se l'erogazione va a buon fine il credito scende di 2 euro. Se **prima** dell'erogazione il credito era di almeno 4 euro, dopo ne restano almeno 2 e si torna in Attesa scelta prodotto: il cliente può scegliere un altro prodotto. Altrimenti si torna in Attesa. Se l'erogazione fallisce (*time out*) si torna alla scelta del prodotto senza scalare nulla.

Dal diagramma al codice

Un diagramma di stato a livello di dettaglio si può implementare con una catena di **`switch` annidati**: il primo sceglie lo stato corrente, il secondo l'evento arrivato.

```
switch (statoCorrente) {
  case STATO_1:
    switch (evento) {
      case EVENTO_1:
        if (guardia1) {
          azioneEvento1();
          statoCorrente = STATO_2;
        }
        break;
      // ... altri eventi
    }
    break;
  case STATO_2:
    // ...
}
```

Funziona, ma con molti stati ed eventi lo `switch` diventa enorme e ogni modifica rischia di rompere qualcosa altrove. Nei sistemi orientati agli oggetti c'è una soluzione migliore.

Pattern State

Il diagramma di stato si traduce in:

- una **classe generale** degli stati, che dichiara un metodo per **ogni evento** ma lo implementa **senza fare nulla**;
- una serie di **classi derivate**, una per ogni stato del diagramma, che ridefiniscono (*override*) **solo** i metodi degli eventi che hanno un effetto in quello stato;
- una classe **contenitore** (il *contesto*), che ricorda lo stato corrente e **delega** ogni evento a lui.

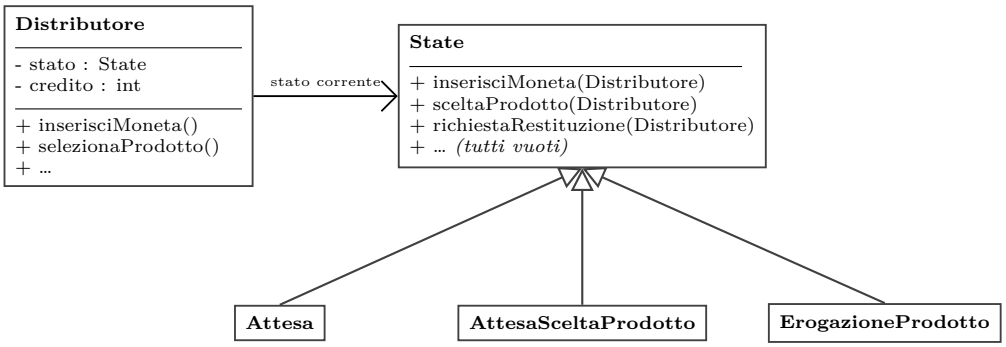


Figura 9.2: Il pattern State applicato al distributore: una sottoclasse di State per ogni stato del diagramma.

Il contesto tiene lo stato corrente e il credito, e inoltra ogni evento allo stato corrente passandogli se stesso:

```java
public class Distributore {
    private State stato;
    static State attesa = new Attesa();
    static State attesaSceltaProdotto = new AttesaSceltaProdotto();
    static State erogazioneProdotto = new ErogazioneProdotto();
    private int credito = 0;

    public Distributore() { setState(attesa); }
    void setState(State nuovoStato) { this.stato = nuovoStato; }
    void setCredito(int credito) { this.credito = credito; }
    int getCredito() { return credito; }

    // ogni evento viene delegato allo stato corrente
    public void inserisciMoneta()   { stato.inserisciMoneta(this); }
    public void selezionaProdotto() { stato.sceltaProdotto(this); }
    public void richiestaRestituzioneCredito() { stato.richiestaRestituzione(this); }
    public void spegniDistributore() { stato.spegniDistributore(this); }
    public void prelevaProdotto()    { stato.prelevaProdotto(this); }
    public void timeoutErogazione()  { stato.timeOutErogazione(this); }
}
```

La classe generale degli stati non fa nulla: in uno stato, un evento non previsto viene semplicemente ignorato.

```java
public class State {
    public void inserisciMoneta(Distributore distributore) {}
    public void richiestaRestituzione(Distributore distributore) {}
    public void sceltaProdotto(Distributore distributore) {}
    public void spegniDistributore(Distributore distributore) {}
    public void prelevaProdotto(Distributore distributore) {}
    public void timeOutErogazione(Distributore distributore) {}
}
```

Ogni stato ridefinisce solo gli eventi che lo riguardano. Ecco lo stato **Attesa**:

```java
public class Attesa extends State {
    @Override
    public void inserisciMoneta(Distributore distributore) {
        distributore.setCredito(distributore.getCredito() + 1);
        System.out.println("Credito attuale: " + distributore.getCredito());
        if (distributore.getCredito() >= 2) {       // guardia
            distributore.setState(Distributore.attesaSceltaProdotto);  // transizione
        }
    }
    @Override
    public void richiestaRestituzione(Distributore distributore) {
        System.out.println("Ritira il tuo credito: " + distributore.getCredito());
        distributore.setCredito(0);
    }
    @Override
    public void sceltaProdotto(Distributore distributore) {
        System.out.println("Non hai credito sufficiente");
    }
    @Override
    public void spegniDistributore(Distributore distributore) {
        System.out.println("Ritira il tuo credito: " + distributore.getCredito());
        distributore.setCredito(0);
        System.out.println("Spengo il distributore");
        System.exit(0);
    }
    @Override
    public void prelevaProdotto(Distributore distributore) {
        System.out.println("Il prodotto non e' pronto");
    }
}
```

Nel diagramma	Nel codice (pattern State)
Stato	Una sottoclasse di `State` (`Attesa`, …)
Evento (trigger)	Un metodo di `State` (`inserisciMoneta`, …)
Guardia	Un `if` dentro il metodo
Azione	Il corpo del metodo (`setCredito`, `println`, …)
Transizione	La chiamata `setState(…)` verso il nuovo stato
Evento ignorato in uno stato	Metodo non ridefinito: resta quello vuoto di `State`

Tabella 9.1: Corrispondenza tra diagramma di stato e codice.

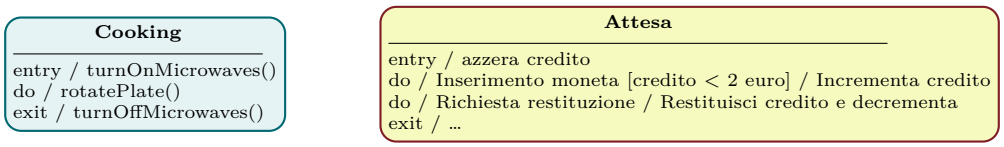
Perché è meglio dello switch

È il **polimorfismo** ad adattare il comportamento allo stato corrente: `stato.inserisciMoneta(this)` esegue codice diverso a seconda di quale oggetto è in `stato`. Il codice di ogni stato è raccolto in una sola classe, e aggiungere uno stato significa aggiungere una classe, senza toccare un unico `switch` gigante. Il codice completo dell'esempio è su Teams.

Attività interne

Uno stato può contenere, facoltativamente, una lista di **attività interne**. Ogni attività ha un'etichetta che dice **quando** va eseguita; tre etichette sono riservate:

- **entry /** attività eseguita all'**ingresso** nello stato;
- **do /** attività eseguita **finché** il sistema resta nello stato (dopo che le attività di entry sono terminate);
- **exit /** attività eseguita all'**uscita** dallo stato.



Il forno a microonde accende le microonde quando inizia a cuocere, fa girare il piatto per tutta la cottura e spegne le microonde quando smette, qualunque sia il motivo per cui smette.

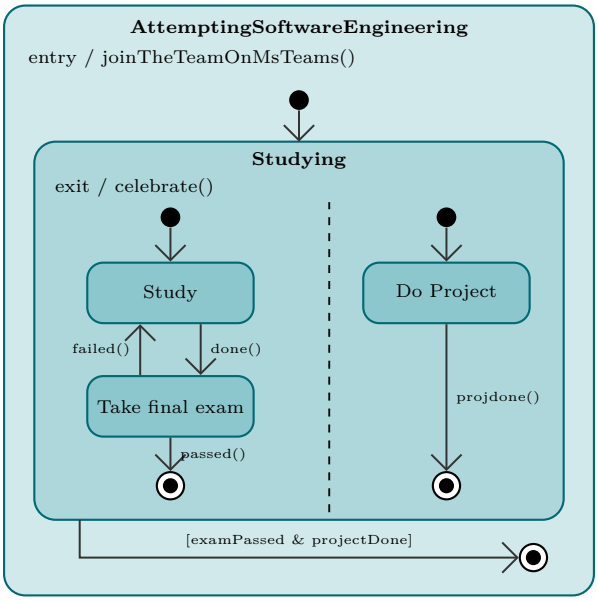
Autotransizione o attività interna?

Le due cose sembrano equivalenti, ma non lo sono. Un'**autotransizione** (una freccia che esce dallo stato e vi rientra) **esce** dallo stato e **rientra**: esegue quindi le attività di **exit** e poi di **entry**. Una reazione **interna**, scritta dentro lo stato, non lo lascia mai. Nello stato Attesa qui sopra la differenza è decisiva: se l'inserimento di una moneta fosse un'autotransizione, ogni moneta farebbe rieseguire `entry / azzera credito` e il credito tornerebbe a zero a ogni moneta!
Nella notazione UML standard le reazioni agli eventi dentro uno stato si scrivono `evento [guardia] / azione`, senza `do`, che resta riservato a un'attività continua come `rotatePlate()`; la slide del distributore usa `do` anche per queste reazioni.

Stati compositi e regioni parallele

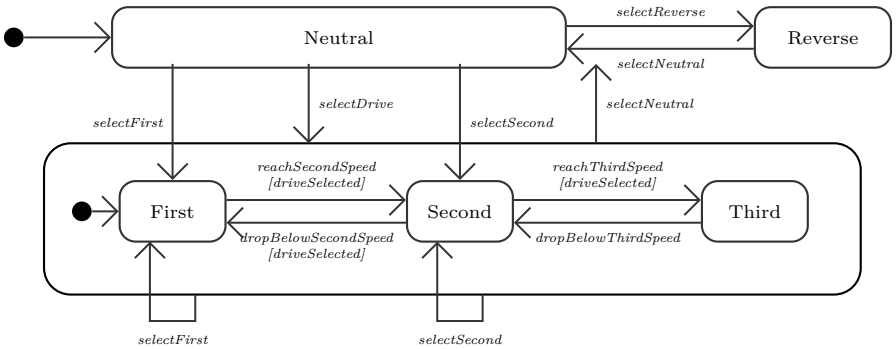
- Gli **stati compositi** contengono, al loro interno, un intero diagramma di stato.
- Uno stato composito può contenere **più regioni**, separate da una linea tratteggiata, che rappresentano comportamenti che si svolgono **in parallelo**: in ogni regione è attivo uno stato.
- Quando si esce da uno stato composito, **tutte** le sue regioni vengono terminate.
- Analogamente esistono gli **stati di attività** (*activity states*), che contengono un diagramma di attività.

Esempio 9.10 — Superare Ingegneria del Software



Entrando nello stato composito ci si iscrive al team su MS Teams (`entry`) e si entra in **Studying**. Qui due regioni procedono **in parallelo**: da una parte si studia e si sostiene l'esame (se lo si fallisce, si torna a studiare), dall'altra si fa il progetto. Quando esame e progetto sono entrambi superati si esce da Studying, eseguendo `exit / celebrate()`.

Esempio 9.11 — Il cambio automatico di un'auto



Lo stato composito (senza nome) raccoglie le marce avanti. Le transizioni che partono dal suo **bordo** valgono per **tutti** i suoi sottostati:

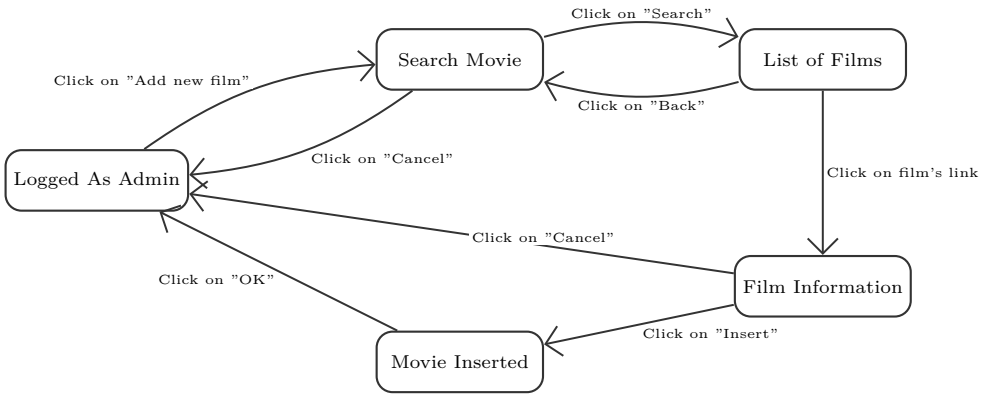
- `selectNeutral` riporta in folle da qualunque marcia: basta una freccia invece di tre;
- `selectFirst` e `selectSecond`, partendo dal bordo, portano in prima o in seconda da qualunque marcia.

Una transizione che arriva sul bordo (`selectDrive`) entra dallo stato iniziale interno, cioè in First; una transizione che punta direttamente a un sottostato (`selectSecond` da Neutral) entra proprio in quel sottostato.

Diagrammi di stato e interfacce grafiche

I diagrammi di stato sono particolarmente utili per modellare i sistemi **guidati dagli eventi** (*event-driven*). Un caso speciale molto importante sono le **interfacce grafiche** (GUI): uno stato corrisponde a una **finestra** (o schermata), un evento trigger a un'azione dell'utente (un tasto premuto, un clic del mouse, …).

Esempio 9.12 — Una GUI per gestire un archivio di film (percorso principale)



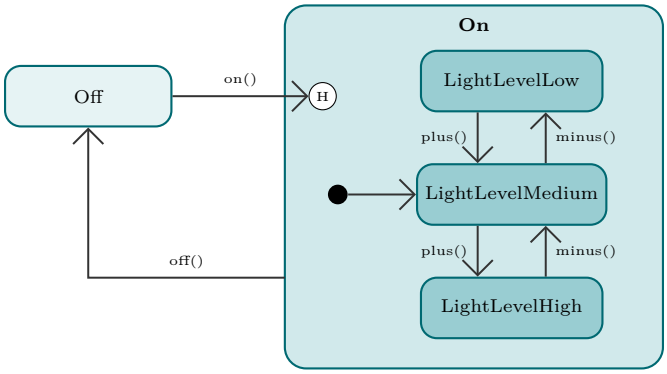
Le slide mostrano un modello più completo, con altre schermate combinate (ad esempio la scheda del film insieme alla lista dei film); qui è riportato solo il percorso per inserire un nuovo film. Un modello così permette, ad esempio, di verificare che da ogni schermata si possa tornare indietro, o di generare i casi di test dell'interfaccia percorrendo le transizioni.

Lo pseudostato history

Pseudostato history (shallow)

Si disegna come una **H** cerchiata. Rappresenta lo **stato più recentemente attivo** all'interno di uno stato composito: entrare nel composito attraverso la H significa tornare nel sottostato in cui ci si trovava quando se ne è usciti. Ricorda però solo il sottostato di primo livello, **non** i sottostati di quel sottostato (per questo è detto *shallow*, superficiale). Può comparire solo negli stati compositi, e al più una volta per regione.

Esempio 9.13 — Una lampada con tre livelli di luce

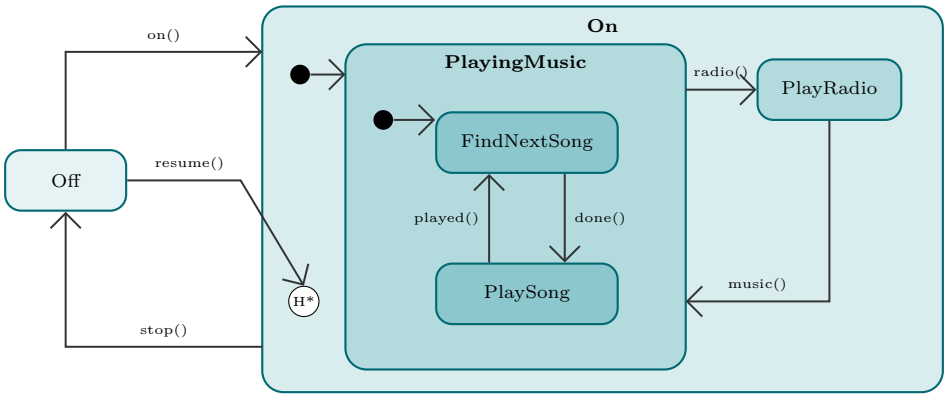


La prima volta che si accende la lampada non c'è nessuna storia da ricordare, e si parte dallo stato iniziale interno: **LightLevelMedium**. Da acceso, `off()` spegne la lampada da qualunque livello (la transizione parte dal bordo). Alla riaccensione, `on()` entra attraverso la **H**, e la lampada torna al livello che aveva quando è stata spenta.

Pseudostato history (deep)

Si disegna come **H*** cerchiato. Funziona come la history shallow, ma ripristina l'**intera configurazione** della regione: lo stato attivo a **tutti** i livelli di annidamento, compresi i sottostati dei sottostati.

Esempio 9.14 — Un lettore musicale con la radio



`on()` accende il lettore dall'inizio: si entra in PlayingMusic e si cerca la prossima canzone. `resume()` invece entra attraverso **H***: se il lettore era stato fermato mentre suonava una canzone, torna in **PlaySong**, cioè nel sottostato **di secondo livello** (On → PlayingMusic → PlaySong). Una history shallow avrebbe ricordato solo PlayingMusic, e si sarebbe ripartiti dal suo stato iniziale, FindNextSong. Se invece era stato fermato mentre ascoltava la radio, torna in PlayRadio.

Il diagramma di attività

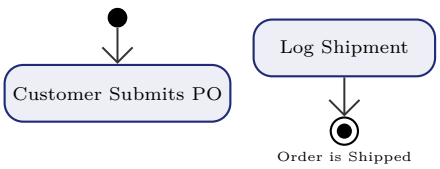
Un **diagramma di attività** si può usare in fasi diverse dello sviluppo:

- per modellare un **flusso di lavoro** (*workflow*);
- nella fase di **analisi**, per modellare un **problema reale**;
- nella **progettazione di alto livello**, per modellare l'interazione tra oggetti concettuali nella soluzione di un problema;
- nella **progettazione di dettaglio**, per modellare un **algoritmo**, eventualmente concorrente, da implementare.

È un'estensione del classico **diagramma di flusso** (*flowchart*) che introduce, in particolare, le **esecuzioni concorrenti**. In UML è nato come estensione del diagramma di stato, ma nelle versioni più recenti ne è stato completamente separato, quindi ha senso studiarlo a parte.

Attività e flussi

- Le **attività** sono le azioni di elaborazione che il sistema deve eseguire; si disegnano come **rettangoli arrotondati** con il nome dell'attività.
- Il **flusso** tra le attività si disegna con frecce entranti e uscenti.
- Per convenzione, l'**ingresso** nell'attività coincide con il suo **inizio** e l'**uscita** con il suo **completamento**.
- Anche qui un nodo **iniziale** e un nodo **finale** delimitano il diagramma.



Decisioni: branch e merge

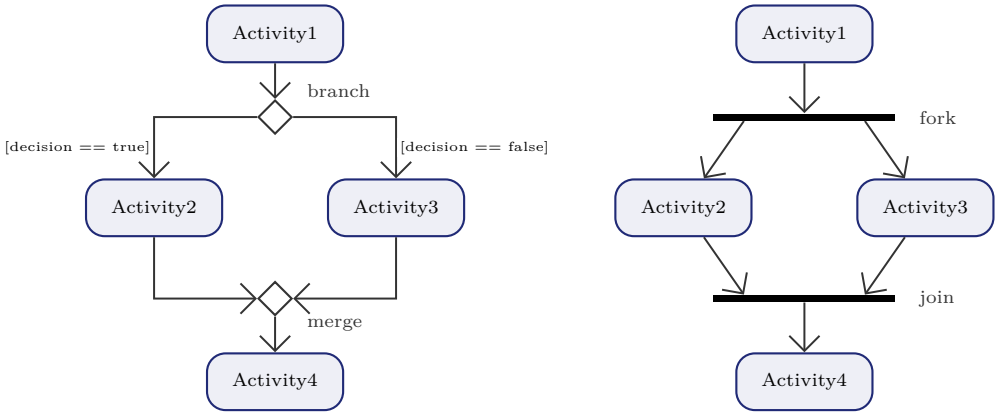
Branch e merge

Il **branch** (decisione) corrisponde a un **IF**. Si disegna come un **rombo** con un solo arco entrante e più archi uscenti; ogni arco uscente ha una **condizione di guardia** tra parentesi quadre, e le condizioni devono essere **mutuamente esclusive**.
Il **merge** è l'elemento duale e corrisponde all'**END IF**: è ancora un rombo, ma con più archi entranti e un solo arco uscente.

Concorrenza: fork e join

Fork e join

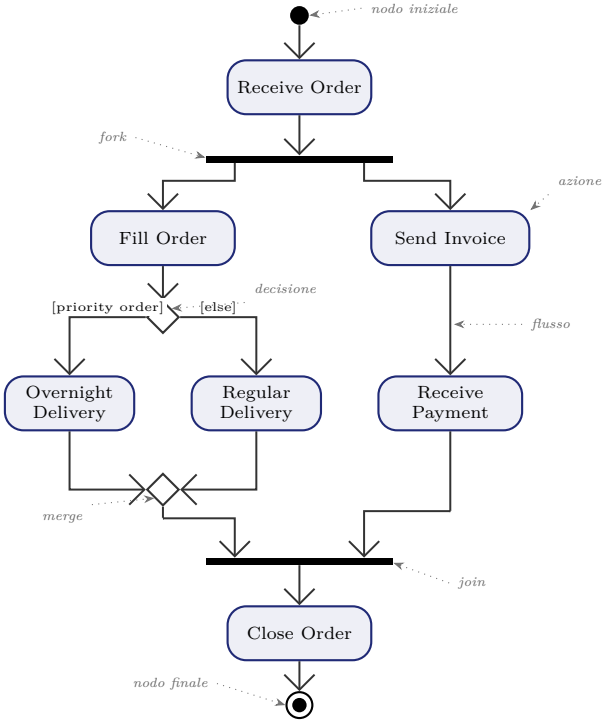
Il **fork** separa il flusso in due o più **thread** che possono procedere **in modo concorrente**. Si disegna come una **barra** in cui entra un solo arco ed escono più archi, senza etichette: si intende che vengono avviati tutti i thread insieme.
Il **join** è l'elemento duale: una barra in cui entrano più archi ed esce un solo arco. Si prosegue solo quando **tutti** i flussi entranti sono arrivati. Fork e join corrispondono esattamente alle primitive omonime della programmazione concorrente.



Il rombo sceglie, la barra sdoppia

Dopo un **branch** si percorre **uno solo** dei rami (quello con la guardia vera). Dopo un **fork** si percorrono **tutti** i rami, contemporaneamente. Allo stesso modo il **merge** lascia passare il primo flusso che arriva, mentre il **join** aspetta che arrivino tutti.

Esempio 9.15 — La gestione di un ordine



Ricevuto l'ordine, due flussi procedono **in parallelo**: da una parte si prepara l'ordine e lo si spedisce (con consegna espressa se l'ordine è prioritario, normale altrimenti), dall'altra si invia la fattura e si riceve il pagamento. L'ordine si chiude solo quando **entrambi** i flussi sono terminati (join).

Esempio 9.16 — Fork e join nel codice

Il diagramma della gestione di un ordine si traduce quasi riga per riga in Java: ogni ramo del fork è un compito eseguito in parallelo, il join è l'attesa che finiscano entrambi.

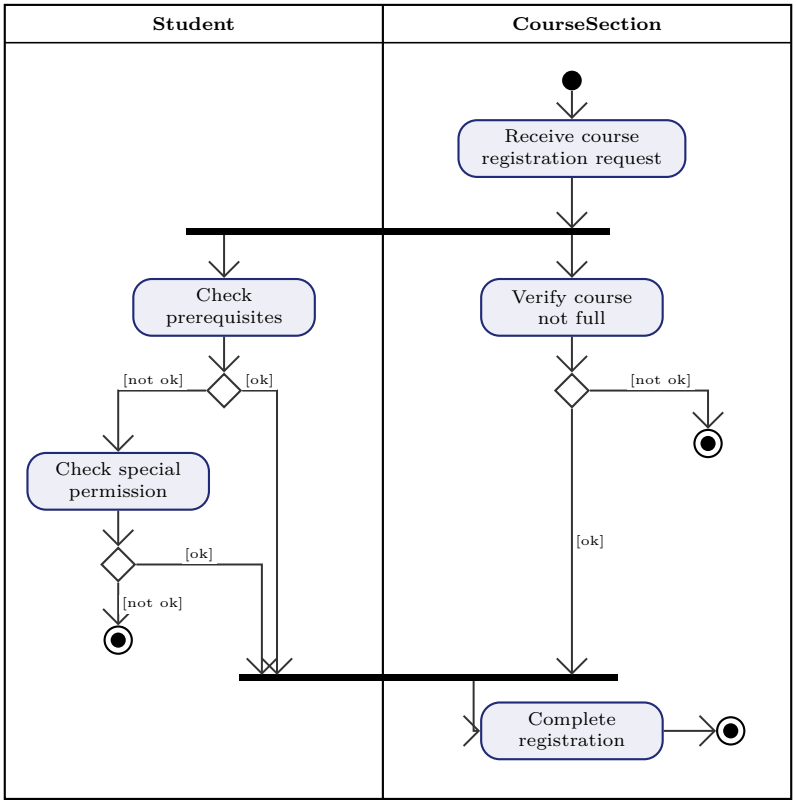
GestioneOrdini.java

```java
public void gestisci(Ordine ordine) {
  // fork: i due flussi partono insieme
  CompletableFuture<Void> spedizione = CompletableFuture.runAsync(() -> {
    prepara(ordine);
    if (ordine.prioritario()) {              // branch [priority order]
      spedizioneEspressa(ordine);
    } else {                                 // [else]
      spedizioneNormale(ordine);
    }
  });
  CompletableFuture<Void> pagamento = CompletableFuture.runAsync(() -> {
    inviaFattura(ordine);
    riceviPagamento(ordine);
  });
  // join: si prosegue solo quando entrambi i flussi sono arrivati
  CompletableFuture.allOf(spedizione, pagamento).join();
  chiudi(ordine);
}
```

Le swimlane

I diagrammi di attività possono modellare anche le **interazioni tra classi diverse** (o tra ruoli, reparti, sistemi). Il diagramma si divide in **corsie verticali** (*swimlane*), e in ogni corsia compaiono le attività che riguardano una particolare classe.

Esempio 9.17 — L'iscrizione a un corso

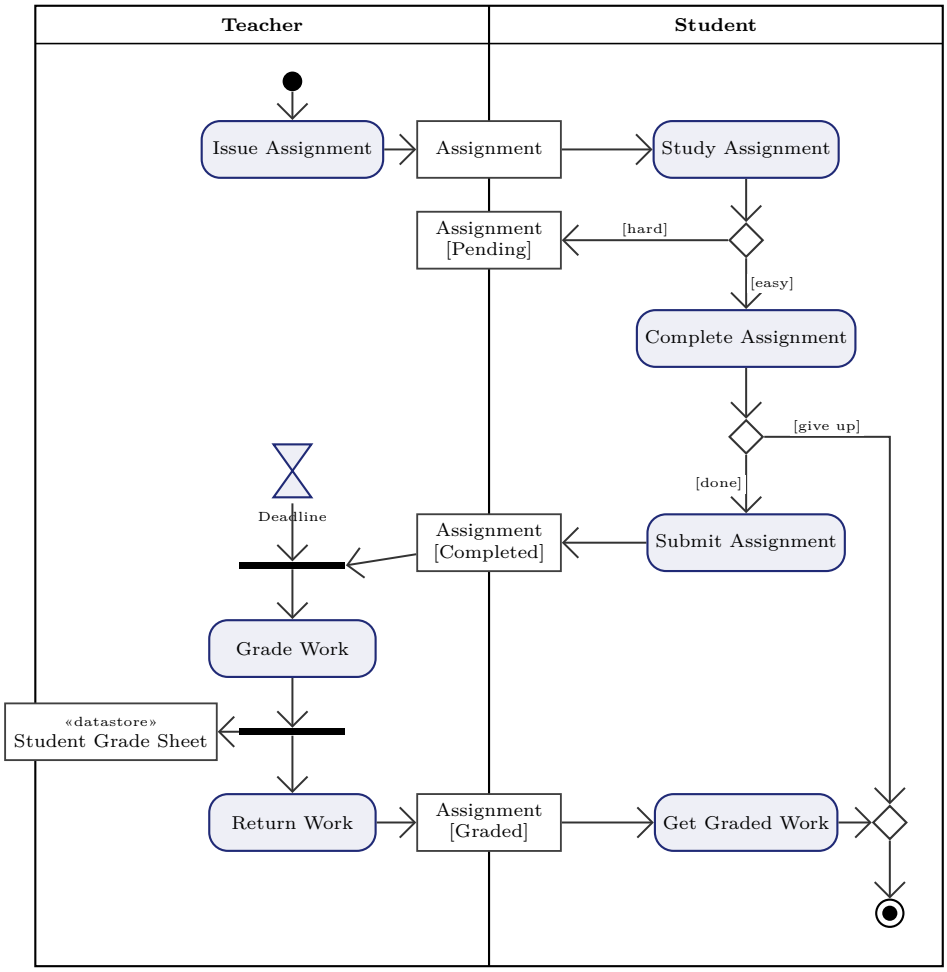


Ricevuta la richiesta, due controlli partono **in parallelo**: il sistema del corso (**CourseSection**) verifica che il corso non sia pieno, mentre sullo studente (**Student**) si controllano i prerequisiti; se mancano, si controlla se ha un permesso speciale. L'iscrizione si completa solo se **entrambi** i controlli vanno a buon fine (join). Se uno dei due fallisce, il flusso arriva a un nodo finale.

Il nodo finale chiude tutto

Quando un flusso raggiunge un **nodo finale di attività**, termina l'**intera** attività, compresi gli altri flussi concorrenti ancora in corso. Nell'esempio, se il corso è pieno l'iscrizione si interrompe subito, anche se il controllo dei prerequisiti non è ancora finito.

Esempio 9.18 — Il compito tra docente e studente: nodi oggetto e datastore



I rettangoli a cavallo tra le corsie sono **nodi oggetto**: rappresentano l'oggetto che passa da un'attività all'altra (qui il compito), e tra parentesi quadre il suo **stato** in quel momento (*Pending*, *Completed*, *Graded*). Lo stereotipo **«datastore»** indica un archivio persistente: i voti restano nel registro anche dopo che l'attività è terminata. Il docente corregge solo quando sono arrivati sia il compito sia la scadenza (join con il segnale di tempo).

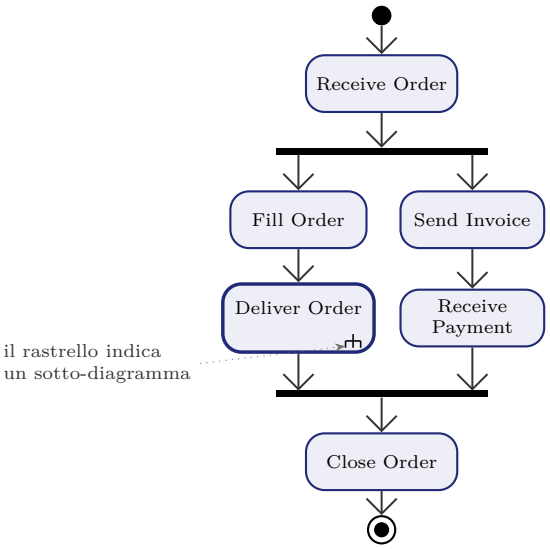
Si noti anche una scelta discutibile del diagramma originale: se il compito è difficile (*[hard]*) il flusso finisce nel nodo oggetto *Assignment [Pending]* e lì si ferma, senza arrivare né al docente né a un nodo finale. Andrebbe chiarito che cosa succede a un compito rimasto in sospeso.

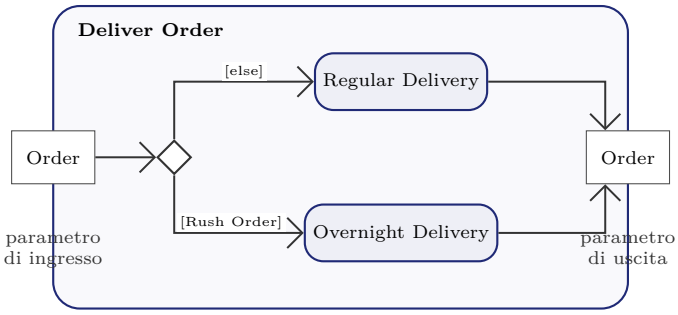
Altri esempi nelle slide

Le slide riportano altri due diagrammi con swimlane: il prelievo al bancomat (corsie Customer, ATM Machine e Bank) e un processo di vendita con le corsie Customer, Sales, Contracts, Legal e Fulfillment. Vale la pena provare a leggerli con le regole di questo capitolo.

Macro-attività

Il limite dei diagrammi di attività è probabilmente la descrizione **troppo dettagliata** dei problemi. Per gestire diversi livelli di astrazione si possono racchiudere i compiti complessi in **macro-attività** generiche, da dettagliare separatamente in altri diagrammi.





Nel diagramma generale *Deliver Order* è un'unica attività, marcata con il simbolo del **rastrello**, che indica che ha un diagramma di dettaglio. Il dettaglio, a destra, è un'attività con un **nome** e dei **parametri**: i rettangoli sul bordo sono l'ordine in ingresso e quello in uscita. Dentro si sceglie tra consegna normale ed espressa.

I segnali

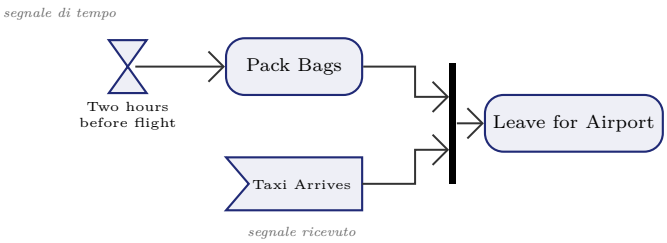
I diagrammi di attività, a differenza di quelli di stato, non mostrano esplicitamente gli **eventi**. Per compensare questa mancanza sono stati introdotti i **segnali**.

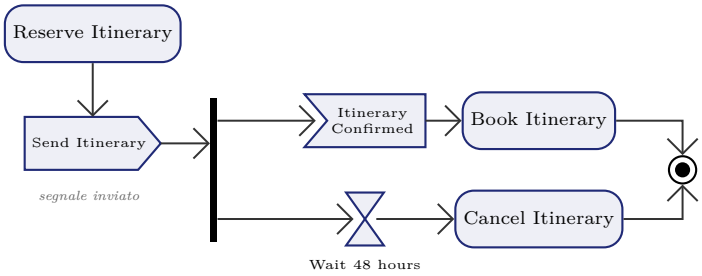
Segnale

Un **segnale** indica un evento **esterno** e **asincrono** che influenza l'esecuzione di un'attività. Un caso tipico sono i **segnali di tempo**. Simboli diversi indicano:

- i segnali **inviati** (*send*): un pentagono a punta;
- i segnali **ricevuti** (*accept*): un pentagono con una rientranza;
- i segnali di **tempo** (*timer*): una clessidra.

Esempio 9.19 — Segnali di tempo, ricevuti e inviati





- **In alto**: due ore prima del volo (segnale di tempo) si fanno le valigie; si parte per l'aeroporto solo quando le valigie sono pronte **e** il taxi è arrivato (segnale ricevuto, poi join).
- **In basso**: si prenota un itinerario e lo si invia (segnale inviato); poi due flussi partono in parallelo. Se arriva la conferma, si prenota; se passano 48 ore senza conferma, si annulla. Entrambi portano al nodo finale, che chiude l'intera attività: **vince il primo** dei due eventi, e l'altro flusso viene interrotto.

Per approfondire

Martin Fowler, *UML Distilled*: capitolo 10 (diagrammi di stato) e capitolo 11 (diagrammi di attività). La specifica completa è l'OMG UML 2.5 (`https://www.omg.org/spec/UML/2.5/PDF/`); un altro riferimento è I. Jacobson, J. Rumbaugh, G. Booch, *The Unified Modeling Language Reference Manual*.

In sintesi

- Il **diagramma dei componenti** mostra i componenti (file, eseguibili, database, librerie, sottosistemi), le **interfacce** fornite (pallina) e richieste (presa) e le **porte**; non dice dove girano.
- Il **diagramma di deployment** mostra l'architettura **fisica**: **nodi** (parallelepipedi, *istanza : tipo*), componenti e **artefatti** al loro interno, **connessioni** fisiche; i **valori etichettati** `{nome = valore}` aggiungono dettagli, e un nodo può contenere **ambienti di esecuzione**.
- Il **diagramma di stato** descrive come un sistema o un oggetto reagisce agli eventi. Le transizioni hanno etichetta **trigger** [**guardia**] **/ azioni**; se più transizioni sono attivabili ne scatta una sola, in modo non deterministico.
- Un diagramma di stato si implementa con **switch annidati** o, meglio, con il **pattern State**: una sottoclasse per stato, un metodo per evento, e il polimorfismo al posto dello switch.
- Gli stati possono avere attività **entry**, **do** ed **exit**, essere **compositi** (anche con **regioni parallele**) e usare lo pseudostato **history** per ricordare il sottostato attivo: **H** solo al primo livello, **H*** a tutti i livelli.
- Il **diagramma di attività** estende il diagramma di flusso con la concorrenza: **branch/merge** (rombo, un solo ramo), **fork/join** (barra, tutti i rami), **swimlane**, **nodi oggetto** e **datastore**, **macro-attività** e **segnali** (tempo, ricevuti, inviati).

Parte V

Progettazione

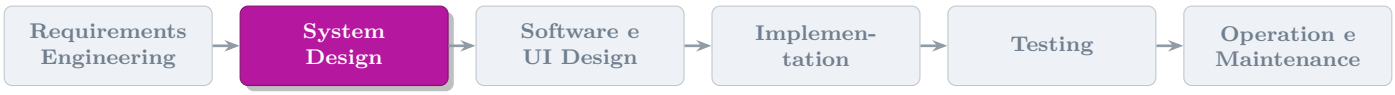
System Design e buone pratiche di progettazione

Con i requisiti e i modelli di analisi abbiamo descritto **che cosa** il sistema deve fare. Da qui in avanti ci occupiamo di **come** farlo. Questo capitolo raccoglie le lezioni dedicate al *System Design* e segue due fili:

- le **attività** della progettazione di sistema: fissare gli obiettivi di progetto e scomporre il sistema in sottosistemi;
- le **regole di buona progettazione**, valide per qualunque software orientato agli oggetti: coesione, accoppiamento, Legge di Demetra e progettazione guidata dalle responsabilità.

La fase era già stata presentata a grandi linee nel Capitolo 3 (*Fase 2: System Design*); qui la riprendiamo in dettaglio.

Dove siamo nel processo



Nel modello a cascata il System Design segue la Requirements Engineering. In ingresso riceve i requisiti, i casi d'uso e il modello di dominio; in uscita produce l'**architettura del sistema**. I requisiti vengono assegnati ai sottosistemi software, i sottosistemi alle risorse hardware, e si scelgono i **pattern architetturali** da adottare.

System Design

Il **System Design** è l'insieme delle attività con cui l'ingegnere del software trasforma il **modello di analisi** nel **modello di progetto del sistema**.
L'obiettivo finale del System Design e del successivo Object Design è produrre un nuovo documento, scritto in linguaggio **tecnico e formale**, da consegnare ai programmatori perché possano implementare il software descritto nel documento dei requisiti.

La progettazione si svolge su due livelli, che corrispondono alle fasi 2 e 3 della cascata.

System Design

1. definire gli **obiettivi di progetto**;
2. definire l'**architettura**, scomponendo il sistema in sottosistemi, che team diversi possono implementare **in parallelo**;
3. scegliere le **strategie** di costruzione: hardware e software, gestione dei dati persistenti, flusso di controllo globale, politiche di controllo degli accessi…

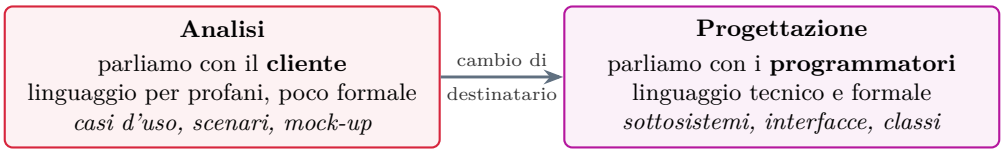
→

Object Design

- data l'architettura, specificare i **dettagli implementativi** dei singoli sottosistemi;
- scegliere le strategie di implementazione migliori, ad esempio i **design pattern**.

Cambia l'interlocutore

Con la progettazione il punto di vista cambia radicalmente. Finora abbiamo lavorato **per il cliente** e con il cliente, usando un linguaggio da non addetti ai lavori. Nel System Design i nostri interlocutori sono i **team di programmatori** che dovranno implementare il sistema: chi legge i nostri documenti è un esperto di informatica, e il linguaggio diventa tecnico.



Le attività della progettazione di sistema

Il System Design si articola in tre macro-attività.

1. **Identificare gli obiettivi di progetto.** Gli sviluppatori stabiliscono quali caratteristiche di qualità vanno ottimizzate e con quale priorità.
2. **Progettare la scomposizione in sottosistemi.** A partire dai casi d'uso e dai modelli di analisi, si divide il sistema in parti più piccole, usando stili architetturali standard.
3. **Raffinare la scomposizione per soddisfare gli obiettivi.** La prima scomposizione di solito non soddisfa gli obiettivi di progetto: la si raffina finché non li soddisfa.

Nel dettaglio, la progettazione di sistema tocca otto aspetti, riassunti in figura. In questo capitolo trattiamo i primi due, evidenziati, insieme alle regole generali che guidano la scomposizione.

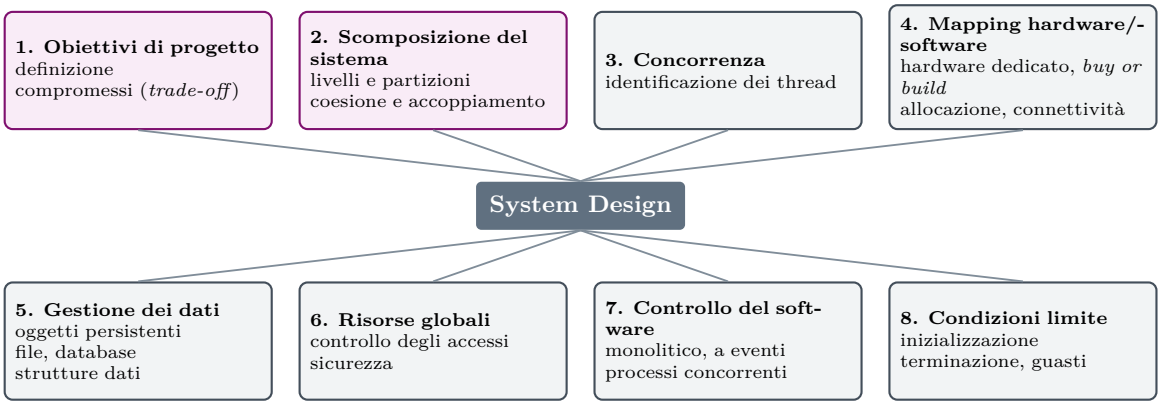


Figura 10.1: Gli otto aspetti del System Design. In questo capitolo: obiettivi di progetto e scomposizione.

Gli obiettivi di progetto

Identificare le **qualità** su cui il sistema deve puntare è il primo passo della progettazione.

- Molti obiettivi di progetto derivano dai **requisiti non funzionali** (Capitolo 5) o dal dominio applicativo; altri sono indicati direttamente dal cliente.
- Vanno **formalizzati esplicitamente**, perché ogni decisione di progetto importante deve essere presa in base allo **stesso insieme di criteri**. Se ogni sviluppatore ottimizza ciò che gli sembra più importante, il risultato è incoerente.

I criteri di progetto

Gli obiettivi si scelgono da un lungo elenco di qualità desiderabili, organizzabili in cinque gruppi. Molte di queste qualità le abbiamo già incontrate nel modello ISO/IEC 25010 del Capitolo 4: nella terza colonna indichiamo la caratteristica corrispondente.

Gruppo	Esempi di criteri	In ISO/IEC 25010
Prestazioni	tempo di risposta, throughput, memoria richiesta	efficienza prestazionale
Affidabilità	robustezza, disponibilità, tolleranza ai guasti, sicurezza	affidabilità, sicurezza
Costi	di sviluppo, di manutenzione, di gestione	(non è una qualità del prodotto)
Manutenzione	estendibilità, modificabilità, adattabilità, portabilità	manutenibilità, flessibilità
Usabilità	facilità d'uso e di apprendimento	usabilità

I compromessi di progetto

Nella pratica si riesce a tenere conto solo di un **piccolo sottoinsieme** di questi criteri: non è realistico, ad esempio, sviluppare un software che sia allo stesso tempo molto sicuro e poco costoso. Gli sviluppatori devono quindi **dare priorità** agli obiettivi, tenendo conto anche di vincoli gestionali come i tempi e il budget. I compromessi più frequenti sono quattro.

Memoria *vs* velocità
Se il software non rispetta i requisiti di tempo di risposta o throughput, si usa più memoria per renderlo veloce (cache, ridondanza). Se non rispetta quelli di memoria, si comprimono i dati a scapito della velocità.

Tempi di rilascio *vs* funzionalità
Se i tempi sono stretti, si può rilasciare in tempo una versione con **meno funzionalità** di quelle richieste, rimandando le altre.

Tempi di rilascio *vs* qualità
Con tempi stretti, il project manager può rilasciare nei tempi un software con difetti noti e correggerli dopo, oppure rilasciarlo in ritardo con meno difetti.

Tempi di rilascio *vs* personale
Può essere necessario **aggiungere persone** al progetto per aumentarne la produttività (ma attenzione, vedi il riquadro qui sotto).

La legge di Brooks

Aggiungere persone funziona solo se lo si fa **per tempo**. Fred Brooks, in *The Mythical Man-Month* (1975), osservò che *aggiungere personale a un progetto software in ritardo lo fa ritardare ancora di più*: i nuovi arrivati vanno formati da chi già lavora al progetto, e il numero di canali di comunicazione cresce con il quadrato delle persone. Il compromesso tempi/personale va quindi valutato all'inizio, non a progetto già in ritardo.

Esempio 10.1 — Obiettivi di progetto per la gestione delle emergenze

Gli esempi delle slide su database e sottosistemi (*Resource*, *Map*, *Incident Management*) vengono da un sistema per la gestione delle emergenze: gli operatori registrano gli incidenti, consultano una mappa e assegnano le risorse (ambulanze, vigili del fuoco). Per un sistema del genere una scelta ragionevole di priorità è:

1. **affidabilità e disponibilità**: il sistema deve funzionare proprio quando c'è un'emergenza;
2. **tempo di risposta**: un incidente registrato deve comparire subito sulla mappa degli operatori;
3. **usabilità**: gli operatori lavorano sotto stress, l'interfaccia non deve indurli in errore.

Il **costo** passa in secondo piano: si accetta di spendere di più, ad esempio per server ridondanti, pur di garantire la disponibilità. È un compromesso costo/affidabilità deciso **una volta per tutte** e scritto nel documento di progetto, in modo che ogni scelta successiva lo rispetti.

Criteri generali: progettare per il cambiamento

Molti criteri di progetto dipendono strettamente dal problema da risolvere: trovare il miglior equilibrio fra i vincoli è compito dell'analista, che si basa sulla propria esperienza e sensibilità. Esistono però anche criteri **generali**, validi per qualunque software orientato agli oggetti. Applicati correttamente, migliorano in modo significativo la **qualità interna** del codice senza introdurre svantaggi particolari. Il resto del capitolo è dedicato a questi criteri.

Il primo è un atteggiamento: quando progettiamo una classe dobbiamo sforzarci di pensare a quali **cambiamenti** potranno essere richiesti in futuro.

Progettare per anticipare il cambiamento

Le scelte iniziali devono **facilitare l'evoluzione futura** del software (*designing to anticipate change*). È uno dei requisiti fondamentali di un buon progetto, ed è anche un criterio con cui il progetto viene valutato.

Ricordiamo dal Capitolo 2 che la manutenzione rappresenta la parte più consistente del costo di un software: un progetto che rende economici i cambiamenti ripaga questo investimento per tutta la vita del sistema.

La scomposizione in sottosistemi

Per ridurre la complessità della soluzione si scompone il sistema in parti più piccole, i **sottosistemi**.

- Un sottosistema corrisponde tipicamente alla porzione di lavoro che un **singolo sviluppatore o un team** può svolgere in modo indipendente.
- Se i sottosistemi sono relativamente indipendenti, i team possono lavorarci con un **costo di comunicazione** minimo.
- Se un sottosistema è ancora complesso, si applica di nuovo lo stesso principio e lo si scompone in sottosistemi più semplici.

Perché scomporre conviene

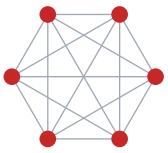
Indichiamo con P un problema, con $C(P)$ la sua complessità e con $E(P)$ lo sforzo necessario a risolverlo. Dati due problemi P_1 e P_2, è intuitivo che

$$C(P_1) > C(P_2) \;\Rightarrow\; E(P_1) > E(P_2).$$

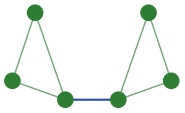
Si osserva però anche che la complessità di due problemi affrontati **insieme** è maggiore della somma delle loro complessità prese separatamente:

$$C(P_1 + P_2) > C(P_1) + C(P_2) \qquad \text{e quindi} \qquad E(P_1 + P_2) > E(P_1) + E(P_2).$$

Il motivo sono le **interazioni**: risolvendo i due problemi insieme bisogna tenere conto anche di come ogni parte dell'uno influenza ogni parte dell'altro. In figura, sei elementi tutti potenzialmente legati fra loro hanno $6 \cdot 5/2 = 15$ collegamenti possibili; divisi in due gruppi da tre, con un solo punto di contatto, i collegamenti sono $3 + 3 + 1 = 7$. È l'argomento a favore del *divide et impera*, a patto che i pezzi siano davvero **indipendenti**: è quello che misureremo con coesione e accoppiamento.



insieme: 15 legami



scomposti: 7 legami

Modellare i sottosistemi

In UML un sottosistema si rappresenta con un **package** (Capitolo 8): una collezione di classi, associazioni, operazioni e vincoli fra loro correlati. Anche i linguaggi offrono costrutti per lo stesso concetto: i `package` in Java, i `namespace` in C++ e C#.

Servizi e interfaccia di un sottosistema

Un sottosistema è caratterizzato dai **servizi** che offre agli altri sottosistemi. L'insieme delle operazioni disponibili agli altri sottosistemi forma l'**interfaccia del sottosistema**: comprende i nomi delle operazioni, i loro parametri, i loro tipi e i valori restituiti.

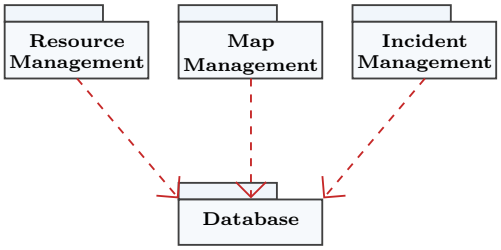
Il System Design si concentra sulla definizione dei servizi offerti da ogni sottosistema, in termini di **operazioni**, **parametri** e **comportamento ad alto livello**. I dettagli interni restano all'Object Design.

Esempio 10.2 — Accesso diretto al database o tramite un sottosistema dedicato

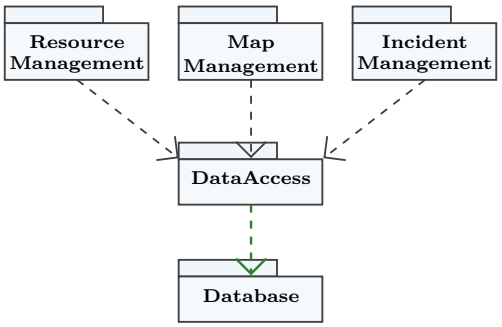
Torniamo al sistema per le emergenze. Nella prima versione (a sinistra) tutti i sottosistemi accedono **direttamente** al database.

- Non si riesce a specificare con chiarezza le responsabilità dei moduli: chi *gestisce* i dati, chi li *salva*?
- Tutti i sottosistemi dipendono dall'interfaccia del database: se cambia il DBMS, va modificato il codice di **tutte** le classi che vi accedono.

Nella seconda versione (a destra) si introduce il sottosistema **DataAccess**, l'unico che gestisce l'input/output con il database. Se cambia il DBMS, si modifica **solo** DataAccess.



tre dipendenze dal DBMS



una sola dipendenza dal DBMS

Nelle slide il testo dice *«modifico solo il sottosistema Storage»*, ma nel diagramma lo stesso sottosistema si chiama *DataAccess*: sono la stessa cosa.

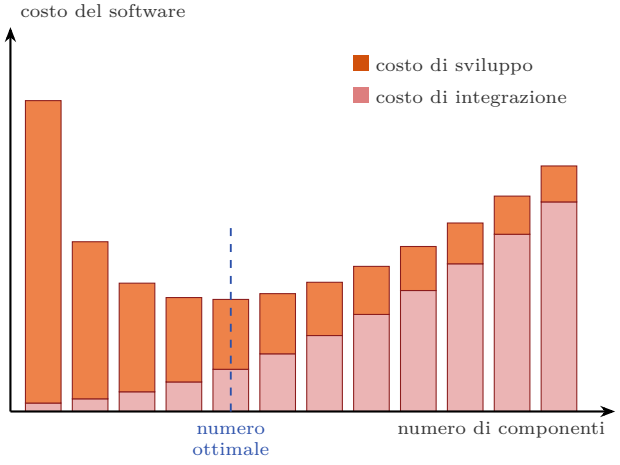
Quanti sottosistemi?

L'esempio ha ridotto le relazioni fra i sottosistemi, ma ha **aumentato la complessità**: c'è un sottosistema in più da progettare, implementare e attraversare a ogni accesso ai dati. Inseguendo la riduzione delle dipendenze si rischia di aggiungere livelli di astrazione che costano tempo di sviluppo e di esecuzione.

Il grafico, tratto da Pressman, mostra il compromesso. Al crescere del numero di componenti:

- il **costo di sviluppo** di ciascun componente diminuisce, perché ogni componente è più piccolo e semplice;
- il **costo di integrazione** aumenta, perché ci sono più interfacce da definire e più pezzi da far collaborare.

Il costo totale ha quindi un **minimo**: troppi pochi componenti sono difficili da sviluppare, troppi sono difficili da integrare.



L'indipendenza funzionale

La divisione in sottosistemi è guidata da due fattori qualitativi fondamentali, la **coesione** e l'**accoppiamento**. L'obiettivo è l'**indipendenza funzionale** dei sottosistemi, che si massimizza con **alta coesione** e **basso accoppiamento**.

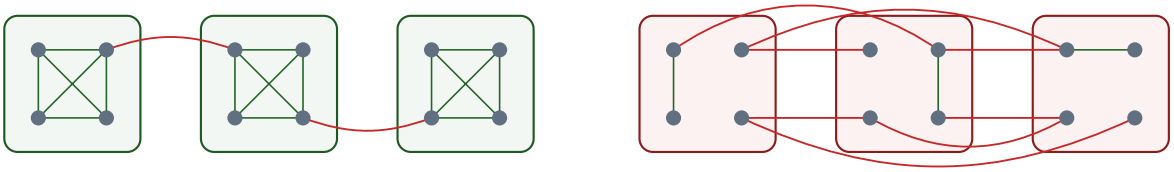


Figura 10.2: Gli stessi elementi raggruppati in due modi. In verde i legami interni ai moduli, in rosso quelli fra moduli. A sinistra ogni modulo è quasi autosufficiente; a destra quasi ogni legame attraversa i confini.

Coesione

Coesione

La **coesione** misura quanto sono **strettamente correlate e focalizzate** le responsabilità (o i servizi offerti) di un modulo. Se ogni unità è responsabile di un **unico compito**, diciamo che ha **alta coesione**.

L'alta coesione è una proprietà desiderabile del codice, perché permette di:

- capire meglio il ruolo di una classe;
- riusare una classe;
- mantenere una classe;
- limitare e concentrare le modifiche al codice;
- usare nomi appropriati, efficaci e comunicativi.

La coesione si valuta a due livelli di **granularità**:

- **coesione dei metodi**: un metodo dovrebbe essere responsabile di un solo compito ben definito;
- **coesione delle classi**: ogni classe dovrebbe rappresentare un solo concetto ben definito.

Il principio di singola responsabilità

Single Responsibility Principle (SRP)

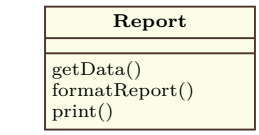
Una classe dovrebbe avere **una sola responsabilità**, cioè **un solo motivo per cambiare**.

Una responsabilità è quindi un **motivo di cambiamento**: le responsabilità di una classe sono i suoi *assi di cambiamento*. Se una classe ha due responsabilità, nel progetto queste sono **accoppiate** e devono cambiare insieme. Una classe con più di una responsabilità diventa **fragile**: si rompe ogni volta che cambia uno qualunque dei requisiti da cui dipende.

Esempio 10.3 — Un report con troppe responsabilità

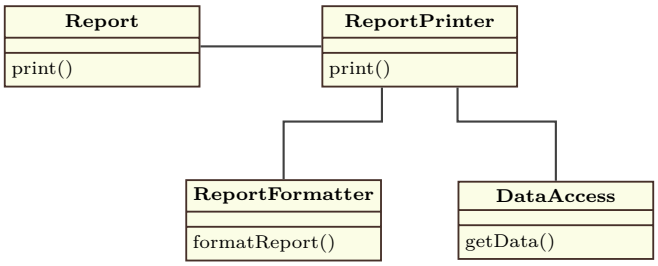
La classe `Report` recupera i dati, li formatta e stampa il report. L'esempio delle slide è in C#; qui lo riportiamo in Java.

```java
public class Report {
  private List<ReportDataElement> getData() {
    System.out.println("Getting data...");
    return new ArrayList<>();
  }
  private void formatReport() {
    System.out.println("Formatting report...");
  }
  public void print() {
    getData();
    formatReport();
    System.out.println("Printing report...");
  }
}
```



Tre motivi per cambiare:
la sorgente dei dati, il formato,
il dispositivo di stampa.

Applicando l'SRP si separano le tre responsabilità in tre classi; `Report` si limita a chiedere la stampa.




```java
public class DataAccess {
  public List<ReportDataElement> getData() {
    System.out.println("Getting data...");
    return new ArrayList<>();
  }
}

public class ReportFormatter {
  public void formatReport() {
    System.out.println("Formatting report...");
  }
}
```

```java
public class ReportPrinter {
  public void print() {
    DataAccess dataAccess = new DataAccess();
    ReportFormatter reportFormatter =
        new ReportFormatter();
    dataAccess.getData();
    reportFormatter.formatReport();
    System.out.println("Printing report...");
  }
}

public class Report {
  public void print() {
    ReportPrinter reportPrinter = new ReportPrinter();
    reportPrinter.print();
  }
}
```

Ora ogni motivo di cambiamento tocca una sola classe:

Se cambia...	si modifica solo...
la sorgente dei dati (un altro database, un file, un servizio web)	`DataAccess`
il formato del report (PDF, HTML, tabella)	`ReportFormatter`
il modo di stampare (stampante, schermo, e-mail)	`ReportPrinter`

Coesione sì, ma l'accoppiamento?

Nella versione divisa `ReportPrinter` crea con `new` le classi **concrete** `DataAccess` e `ReportFormatter`: per stampare un report letto da un file invece che da un database bisognerebbe comunque modificare `ReportPrinter`. La coesione è migliorata, ma l'accoppiamento resta alto. Vedremo nella sezione sull'accoppiamento come ridurlo con le **interfacce**.
Si noti anche che nel codice delle slide i dati letti da `getData()` non vengono passati a `formatReport()`: è uno schema che si limita a stampare messaggi. In una versione reale i dati scorrerebbero da `DataAccess` a `ReportFormatter`, ad esempio con `formatReport(dati)`.

I vantaggi della coesione

- **Riuso**: se tutte le classi rispettano l'SRP, è più probabile riusarne qualcuna.
- **Chiarezza**: il codice è più pulito, perché le classi non fanno cose inattese.
- **Nomi**: se una classe ha una sola responsabilità, trovarle un buon nome è facile.
- **Leggibilità**: chiarezza, nomi migliori e file più corti migliorano molto la leggibilità.

Indicatori di alta coesione

- la classe ha responsabilità moderate, limitate a una sola area funzionale;
- collabora con altre classi per completare i compiti;
- ha un numero limitato di metodi, cioè di funzionalità fortemente correlate;
- tutti i metodi sembrano appartenere allo stesso insieme, con uno scopo complessivo simile;
- la classe è facile da capire.

Campanelli d'allarme

- il nome contiene «e» o termini generici (`GestoreUtentiEStampe`, `Manager`, `Utils`);
- gruppi di metodi usano gruppi di attributi **disgiunti**: sono due classi in una;
- per descrivere la classe serve una frase con molte virgole;
- una modifica a un requisito qualunque finisce sempre per toccare quella classe.

Tipi e livelli di coesione

Non tutta la coesione è uguale. A seconda del **motivo** per cui gli elementi di un modulo stanno insieme, si distinguono sette tipi di coesione, ordinati dal più debole al più forte. Più il legame è debole, più il raggruppamento è casuale e più il modulo tende a cambiare per ragioni diverse.

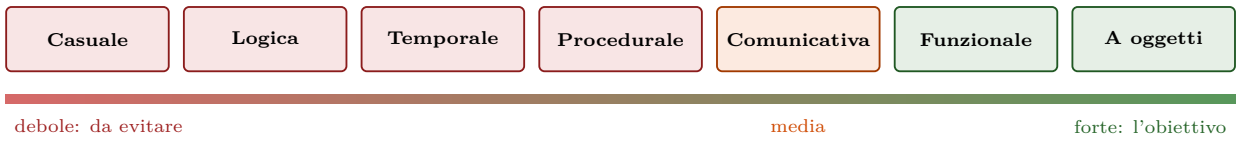


Figura 10.3: I sette livelli di coesione, dal più debole al più forte.

Tipo	Gli elementi stanno insieme perché...	Esempio
Casuale (*accidental*)	... capita: sono semplicemente raggruppati, senza un legame reale	una classe `Utils` con `formattaData`, `calcolaIva` e `inviaEmail`
Logica	svolgono attività **logicamente simili** (gestione degli errori, delle stampe...), anche se su dati e per scopi diversi	un modulo `Stampe` che stampa fatture, etichette e report, scegliendo con un parametro quale
Temporale	vanno eseguiti **nello stesso intervallo di tempo**	un metodo `init()` che apre il database, carica la configurazione e azzera i contatori
Procedurale	vanno eseguiti **in un certo ordine**	`leggiInput()`, poi `valida()`, poi `salva()`, messi insieme solo perché vengono uno dopo l'altro
Comunicativa	lavorano su un **insieme di dati comune**	visualizzazione, logica e salvataggio dello stesso `Ordine` nello stesso modulo
Funzionale	contribuiscono tutti a **una sola attività**, e non fanno altro	un modulo che calcola il codice fiscale
A oggetti	ogni operazione **modifica o ispeziona gli attributi** dell'oggetto	una classe `ContoCorrente` con `deposita`, `preleva`, `getSaldo`

La coesione **comunicativa** è a metà strada: i dati sono condivisi, ma le responsabilità sono diverse (mostrare, elaborare, salvare), e quindi i motivi di cambiamento sono più d'uno. La coesione a cui puntiamo con il Single Responsibility Principle è quella **funzionale**, e nelle classi quella **a oggetti**.

Temporale e procedurale sono i casi più insidiosi

Un metodo `init()` o una sequenza «leggi, valida, salva» sembrano ben organizzati, perché il codice si legge dall'alto in basso senza salti. Il problema è che il motivo per cui le istruzioni stanno insieme è il **momento** in cui vengono eseguite, non il **compito** che svolgono: se cambia il modo di validare, si tocca lo stesso modulo che si occupa di salvare. Spesso la soluzione è lasciare lì solo il coordinamento, e delegare ogni passo a un modulo funzionalmente coeso.

Esempio 10.4 — Dalla coesione temporale alla coesione funzionale

Un `init()` tipico: le istruzioni stanno insieme solo perché vanno eseguite all'avvio.

Applicazione.java

```java
public void init() throws Exception {
  connessione = DriverManager.getConnection(URL_DB);      // database
  Properties p = new Properties();                        // configurazione
  p.load(new FileInputStream("app.properties"));
  timeout = Integer.parseInt(p.getProperty("timeout"));
  contatoreAccessi = 0;                                   // statistiche
  contatoreErrori = 0;
}
```

Se cambia il formato della configurazione o il database, si modifica la stessa classe che tiene le statistiche. Dopo il refactoring all'avvio resta solo il **coordinamento**, e ogni passo sta in un modulo funzionalmente coeso:

Applicazione.java

```java
public void avvia() {
  database.connetti();                      // Database: solo la connessione
  configurazione.carica("app.properties");  // Configurazione: solo leggere e validare le impostazioni
  statistiche.azzera();                     // Statistiche: solo i contatori
}
```

Antipattern

Un **antipattern** è una cattiva pratica che si osserva **comunemente** nel software reale: una soluzione che sembra ragionevole sul momento, ma le cui conseguenze negative superano i vantaggi.

La **God Class** (o *God Object*) è l'antipattern opposto all'alta coesione. Una God Class:

- ha una conoscenza **eccessiva** del sistema;
- gestisce **molte responsabilità non correlate**;
- fa da **snodo centrale** per tutte le operazioni: le altre classi sono poco più che contenitori di dati, e tutta la logica passa da lei.

Le conseguenze:

- **difficile da mantenere**: una singola modifica si propaga in modo imprevedibile a tutto il sistema;
- **soggetta a errori**: un bug nella God Class colpisce molte aree diverse;
- **difficile da scalare**: una struttura monolitica ostacola l'elaborazione distribuita o parallela, e anche il lavoro in parallelo dei team, che finiscono per modificare tutti lo stesso file.



Le altre classi sono solo dati: tutta la logica sta in un'unica classe.

La cura è quella vista con il Single Responsibility Principle: individuare i motivi di cambiamento e distribuire le responsabilità, ognuna nella classe che possiede i dati su cui lavora (`calcolaMedia` in `Studente`, `prenotaAula` in un servizio di prenotazione, `salvaSuDb` in un *repository*…).

Accoppiamento

Accoppiamento

L'**accoppiamento** (*coupling*) misura quanto una classe è **connessa** ad altre classi, quanto ne **conosce** i dettagli e quanto ne **dipende**.
Due o più classi sono accoppiate quando è impossibile riusarne una senza riusare anche le altre. Se due classi dipendono strettamente e in molti dettagli l'una dall'altra, si dicono **fortemente accoppiate**; altrimenti **debolmente accoppiate**.
Per avere codice di qualità bisogna puntare a un **basso accoppiamento**.

Il basso accoppiamento permette di:

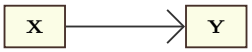
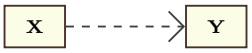
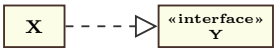
- capire il codice di una classe senza leggere i dettagli delle altre;
- modificare una classe senza che le modifiche si ripercuotano sulle altre;
- riusare facilmente una classe senza dover importare anche altre classi;
- testare facilmente una classe, indipendentemente dalle altre.

In una parola, migliora la **manutenibilità** del software.

Un certo grado di accoppiamento è però **inevitabile**: nel paradigma a oggetti il sistema funziona proprio perché gli oggetti si scambiano messaggi, e per mandare un messaggio a un oggetto bisogna conoscerlo. L'accoppiamento va quindi **limitato** dove possibile, non eliminato. La linea guida è assegnare le responsabilità delle classi in modo da ottenere un accoppiamento basso; la Legge di Demetra, più avanti, ne è un'applicazione concreta.

Le forme di accoppiamento

Date due classi X e Y, X è accoppiata a Y se si verifica almeno uno di questi casi. Accanto riportiamo come la dipendenza si vede in un diagramma delle classi (Capitolo 7).

	Caso	In UML
1	X ha un **attributo** di tipo Y	
2	X ha un **metodo** che usa un elemento di tipo Y (parametro, variabile locale, valore restituito…)	
3	X è una **sottoclasse** (anche indiretta) di Y	
4	X **implementa** un'interfaccia di tipo Y	

Il caso 3, l'**ereditarietà**, è quello che produce l'accoppiamento più forte. Una sottoclasse eredita attributi e metodi della superclasse e spesso ne dipende anche nei dettagli di implementazione: una modifica alla superclasse può rompere tutte le sottoclassi, anche quelle che non si sono mai occupate della parte modificata. L'interfaccia (caso 4), invece, impegna solo a rispettare un **contratto**, senza portarsi dietro alcuna implementazione.

Esempio 10.5 — Le quattro forme di accoppiamento in una classe

X.java

```java
public class X extends Base implements Stampabile {   // 3: sottoclasse di Base   4: implementa Stampabile
  private Y y;                                          // 1: attributo di tipo Y

  public Z converti(W w) {                              // 2: W e Z usati in un metodo
    return new Z(w.valore());
  }
}
```

`X` è accoppiata a cinque tipi: una modifica a `Base` (il legame più forte), a `Stampabile`, a `Y`, a `W` o a `Z` può costringere a modificarla.

Pensare per interfacce

Le interfacce sono uno strumento centrale per ridurre l'accoppiamento, e la libreria standard di Java ne contiene centinaia di esempi d'uso corretto (`List`, `Comparable`, `Runnable`…). I loro vantaggi principali sono due.

- **Fissano uno standard.** Un'interfaccia definisce un **contratto** che favorisce il riuso. Un oggetto che implementa un'interfaccia promette di rispettare quello standard; l'oggetto che lo usa si chiama **consumatore** (*consumer*). L'interfaccia è un contratto fra un oggetto e i suoi consumatori.
- **Introducono un livello di astrazione** che rende i programmi più facili da capire: permettono di ragionare sul comportamento generale del codice senza entrare nei dettagli specifici.

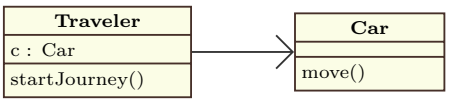
Vediamo l'effetto di un'interfaccia su un esempio concreto.

Esempio 10.6 — Il viaggiatore e l'automobile

```java
class Traveler {
  Car c = new Car();

  void startJourney() {
    c.move();
  }
}

class Car {
  void move() {
    // logica...
  }
}
```



`Traveler` ha un attributo di tipo `Car`: è il caso 1, un accoppiamento forte.

- Che cosa succede se vogliamo riusare `Traveler` con un mezzo di trasporto diverso da `Car`? Dobbiamo modificarlo.
- Che cosa succede se cambiano i metodi di `Car`? Dobbiamo modificare anche `Traveler`.

Introduciamo un'interfaccia `Vehicle`, che `Car` e `Bike` implementano.

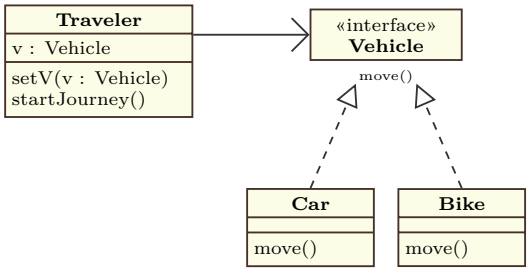
```java
interface Vehicle {
  void move();
}

class Car implements Vehicle {
  public void move() { /* logica */ }
}

class Bike implements Vehicle {
  public void move() { /* logica */ }
}

class Traveler {
  Vehicle v;

  public void setV(Vehicle v) {
    this.v = v;
  }
  void startJourney() {
    v.move();
  }
}
```



`Traveler` non conosce più né `Car` né `Bike`.

L'interfaccia `Vehicle` ha **spezzato** l'accoppiamento fra `Traveler` e `Car`. Le conseguenze:

- `Traveler` ora è accoppiato a un'**interfaccia**, non a una classe concreta;
- tutte le implementazioni di `Vehicle` funzionano con `Traveler` senza modificarne il codice;
- si potranno usare in futuro implementazioni di `Vehicle` che oggi non esistono ancora (**anticipare il cambiamento**);
- serve però un'**entità esterna** che fornisca (*inietti*) l'implementazione concreta: qui il metodo `setV`, chiamato ad esempio così:

  ```java
  Traveler t = new Traveler();
  t.setV(new Bike());      // chi crea Traveler sceglie il mezzo
  t.startJourney();
  ```

- la complessità del codice è aumentata: c'è un tipo in più da conoscere.

Iniezione con il costruttore

Nella versione delle slide, se qualcuno chiama `startJourney()` senza aver prima chiamato `setV(...)`, l'attributo `v` vale `null` e il programma termina con una `NullPointerException`. Quando la dipendenza è **obbligatoria**, conviene passarla nel **costruttore**: `public Traveler(Vehicle v) { this.v = v; }`. Così non può esistere un `Traveler` senza mezzo di trasporto. Questa tecnica, con cui chi usa una classe le fornisce dall'esterno gli oggetti di cui ha bisogno, si chiama **dependency injection**. Applicata a `ReportPrinter` dell'esempio sul report, risolverebbe il problema di accoppiamento segnalato lì.

UN ESEMPIO COMPLETO: IL RAGAZZO DEI GIORNALI

L'esempio classico del *Paperboy* (il ragazzo che consegna i giornali e passa a riscuotere) mostra un caso di accoppiamento eccessivo molto comune, e come correggerlo.

```java
public class Customer {
  private String firstName;
  private String lastName;
  private Wallet myWallet;

  public String getFirstName() {
    return firstName;
  }
  public String getLastName() {
    return lastName;
  }
  public Wallet getWallet() {
    return myWallet;
  }
}
```

```java
public class Wallet {
  private float value;

  public float getTotalMoney() {
    return value;
  }
  public void setTotalMoney(float newValue) {
    value = newValue;
  }
  public void addMoney(float deposit) {
    value += deposit;
  }
  public void subtractMoney(float debit) {
    value -= debit;
  }
}
```

Ecco il codice di un metodo della classe `Paperboy`, che riscuote due dollari.

```java
float payment = 2.00f;                    // "voglio i miei due dollari!"
Wallet theWallet = myCustomer.getWallet();
if (theWallet.getTotalMoney() > payment) {
  theWallet.subtractMoney(payment);
} else {
  // ripassare piu' tardi a riscuotere
}
```

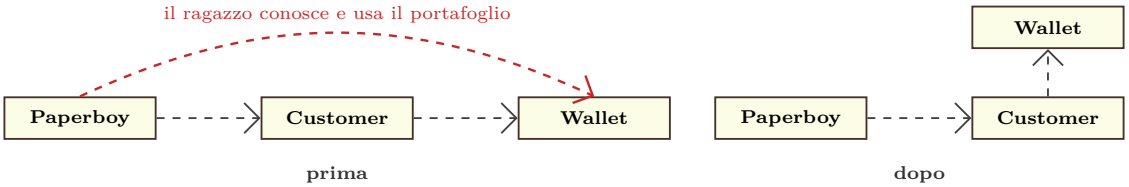
Perché è sbagliato

1. **Non rispecchia il mondo reale.** Tradotto in linguaggio naturale, il codice dice che quando il ragazzo passa a riscuotere, il cliente gli lascia **prendere il portafoglio** dalla tasca e sfilarne due dollari, fidandosi che prenda solo quanto dovuto. Se in futuro `Wallet` contenesse anche le carte di credito, il ragazzo avrebbe accesso pure a quelle. Il problema di fondo è che il `Paperboy` è **esposto a più informazioni di quante gliene servano**.
2. **Accoppia tre classi.** `Paperboy` ora sa che il cliente ha un portafoglio e può manipolarlo: per compilarlo servono sia `Customer` sia `Wallet`. Le tre classi sono **fortemente accoppiate**, e una modifica a `Wallet` può costringere a modificare le altre due.

3. **È fragile.** Supponiamo che a un cliente rubino il portafoglio, e che un collega decida di modellare il furto impostando il portafoglio a `null`:

```
victim.setWallet(null);
```

È una scelta ragionevole: né la documentazione né il codice impongono che il portafoglio esista. Ma il codice del `Paperboy` **dà per scontato** che ci sia, e chiamando un metodo su `null` ottiene una `NullPointerException`. Si potrebbe controllare `null` prima di ogni chiamata, ma il codice del ragazzo diventerebbe ancora più confuso: *«se il cliente ha un portafoglio, guarda quanto c'è dentro; se può pagarmi, prendi i soldi»*.



La versione migliorata

`Customer` non offre più `getWallet()`, ma un metodo `getPayment()`: il ragazzo **chiede** un pagamento, e il cliente decide come farlo.

```
public class Customer {
  private String firstName;
  private String lastName;
  private Wallet myWallet;

  // getFirstName(), getLastName() come prima

  public float getPayment(float bill) {
    if (myWallet != null
        && myWallet.getTotalMoney() >= bill) {
      myWallet.subtractMoney(bill);
      return bill;
    }
    return 0;
  }
}
```

```
// in un metodo di Paperboy
float payment = 2.00f;
float paidAmount =
    myCustomer.getPayment(payment);
if (paidAmount == payment) {
  // ringrazia e lascia la ricevuta
} else {
  // ripassa piu' tardi
}
```

Il codice delle slide va corretto

Il metodo `getPayment` delle slide non compila:

- usa le variabili `theWallet` e `payment`, che appartengono al codice del `Paperboy`: dentro `Customer` vanno usati l'attributo `myWallet` e il parametro `bill`;
- se il portafoglio manca o non contiene abbastanza soldi, il metodo termina **senza restituire nulla**, e Java segnala *missing return statement*: serve un `return 0` finale.

Inoltre, sia nelle slide sia nel codice originale, la condizione `getTotalMoney() > payment` impedisce di pagare a un cliente che ha **esattamente** due dollari: la condizione corretta è `>=`. Infine, in un'applicazione reale le somme di denaro non si rappresentano con `float`, che introduce errori di arrotondamento, ma con `BigDecimal` o con un intero di centesimi.

Perché è meglio

1. **Modella meglio la realtà.** Il codice del `Paperboy` ora *chiede* al cliente un pagamento, e non ha accesso diretto al portafoglio.
2. **Isola i cambiamenti.** La classe `Wallet` può cambiare e il ragazzo ne è completamente isolato. Se cambiasse l'interfaccia di `Wallet`, andrebbe aggiornato solo `Customer`; finché l'interfaccia di `Customer` resta la stessa, nessun suo cliente si accorgerà che ha un portafoglio nuovo. Il codice è più manutenibile perché le modifiche non si **propagano a catena** in un grande progetto.
3. **Libera l'implementazione.** È forse la ragione più «a oggetti»: ora siamo liberi di cambiare l'implementazione di

`getPayment()`. Nella realtà, quando passa il ragazzo, il cliente potrebbe prendere i due dollari da un barattolo di monete, cercarli fra i cuscini del divano o farseli prestare dal coinquilino. Tutto questo è **logica di business** che non riguarda il ragazzo: può stare dentro `getPayment()` e cambiare in futuro senza toccare il codice del `Paperboy`.

Ma Customer non è diventato più complesso?

Un'obiezione legittima: abbiamo reso `Customer` più complesso. Ma nella prima versione era il `Paperboy` a dover conoscere e manipolare il `Wallet`. Se `Customer` è più complesso, perché i suoi clienti ora sono **meno** complessi? Perché la complessità è sempre la stessa: è solo **contenuta** meglio nell'oggetto giusto ed esposta in modo sano. Prima esisteva nel codice dei clienti, magari ripetuta in più punti; ora compare **una volta sola**, e si mantiene nello stesso posto in cui avvengono i cambiamenti che potrebbero romperla.

La Legge di Demetra

La **Legge di Demetra** (*Law of Demeter*) è un'altra linea guida fondamentale per ottenere un basso accoppiamento. Prende il nome dal progetto Demeter, uno dei primi grandi sistemi a oggetti, sviluppato a Boston alla fine degli anni Ottanta (le slide indicano la Boston University, ma il gruppo di Karl Lieberherr che formulò la legge lavorava alla Northeastern University).

L'idea di fondo è spingere l'**information hiding** al massimo: niente catene del tipo `foo.getA().getB().getC()`. Più lungo è il percorso di chiamate che il programma attraversa, più il programma è fragile ai cambiamenti: ogni classe lungo la catena è una classe da cui dipendiamo.

Legge di Demetra

Un metodo M di un oggetto O dovrebbe inviare messaggi **solo** a:

1. l'oggetto stesso (`this`);
2. i **parametri** di M;
3. gli oggetti **creati** all'interno di M;
4. i **componenti diretti** di O, cioè i suoi attributi.

In breve: *parla solo con i tuoi amici più stretti, non con gli amici degli amici.*

Esempio 10.7 — Chiamate permesse e vietate

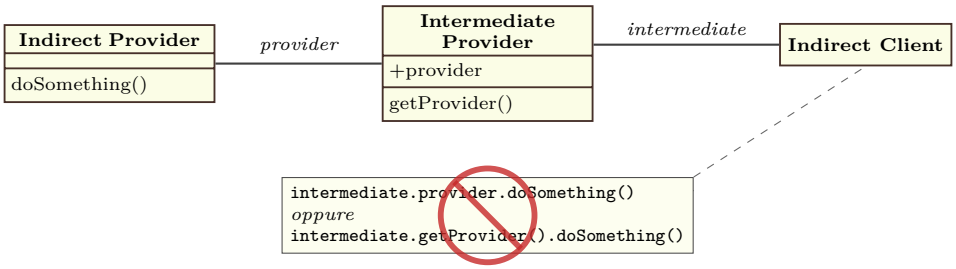
```
class Driver {
  private Car car;                        // componente diretto

  void drive(Road road) {                 // parametro
    this.checkMirrors();                  // OK (1): this
    road.getSpeedLimit();                 // OK (2): parametro
    Route r = new Route(); r.compute();   // OK (3): creato qui
    car.increaseSpeed();                  // OK (4): componente diretto
    car.getEngine().getCarburator()
       .openFuelValve();                  // NO: oggetti estranei
  }
}
```

L'ultima chiamata attraversa `Car`, `Engine` e `Carburator`: `Driver` dipende così da tre classi invece che da una, e da come sono collegate fra loro.

Com'è fatta una violazione

In figura un *client* raggiunge un fornitore indiretto passando attraverso un intermediario. Le due scritture nella nota sono equivalenti: usare un *getter* invece dell'accesso diretto all'attributo **non** rimedia alla violazione, perché il client continua a conoscere la struttura interna dell'intermediario.



Eliminare il codice di navigazione

La correzione consiste nel **spostare il comportamento** verso l'oggetto che possiede i dati, un passo alla volta. L'esempio (da Oscar Nierstrasz) è un'auto che accelera aprendo la valvola del carburatore.

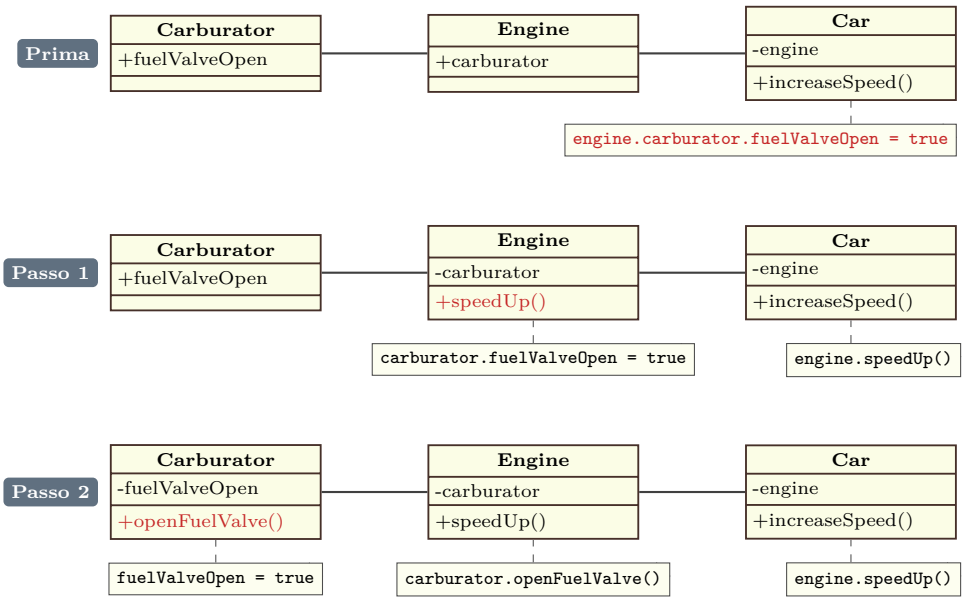


Figura 10.4: Eliminare il codice di navigazione. A ogni passo un oggetto smette di frugare nella struttura di un altro e gli chiede invece un servizio (in rosso i metodi aggiunti; nelle note il codice dei metodi).

- **Prima**: `Car` attraversa `Engine` e scrive direttamente l'attributo del `Carburator`. Conosce la struttura interna di due classi.
- **Passo 1**: `Engine` offre il servizio `speedUp()`. `Car` parla solo con il suo componente diretto, l'`engine`; l'attributo `carburator` può diventare privato.
- **Passo 2**: anche `Carburator` offre un servizio, `openFuelValve()`, e il suo attributo diventa privato. Ora ogni classe conosce solo i propri vicini immediati.

Il Paperboy e Demetra

La prima versione del `Paperboy` violava proprio la Legge di Demetra: `myCustomer.getWallet().subtractMoney(...)` manda un messaggio al `Wallet`, che non è né un parametro né un componente del ragazzo. La correzione è la stessa dell'auto: spostare il comportamento (`getPayment()`) nell'oggetto che possiede i dati.
Attenzione a non ridurre la regola al conteggio dei punti: `builder.append("a").append("b")` o una catena di operazioni su uno *stream* non violano la Legge di Demetra, perché ogni chiamata restituisce lo stesso oggetto o un oggetto dello stesso tipo, e non si naviga nella struttura interna di oggetti estranei.

PROGETTAZIONE GUIDATA DALLE RESPONSABILITÀ

La **progettazione guidata dalle responsabilità** (*responsibility-driven design*) ragiona sul sistema in termini di tre concetti: **classe**, **responsabilità** e **collaborazione**. Mette insieme le regole viste finora.

ogni classe ha responsabilità e ruoli **chiaramente definiti**	la classe che **possiede i dati** è responsabile di elaborarli	ogni classe è responsabile della **gestione dei propri dati**
alta coesione	**basso accoppiamento**	**incapsulamento**

Lo strumento che facilita questo modo di ragionare sono le **schede CRC**, già incontrate nel Capitolo 7 per identificare le classi del modello di dominio.

Le schede CRC

Le schede CRC (*Class–Responsibility–Collaboration*) sono state introdotte da Kent Beck e Ward Cunningham per insegnare la progettazione a oggetti. A ogni classe del diagramma delle classi di progetto si associa una scheda che riporta:

- il **nome** della classe;
- le sue **responsabilità**: le conoscenze che la classe mantiene o i servizi che offre;
- i suoi **collaboratori**: le altre classi di cui la classe deve sfruttare conoscenze o servizi per assolvere le proprie responsabilità.

Nome della classe	
Superclassi	
Sottoclassi	
Responsabilità	Collaboratori

La scheda CRC completa delle slide.

Rispetto alla versione del Capitolo 7 questa scheda riporta anche **superclassi** e **sottoclassi**: in progettazione la gerarchia è decisa, e va tenuta sotto controllo perché l'ereditarietà è la forma di accoppiamento più forte.
Le schede CRC hanno due conseguenze pratiche, che ne fanno uno strumento di **diagnosi**:

- se non si riescono a specificare con chiarezza le responsabilità di una classe, c'è un **errore di progetto** (bassa coesione);
- se una classe ha bisogno di **troppi collaboratori**, c'è un potenziale errore di progetto (alto accoppiamento).

Esempio 10.8 — Le schede CRC del Paperboy

Ecco le schede della versione migliorata. Si noti che `Wallet` non compare fra i collaboratori del `Paperboy`: la scheda rende visibile a colpo d'occhio la riduzione dell'accoppiamento.

Paperboy		**Customer**	
consegna i giornali riscuote il pagamento	Customer	conosce nome e cognome paga un conto (`getPayment`)	Wallet

Wallet	
conosce la somma contenuta aggiunge e toglie denaro	—

In sintesi

- Il **System Design** trasforma il modello di analisi nel modello di progetto, scritto per i **programmatori**: obiettivi di progetto, architettura in sottosistemi, strategie di costruzione. L'**Object Design** ne specifica poi i dettagli.
- Gli **obiettivi di progetto** derivano soprattutto dai requisiti non funzionali, vanno **formalizzati** e ordinati per priorità, perché si possono ottimizzare solo pochi criteri: memoria/velocità, tempi/funzionalità, tempi/qualità, tempi/personale sono i compromessi tipici.
- Si **scompone** perché $C(P_1 + P_2) > C(P_1) + C(P_2)$; ogni sottosistema offre i suoi servizi tramite un'**interfaccia**. Il numero di componenti ottimale bilancia costo di sviluppo e costo di integrazione.
- L'obiettivo è l'**indipendenza funzionale**: **alta coesione** (una sola responsabilità, un solo motivo per cambiare)

e **basso accoppiamento**.

- La coesione va dalla **casuale** (la più debole) alla **funzionale** e **a oggetti** (le più forti); la **God Class**, che concentra in sé tutte le responsabilità, è l'antipattern da evitare.
- L'accoppiamento nasce da attributi, parametri e variabili, **ereditarietà** (il più forte) e interfacce; le **interfacce** con l'iniezione delle dipendenze lo riducono.
- La **Legge di Demetra**: un metodo parla solo con `this`, i parametri, gli oggetti che crea e i componenti diretti. Le catene `a.getB().getC()` si eliminano spostando il comportamento verso chi possiede i dati.
- Nella **progettazione guidata dalle responsabilità** le **schede CRC** servono da diagnosi: responsabilità poco chiare indicano bassa coesione, troppi collaboratori un accoppiamento alto.

Architetture software

Nel capitolo precedente abbiamo visto perché e come scomporre un sistema in sottosistemi, guidati da coesione e accoppiamento. Qui facciamo un passo in più: vediamo come i sottosistemi si **organizzano** in una struttura complessiva, l'**architettura**, e quali sono le architetture standard da cui si parte nella pratica. Nello schema degli otto aspetti del System Design (Capitolo 10) ci troviamo al passo 2, *scomposizione del sistema*: livelli e partizioni.

Che cos'è un'architettura software

Architettura software

L'**architettura software** di un sistema è la sua struttura (o l'insieme delle sue strutture), formata da **elementi software**, dalle loro **proprietà visibili dall'esterno** e dalle **relazioni** fra di essi.
Quali strutture considerare, e come rappresentarle, dipende dagli obiettivi di ingegneria che si perseguono.

La definizione, dovuta a Bass, Clements e Kazman (*Software Architecture in Practice*), ha tre conseguenze importanti.

1. L'architettura è un'astrazione

L'architettura definisce gli elementi e il modo in cui **interagiscono**, e tace di proposito sui dettagli puramente locali: ciò che è privato di un elemento **non** è architetturale. Contano invece le proprietà **visibili dall'esterno**, cioè le ipotesi che un elemento può fare su un altro:

- i servizi **forniti** e i servizi **richiesti**;
- le caratteristiche di **prestazione**;
- la **gestione dei guasti**;
- l'**uso delle risorse**.

Sono le stesse informazioni che, nel diagramma dei componenti del Capitolo 9, stanno nelle interfacce fornite e richieste.

2. Ogni sistema ha un'architettura

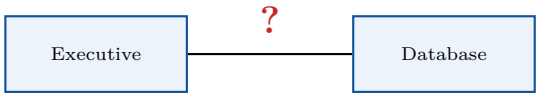
Ogni sistema è composto di elementi in relazione fra loro, quindi ha un'architettura; nel caso limite, un solo elemento in relazione solo con sé stesso. Ma **avere** un'architettura è diverso dall'avere un'architettura **nota a tutti**: l'architettura è una cosa, la sua **specifica** un'altra.

Se non la progetti, ne avrai una comunque

Se non si sviluppa esplicitamente un'architettura, il sistema ne avrà comunque una: quella che emerge dalle scelte prese giorno per giorno da chi scrive il codice. E potrebbe non piacerci affatto.

3. Un disegno a scatole e frecce non basta

Ne segue che un disegno fatto di **scatole e linee** non è, da solo, un'architettura: è soltanto un punto di partenza. Vedendo una scatola con scritto «database» si può immaginare che cosa faccia, ma niente di più.



Che servizi offre *Database*? Chi chiama chi? Quante richieste al secondo regge? Che succede se non risponde? Senza queste **specifiche e proprietà** il disegno non dice nulla di verificabile.

Inoltre un sistema ha **molte strutture** (o *viste*): la scomposizione in moduli, i componenti in esecuzione, l'allocazione sui nodi hardware… Nessuna struttura da sola *è* l'architettura, e l'insieme delle strutture da considerare non è fissato: si sceglie ciò che serve per l'analisi, la comunicazione o la comprensione. I diagrammi UML visti finora ne sono un esempio: package (Capitolo 8), componenti e deployment (Capitolo 9) sono viste diverse della stessa architettura.

L'architettura nel System Design

L'architettura di un sistema si definisce nella prima fase del System Design, la **progettazione architetturale**. Lo scopo principale è scomporre il sistema in sottosistemi, perché:

- realizzare più componenti distinti è meno complesso che realizzare un sistema monolitico;
- permette lo **sviluppo in parallelo**;
- favorisce modificabilità, riusabilità, portabilità.

Definire un'architettura significa **assegnare le funzionalità ai moduli**: il modulo dell'interfaccia utente, quello di accesso al database, quello della sicurezza, e così via. Anche qui valgono **alta coesione** e **basso accoppiamento**: ogni parte dell'architettura deve offrire servizi fortemente correlati, limitando il numero di altri moduli a cui è legata.

La definizione si svolge da due punti di vista, che insieme portano alla soluzione finale:

1. l'**identificazione dei sottosistemi** e delle loro relazioni;
2. la definizione delle **politiche di controllo** (chi avvia le operazioni, come scorre il flusso di controllo).

Entrambe le scelte sono cruciali, perché sono **difficili da cambiare** a sviluppo avviato: bisognerebbe modificare le interfacce di molti sottosistemi.

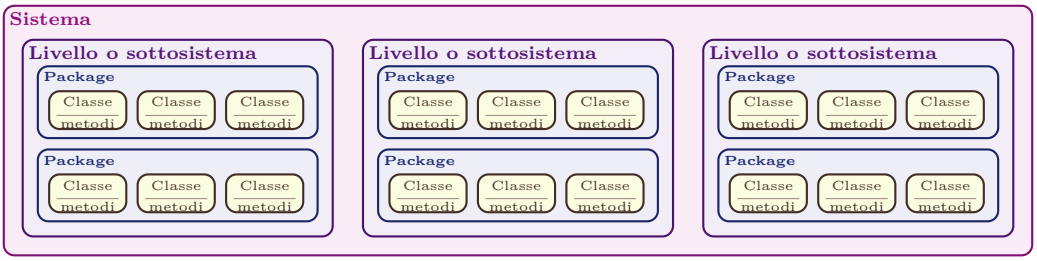
Identificare i sottosistemi

Divide et impera

Affrontare qualcosa di grande tutto in una volta è più difficile che affrontare un insieme di cose più piccole. Dividere il sistema porta quattro vantaggi:

- ogni componente è più piccolo, e quindi più facile da **capire**;
- le parti si possono **sostituire o modificare** senza dover sostituire o modificare profondamente le altre;
- persone diverse possono lavorare su **parti diverse**;
- un singolo ingegnere del software può **specializzarsi**.

La divisione procede per livelli successivi:



- un **sistema** si divide in **livelli** (*layer*) e **sottosistemi**;
- un sottosistema si divide in uno o più **package**;
- un package si divide in **classi**;
- una classe si divide in **metodi**.

Due dimensioni: sottosistemi e livelli

Sottosistemi e livelli sono due modi **ortogonali** di tagliare un sistema.

- La **dimensione funzionale** divide il sistema per *funzioni*: in un'auto, il sistema di alimentazione, quello di propulsione, quello elettrico, quello frenante.
- La **dimensione di astrazione** (o di implementazione) lo divide per *livelli*: dai servizi più astratti, vicini all'utente, a quelli più concreti, vicini all'hardware.

Ogni pezzo del sistema sta all'incrocio di una funzione e di un livello. Un sistema ben progettato è **coerente in entrambe le dimensioni**: un esempio reale è la mappa del kernel Linux, che dispone le funzioni del kernel in colonne e i livelli in righe.

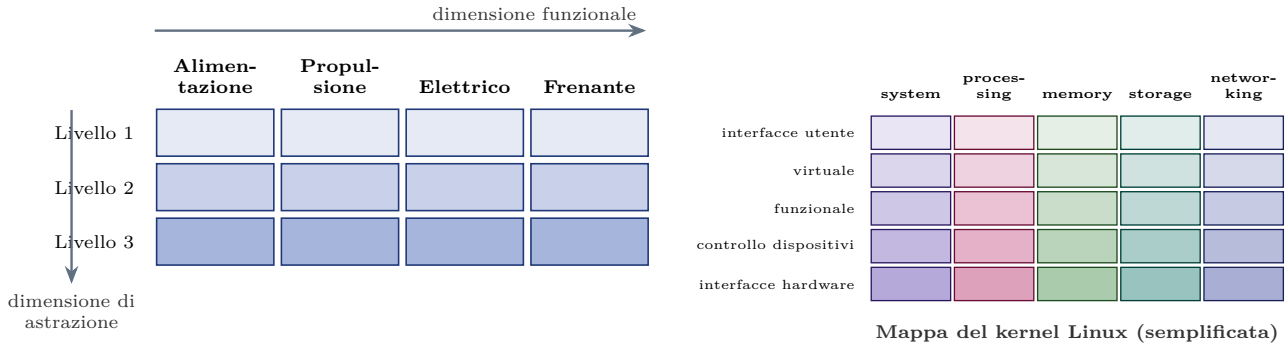


Figura 11.1: A sinistra, i sottosistemi di un'auto (colonne) incrociati con i livelli di astrazione (righe). A destra lo stesso schema nel kernel Linux: funzioni in colonna, livelli in riga.

L'ARCHITETTURA A LIVELLI

Scomposizione gerarchica in livelli

Una scomposizione gerarchica di un sistema è un insieme **ordinato** di **livelli** (*layer*). Un livello è un raggruppamento di sottosistemi che offrono servizi correlati, eventualmente realizzati usando i servizi di altri livelli.

- Un livello può dipendere **solo dai livelli inferiori**.
- Un livello **non sa nulla** dei livelli superiori.

La prima formalizzazione dell'architettura a livelli si deve a **Edsger Dijkstra**, con il sistema operativo *THE* (sviluppato a partire dal 1965 e descritto nel 1968): il sistema va costruito come un insieme di **macchine virtuali**, ognuna costruita **sopra** quella sottostante. Ogni livello offre al livello superiore una «macchina» più comoda di quella che lui stesso riceve dal livello inferiore.

A seconda di quali livelli si possono usare, l'architettura è **chiusa** o **aperta**.

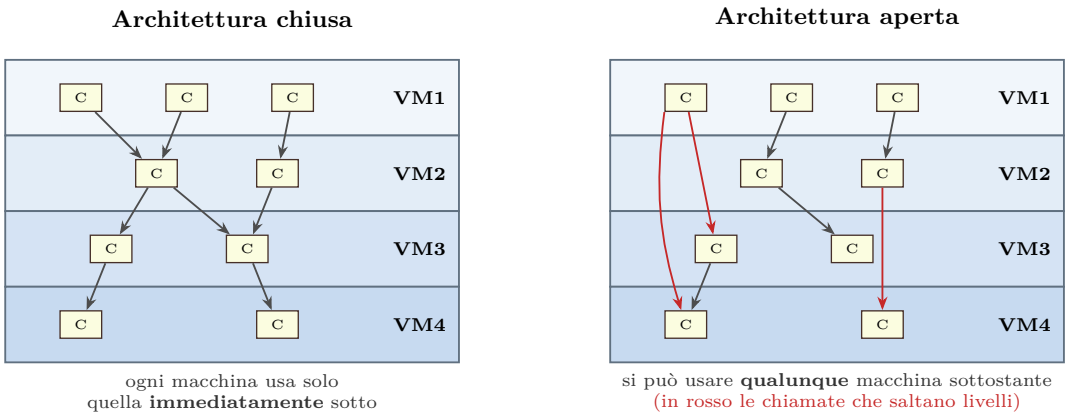


Figura 11.2: Il sistema come pila di macchine virtuali (VM1 in cima, VM4 in fondo).

Architettura chiusa

Ogni livello può chiamare **solo** le operazioni del livello **immediatamente** sottostante.
Obiettivo di progetto: alta **manutenibilità** e **portabilità**. Si può sostituire un livello toccando solo quello sopra; per portare il sistema su un'altra piattaforma basta riscrivere i livelli più bassi.

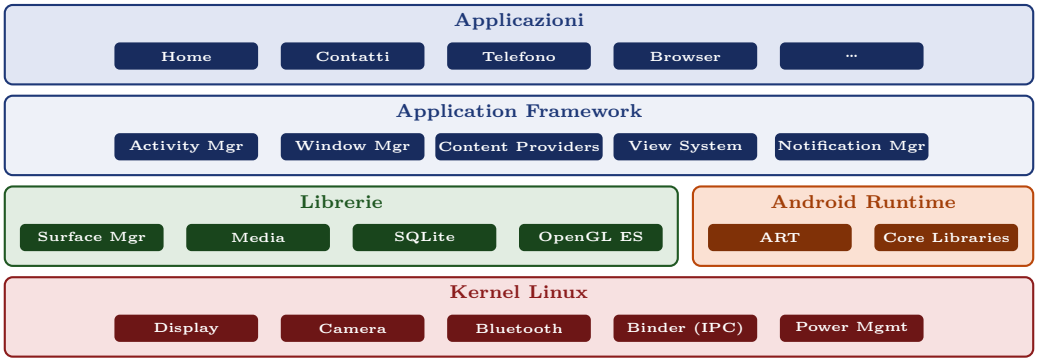
Architettura aperta

Ogni livello può usare i servizi di **tutti** i livelli sottostanti.
Obiettivo di progetto: **efficienza** a tempo di esecuzione. Si evitano i passaggi intermedi, ma ogni livello dipende da più livelli: cambiare quelli in basso costa di più.

È un'applicazione diretta del Capitolo 10: l'architettura chiusa **riduce l'accoppiamento** (ogni livello conosce un solo vicino, come nella Legge di Demetra), quella aperta lo aumenta in cambio di prestazioni.

Esempio 11.1 — Android, un'architettura (quasi) chiusa

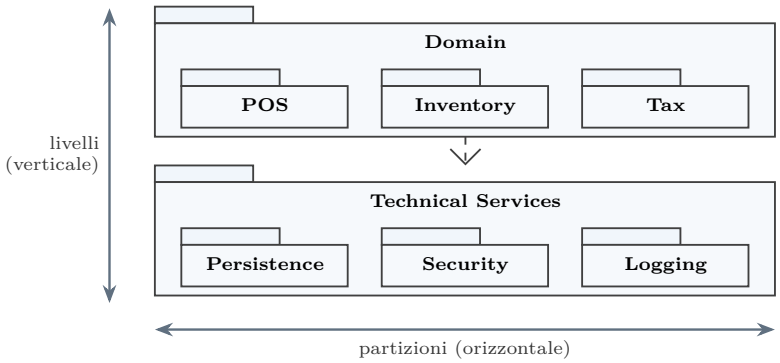
Lo stack del sistema operativo Android è un esempio classico di architettura a livelli: le applicazioni usano il *framework* applicativo, che usa le librerie e il runtime, che si appoggiano al kernel Linux.



Un'app normale non parla con il kernel né con i driver: chiede i servizi al framework (ad esempio la fotocamera), che li ottiene dai livelli sottostanti. A rigore l'architettura non è chiusa al cento per cento: con l'*Android NDK* un'app può usare direttamente librerie native. È la regola, non la legge.

Livelli e partizioni

Dentro un'architettura a livelli, ogni livello può a sua volta essere diviso in **partizioni**: sottosistemi **allo stesso livello** di astrazione, che si occupano di funzioni diverse. Nell'esempio di Larman (un sistema di cassa, POS), il livello del dominio è partizionato in *POS*, *Inventory* e *Tax*; quello dei servizi tecnici in *Persistence*, *Security* e *Logging*.



LE PRINCIPALI ARCHITETTURE

L'ingegneria del software ha codificato diversi **stili architetturali**, da usare come base per progettare un sistema specifico:

- architettura **client/server**;
- architettura a **repository**;
- architettura **a livelli**;
- architettura **three-tier** e **multi-tier**;
- architettura **orientata ai servizi** (SOA);
- ...

Client/server

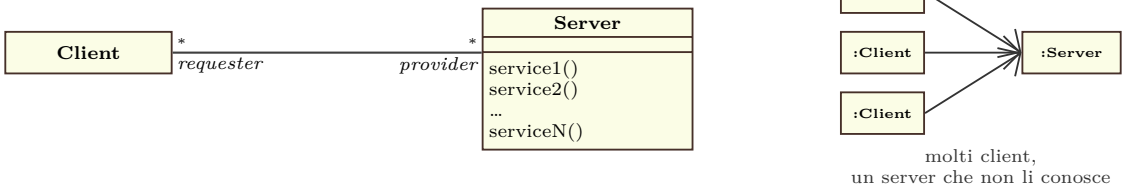
Architettura client/server

Un'architettura **distribuita** in cui dati ed elaborazione sono ripartiti in una rete fra due tipi di nodi.

- I **server** sono elaboratori potenti e dedicati, che offrono servizi specifici: stampa, gestione del file system, compilazione, gestione del traffico di rete, calcolo.
- I **client** sono macchine meno potenti, su cui girano le applicazioni dell'utente, che usano i servizi dei server.

Dal punto di vista della progettazione, un sottosistema detto **server** fornisce servizi a istanze di altri sottosistemi, i **client**, che si occupano dell'interazione con l'utente.

- I client chiamano il server, che esegue dei servizi e restituisce il risultato; la risposta di solito è **immediata**.
- I client conoscono l'**interfaccia** del server (i suoi servizi), il nome e la natura dei server.
- I server **non** conoscono le interfacce dei client, e non hanno bisogno di conoscerne né l'identità né il numero: servono chiunque faccia una richiesta.
- Gli utenti interagiscono **solo con il client**.



L'esempio più familiare è il **Web**: il browser è il client, il server web risponde a chiunque gli invii una richiesta HTTP, senza sapere in anticipo chi o quanti saranno.

Esempio 11.2 — Un server che non conosce i suoi client

Il più piccolo server possibile: offre un solo servizio (l'ora esatta) a chiunque si colleghi, senza sapere chi sia.

ServerOrario.java

```java
try (ServerSocket server = new ServerSocket(5000)) {
  while (true) {
    try (Socket client = server.accept();                    // chiunque, in qualunque numero
         PrintWriter risposta = new PrintWriter(client.getOutputStream(), true)) {
      risposta.println(LocalTime.now());                     // il servizio
    }
  }
}
```

Il client, invece, deve sapere dove si trova il server e che cosa offre (la sua interfaccia: «collegati alla porta 5000 e leggi una riga»):

ClientOrario.java

```java
try (Socket server = new Socket("orologio.example.it", 5000);
     BufferedReader richiesta = new BufferedReader(new InputStreamReader(server.getInputStream()))) {
  System.out.println("Sono le " + richiesta.readLine());
}
```

Nei sistemi reali al posto dei socket si usa quasi sempre HTTP, con le API REST del Capitolo 14; lo schema resta questo.

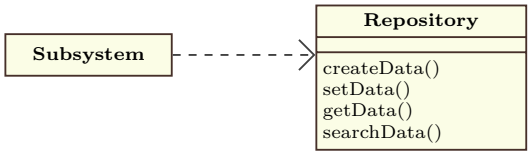
Repository

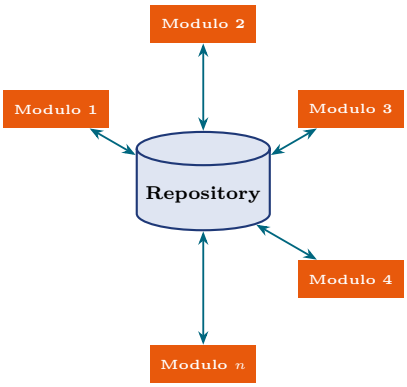
Architettura a repository

I sottosistemi accedono e modificano un'**unica struttura dati** condivisa, il **repository**. I sottosistemi sono «relativamente indipendenti»: interagiscono **solo attraverso il repository**.

Il flusso di controllo può essere dettato:

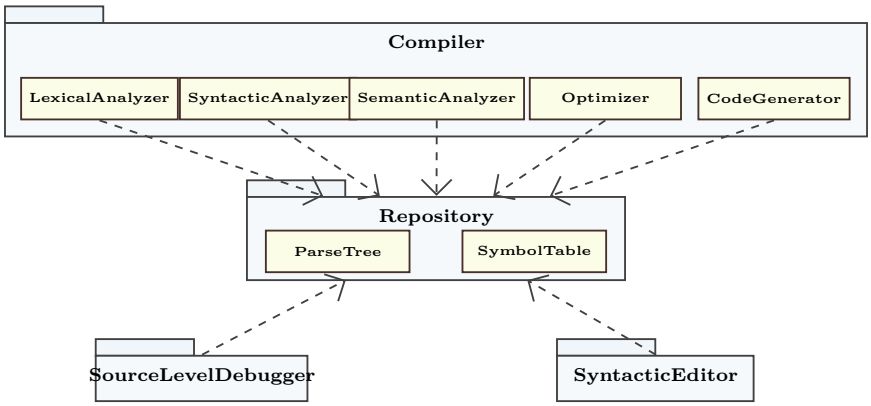
- dal **repository**: è una modifica ai dati memorizzati a far partire un sottosistema (in questo caso si parla anche di architettura a *blackboard*, a lavagna);
- dai **sottosistemi**: ognuno ha il proprio flusso di controllo indipendente, e usa il repository quando gli serve.





Esempio 11.3 — Il compilatore e i DBMS

Due esempi tipici di architettura a repository sono i **DBMS** e i **compilatori moderni**. In un compilatore, analizzatore lessicale, sintattico e semantico, ottimizzatore e generatore di codice non si chiamano fra loro: leggono e scrivono l'**albero sintattico** e la **tabella dei simboli**. Allo stesso repository attingono anche strumenti esterni come il debugger e l'editor sintattico.



Aggiungere un nuovo strumento (un formattatore del codice, un analizzatore di stile) non richiede di modificare il compilatore: basta che legga lo stesso repository.

Vantaggi

- Modo efficiente di condividere **grandi quantità di dati**: si scrivono una volta, tutti li leggono.
- Un sottosistema non deve preoccuparsi di come i dati sono prodotti o usati dagli altri.
- Gestione **centralizzata** di backup, sicurezza, controllo degli accessi, ripristino dagli errori.
- Il modello dei dati condivisi è pubblicato come **schema** del repository: è facile aggiungere nuovi sottosistemi.

Svantaggi

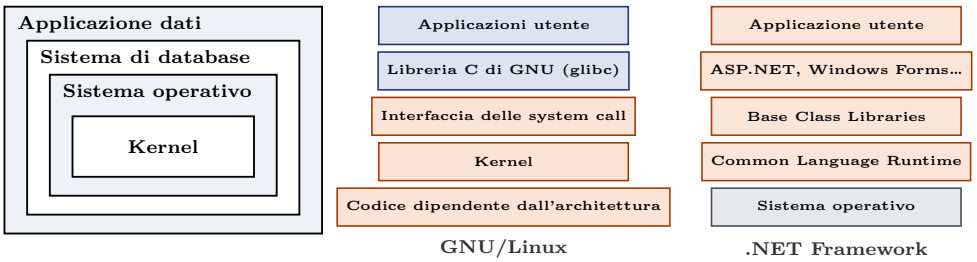
- I sottosistemi devono accordarsi su un modello dei dati di **compromesso**: prestazioni più basse.
- **Evoluzione dei dati**: adottare un nuovo modello è difficile e costoso, perché va applicato all'intero repository e vanno aggiornati tutti i sottosistemi.
- Sottosistemi diversi possono avere esigenze diverse di backup e sicurezza, che il repository non supporta.
- È difficile **distribuire** il repository su più macchine continuando a vederlo come logicamente centralizzato: problemi di ridondanza e di consistenza dei dati.

A livelli

Architettura a livelli

I sottosistemi sono organizzati in **livelli**. Ogni livello:

- usa i servizi del livello sottostante;
- fornisce servizi al livello superiore attraverso **interfacce ben definite**.



Vantaggi

- **astrazione** delle responsabilità (*separation of concerns*);
- livelli inferiori e superiori **isolati** gli uni dagli altri (disaccoppiati);
- **riusabilità** dei livelli.

Svantaggi

- **sovraccarico** (*overhead*) dovuto all'attraversamento di più livelli;
- maggiore **complessità**.

Layer e tier

Le slide osservano che per la maggior parte delle persone i termini *layer* e *tier* sono intercambiabili. Molti autori, però, li distinguono: un **layer** è una suddivisione **logica** del codice (presentazione, logica, dati), un **tier** una separazione **fisica**, cioè su processi o macchine diversi. Un'applicazione può avere tre layer e girare tutta su un solo tier, oppure distribuire ogni layer su un tier diverso, come nell'architettura che vediamo ora.

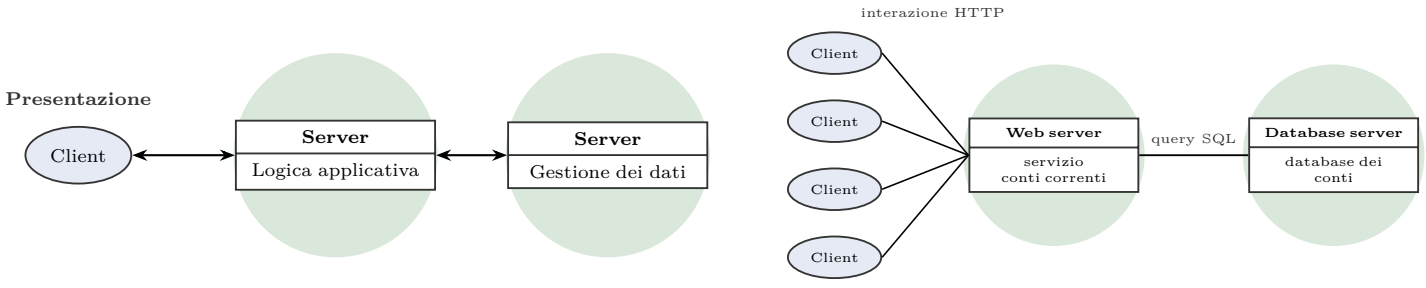
Three-tier

L'architettura **three-tier**, introdotta negli anni Novanta, gestisce **esplicitamente** la logica applicativa (*business logic*) in un livello dedicato. I tre livelli sono:

1. **gestione dei dati**: comunicazione con il DBMS, file XML, …;
2. **logica applicativa**: elaborazione dei dati, regole del dominio;
3. **interfaccia utente**: presentazione dei dati e dei servizi.

Ogni livello ha i propri obiettivi e vincoli di progetto, e **nessun livello fa ipotesi** sulla struttura o sull'implementazione degli altri: il livello 2 non sa come sono rappresentati i dati né com'è fatta l'interfaccia; il livello 3 non sa come funziona la logica applicativa.

- Non c'è comunicazione diretta fra il livello 1 e il livello 3: l'interfaccia utente non riceve né inserisce dati direttamente nel livello dei dati, e ogni informazione, in entrambe le direzioni, è **filtrata dalla logica applicativa**. È un'architettura a livelli **chiusa**.
- I livelli funzionano senza presupporre di far parte di un'applicazione specifica: le applicazioni sono viste come **collezioni di componenti che cooperano**, e ogni componente può far parte contemporaneamente di applicazioni diverse (ad esempio lo stesso database, o lo stesso componente di logica).



Un'architettura three-tier client/server — Un sistema di internet banking

Figura 11.3: L'architettura three-tier: in generale (a sinistra) e in un sistema di internet banking (a destra).

Vantaggi

Flessibilità e modificabilità, grazie ai componenti separati:
- i componenti si possono usare in sistemi diversi;
- modificare un componente non tocca il resto del sistema (a meno che cambi la sua API);
- la ricerca dei bug è più mirata, perché le funzionalità sono separate e isolate;
- aggiungere funzionalità significa estendere solo i componenti coinvolti, o aggiungerne di nuovi.

Interconnettività:
- le API dei componenti evitano gli adattatori del modello client/server: N interfacce diverse possono collegarsi allo stesso servizio;
- applicazioni diverse accedono facilmente ai dati comuni (logiche diverse sullo stesso gestore dei dati).

Gestione dei sistemi distribuiti: la logica applicativa si può aggiornare senza aggiornare i client.

Svantaggi

Dimensione ed efficienza delle applicazioni:
- uso intenso della comunicazione di rete, quindi **latenza** dei servizi;
- la comunicazione fra componenti richiede librerie per lo scambio di informazioni, quindi codice voluminoso.

Integrazione del software *legacy*:
- molte aziende gestiscono i propri dati con vecchio software monolitico;
- è difficile applicare il modello three-tier alle nuove applicazioni che devono usarlo;
- servono **adattatori** per interfacciarsi con il software legacy.

Esempio 11.4 — Tre livelli, in codice

Ogni livello è una classe (in un sistema reale, un package o un componente) che conosce solo quella sotto.

OrdineController.java
```java
public class OrdineController {                    // interfaccia utente
  private final OrdineService service;

  public String mostra(long id) {
    return VistaOrdine.html(service.dettaglio(id));
  }
}
```

OrdineService.java
```java
public class OrdineService {                       // logica applicativa
  private final OrdineRepository repository;

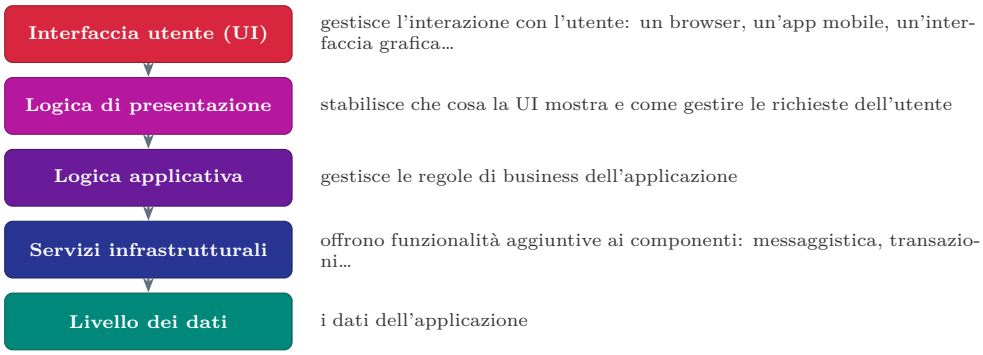
  public Ordine dettaglio(long id) {
    Ordine ordine = repository.trova(id).orElseThrow();
    ordine.applicaScontiInVigore();                // le regole del dominio stanno qui
    return ordine;
  }
}
```

OrdineRepository.java
```java
public class OrdineRepository {                    // gestione dei dati
  public Optional<Ordine> trova(long id) {
    // SELECT ... FROM ordini WHERE id = ?
  }
}
```

Se il controller chiamasse direttamente il repository «per fare prima», l'architettura diventerebbe **aperta**, e l'ordine mostrato all'utente salterebbe le regole del dominio: in questo caso, gli sconti.

Multi-tier

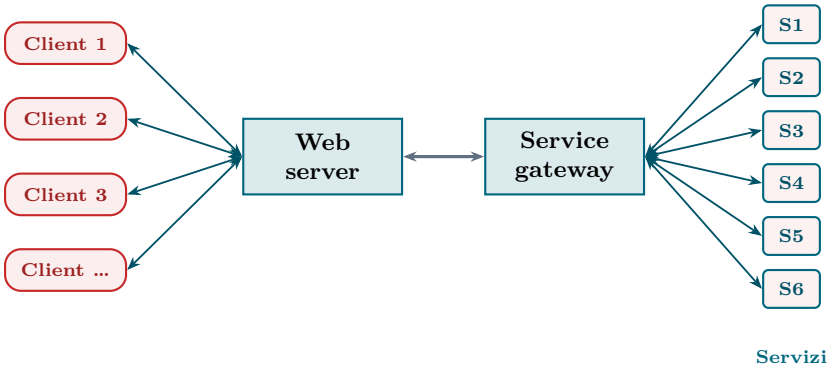
L'architettura **multi-tier** è l'evoluzione della three-tier su N livelli, e ammette configurazioni diverse. I suoi elementi fondamentali sono cinque.



Orientata ai servizi (SOA)

Nell'architettura **orientata ai servizi** (*Service-Oriented Architecture*, SOA) il sistema è un insieme di **servizi** indipendenti, ciascuno con una propria interfaccia, a cui i client accedono attraverso un punto di ingresso comune.

- I servizi sono componenti **senza stato** (*stateless*): non conservano informazioni fra una richiesta e l'altra, quindi si possono **replicare** e **spostare** da un computer all'altro.
- Alla fornitura dei servizi possono partecipare **molti server**.
- Un'architettura orientata ai servizi è di solito più facile da **scalare** quando la domanda cresce (basta aggiungere repliche dei servizi più richiesti) ed è **resiliente** ai guasti (se una replica cade, ne risponde un'altra).



Il *service gateway* smista ogni richiesta al servizio competente: i client vedono un unico punto di accesso e non sanno quante repliche ci siano dietro, né su quali macchine girino.

Confronto

Stile	Idea	Punti di forza	Punti deboli	Esempi
Client/server	i server offrono servizi, i client li chiamano	servizi centralizzati, client semplici	il server è un collo di bottiglia	Web, server di stampa
Repository	i sottosistemi comunicano solo tramite dati condivisi	condivisione di grandi moli di dati, nuovi sottosistemi facili da aggiungere	modello dei dati difficile da cambiare e da distribuire	compilatori, DBMS
A livelli	ogni livello usa solo quelli sottostanti	astrazione, isolamento, riuso	overhead, complessità	sistemi operativi, Android
Three-tier	presentazione, logica e dati su livelli separati	flessibilità, interconnettività	latenza di rete, integrazione del legacy	internet banking, e-commerce
SOA	servizi senza stato dietro un gateway	scalabilità, resilienza ai guasti	complessità di gestione e comunicazione	grandi applicazioni web

In sintesi

- L'**architettura software** è l'insieme delle strutture di un sistema: elementi, proprietà visibili dall'esterno e relazioni. È un'astrazione, ogni sistema ne ha una (meglio progettarla), e un disegno a scatole e frecce non basta.
- Si definisce nella **progettazione architetturale**, identificando i sottosistemi e le politiche di controllo: scelte difficili da cambiare in seguito.
- Il sistema si divide in **livelli** e **sottosistemi**, poi in package, classi e metodi; funzioni e livelli sono due dimensioni ortogonali.
- In un'architettura a livelli ogni livello dipende solo da quelli inferiori. **Chiusa** (solo il livello immediatamente sotto): manutenibilità e portabilità. **Aperta** (tutti i livelli sotto): efficienza.
- Gli stili principali sono **client/server**, **repository**, **a livelli**, **three-tier** e **multi-tier**, **SOA**: ognuno con vantaggi e svantaggi da pesare rispetto agli obiettivi di progetto.

Architetture software moderne

Il capitolo precedente ha presentato gli stili architetturali classici: client/server, repository, livelli, three-tier, SOA. Qui guardiamo le architetture dei sistemi **moderni**, che devono servire milioni di utenti, non fermarsi mai ed evolvere ogni giorno. Il filo conduttore sono le **proprietà** che l'architettura determina (scalabilità, disponibilità, consistenza dei dati) e le tecniche per ottenerle: bilanciamento del carico, replica, code di messaggi, microservizi. Chiudiamo con un tema meno tecnico ma altrettanto decisivo: il legame fra l'architettura e l'**organizzazione** dei team.

Che cos'è, di nuovo, un'architettura

Nel Capitolo 11 abbiamo definito l'architettura come l'insieme delle strutture di un sistema. Martin Fowler ne dà una definizione complementare, centrata sulle **decisioni**.

Architettura software (Fowler)

L'architettura di un sistema software è l'insieme delle **decisioni di progetto principali** sul sistema: il progetto di massima (*blueprint*) della sua costruzione e della sua evoluzione. Sono decisioni che hanno un impatto significativo sulla struttura del sistema, sul suo comportamento e sulle sue interazioni, e sulle sue proprietà non funzionali.

«Principale» indica un grado di importanza tale da dare a una decisione lo **status di decisione architetturale**. Ma quale lo è?

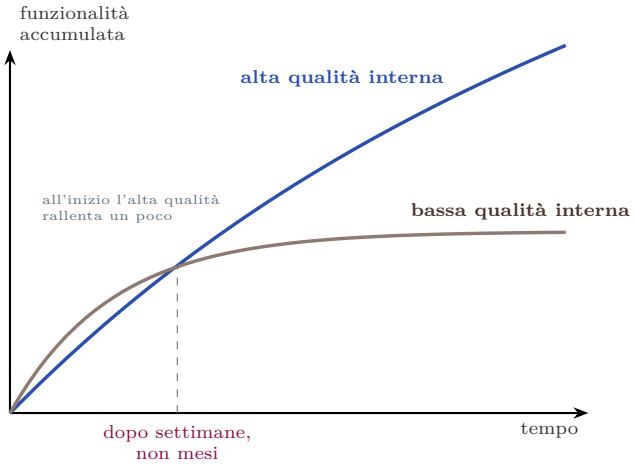
- Scegliere fra MariaDB e MySQL è una decisione architetturale? Probabilmente no: sono due database relazionali quasi intercambiabili, e passare dall'uno all'altro costa poco.
- Scegliere fra un database **relazionale** e uno **NoSQL**? Sì: cambia il modo in cui si modellano i dati, le garanzie di consistenza, il codice che li usa. Tornare indietro costa moltissimo.

L'architettura riguarda ciò che è **difficile da cambiare in seguito**. Non esiste un criterio oggettivo per stabilire quali decisioni siano «principali»; come scrive Fowler, *«l'architettura riguarda le cose importanti, qualunque esse siano»*. Il cuore del pensare un software in modo architetturale è decidere che cosa è importante, e poi spendere energie per mantenere quegli elementi in buone condizioni. Per diventare architetto, uno sviluppatore deve saper riconoscere gli elementi importanti: quelli che, se non tenuti sotto controllo, porteranno con ogni probabilità a problemi seri.

Perché l'architettura conta

Per clienti e utenti l'architettura è un argomento ostico: è qualcosa che **non percepiscono** direttamente. Eppure ha un impatto significativo sulla struttura del sistema, sul suo comportamento e sulle sue proprietà non funzionali. Inoltre un'architettura scadente rende il sistema molto più difficile da sviluppare e da far evolvere, con conseguenze sui costi e sui tempi di consegna delle nuove funzionalità.

Il grafico, da un articolo di Fowler (*Is High Quality Software Worth the Cost?*), mostra la funzionalità accumulata nel tempo da due progetti. Quello con **bassa qualità interna** parte più veloce, ma rallenta presto: ogni nuova funzionalità costa di più, perché il codice è sempre più difficile da capire e modificare. Quello con **alta qualità interna** (una buona architettura) parte un po' più lento, ma poi consegna più in fretta e a costi minori.
Il punto chiave è **quando** le due curve si incrociano: secondo Fowler, dopo **settimane**, non mesi. La qualità interna costa di più all'inizio, ma ripaga molto presto.



Le proprietà che dipendono dall'architettura

Le proprietà su cui l'architettura incide di più sono sei. Le prime tre sono i temi delle prossime sezioni; le altre tre sono misure di prestazione che conviene distinguere bene.

Scalabilità	Disponibilità	Consistenza
Prestazioni	Latenza	Throughput

- La **latenza** è il **tempo** necessario a servire una singola richiesta (ad esempio 200 ms).
- Il **throughput** è la **quantità** di lavoro servita nell'unità di tempo (ad esempio 5000 richieste al secondo).

Le due misure sono legate ma non coincidono: un sistema può servire moltissime richieste al secondo, ciascuna lentamente.

SCALABILITÀ

Scalabilità

Un servizio è **scalabile** se, aggiungendo risorse, le sue prestazioni aumentano in modo **proporzionale** alle risorse aggiunte.

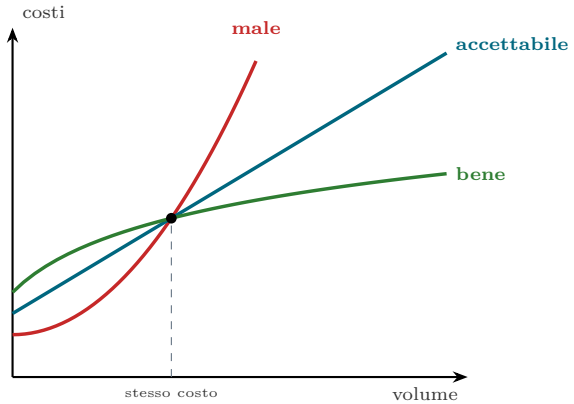
Di solito aumentare le prestazioni significa servire **più unità di lavoro** (più richieste, più utenti), ma può anche significare gestire unità di lavoro **più grandi**, ad esempio quando i dati crescono. Un modo semplice di distinguere prestazioni e scalabilità:

- se c'è un problema di **prestazioni**, il sistema è lento anche per un **singolo** utente;
- se c'è un problema di **scalabilità**, il sistema è veloce per un singolo utente, ma lento **sotto carico**.

La scalabilità è una questione di soldi. Il grafico mostra come crescono i costi al crescere del volume di lavoro (richieste servite, transazioni, utenti contemporanei…) per tre ipotetici sistemi.

- Male: costi che crescono più che proporzionalmente. Il sistema è economico finché è piccolo, ma diventa insostenibile proprio quando ha successo.
- Accettabile: costi che crescono in proporzione al volume.
- Bene: costi che crescono meno che proporzionalmente. Ogni utente in più costa meno del precedente.

Le tre curve si incrociano in un punto: a quel volume i tre sistemi costano uguale. Prima di quel punto il sistema «cattivo» è addirittura il più economico, ed è proprio questa la trappola: le differenze si vedono solo quando il volume cresce.



Scalare orizzontalmente, verticalmente, diagonalmente

Come si serve più lavoro? Le strade sono tre.

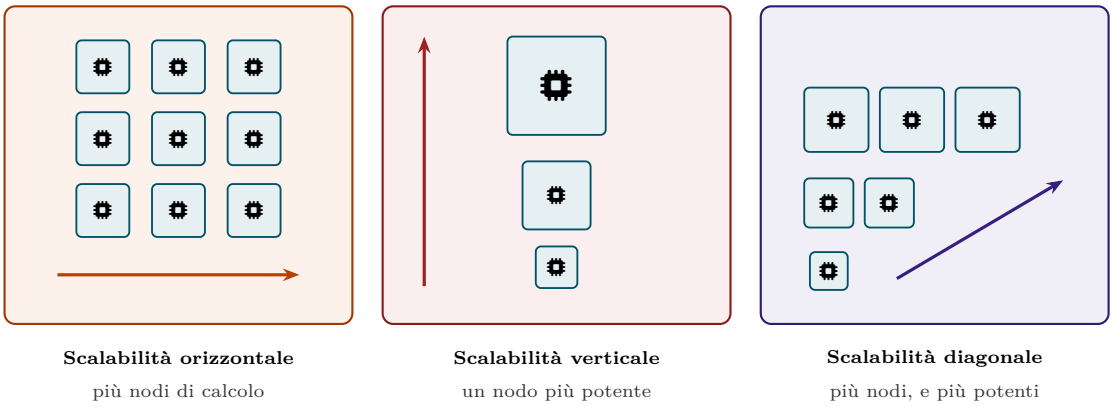


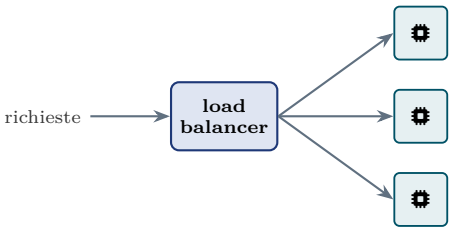
Figura 12.1: Le tre strade per servire più lavoro.

La scalabilità **verticale** è la più semplice (non cambia nulla nel software), ma ha un limite fisico ed economico: oltre una certa soglia una macchina più potente costa moltissimo, o non esiste. Quella **orizzontale** non ha limiti teorici, ma richiede che il software sappia distribuire il lavoro su più nodi.

Il bilanciamento del carico

Quando si scala orizzontalmente (o diagonalmente), serve un componente dedicato che distribuisca le richieste in arrivo fra i nodi di calcolo: il **load balancer** (bilanciatore del carico). Le strategie più comuni per scegliere il nodo sono:

- **casuale**: ogni richiesta va a un nodo scelto a caso;
- **meno carico** (*least loaded*): la richiesta va al nodo meno occupato in quel momento;
- **round robin**: i nodi si alternano a turno, uno dopo l'altro.



Il load balancer è un punto delicato

Tutto il traffico passa da lì, quindi può diventare un **collo di bottiglia**. E un singolo load balancer è un **single point of failure**: se si guasta, si ferma tutto, anche se i nodi dietro funzionano. Nei sistemi reali i load balancer stessi vengono replicati.

Esempio 12.1 — Round robin in Java, e un load balancer vero

La strategia round robin sta in poche righe. Il contatore è atomico perché le richieste arrivano da più thread insieme: con un `int` normale due richieste potrebbero leggere lo stesso valore e finire sullo stesso nodo.

RoundRobin.java

```java
public class RoundRobin {
  private final List<URI> nodi;
  private final AtomicInteger prossimo = new AtomicInteger();

  public RoundRobin(List<URI> nodi) {
    this.nodi = nodi;
  }

  public URI scegli() {                       // n1, n2, n3, n1, n2...
    return nodi.get(Math.floorMod(prossimo.getAndIncrement(), nodi.size()));
  }
}
```

Nella pratica non lo si scrive: si configura un load balancer esistente, come nginx. La strategia è una riga, e il nodo di riserva realizza il fail-over della prossima sezione.

nginx.conf

```
upstream negozio {
    least_conn;                     # il nodo meno carico (senza questa riga: round robin)
    server 10.0.0.11:8080;
    server 10.0.0.12:8080;
    server 10.0.0.13:8080 backup;   # usato solo se gli altri sono giu'
}

server {
    listen 80;
    location / {
        proxy_pass http://negozio;  # ogni richiesta va a uno dei nodi del gruppo
    }
}
```

DISPONIBILITÀ

Disponibilità

La **disponibilità** (*availability*) è la **frazione di tempo** in cui un sistema o un servizio è operativo e accessibile.

La disponibilità è cruciale per i sistemi che devono funzionare in modo continuo: i sistemi **business-critical**, dove un fermo causa perdite economiche e danni d'immagine, e i sistemi **safety-critical**, dove sono in gioco persone. Nel modello ISO/IEC 25010 del Capitolo 4 è una delle sotto-caratteristiche dell'affidabilità. Un sistema può diventare indisponibile per:

- problemi **software** (l'applicazione smette di rispondere);
- **carichi eccessivi** (non ci sono abbastanza risorse di calcolo);
- perdita della **connessione di rete**;
- **guasti hardware**.

Quantificare la disponibilità: i «nove»

La disponibilità si esprime come percentuale di tempo in cui il servizio è attivo (*uptime*), e si misura in **numero di nove**: un servizio disponibile al 99,99% ha «quattro nove». Ogni nove in più divide per dieci il fermo annuo tollerato.

Nove	Disponibilità	Fermo tollerato in un anno
uno	90%	circa 36 giorni e mezzo
due	99%	circa 3 giorni e 15 ore
tre	99,9%	circa 8 ore e 45 minuti
quattro	99,99%	circa 52 minuti
cinque	99,999%	circa 5 minuti
sei	99,9999%	circa 31 secondi

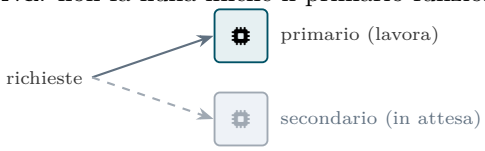
Il calcolo è semplice: un anno ha circa $365 \times 24 = 8760$ ore, e il fermo tollerato è la frazione restante. Con tre nove, $8760 \times 0{,}001 = 8{,}76$ ore, cioè circa 8 ore e 45 minuti (le slide riportano 8 ore e 41 minuti: la piccola differenza dipende da come si conta la durata dell'anno).

Come si ottiene un'alta disponibilità: il fail-over

L'idea è non dipendere da un'unica istanza di un componente. Le tecniche di **fail-over** sono due.

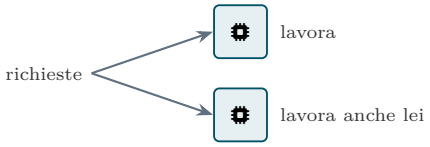
Fail-over attivo-passivo

Se il componente **primario** si guasta, un'istanza **secondaria** ne prende il posto. La secondaria di solito è **passiva**: non fa nulla finché il primario funziona.



Fail-over attivo-attivo

Esistono fin dall'inizio due o più istanze, che si dividono il lavoro (come nella scalabilità orizzontale). Se una si guasta, le altre si fanno carico anche della sua parte.



Esempio 12.2 — Un fail-over attivo-passivo con il battito cardiaco

Il primario manda un segnale di vita (*heartbeat*) ogni secondo; una sentinella controlla periodicamente quando è arrivato l'ultimo e, se tace da troppo, promuove il secondario.

Sentinella.java

```java
public class Sentinella {
  private static final Duration TOLLERANZA = Duration.ofSeconds(3);
  private volatile Instant ultimoBattito = Instant.now();
  private final Nodo primario, secondario;
  private volatile Nodo attivo;

  public Sentinella(Nodo primario, Nodo secondario) {
    this.primario = primario;
    this.secondario = secondario;
    this.attivo = primario;
  }

  public void battito() {                    // chiamato dal primario ogni secondo
    ultimoBattito = Instant.now();
  }

  public void controlla() {                  // eseguito periodicamente
    if (attivo == primario && Instant.now().isAfter(ultimoBattito.plus(TOLLERANZA))) {
      attivo = secondario;                   // fail-over
      allarme.notifica("primario muto da 3 secondi: attivato il secondario");
    }
  }
}
```

Tre secondi sono un compromesso: con una tolleranza più corta si scatta per un rallentamento passeggero, con una più lunga si resta fermi più a lungo prima di reagire.

Osservabilità e monitoraggio

Con il fail-over, la durata del fermo dipende da quanto in fretta si reagisce al guasto.

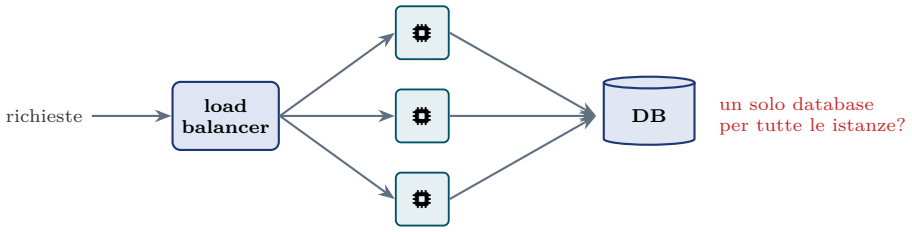
- **Quanto in fretta ci accorgiamo che un componente si è guastato?** E, prima ancora, **come** ce ne accorgiamo? Di solito con gli **heartbeat**: segnali o controlli periodici che verificano che il sistema funzioni normalmente. Se un componente smette di «battere», lo si considera guasto.
- **Quanto ci mette la secondaria a partire?** Può essere in *hot standby* (già avviata e pronta a subentrare) o in *cold standby* (spenta, da avviare al momento). Nel fail-over attivo-attivo il problema non si pone: le altre istanze sono già in funzione.

Osservabilità

L'**osservabilità** è il grado in cui riusciamo a capire che cosa sta succedendo all'interno di un sistema software mentre è in esecuzione, a partire dai dati che produce (log, metriche, tracce delle richieste). È una caratteristica importante di un'architettura; strumenti come *OpenTelemetry* servono proprio a raccogliere questi dati in modo uniforme.

Dati e consistenza

Scalare orizzontalmente le istanze dell'applicazione è stato relativamente facile: basta un nuovo server con l'applicazione e un load balancer davanti. Ma quasi tutte le applicazioni hanno anche dati **persistenti**, di solito in un database relazionale. Tutte le istanze, dietro il load balancer, usano lo stesso database: e se è il database a non reggere più il carico? Scalare i dati è molto meno semplice.



Le proprietà ACID

Proprietà ACID

Le transazioni dei database relazionali (RDBMS) godono di quattro proprietà.

- **Atomicità** (*Atomicity*): ogni transazione è «tutto o niente».
- **Consistenza** (*Consistency*): ogni transazione porta il database da uno stato valido a un altro stato valido.
- **Isolamento** (*Isolation*): eseguire transazioni in modo concorrente dà lo stesso risultato che eseguirle in sequenza.
- **Durabilità** (*Durability*): una volta confermata (*commit*), una transazione resta tale, anche in caso di guasto.

Sono proprio queste garanzie a rendere impossibile scalare un RDBMS orizzontalmente in modo semplice: se i dati stanno su più macchine, garantire isolamento e consistenza richiede che le macchine si coordinino a ogni scrittura. Le strade sono:

- scalarlo **verticalmente**, dandogli più risorse di calcolo;
- usare tecniche specifiche: **replica** (master-slave e master-master), **federazione** e **sharding**, **denormalizzazione**.

Replica master-slave e master-master

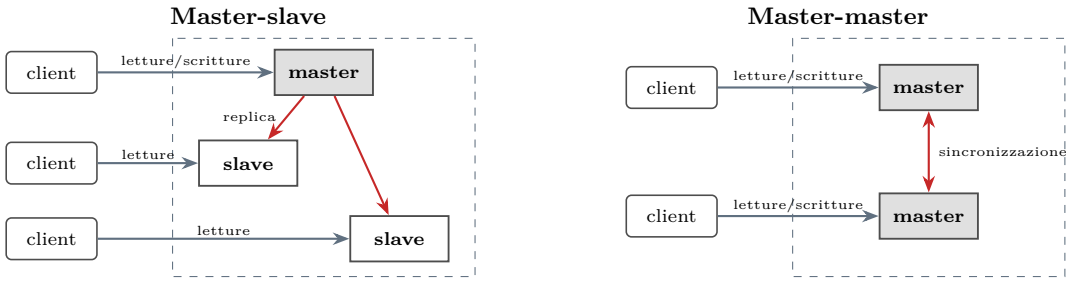


Figura 12.2: Le due forme di replica (rielaborate dalle slide *Scalability, availability, stability, patterns*). In rosso il lavoro di coordinamento che introduce un **sovraccarico**.

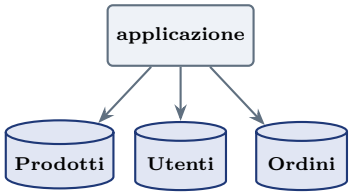
- **Master-slave.** Ci sono più istanze del RDBMS: un **master** e uno o più **slave**. Il master serve letture e scritture e replica ogni scrittura su ogni slave; gli slave sono **repliche in sola lettura**. *Disponibilità*: se il master si guasta, il sistema può continuare in sola lettura, e uno slave può eventualmente essere promosso a master. Funziona bene quando ci sono **molte letture e poche scritture**.
- **Master-master.** Tutte le istanze sono master e servono sia letture sia scritture. A ogni scrittura i master si coordinano e si sincronizzano, mentre le letture sono gestite in modo indipendente. *Disponibilità*: se un master si guasta, il sistema continua con l'altro. La sincronizzazione però introduce un **sovraccarico**.

Federazione e sharding

Le slide le elencano fra le tecniche di replica, ma a rigore sono tecniche di **partizionamento**: invece di copiare gli stessi dati su più database, si **dividono** i dati fra più database.

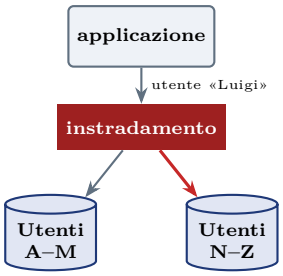
Federazione. Un unico grande database viene diviso in più database **per funzione**: invece di un database monolitico, uno per i prodotti, uno per gli utenti, uno per gli ordini. Gli svantaggi:

- non serve a nulla se lo schema ha un'unica tabella molto grande;
- è più difficile fare *join* fra dati che stanno in database diversi;
- è più difficile far rispettare vincoli fra tabelle (ad esempio le chiavi esterne);
- può richiedere di denormalizzare.



Sharding. I dati vengono distribuiti su più database in modo che ciascuno gestisca solo un **sottoinsieme** dei record della stessa tabella: ad esempio gli utenti con il nome dalla A alla M in un database, quelli dalla N alla Z in un altro. Gli svantaggi:

- la logica applicativa diventa più complessa, perché deve sapere dove sta ogni dato;
- gli *shard* possono **sbilanciarsi** (in Italia i cognomi che iniziano per R sono molti di più di quelli che iniziano per Q);
- unire i dati è difficile, e far rispettare i vincoli lo è ancora di più.



Esempio 12.3 — La logica di instradamento dello sharding

«La logica applicativa deve sapere dove sta ogni dato»: in pratica, una funzione che dalla chiave sceglie lo shard.

Shard.java

```java
// per intervalli: semplice da capire, ma gli shard si sbilanciano
public DataSource shardPer(String cognome) {
  char iniziale = Character.toUpperCase(cognome.charAt(0));
  return iniziale <= 'M' ? shardAM : shardNZ;
}

// per hash: le chiavi si distribuiscono in modo uniforme sugli shard
public DataSource shardPer(long idUtente) {
  return shard.get(Math.floorMod(Long.hashCode(idUtente), shard.size()));
}
```

Lo sharding per hash risolve lo sbilanciamento ma ne crea un altro problema: se si aggiunge uno shard, cambia `shard.size()` e quasi tutte le chiavi cambiano destinazione, quindi vanno spostate. Per questo i sistemi reali usano varianti come il *consistent hashing*, in cui aggiungere uno shard sposta solo una piccola parte dei dati.

Denormalizzazione

La **denormalizzazione** migliora le prestazioni in lettura a scapito di quelle in scrittura (e, naturalmente, della normalizzazione). Si scrivono **copie ridondanti** degli stessi dati in più tabelle, per evitare *join* costosi. Il prezzo è che mantenere le copie consistenti fra loro diventa un problema. Alcuni RDBMS, come PostgreSQL, offrono le **viste materializzate**, che memorizzano il risultato di un'interrogazione e si occupano di gestire le copie ridondanti (in PostgreSQL la vista va aggiornata esplicitamente con `REFRESH MATERIALIZED VIEW`; altri DBMS la aggiornano automaticamente).

I database NoSQL e la consistenza finale

I database **NoSQL** sono più scalabili, a costo di rinunciare a vere transazioni ACID (e a parte delle capacità di interrogazione). Le loro proprietà si riassumono spesso con l'acronimo **BASE**:

- **Basically Available**: il sistema garantisce la disponibilità;
- **Soft state**: lo stato del sistema può cambiare nel tempo, anche senza input;
- **Eventual consistency**: il sistema diventa consistente **dopo un certo tempo**, a patto di non ricevere nuovi input in quel periodo.

Consistenza finale (eventual consistency)

Garanzia informale che, se un dato non riceve nuovi aggiornamenti, **prima o poi** tutti gli accessi a quel dato restituiranno l'ultimo valore scritto.

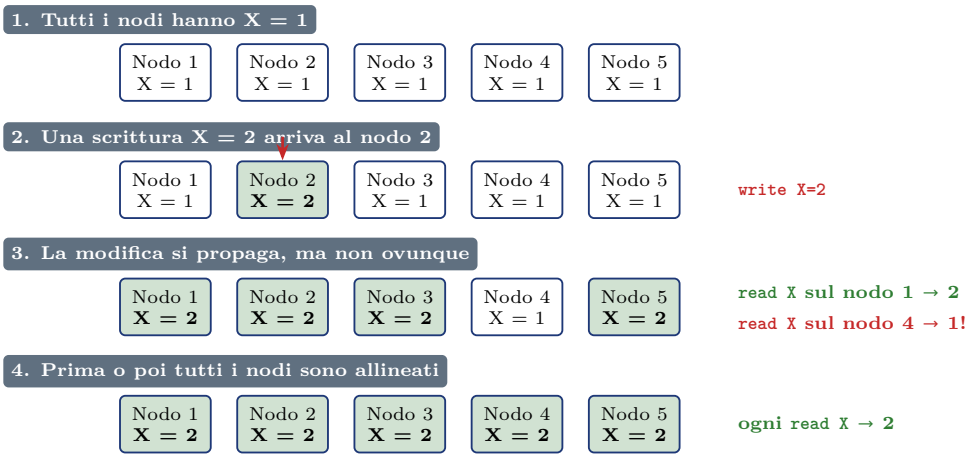


Figura 12.3: La consistenza finale in un cluster NoSQL di cinque nodi. Al passo 3 due letture contemporanee danno risultati diversi: è il prezzo della scalabilità.

Esistono diversi modelli di database NoSQL:

Modello	Come organizza i dati	Esempi
Chiave-valore	coppie chiave/valore, tipicamente in memoria	Redis, Memcached
Documentali	documenti (XML, JSON…) che contengono tutte le informazioni di un oggetto	MongoDB
A grafo	ogni record è un nodo, le relazioni fra record sono archi	Neo4J
Vettoriali	memorizzazione e ricerca di grandi vettori, spesso prodotti da applicazioni di intelligenza artificiale	Milvus

Elaborazione asincrona

Man mano che un sistema cresce, può capitare di dover svolgere elaborazioni che non si possono fare **in linea** mentre si serve una richiesta, perché introdurrebbero una latenza inaccettabile. Nell'esempio, a ogni richiesta che produce un errore, l'applicazione chiede a un `ErrorHandler` di registrarlo, avvisare gli amministratori e analizzarlo, e solo dopo risponde al client.

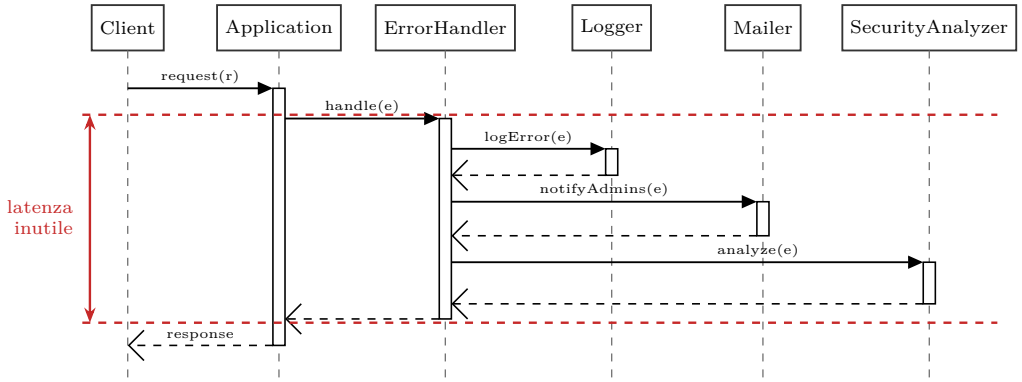


Figura 12.4: Elaborazione sincrona: il client aspetta che l'errore sia registrato, notificato e analizzato, anche se a lui non serve.

Code di messaggi e message broker

L'asincronismo aiuta a ridurre i tempi di risposta. Un modo di realizzarlo sono le **code di messaggi** (*Message Queue*, MQ): componenti progettati per ricevere, conservare e consegnare messaggi. Quando un compito è troppo lento per essere svolto in linea:

1. l'applicazione **pubblica** un messaggio in una coda, con i dettagli del compito da svolgere;
2. un **worker**, in modo asincrono e magari più tardi, fa da **consumatore**: preleva il messaggio dalla coda e svolge davvero il compito;
3. l'utente **non resta bloccato** ad aspettare la fine del compito lungo, che viene completato in background appena uno dei worker (che possono essere molti) è disponibile.

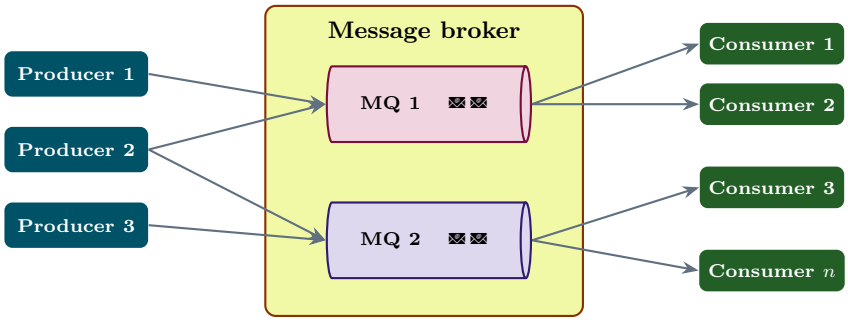


Figura 12.5: Produttori e consumatori non si conoscono: comunicano solo attraverso le code del broker.

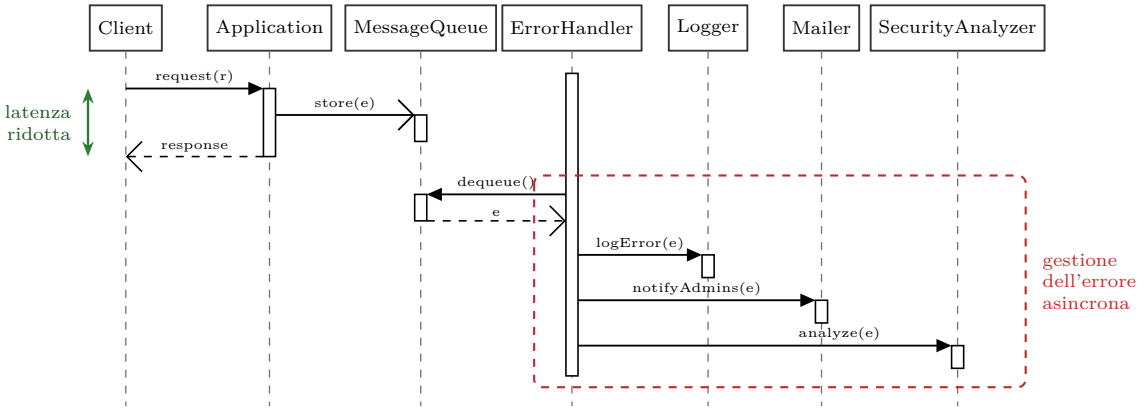


Figura 12.6: Con una coda di messaggi l'applicazione deposita l'errore e risponde subito; l'`ErrorHandler` lo preleva e lo gestisce per conto suo.

Fra i **message broker** open source più diffusi ci sono RabbitMQ, Apache ActiveMQ e BlazingMQ.

I message broker non sono oggetti banali

Internamente sono organizzati in architetture distribuite complesse; l'instradamento dei messaggi può essere più sofisticato di una semplice interrogazione da parte dei consumatori; e broker e configurazioni diversi garantiscono **proprietà diverse**: la consegna di un messaggio può essere garantita o no, e lo stesso messaggio può essere consegnato a **più di un** consumatore. Bisogna valutare con attenzione questi aspetti rispetto al problema da risolvere: un messaggio consegnato due volte, ad esempio, non deve far addebitare due volte un pagamento.

Esempio 12.4 — Produttore e consumatori, in miniatura

Dentro un solo programma Java una coda di messaggi si può simulare con una `BlockingQueue`: l'applicazione web pubblica il compito lento e risponde subito, mentre alcuni worker lo svolgono in background.

Ordini.java

```java
BlockingQueue<Ordine> coda = new LinkedBlockingQueue<>();

// produttore: risponde all'utente senza aspettare PDF ed e-mail
public String conferma(Ordine ordine) {
  coda.offer(ordine);
  return "Ordine ricevuto: riceverai una e-mail di conferma";
}

// consumatore: lo stesso codice gira in quattro worker
Runnable worker = () -> {
  try {
    while (true) {
      Ordine ordine = coda.take();            // aspetta il prossimo messaggio
      generaFatturaPdf(ordine);
      inviaEmail(ordine.cliente(), "Ordine confermato");
    }
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();       // il worker viene fermato
  }
};
ExecutorService pool = Executors.newFixedThreadPool(4);
for (int i = 0; i < 4; i++) {
  pool.submit(worker);
}
```

Un message broker come RabbitMQ porta la coda **fuori** dal programma: produttori e consumatori possono stare su macchine diverse, scalare in modo indipendente, e i messaggi sopravvivono al riavvio di entrambi. Resta il problema della consegna doppia: un consumatore che riceve due volte lo stesso messaggio deve accorgersene (deve essere *idempotente*).

ConsumatorePagamenti.java

```java
if (pagamentiGiaRegistrati.add(messaggio.id())) {   // add restituisce false se l'id c'era gia'
  addebita(messaggio.carta(), messaggio.importo());
}
```

Un altro scenario: lo stesso utente su più dispositivi

Le code non servono solo a spostare in background i compiti lenti. Un altro scenario tipico è quello delle applicazioni a cui lo **stesso utente** accede da **più dispositivi**, anche contemporaneamente: Microsoft Teams, per esempio, può essere aperto nello stesso momento sul telefono, sul portatile e nel browser. Quando arriva un nuovo messaggio deve comparire su tutti i dispositivi; quando l'utente lo legge su uno, gli altri devono smettere di segnalarlo come non letto.

Se fosse il servizio di chat a chiamare i dispositivi uno per uno, dovrebbe sapere in ogni momento quali sono connessi, aspettare la risposta di ciascuno e gestire quelli momentaneamente irraggiungibili (un telefono in galleria). Con un message broker, invece, il servizio **pubblica** l'evento **una sola volta**; ogni dispositivo connesso è un **consumatore** iscritto agli eventi del proprio utente, e il broker consegna a ciascuno la sua copia. È proprio il caso, citato nel riquadro precedente, in cui lo stesso messaggio arriva a **più** consumatori: qui è esattamente ciò che si vuole. E se il broker conserva i messaggi per i consumatori disconnessi, un dispositivo che si riconnette trova in coda gli eventi che si era perso.

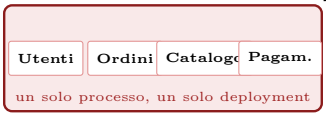


Figura 12.7: Lo stesso utente collegato da tre dispositivi: il servizio pubblica l'evento una volta sola e il broker lo consegna a ciascun dispositivo.

MICROSERVIZI

Architettura monolitica

- componenti e moduli **strettamente impacchettati** in un'unica unità;
- un'unica **codebase**;
- un'unica unità di **deployment**, autosufficiente;
- le responsabilità di business sono **accoppiate**.



Architettura a microservizi

- il software è strutturato come un insieme di **servizi**;
- ogni servizio ha la **propria codebase**, anche in linguaggi diversi (*poliglotta*);
- ogni servizio si può **rilasciare in modo indipendente**;
- i servizi sono organizzati attorno alle **capacità di business**;
- i servizi sono **debolmente accoppiati**.



Perché mai dovrei farlo?

All'inizio, o per software poco complesso, il monolite ha vantaggi concreti:

- **deployment facile**: c'è un solo artefatto da costruire (ad esempio un unico file WAR da caricare su un application server come Apache Tomcat, che parla con un unico database);
- **sviluppo facile**: un'unica codebase, e una gestione dei dati semplice.

I problemi arrivano quando il monolite **cresce in complessità**. Le slide lo paragonano a Sisifo, condannato a spingere in eterno un masso su per un monte:

- l'architettura interna diventa sempre più difficile da mantenere e da far evolvere;
- i nuovi rilasci richiedono sempre più tempo (mesi), e aggiungere funzionalità diventa lento;
- il ciclo di *build*, test e rilascio diventa lungo e complicato: *chi ha rotto la build?*;
- il deployment diventa sempre più difficile: *di chi è il modulo che non funziona?*;
- l'innovazione si ferma.

Le motivazioni: consegnare spesso e bene

Il mondo di oggi cambia rapidamente, le aziende devono essere agili e innovare in fretta, e il software deve essere consegnato **rapidamente, frequentemente e in modo affidabile**. Le quattro metriche **DORA** (*DevOps Research and Assessment*) misurano proprio questo:

Metrica	Che cosa misura
Deployment Frequency	quanto spesso si rilascia in produzione
Lead Time for Changes	quanto tempo impiega un *commit* ad arrivare in produzione
Change Failure Rate	la percentuale di rilasci che causano un guasto in produzione
Time to Restore Service	quanto tempo serve per riprendersi da un guasto in produzione

Dividere il monolite in microservizi aiuta a rendere gestibili i sistemi grandi:

- ogni microservizio è molto più piccolo del sistema intero, quindi più facile da capire, mantenere, far evolvere e testare;
- ogni microservizio si può costruire e rilasciare in modo indipendente;
- una nuova funzionalità probabilmente tocca un solo microservizio;
- i rilasci sono più frequenti: per aggiungere una funzionalità basta rilasciare di nuovo un solo microservizio.

Scalare un monolite e scalare dei microservizi

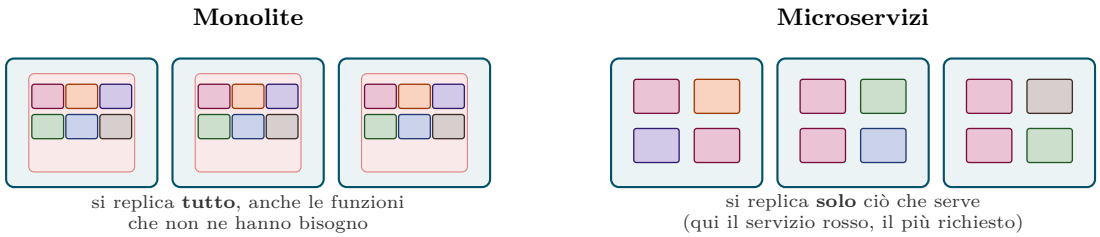


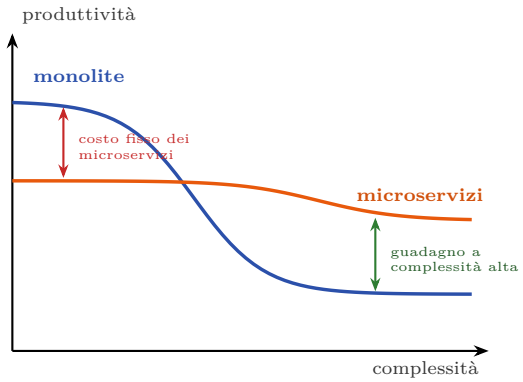
Figura 12.8: Il monolite mette tutte le funzionalità in un unico processo e scala replicandolo per intero; i microservizi mettono ogni funzionalità in un processo distinto e scalano distribuendo i servizi sui server, replicando quelli che servono.

Conviene davvero?

Non prendete neanche in considerazione i microservizi, a meno che il sistema non sia complesso. La complessità che spinge verso i microservizi può venire da:

- le dimensioni pure, con team di sviluppo molto grandi;
- la **multi-tenancy** (lo stesso sistema serve molti clienti separati);
- il dover supportare modelli di interazione diversi (web, app, API per terzi…);
- la necessità di scalare.

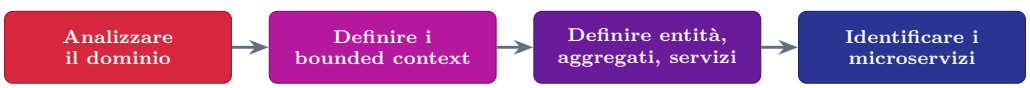
Il grafico (Fowler, *Microservice Premium*) lo mostra: per sistemi poco complessi, il sovraccarico dei microservizi **riduce** la produttività. Quando la complessità cresce, la produttività del monolite crolla; quella dei microservizi cala molto meno, grazie al minore accoppiamento.



Adottare i microservizi significa rispondere a quattro domande: come progettare i microservizi, come gestire la comunicazione fra servizi, come gestire i dati, come organizzare i team di sviluppo. Le vediamo una alla volta.

Progettare i microservizi

Definire i **confini** di ogni servizio è una delle sfide più grandi. La regola generale è che un servizio dovrebbe fare «**una cosa sola**» (è il Single Responsibility Principle del Capitolo 10, applicato ai servizi), ma nessun procedimento meccanico garantisce il risultato giusto. Viene in aiuto il **Domain-Driven Design** (DDD), un approccio alla progettazione centrato sul modello del dominio. Le slide lo chiamano *Domain-Driven Development*, ma il nome corretto, introdotto da Eric Evans nel 2003, è *Domain-Driven* ***Design***. Il procedimento ha quattro passi.



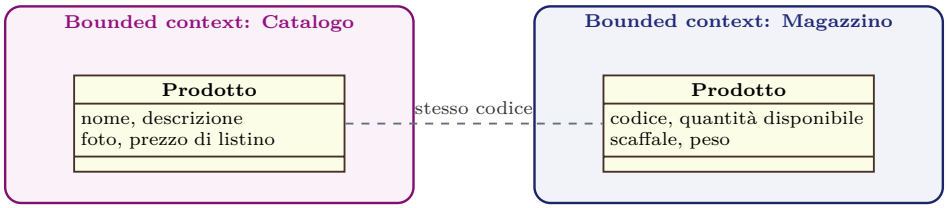
1. **Analizzare il dominio.** Si comprendono i requisiti funzionali; il risultato è una descrizione informale del dominio del

sistema, che si può raffinare in un insieme di modelli di dominio (Capitolo 7).

2. **Definire i bounded context.** Ogni **bounded context** (contesto delimitato) contiene un modello di dominio che rappresenta un particolare **sottodominio**. Unificare tutto il dominio in un solo modello potrebbe non essere né possibile né conveniente.
3. **Definire entità, aggregati e servizi di dominio.** Per ogni bounded context si applicano i pattern del DDD:
   - le **entità**: oggetti con un'**identità** che persiste nel tempo (un cliente resta lo stesso cliente anche se cambia indirizzo);
   - gli **aggregati**: confini di **consistenza** attorno a una o più entità, che modellano gli invarianti da rispettare in una transazione (un ordine con le sue righe: il totale deve sempre essere la somma delle righe);
   - i **servizi di dominio**: oggetti che implementano logica senza conservare uno stato.
4. **Identificare i microservizi.** Si parte da un bounded context: le funzionalità di un microservizio non dovrebbero estendersi su più bounded context. Aggregati e servizi di dominio sono buoni candidati a diventare microservizi.

Esempio 12.5 — La stessa parola, due contesti

In un e-commerce, «prodotto» significa cose diverse a seconda di chi ne parla.



Forzare un unico modello «Prodotto» per tutto il sistema produrrebbe una classe enorme, che cambia ogni volta che cambia il catalogo *o* il magazzino: è la God Class del Capitolo 10, a livello di sistema. Con due bounded context, ognuno ha il proprio modello, e i due condividono solo l'identificativo. Catalogo e Magazzino sono due buoni candidati a diventare due microservizi.

La comunicazione fra microservizi

In un monolite i componenti comunicano con **chiamate di metodo**. In un'architettura a microservizi la comunicazione dovrebbe essere **indipendente dalla tecnologia**: spesso i team usano i protocolli e le tecnologie su cui è costruito il Web. La comunicazione può essere:

- **sincrona**: REST (API HTTP orientate alle risorse), Apache Thrift (che supporta più protocolli: binari, basati su JSON…);
- **asincrona**: code di messaggi.

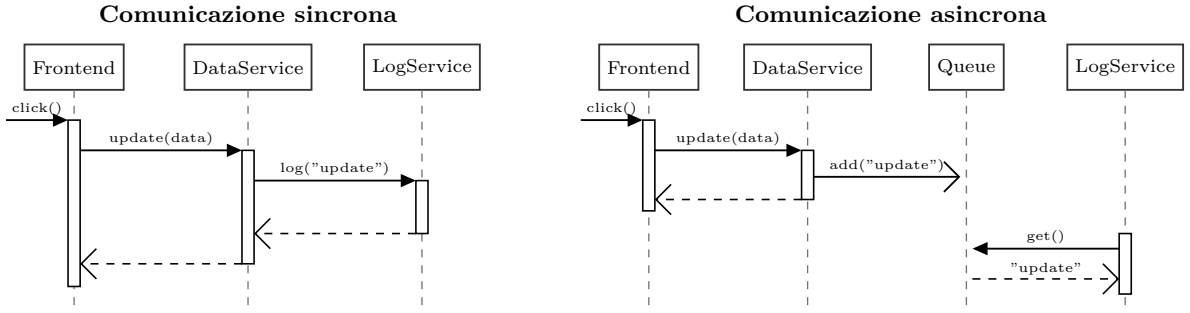


Figura 12.9: A sinistra il `DataService` aspetta il `LogService` prima di rispondere; a destra deposita il messaggio nella coda e risponde subito, e il `LogService` lo preleva quando vuole.

Le chiamate sincrone possono penalizzare le prestazioni, e soprattutto, quando sono molte, si manifesta un **effetto moltiplicativo** dei fermi: se per servire una richiesta devono funzionare tutti i servizi di una catena, la disponibilità complessiva è il **prodotto** delle disponibilità dei singoli servizi. O si rende asincrona la comunicazione, o si gestiscono i fermi.

Si moltiplicano le disponibilità, non i fermi

Le slide dicono che il fermo del sistema diventa «il prodotto dei fermi dei singoli servizi». A moltiplicarsi, in realtà, sono le **disponibilità**: se una richiesta attraversa in modo sincrono n servizi indipendenti, con disponibilità $A_1, \dots, A_n$, la disponibilità complessiva è $A = A_1 \cdot A_2 \cdots A_n$. Con dieci servizi al 99,9% ciascuno, $A = 0{,}999^{10} \approx 99{,}0\%$: si passa da circa 9 ore a circa 3 giorni e mezzo di fermo all'anno. Il fermo complessivo è quindi circa la **somma** dei fermi, non il prodotto.

Esempio 12.6 — Una chiamata sincrona che sa gestire il fermo

Un servizio che ne chiama un altro in modo sincrono non deve aspettarlo per sempre, né propagare il suo fermo a chi lo ha chiamato. Due difese minime: un **timeout** e una **risposta degradata**.

ClientCatalogo.java

```java
public Prodotto prodotto(long id) throws InterruptedException {
  HttpRequest richiesta = HttpRequest.newBuilder(URI.create("http://catalogo/prodotti/" + id))
      .timeout(Duration.ofMillis(300))        // un servizio lento conta come un servizio fermo
      .GET()
      .build();
  try {
    HttpResponse<String> risposta = client.send(richiesta, HttpResponse.BodyHandlers.ofString());
    return Prodotto.daJson(risposta.body());
  } catch (IOException e) {                   // timeout, connessione rifiutata...
    return cache.ultimoNoto(id);              // meglio un prezzo di ieri che una pagina d'errore
  }
}
```

La gestione dei dati

I monoliti sono tipicamente associati a un unico database **centralizzato**, spesso anche per via dei costi di licenza dei DBMS. Anche i microservizi possono lavorare con un database centralizzato, ma allora le modifiche allo schema vanno **coordinate** fra i team, e i servizi tornano ad essere accoppiati attraverso i dati.

L'alternativa è un **database per ogni servizio**: ogni microservizio (e ogni team) è davvero indipendente, e si può scegliere per ognuno il tipo di database più adatto (*polyglot persistence*: un database relazionale per gli ordini, uno a documenti per il catalogo, uno chiave-valore per le sessioni…).

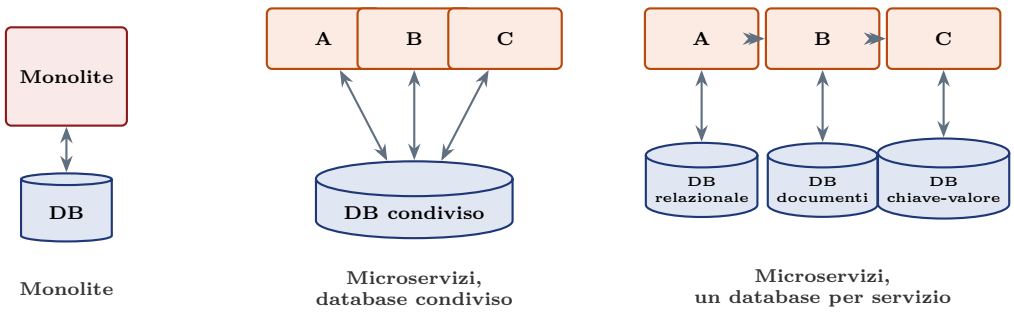


Figura 12.10: Tre modi di gestire i dati. Nel terzo i servizi si scambiano dati solo attraverso le loro interfacce (tratteggio).

Con i dati distribuiti su molti database, garantire una **consistenza forte** diventa sempre più difficile: spesso ci si accontenta della **consistenza finale** vista per i database NoSQL. A un certo punto nel futuro, tutte le letture di un'entità restituiranno il valore aggiornato.

I microservizi danno...	...ma non gratis
confini forti fra i moduli	più complessità (un sistema distribuito)
rilasci indipendenti	consistenza solo finale
libertà di scelta delle tecnologie	più complessità operativa (molti servizi da rilasciare e monitorare)

Bisogna quindi avere una **buona ragione** per adottarli. Un caso spesso citato è quello di Istio, un noto progetto open source che nel 2020 è **tornato** da un'architettura a microservizi a una monolitica, perché la complessità operativa superava i benefici (Mendonça et al., *The Monolith Strikes Back*, IEEE Software, 2021).

Un caso reale: un'applicazione a microservizi



Per vedere che cosa succede quando questi concetti incontrano la pratica, le slide presentano un'esperienza recente: lo sviluppo di un'applicazione complessa a microservizi nell'ambito del corso di *Software Architecture Design*.

Il progetto

- L'applicazione è stata sviluppata nell'arco di **tre edizioni successive** del corso. Nella prima, i team hanno implementato i singoli microservizi, **separatamente**; nella seconda e nella terza i servizi sono stati integrati e arricchiti di nuove funzionalità. Infine alcuni ricercatori a contratto l'hanno messa in produzione e hanno risolto diversi problemi di prestazioni e di sicurezza.
- Il prodotto finale è un insieme di **immagini di container** da eseguire su **Docker**.
- L'applicazione è un **gioco** per imparare la **copertura del codice** (*code coverage*). Dai nomi dei servizi e delle API se ne intuisce la meccanica: lo studente scrive casi di test per una classe sotto test (*class under test*), il sistema li compila e ne misura la copertura, e il risultato è confrontato con quello di avversari automatici, i *robot*, i cui test sono generati con strumenti come EvoSuite.

L'architettura

Il diagramma di deployment mostra la struttura concettuale. Il client è un browser, che comunica con il server in HTTP sulla porta 80. Sul server, sopra il sistema operativo, gira Docker, che fa da ambiente di esecuzione: ogni microservizio è un **container**. Le sigle T1, …, T8 indicano il **team** che ha sviluppato ciascun servizio (T23 è il servizio nato dall'unione del lavoro dei team 2 e 3).

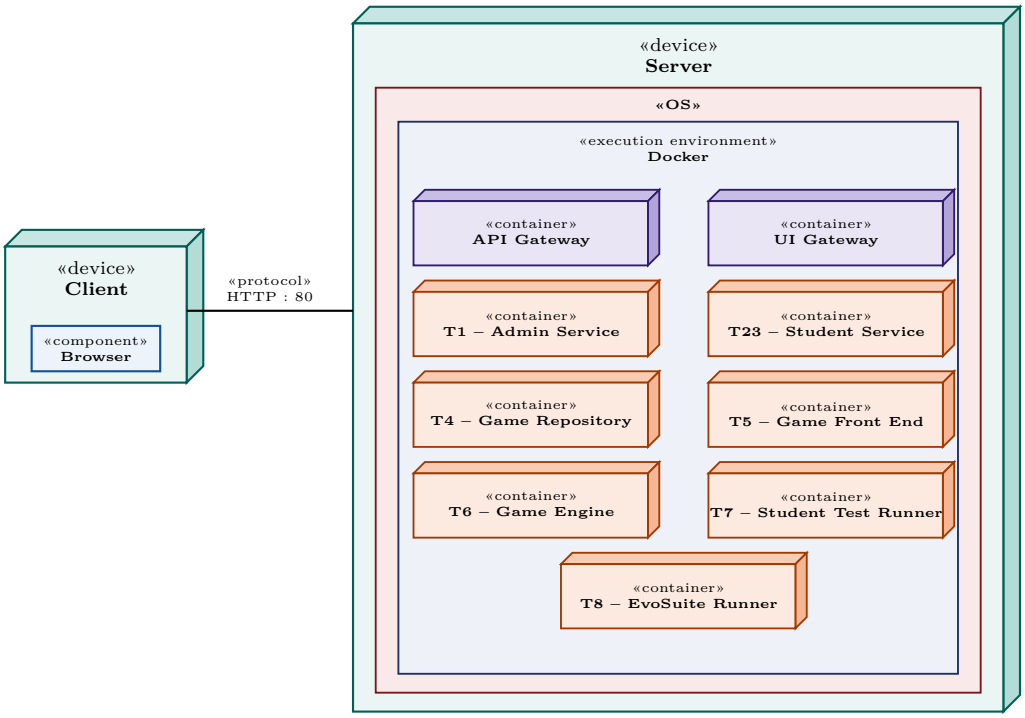


Figura 12.11: Il diagramma di deployment dell'applicazione (ridisegnato dalle slide): un container Docker per ogni microservizio, più due gateway.

Il diagramma dei componenti mostra chi chiama chi. Due componenti fanno da **gateway**: l'*UI Gateway* porta le richieste delle pagine ai tre servizi che producono l'interfaccia (il front end del gioco, il servizio degli studenti e quello di amministrazione), mentre l'*API Gateway* instrada le richieste di compilazione e di calcolo della copertura. Molte chiamate, però, vanno **direttamente** da un servizio all'altro: tutte sono chiamate REST sincrone.

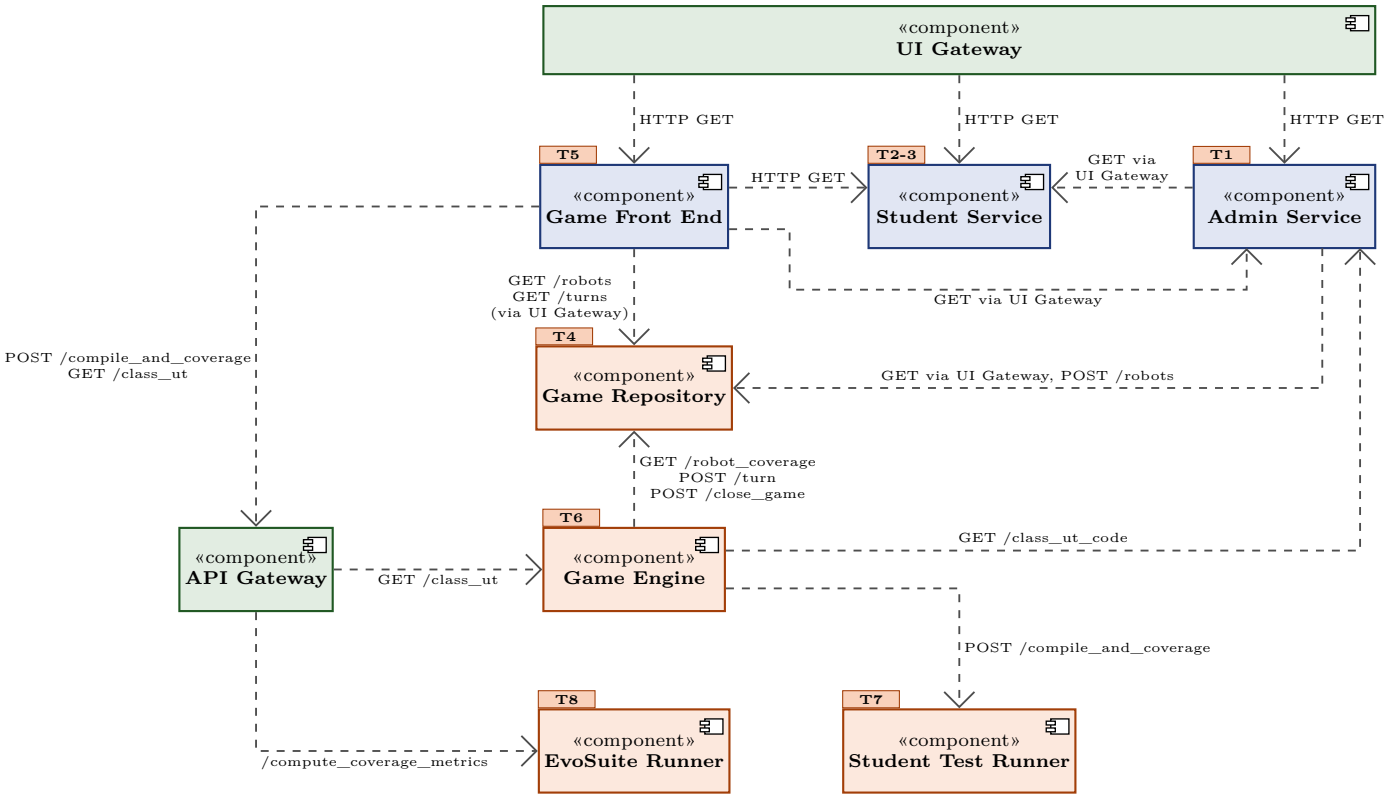


Figura 12.12: Il diagramma dei componenti (ridisegnato e semplificato dalle slide). Le frecce tratteggiate sono chiamate REST, con il metodo HTTP e l'endpoint invocato; le etichette T1, …, T8 indicano il team che ha sviluppato il servizio.

Gli anti-pattern individuati

Analizzando il sistema sono stati individuati molti **anti-pattern**: soluzioni ricorrenti che sembrano ragionevoli ma producono più problemi di quanti ne risolvano. Il numero accanto al nome è quello dell'anti-pattern nel catalogo usato per l'analisi; l'ultima colonna dice in quale servizio è stato trovato, oppure se riguarda **tutto il sistema**. Le slide li raggruppano in quattro famiglie.

Scomposizione

Anti-pattern	Che cosa succede	Dove
Progettazione interna dei servizi		
No API-versioning (n. 6)	il sistema espone le sue API senza alcuna strategia di **versionamento**: ogni modifica rischia di rompere chi le usa	tutto il sistema
Ambiguous service (n. 9)	le **responsabilità** di un servizio non sono definite con chiarezza: confusione e funzionalità sovrapposte	T23
Scomposizione fra servizi		
Chatty service (n. 17)	un servizio comunica **troppo** con gli altri, con molte chiamate a grana fine per raccogliere i dati che gli servono: cresce il tempo di risposta e peggiorano le prestazioni dell'intero sistema	T6

Sono problemi di **confini**. Senza versioni delle API viene meno il vantaggio principale dei microservizi, il rilascio indipendente: un servizio non può cambiare senza costringere gli altri ad adeguarsi. Un servizio dalle responsabilità ambigue e uno che passa il tempo a interrogare gli altri sono il sintomo di bounded context tracciati male: dati e operazioni che stanno insieme sono finiti in servizi diversi. Non a caso il servizio chiacchierone è il motore di gioco (T6), che nel diagramma dei componenti chiama quasi tutti.

Interazione fra servizi

Anti-pattern	Che cosa succede	Dove
On-line only (n. 25)	il sistema si affida **solo** alla comunicazione sincrona fra microservizi ed evita l'elaborazione batch anche per i compiti lunghi o pesanti: colli di bottiglia e inefficienze	T1
Hardcoded endpoint (n. 28)	indirizzi, porte o endpoint dei microservizi sono **scritti** nel codice, nei file di configurazione o nelle variabili d'ambiente: tra l'altro, è difficile scalare	T1, T5, T6
No API gateway (n. 29)	manca un API gateway centralizzato: i client devono chiamare **direttamente** i servizi, e sicurezza e instradamento si complicano	tutto il sistema
Wobbly service interactions (n. 30)	un servizio interagisce con altri servizi, o con i router dei messaggi, senza meccanismi per **tollerarne i guasti**	tutto il sistema
Timeout (n. 31)	chi usa un servizio spesso non riesce a connettersi, probabilmente perché la sua **indisponibilità** non è gestita come si deve	tutto il sistema

Qui ritornano quasi tutti i temi del capitolo. *On-line only* è esattamente il problema che le **code di messaggi** risolvono: un compito lungo svolto in linea fa aspettare l'utente. Con gli endpoint scritti nel codice non si può mettere un **load balancer** davanti a più istanze di un servizio, cioè non si può scalare orizzontalmente. Il gateway, nei diagrammi, c'è; ma non è l'**unico** punto d'ingresso, visto che diverse chiamate lo scavalcano e alcuni servizi sono raggiungibili direttamente dall'esterno. Infine *Wobbly service interactions* e *Timeout* riguardano la **disponibilità**: in un sistema distribuito i guasti parziali sono la normalità, e un servizio che non sa reagire all'assenza di un altro trascina con sé tutto il sistema.

Sicurezza

Anti-pattern	Che cosa succede	Dove
Unauthenticated traffic (n. 33)	le richieste alle API, sia dei client esterni sia fra microservizi interni, non sono **autenticate** come si deve	tutto il sistema
Multiple user authentication (n. 34)	il sistema offre **più punti** di autenticazione degli utenti: aumentano la superficie d'attacco e i rischi	T1, T23
Publicly accessible microservices (n. 35)	i microservizi sono esposti **direttamente** ai client esterni: senza controlli adeguati possono rivelare informazioni sensibili, e il rischio di attacchi cresce	T1, T23, T4, T8
Unnecessary privileges to microservices (n. 36)	i microservizi hanno più accessi, permessi o funzionalità di quanti ne servano al loro compito	tutto il sistema
Insufficient access control (n. 37)	manca un **controllo degli accessi** adeguato: dati sensibili e funzioni di business restano esposti e la riservatezza è compromessa	tutto il sistema
Non-secured service-to-service communications (n. 39)	le interazioni fra servizi non sono **cifrate** né protette in altro modo: i dati si possono intercettare, alterare, spiare	tutto il sistema
Hardcoded secrets (n. 42)	dati di configurazione sensibili, come password o chiavi delle API, sono scritti **direttamente** nel codice sorgente o negli script di deploy	T1, T23, T4

Un monolite ha un solo ingresso da proteggere; un sistema a microservizi ne ha potenzialmente uno per servizio, più tutte le comunicazioni interne, che viaggiano in rete. Molti di questi anti-pattern hanno la stessa cura: un **gateway** che sia davvero l'unico punto d'ingresso, con un'**autenticazione centralizzata**, servizi raggiungibili solo dalla rete interna, comunicazioni cifrate e segreti custoditi fuori dal codice.

Organizzazione dei team

Anti-pattern	Che cosa succede	Dove
Golden hammer (n. 44)	una tecnologia familiare viene applicata a **ogni** problema, adatta o no: gli sviluppatori riusano quella che conoscono bene anche dove non serve	tutto il sistema
Too many standards (n. 46)	si usano troppi linguaggi, protocolli o framework diversi. I microservizi permettono la diversità tecnologica, ma un'eterogeneità **eccessiva** rende difficile la manutenzione, soprattutto quando gli sviluppatori cambiano, e complica integrazione ed evoluzione	T1, T23, T4, T8
Inadequate techniques support (n. 47)	il team usa strumenti, processi o metodi poco adatti a costruire e gestire microservizi, spesso pur avendo a disposizione alternative migliori	tutto il sistema
No CI/CD (n. 53)	non si adottano l'**integrazione continua** e la consegna (o il rilascio) continui	tutto il sistema
Manual configuration (n. 54)	host, servizi e istanze sono configurati **a mano**: non c'è una separazione netta fra il codice e la gestione della configurazione, e l'automazione è ostacolata	tutto il sistema
Insufficient monitoring (n. 55) e Local logging (n. 58)	prestazioni e guasti dei microservizi non sono **monitorati**, quindi i problemi sono difficili da scoprire e da risolvere; e ogni servizio gestisce i propri log per conto suo, senza aggregarli in un punto unico	tutto il sistema
Dismiss documentation (n. 56)	le API esposte non sono **documentate** a dovere: diventa difficile avere una visione d'insieme del sistema	tutto il sistema

Too many standards è l'altra faccia di uno dei vantaggi dei microservizi, la libertà di scelta delle tecnologie, e qui il ricambio degli sviluppatori è estremo: a ogni edizione del corso arrivano studenti nuovi. Senza CI/CD e con la configurazione fatta a mano non si possono migliorare le metriche **DORA**, che erano la ragione stessa per adottare i microservizi. Senza monitoraggio e senza log centralizzati manca l'**osservabilità** su cui si basa il fail-over.

Che cosa insegna questo caso

Quasi tutti gli anti-pattern riguardano **tutto il sistema**, non un singolo servizio: versioni delle API, gateway, autenticazione, tolleranza ai guasti, CI/CD, log centralizzati. Sono le decisioni che stanno **fra** i servizi, e che nessun team, sviluppando il proprio servizio per conto suo (qui addirittura in edizioni diverse del corso), aveva il compito di prendere. È la «complessità operativa» del prezzo dei microservizi, e insieme un'illustrazione della legge di Conway, a cui è dedicata la prossima sezione: team che non comunicano producono servizi che non si integrano.

Architettura e organizzazione dei team

Che cosa c'entra l'organizzazione dei team con l'architettura? Moltissimo.

Legge di Conway

«Qualunque organizzazione che progetta un sistema produrrà un progetto la cui struttura è una copia della struttura di comunicazione dell'organizzazione.»

— Melvin Conway, *How do committees invent?*, Datamation, 1968

Il motivo è che l'accoppiamento del software è reso possibile e incoraggiato dalla **comunicazione fra persone**. Se si può parlare con l'autore di un pezzo di codice, è più facile capire davvero quel codice; e quindi è più facile che il proprio codice interagisca con esso, cioè che vi si **accoppi**. Due team che non si parlano produrranno, quasi inevitabilmente, due moduli con un'interfaccia stretta fra loro.

Come si organizza un team grande

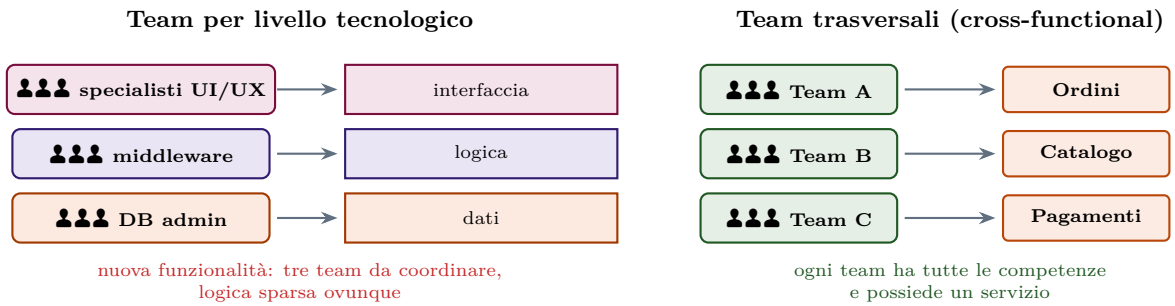


Figura 12.13: La legge di Conway all'opera: team divisi per tecnologia producono un'architettura a livelli; team trasversali producono servizi organizzati attorno alle capacità di business.

Spesso il management organizza i team in base al **livello tecnologico**: specialisti di interfaccia, di middleware, di database. Team funzionali divisi per livello portano ad **architetture a livelli**. E se serve aggiungere una nuova funzionalità? Bisogna coordinare tutti i team, e la logica finisce un po' ovunque.

I microservizi favoriscono invece team **piccoli e indipendenti**, e **trasversali** (*cross-functional*): ogni team include l'intera gamma di competenze necessarie (interfaccia, logica, dati, test, rilascio) e possiede uno o più servizi.

Che fare con la legge di Conway

- **Ignorarla**: non tenerne conto, perché non se ne è mai sentito parlare o perché si pensa che non valga nel proprio caso. *Spoiler*: vale.
- **Accettarla**: riconoscerne l'impatto, e fare in modo che l'architettura non si scontri con i canali di comunicazione del team.
- **La manovra di Conway inversa** (*Inverse Conway Maneuver*): cambiare la struttura di comunicazione del team per **favorire l'architettura desiderata**. Se vogliamo tre servizi indipendenti, organizziamo tre team indipendenti.

Il messaggio da portare a casa

La scomposizione di un sistema e la scomposizione dell'organizzazione che lo sviluppa **non si possono fare indipendentemente**: né all'inizio del progetto, né durante tutto il suo ciclo di vita. Far evolvere l'architettura e riorganizzare le persone devono andare di pari passo.

In sintesi

- L'architettura è l'insieme delle **decisioni principali**: quelle difficili da cambiare in seguito. Una buona qualità interna costa all'inizio, ma ripaga in poche settimane.
- **Scalabilità**: le prestazioni crescono in proporzione alle risorse. Si scala orizzontalmente, verticalmente o diagonalmente; per scalare orizzontalmente serve un **load balancer**, che non deve diventare un collo di bottiglia o un single point of failure.
- **Disponibilità**: si misura in «nove» e si ottiene con il **fail-over** attivo-passivo o attivo-attivo, con heartbeat e osservabilità per accorgersi dei guasti.
- **Dati**: le proprietà **ACID** rendono difficile scalare un RDBMS; le tecniche sono replica (master-slave, master-master), federazione, sharding, denormalizzazione. I database **NoSQL** (BASE) scalano meglio rinunciando alla consistenza immediata: **consistenza finale**.
- Le **code di messaggi** rendono asincrone le elaborazioni lente e riducono la latenza; un broker serve anche a consegnare lo stesso evento a più consumatori, per esempio a tutti i dispositivi di un utente.
- I **microservizi** danno confini forti, rilasci indipendenti e libertà tecnologica, al prezzo di complessità, consistenza finale e costi operativi: vanno adottati solo per sistemi complessi, progettandone i confini con il DDD (bounded context, aggregati).
- Il **caso reale** di un gioco a microservizi mostra gli **anti-pattern** tipici (servizi chiacchieroni, endpoint scritti nel codice, nessun gateway unico, autenticazione sparsa, niente CI/CD né log centralizzati): quasi tutti riguardano ciò che sta **fra** i servizi.
- **Legge di Conway**: la struttura del sistema copia quella della comunicazione dell'organizzazione. Architettura e organizzazione dei team vanno progettate insieme.

Parte VI

Infrastruttura e servizi

Virtualizzazione e cloud computing

Nel capitolo precedente abbiamo visto le architetture che servono milioni di utenti: repliche, bilanciatori di carico, microservizi. Tutte presuppongono di avere a disposizione **molte macchine**, da accendere, spegnere e spostare con facilità. Questo capitolo racconta da dove vengono quelle macchine: la **virtualizzazione**, che trasforma un server fisico in tanti server «virtuali», e il **cloud computing**, che vende quelle risorse come un servizio. È il punto di vista del **deployment**: l'ultimo passo del System Design, in cui si decide su quale hardware e software gireranno i sottosistemi (il diagramma di deployment del Capitolo 9).

Perché la virtualizzazione

Virtualizzazione

La **virtualizzazione** consiste nel creare una versione **virtuale** di una risorsa che normalmente è fornita fisicamente e appartiene a un sistema: un computer, un disco, una rete. È una tecnica per aumentare l'**efficacia** e l'**efficienza** dei data center complessi.

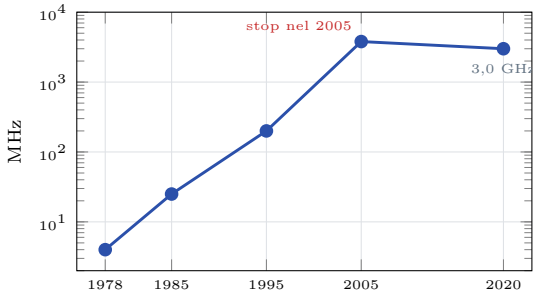
L'idea è vecchia: risale agli anni Sessanta, ai mainframe IBM che andavano divisi fra molti utenti. È tornata di moda una quindicina di anni fa grazie a un cambiamento nei processori, che conviene raccontare dall'inizio.

L'evoluzione delle CPU

I microprocessori hanno sempre visto crescere rapidamente le prestazioni e calare i costi. Per decenni la strategia per aumentare le prestazioni è stata una sola: aumentare la **frequenza** del processore, cioè la sua «velocità».

- 1978: Intel 8086, 4 MHz;
- 1985: Intel 386, 25 MHz (circa 6 volte tanto);
- 1995: Intel Pentium Pro, 200 MHz (circa 8 volte);
- 2005: Intel Pentium 4, 3,8 GHz (circa 19 volte);
- 2020: Intel i9-10980XE, 3,0 GHz.

Nel 2005 la corsa si **ferma**: quindici anni dopo un processore di fascia alta ha una frequenza addirittura più bassa. Si erano raggiunti i limiti fisici del silicio (calore e consumi crescono troppo con la frequenza).



La soluzione è stata **moltiplicare le unità di calcolo**: invece di un processore più veloce, più processori (i **core**) sullo stesso chip. Nel 2020 l'Intel i9-10980XE a 3,0 GHz ha 18 unità di calcolo, l'AMD Ryzen Threadripper 3970X a 3,7 GHz ne ha 32.

Il problema dei multi-core

Tradizionalmente i programmi sono stati scritti per girare su un computer con **una sola CPU**, e la grande maggioranza delle applicazioni è fatta di programmi **sequenziali**. Un programma sequenziale usa un core alla volta, per quanti ne abbia la macchina. Come si sfruttano allora più unità in parallelo?

Esempio 13.1 — Un programma sequenziale non usa i core in più

Il programma somma i resti di tre miliardi di numeri due volte: la prima con uno *stream* normale, la seconda con uno *stream* parallelo, che divide il lavoro fra tutti i core disponibili.

Somma.java

```java
import java.util.stream.LongStream;

public class Somma {
  public static void main(String[] args) {
    int core = Runtime.getRuntime().availableProcessors();
    System.out.println("Unità di calcolo disponibili: " + core);

    long n = 3_000_000_000L;
    long t0 = System.nanoTime();
    long s1 = LongStream.rangeClosed(1, n).map(x -> x % 7).sum();            // un solo core
    long t1 = System.nanoTime();
    long s2 = LongStream.rangeClosed(1, n).parallel().map(x -> x % 7).sum(); // tutti i core
    long t2 = System.nanoTime();

    System.out.println("Stesso risultato: " + (s1 == s2));
    System.out.printf("Sequenziale: %d ms%n", (t1 - t0) / 1_000_000);
    System.out.printf("Parallelo:   %d ms%n", (t2 - t1) / 1_000_000);
  }
}
```

Terminale

```
$ java Somma.java
Unità di calcolo disponibili: 4
Stesso risultato: true
Sequenziale: 5888 ms
Parallelo:   1301 ms
```

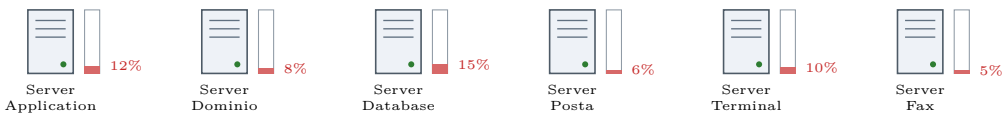
Su una macchina con 4 core la versione parallela è circa quattro volte più veloce. Ma la differenza l'ha fatta il **programmatore**, che ha dovuto riscrivere il calcolo; la prima versione, quella che si scrive di solito, lascia tre core su quattro a guardare.

Riscrivere ogni applicazione in forma parallela non è realistico. La virtualizzazione prende la strada opposta: invece di dividere **un** programma su più core, fa girare sulla stessa macchina **tanti sistemi indipendenti**, ciascuno con i propri programmi sequenziali. Per capire perché questo sia utile bisogna guardare com'era fatto un data center.

Il data center tradizionale

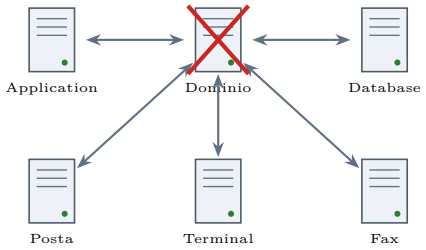
Nel data center tradizionale ogni server offre **un solo servizio** o una sola applicazione (*one workload, one box*): un server per l'applicazione, uno per il dominio (l'autenticazione degli utenti), uno per il database, uno per la posta, uno per il terminal server, uno per il fax. Un nuovo servizio significa un **nuovo server**, con il suo storage.

ogni server consuma corrente, scalda, occupa spazio e va gestito fisicamente; la barra rossa è l'uso medio della CPU



Le conseguenze

- **Risorse sottoutilizzate**. La maggior parte dei server usa in media fra il **5% e il 15%** della propria capacità di calcolo. Per i servizi critici serve inoltre la **ridondanza**: computer di riserva che normalmente non lavorano affatto.
- **Costi di installazione e gestione elevati**. Ogni server consuma molta elettricità, emette calore, occupa spazio e va gestito fisicamente; gli aggiornamenti su larga scala costano molto, e i backup vanno fatti macchina per macchina.
- **Servizi intrinsecamente interconnessi**. Il server di dominio, che autentica gli utenti, viene interrogato da tutti gli altri servizi. Che cosa succede se si guasta?



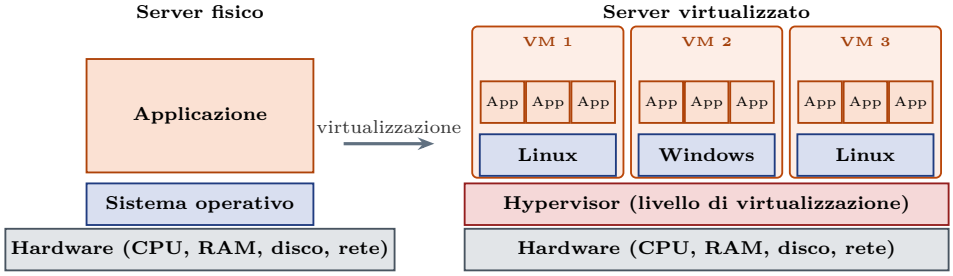
Se un server si guasta, può bloccare i servizi degli altri server che comunicano con lui. Ne derivano:

- l'interruzione delle operazioni legate a quel servizio;
- nei casi più gravi, l'interruzione di **tutte** le attività dell'azienda;
- costi in più (il fermo e il ripristino);
- fermi non sempre prevedibili.

LE MACCHINE VIRTUALI

L'idea

Un server fisico è fatto di una serie di **risorse hardware**, di un **sistema operativo** e di uno o più **programmi applicativi**. L'idea: se la CPU del server ha più unità di calcolo parallele, possiamo «simulare» l'esistenza di **più computer**, indipendenti dall'hardware «reale» sottostante? Ciascuno avrebbe il suo sistema operativo e i suoi programmi, e le unità di calcolo verrebbero finalmente usate.



Hypervisor

L'**hypervisor** è un nuovo strato software che gira **prima** del sistema operativo, direttamente sopra l'hardware. Simula l'esistenza di più computer «virtuali» su un unico computer fisico: fornisce loro le risorse fisiche e le gestisce in modo concorrente a beneficio delle **macchine virtuali** (VM).

Su ogni computer virtualizzato si installa un sistema operativo **normale**, che non ha modo di sapere se sta girando su un computer «vero» o virtuale. Dal suo punto di vista vede una CPU, della memoria, un disco e una scheda di rete: sono risorse virtuali, ritagliate dall'hypervisor da quelle fisiche.

Le proprietà di una macchina virtuale

- **Indipendenza dall'hardware**. L'hypervisor simula una configurazione di server **standard**, nascondendo l'hardware fisico reale. Di conseguenza le VM possono girare su server fisici diversi senza bisogno di essere riconfigurate.
- **Una VM è un file**. Una macchina virtuale è un file che contiene il sistema operativo, le applicazioni, i dati e la rappresentazione della memoria del server. Il file conserva lo stato del server virtuale e permette di copiarlo su un numero qualsiasi di server fisici. Esistono strumenti commerciali che trasformano il contenuto di un server fisico in una VM.
- **Isolamento**. Ogni VM è completamente isolata dalle altre VM che girano sullo stesso server fisico: se una subisce un attacco o va in crash, le altre non ne risentono.
- **Gestione centralizzata**. Tutte le VM di un data center si gestiscono da **un'unica finestra**: la gestione dell'intero data center è centralizzata, e l'amministrazione si semplifica molto.

Esempio 13.2 — Creare e gestire una VM dalla riga di comando

Con VirtualBox, un hypervisor gratuito che si può installare sul proprio portatile, una VM si crea e si configura con il comando `VBoxManage`. Le risorse che le diamo sono numeri in un file di configurazione, non pezzi di hardware.

crea-vm.sh

```
# crea e registra una nuova macchina virtuale
VBoxManage createvm --name web01 --ostype Ubuntu_64 --register
# le assegna 2 CPU virtuali e 4 GB di RAM, presi da quelli del server fisico
VBoxManage modifyvm web01 --cpus 2 --memory 4096
# crea il disco virtuale: un file che puo' crescere fino a 20 GB
DISCO="$HOME/VirtualBox VMs/web01/web01.vdi"
VBoxManage createmedium disk --filename "$DISCO" --size 20480
VBoxManage storagectl web01 --name SATA --add sata
VBoxManage storageattach web01 --storagectl SATA --port 0 --type hdd --medium "$DISCO"
# la avvia senza finestra, come si fa su un server
VBoxManage startvm web01 --type headless
```

Che una VM sia «un file» si vede guardando la sua cartella:

Terminale

```
$ ls -lh ~/"VirtualBox VMs/web01/"
-rw------- 1 anna anna 6.1G web01.vdi     # il disco: occupa solo lo spazio usato davvero
-rw------- 1 anna anna 9.4K web01.vbox    # la configurazione (CPU, RAM, schede di rete), in XML
```

Copiare questi due file su un altro computer con VirtualBox significa **spostare il server**.

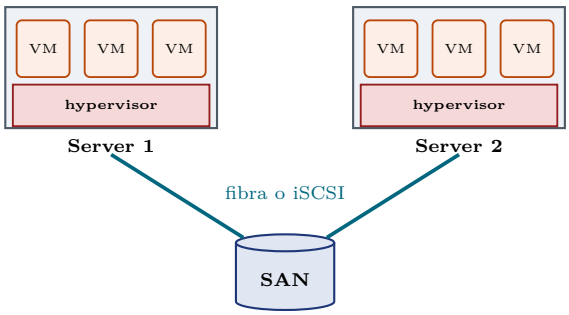
LA VIRTUALIZZAZIONE IN ESERCIZIO

Un server o un'infrastruttura

Nella virtualizzazione di un **singolo server** le macchine virtuali risiedono nello storage del server fisico: dal basso verso l'alto ci sono il server fisico, l'hypervisor, le VM e, dentro ciascuna, il sistema operativo (Microsoft, Linux…) e le applicazioni (gestionali, office, database…). Se però il server si guasta, si fermano tutte le sue VM insieme: abbiamo concentrato in una macchina sola i rischi di sei.

Virtualizzazione enterprise ad alta disponibilità

Un data center enterprise deve avere almeno **due server fisici** e un'unità disco condivisa, la **SAN** (*Storage Area Network*), collegati fra loro ad altissima velocità (fibra ottica o iSCSI). Nella virtualizzazione enterprise le macchine virtuali risiedono nello storage **SAN**, non nei server.



I file delle VM stanno nella SAN, visibile a entrambi i server. Ogni server può quindi avviare **qualunque** VM: per eseguirla basta avere il suo file.

La migrazione da un data center tradizionale avviene **dal fisico al virtuale** (P2V, *physical to virtual*): i sei server della figura iniziale diventano sei VM sui due server dell'infrastruttura, con i loro dischi nella SAN.

Come funziona l'alta disponibilità

La virtualizzazione garantisce la **continuità operativa** grazie alla funzione di **alta disponibilità** (HA, *High Availability*): se un server si guasta, le sue macchine virtuali vengono **migrate immediatamente** sugli altri server disponibili e riavviate. Il fermo si riduce al tempo necessario a riavviare le VM.

Le tecnologie più recenti offrono anche la **fault tolerance** (FT), che garantisce la **continuità** del funzionamento di una VM dopo il guasto di un server, eliminando del tutto il fermo: è attualmente la soluzione migliore per la continuità operativa. È il fail-over del Capitolo 12, applicato alle macchine invece che ai processi.

La migrazione a caldo

Live migration

La migrazione «a caldo» (**live migration**) sposta una VM **in esecuzione** da un server fisico a un altro, senza spegnerla. Permette di fare manutenzione su un server o di bilanciare il carico fra i server mantenendo operativi i server virtuali.

Esempio 13.3 — Una migrazione a caldo con libvirt

Su Linux l'hypervisor KVM si gestisce con `virsh`, il client di *libvirt*. Dovendo spegnere `server1` per manutenzione, si spostano le sue VM su `server2` mentre lavorano:

```
>_ Terminale ×
$ virsh list
 Id   Name     State
----------------------
 1    web01    running
 2    db01     running

$ virsh migrate --live --persistent web01 qemu+ssh://server2/system
$ virsh migrate --live --persistent db01  qemu+ssh://server2/system
```

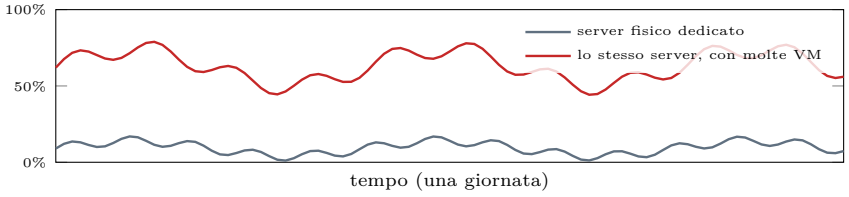
Durante la migrazione la memoria della VM viene copiata sull'altro server mentre la VM continua a girare; alla fine la VM si ferma per una frazione di secondo, giusto il tempo di copiare le ultime pagine modificate. Il disco non va copiato: sta nella SAN, che entrambi i server vedono.

I VANTAGGI DELLA VIRTUALIZZAZIONE

Il consolidamento

Con la virtualizzazione si parla di **consolidamento** del data center: sfruttando al massimo le risorse di calcolo parallele dei processori moderni, si riduce in modo significativo il numero di server necessari a fornire i servizi IT di un'organizzazione.

- **Ottimizzazione dell'hardware**. Sistemi operativi diversi girano contemporaneamente su una sola macchina, e la CPU passa dal 5–15% di un server dedicato a un uso molto più alto.
- **Ottimizzazione dello spazio su disco**. A una VM viene allocato solo lo spazio disco di cui ha strettamente bisogno; il resto del volume resta disponibile per nuove VM.



- **Backup e disaster recovery**. L'intero server (sistema operativo, applicazioni, dati, dispositivi e stato) non è altro che un file: fare copie di sicurezza per il disaster recovery diventa facile.
- **Indipendenza dall'hardware**, come visto sopra.
- **Rilasci semplificati**. Si possono **clonare** le VM «a caldo» per fare test e aggiornamenti senza compromettere l'integrità dell'originale.
- **Riduzione dei costi IT**. Meno server significa risparmiare energia, condizionamento e spazio fisico del data center.
- **Riduzione dei costi del personale**. Con la gestione centralizzata, da un'unica interfaccia di amministrazione, il reparto IT è più produttivo.

Esempio 13.4 — Provare un aggiornamento senza rischi

Prima di aggiornare il sistema operativo del server web, si fotografa lo stato della VM e se ne fa un clone su cui provare l'aggiornamento. Se qualcosa va storto, si torna indietro in pochi secondi.

aggiornamento.sh

```
# una "fotografia" dello stato attuale (disco e configurazione)
VBoxManage snapshot web01 take prima-dell-aggiornamento
# un clone indipendente su cui provare l'aggiornamento
VBoxManage clonevm web01 --name web01-test --register
VBoxManage startvm web01-test --type headless
# ... l'aggiornamento sul clone rompe qualcosa? Si butta via il clone:
VBoxManage unregistervm web01-test --delete
# e se l'avessimo fatto sull'originale, si torna alla fotografia
VBoxManage snapshot web01 restore prima-dell-aggiornamento
# il backup per il disaster recovery e' un file da copiare altrove
VBoxManage export web01 --output web01-backup.ova
```

Esempio 13.5 — Il data center di una pubblica amministrazione a Napoli

- Circa **800 macchine virtuali** in esecuzione in parallelo, con bilanciamento del carico;
- circa **30 server fisici**;
- **2 SAN**, con circa 500 TB di spazio;
- la gestione di 800 macchine da **2 console** di amministrazione.

Il rapporto di consolidamento è di circa $800/30 \approx 27$ VM per server fisico. Senza virtualizzazione sarebbero serviti centinaia di server, ciascuno usato al 5–15%.

Macchine virtuali e container

Il Capitolo 12 ha mostrato un'applicazione a microservizi in cui ogni servizio è un **container** Docker. Un container non è una VM: non contiene un sistema operativo intero, ma solo l'applicazione con le sue librerie, e **condivide il kernel** del sistema che lo ospita. È più leggero (si avvia in un secondo, pesa decine di MB invece di GB) e meno isolato. Le due tecniche convivono: nel cloud i container girano quasi sempre dentro macchine virtuali.

Dockerfile

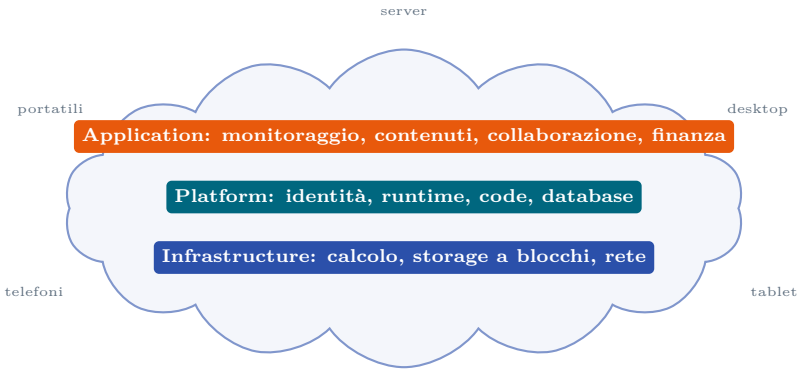
```
# si parte da un'immagine con Java gia' installato
FROM eclipse-temurin:21-jre
WORKDIR /app
# si aggiunge solo l'applicazione: niente sistema operativo
COPY target/negozio.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Il cloud computing

Cloud computing

Nel **cloud computing** le risorse IT sono fornite **come un servizio**: potenza di calcolo, storage, database, code di messaggi. Si usano quando servono e si pagano per quanto si usano.

- I cloud sfruttano le **economie di scala** dell'hardware di largo consumo: storage economico, reti ad alta banda, processori multicore.
- I data center sono **distribuiti geograficamente**.
- Le offerte principali sono quelle di Microsoft, Amazon, Google e pochi altri.



La virtualizzazione è la tecnica che rende possibile il cloud: un fornitore con centinaia di migliaia di server fisici li divide in milioni di VM e le affitta. A cambiare, per chi le usa, è il **modello economico**: non si comprano server, si noleggia capacità.

Scalabilità ed elasticità

Scalabilità ed elasticità

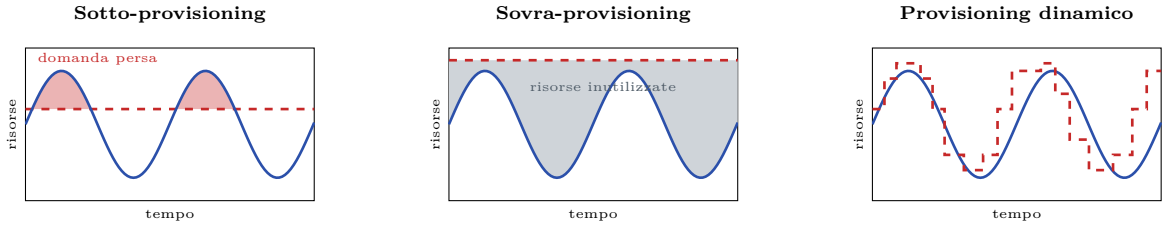
- La **scalabilità** è la capacità di un sistema, di una rete o di un processo di gestire quantità di lavoro crescenti in modo armonioso, o di essere ingrandito facilmente.
- L'**elasticità** è la capacità di far **crescere o ridurre** dinamicamente le risorse dell'infrastruttura secondo il bisogno, per adattarsi ai cambiamenti del carico in modo **autonomo**, massimizzando l'uso delle risorse.

La scalabilità (Capitolo 12) dice che **si può** crescere; l'elasticità aggiunge che si cresce e si **decresce** da soli, seguendo il carico. Le due proprietà si ottengono con il **provisioning dinamico** e con un progetto **multi-tenant**, in cui più clienti (*tenant*) condividono la stessa infrastruttura restando separati.

Provisioning dinamico

Nel **provisioning dinamico** le istanze di calcolo vengono allocate o rilasciate (*provisioned*) da una console di amministrazione o da un'applicazione client, dall'amministratore dei server o della rete o da qualunque altro utente abilitato, invece di essere acquistate e installate.

Nel modello tradizionale la capacità si decide **in anticipo**, e si sbaglia in uno dei due modi:



- Chi **sottostima** l'uso del sistema compra troppo poco (*under-provisioning*): nei picchi la domanda supera la capacità, e si perdono ricavi; se succede spesso, gli utenti se ne vanno e la domanda stessa cala (si perdono gli **utenti**).
- Chi lo **sovrastima** compra troppo (*over-provisioning*): la capacità copre sempre i picchi, ma per la maggior parte del tempo le risorse restano inutilizzate (e pagate).

La soluzione è allocare le risorse **dinamicamente**, per seguire le variazioni stagionali della domanda, le differenze fra settori diversi e i picchi improvvisi dovuti a eventi straordinari (un saldo, una diretta, un esame con mille iscritti che si prenotano nello stesso minuto).

Esempio 13.6 — Una regola di elasticità, in codice e in configurazione

Un *autoscaler* misura periodicamente l'uso medio della CPU delle istanze e ne calcola il numero adatto. La regola di Kubernetes è semplice: se l'uso è il doppio dell'obiettivo, servono il doppio delle istanze.

Autoscaler.java

```java
public class Autoscaler {
  private final int min, max;           // limiti decisi da chi paga
  private final double cpuObiettivo;    // per esempio 60%

  public Autoscaler(int min, int max, double cpuObiettivo) {
    this.min = min;
    this.max = max;
    this.cpuObiettivo = cpuObiettivo;
  }

  /** Quante istanze servono perche' l'uso medio della CPU torni all'obiettivo. */
  public int istanzeDesiderate(int istanzeAttuali, double cpuMedia) {
    int desiderate = (int) Math.ceil(istanzeAttuali * cpuMedia / cpuObiettivo);
    return Math.max(min, Math.min(max, desiderate));
  }
}
```

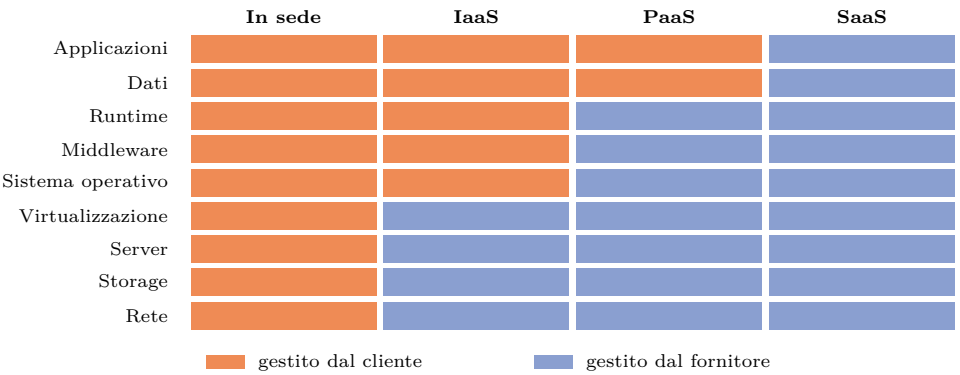
Con `new Autoscaler(2, 20, 60)`: di giorno 4 istanze al 90% diventano $\lceil 4 \cdot 90/60 \rceil = 6$; di notte 6 istanze al 15% diventerebbero $\lceil 6 \cdot 15/60 \rceil = 2$, il minimo. In un cloud non si scrive questo codice: lo si **dichiara**, e il fornitore lo esegue.

autoscaler.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: negozio-web
spec:
  scaleTargetRef:           # che cosa scalare: il servizio web del negozio
    apiVersion: apps/v1
    kind: Deployment
    name: negozio-web
  minReplicas: 2            # mai meno di due, per la disponibilita'
  maxReplicas: 20           # mai piu' di venti, per il conto a fine mese
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

I MODELLI DI SERVIZIO

Il cloud si offre a tre livelli, impilati uno sull'altro: l'infrastruttura (**IaaS**), la piattaforma (**PaaS**) e il software (**SaaS**). Salendo, il fornitore si prende cura di una parte sempre più grande della pila, e il cliente di una parte sempre più piccola.

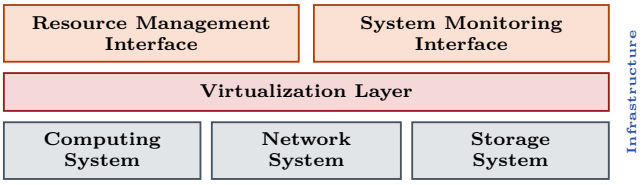


Infrastructure as a Service (IaaS)

IaaS

Al cliente viene data la possibilità di allocare **elaborazione, storage, reti** e altre risorse di calcolo fondamentali, su cui può installare ed eseguire software arbitrario, compresi sistemi operativi e applicazioni. Il cliente non gestisce né controlla l'infrastruttura cloud sottostante, ma ha il controllo dei **sistemi operativi**, dello storage, delle applicazioni installate ed eventualmente un controllo limitato di alcuni componenti di rete.

Esempi: Amazon EC2, Eucalyptus, OpenNebula.



La tecnica che rende possibile l'IaaS è la **virtualizzazione**: un'astrazione delle risorse logiche rispetto alle risorse fisiche sottostanti. Sposta il sistema operativo sopra l'hypervisor; più sistemi operativi condividono l'hardware fisico e offrono servizi diversi, migliorando utilizzo, disponibilità, sicurezza e comodità. È esattamente la figura del server virtualizzato vista all'inizio del capitolo: l'IaaS affitta le VM.

Esempio 13.7 — Due VM in affitto

Con la riga di comando di AWS si noleggiano due macchine virtuali EC2. Scegliamo noi l'immagine del sistema operativo e la taglia; da lì in poi il sistema operativo è nostro: aggiornamenti, Java, sicurezza.

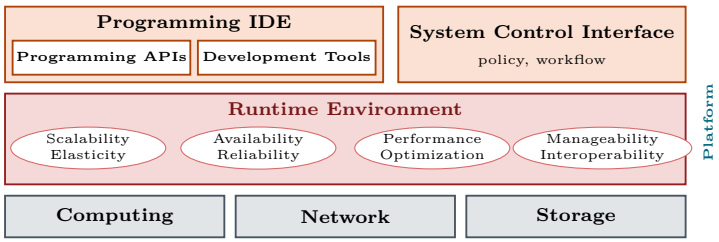
```
>_ Terminale ×
# due VM con 2 vCPU e 2 GB di RAM, partendo da un'immagine Ubuntu
$ aws ec2 run-instances --image-id ami-0abcdef1234567890 \
      --instance-type t3.small --count 2 --key-name chiave-ssh
# ci si collega come a un server qualsiasi, e si installa cio' che serve
$ ssh ubuntu@203.0.113.10 'sudo apt install -y openjdk-21-jre'
# quando non servono piu' si restituiscono, e si smette di pagare
$ aws ec2 terminate-instances --instance-ids i-0a1b2c3d4e5f60718 i-0f1e2d3c4b5a69788
```

Platform as a Service (PaaS)

PaaS

Al cliente viene data la possibilità di installare (*deploy*) sull'infrastruttura cloud applicazioni create da lui o acquistate, scritte con i linguaggi e gli strumenti supportati dal fornitore. Il cliente non gestisce né controlla l'infrastruttura sottostante (rete, server, sistemi operativi, storage), ma ha il controllo delle **applicazioni** installate ed eventualmente della configurazione dell'ambiente che le ospita.

Esempi: Microsoft Azure, Google App Engine, Hadoop.



La tecnica che rende possibile il PaaS è il progetto dell'**ambiente di esecuzione** (*runtime environment*): l'insieme dei servizi software disponibili, di solito realizzato da una collezione di librerie. Le sue proprietà tipiche sono la gestibilità e l'interoperabilità, le prestazioni e l'ottimizzazione, la disponibilità e l'affidabilità, la scalabilità e l'elasticità. Gli utenti sviluppano i propri servizi con un **IDE** che deve integrare tutte le funzionalità dell'ambiente di esecuzione e offrire strumenti di sviluppo come profiler, debugger e ambienti di test. Le API di programmazione cambiano da un fornitore all'altro, ma le operazioni comuni restano: calcolo, storage e comunicazione.

Esempio 13.8 — Consegnare il codice, non le macchine

Con Google App Engine non si noleggiano VM: si consegna l'applicazione e un file che descrive l'ambiente che le serve. Sistema operativo, Java, bilanciamento del carico e numero di istanze sono affare del fornitore.

app.yaml

```yaml
runtime: java21                 # l'ambiente di esecuzione lo fornisce la piattaforma
instance_class: F2
automatic_scaling:              # l'elasticita' si chiede, non si costruisce
  min_instances: 1
  max_instances: 10
  target_cpu_utilization: 0.65
```

Terminale

```
$ mvn package                   # si compila l'applicazione, come sempre
$ gcloud app deploy             # e la si consegna alla piattaforma
```

Rispetto all'esempio IaaS non c'è più nessun `ssh` né `apt install`: il confine fra ciò che gestiamo noi e ciò che gestisce il fornitore si è alzato di due strati.

Software as a Service (SaaS)

SaaS

Al cliente viene data la possibilità di usare le **applicazioni del fornitore**, che girano su un'infrastruttura cloud. Le applicazioni sono accessibili da vari dispositivi attraverso un'interfaccia leggera, come un browser (per esempio la posta via web). Il cliente non gestisce né controlla l'infrastruttura né le singole funzionalità dell'applicazione, con l'eventuale eccezione di alcune impostazioni specifiche dell'utente.

Esempi: Google Apps (Gmail, Google Docs, Google Sites…), Salesforce.com, EyeOS.

Le applicazioni tradizionali devono portare la propria interfaccia su una piattaforma **web**. Si trovano SaaS in domini diversi:

- applicazioni **generali**: suite per l'ufficio, multimedia, messaggistica istantanea;
- applicazioni **aziendali**: ERP, CRM, sistemi di compravendita;
- applicazioni **scientifiche**: simulazioni aerospaziali, simulazioni biochimiche;
- applicazioni per la **pubblica amministrazione**: sistemi sanitari nazionali, servizi di trasporto pubblico.

Per l'utente un SaaS è solo un sito. Per lo sviluppatore che deve integrarlo nel proprio sistema è spesso una **API**: per esempio, il calendario di un utente si legge con una richiesta HTTP, senza installare nulla.

```
>_ Terminale ×
1 $ curl -H "Authorization: Bearer $TOKEN" \
2        https://www.googleapis.com/calendar/v3/users/me/calendarList
```

Il prossimo capitolo spiega come sono fatte queste API, e che cos'è quel token.

I MODELLI DI DEPLOYMENT



Come si dispiega un sistema cloud? Ci sono tre modelli principali: cloud **pubblico**, **privato** e **ibrido**. Ciascuno può avere le caratteristiche viste finora; si differenziano soprattutto per la **portata** e l'**accesso** ai servizi cloud, cioè per chi può usarli.

Cloud pubblico

Cloud pubblico

L'infrastruttura cloud è messa a disposizione del **pubblico** o di un grande gruppo industriale, ed è di proprietà di un'organizzazione che vende servizi cloud. È detto anche cloud esterno o *multi-tenant*: un ambiente accessibile apertamente.

Caratteristiche: infrastruttura omogenea, politiche comuni, risorse condivise e multi-tenant, infrastruttura presa in affitto, economie di scala.

Cloud privato

Cloud privato

L'infrastruttura cloud è gestita **solo per un'organizzazione**. Può essere amministrata dall'organizzazione stessa o da terzi, e trovarsi nella sede (*on premise*) o altrove. Detto anche cloud interno, limita volutamente l'accesso alle risorse a chi appartiene all'organizzazione che lo possiede.

Caratteristiche: infrastruttura eterogenea, politiche personalizzate, risorse dedicate, infrastruttura interna, controllo completo da un capo all'altro. Il data center della pubblica amministrazione di Napoli è, di fatto, un cloud privato.

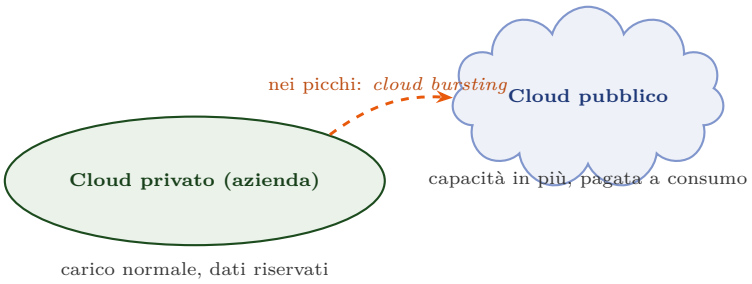
	Cloud pubblico	**Cloud privato**
Infrastruttura	omogenea	eterogenea
Politiche	definite in comune	personalizzate
Risorse	condivise, multi-tenant	dedicate
Costi	spesa operativa (si paga l'uso)	spesa in conto capitale (si compra)
Economia	grandi economie di scala	controllo completo

Cloud ibrido

Cloud ibrido

L'infrastruttura è la **composizione** di due o più cloud (privati o pubblici) che restano entità distinte, ma sono legati da una tecnologia, standard o proprietaria, che permette di spostare dati e applicazioni fra l'uno e l'altro.

L'esempio tipico è il **cloud bursting**: un'azienda serve il carico normale con il proprio cloud privato e, nei picchi, «trabocca» sul cloud pubblico, pagando la capacità in più solo quando serve. È il provisioning dinamico della sezione precedente, applicato fra due cloud.



L'ecosistema del cloud

Mettendo insieme modelli di servizio e di deployment si ottiene la mappa del mercato: cloud pubblici, privati, ibridi e **di comunità** (condivisi da organizzazioni con esigenze comuni), e a ogni livello fornitori diversi.

Livello	Esempi di fornitori e prodotti
SaaS	Google Apps, Salesforce, Microsoft Dynamics CRM
PaaS	Microsoft Azure, Google App Engine, Force.com, IBM, Hadoop
IaaS	Amazon Web Services, Eucalyptus, GoGrid, Sun, AT&T

In sintesi

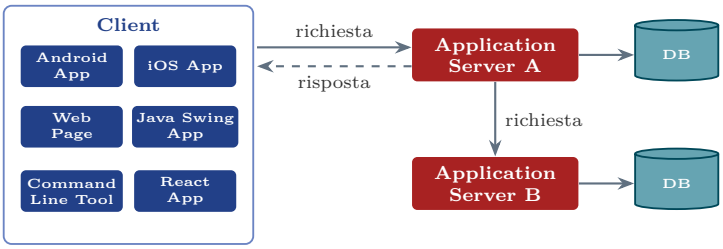
- Dal 2005 le CPU non diventano più veloci ma **più numerose**: i programmi sequenziali non sfruttano i core in più.
- Il data center tradizionale (un servizio per server) spreca risorse (5–15% di CPU), costa molto e ha servizi interdipendenti: un guasto può fermare tutto.
- La **virtualizzazione** introduce l'**hypervisor**, che simula più computer su uno fisico. Una VM è indipendente dall'hardware, isolata, gestita centralmente, ed è **un file**.
- Con due server e una **SAN** si ottengono alta disponibilità (riavvio delle VM su un altro server), fault tolerance e **live migration**; con il consolidamento calano server, energia e personale.
- Il **cloud computing** vende le risorse IT come servizio. L'**elasticità** (crescere e decrescere da soli) si ottiene con il **provisioning dinamico**, contro il sotto- e il sovra-provisioning.
- Modelli di servizio: **IaaS** (si affittano VM: il sistema operativo è del cliente), **PaaS** (si consegna il codice), **SaaS** (si usa l'applicazione). Modelli di deployment: cloud **pubblico**, **privato**, **ibrido** (con il cloud bursting).

Il paradigma REST

Nei capitoli sulle architetture abbiamo diviso i sistemi in sottosistemi, servizi e microservizi che comunicano fra loro; nel capitolo sul cloud abbiamo visto che un SaaS, per uno sviluppatore, è spesso un'API. Resta da capire **come** comunicano: con quale protocollo, con quali messaggi, con quali regole. Oggi la risposta più diffusa è **REST**: un modo di usare il protocollo del Web, HTTP, per far dialogare programmi.

Il contesto architetturale

Il software ha bisogno di comunicare attraverso Internet: secondo un rapporto di Akamai del 2019, l'**83%** del traffico web è fatto di **chiamate ad API**, non di pagine richieste da persone. I client sono di tutti i tipi (app Android e iOS, applicazioni desktop in Java Swing, pagine web, app React, strumenti a riga di comando) e parlano con uno o più application server, ciascuno con il proprio database.



Anche un server, quindi, può essere il client di un altro: è il caso dei microservizi del Capitolo 12.

Frontend e backend

Spesso un'applicazione è divisa in due componenti:

- il **backend**, responsabile dei **dati** e della **logica di business**, sul lato server;
- il **frontend**, responsabile dell'**interfaccia utente**, sul lato client. Può essere un'app nativa per dispositivi mobili o desktop, oppure una pagina HTML «più furba», che costruisce un'interfaccia interattiva con JavaScript (le *Single Page Web App*).

I due componenti comunicano attraverso Internet. È l'architettura client/server del Capitolo 11, in cui il «protocollo» fra le due parti è proprio l'API.

Application Programming Interface

API

Le **API** (*Application Programming Interface*) sono i modi in cui i programmi comunicano fra loro.

Come si gestisce la comunicazione attraverso Internet? Le strade possibili sono tre.

1. **Definire a mano un protocollo** sopra TCP o UDP, aprendo socket e così via. Non è molto conveniente e non è

interoperabile: per integrare n API bisognerebbe imparare n protocolli diversi.

2. Usare protocolli e architetture **dedicati**, come CORBA, Java RMI o SOAP. Esistono (o sono esistiti), ma reinventano un'alternativa al Web.
3. Usare semplicemente **i protocolli del Web**: HTTP.

REST

REST

REST (*Representational State Transfer*) **non** è uno standard né un protocollo: è uno **stile architetturale**, cioè un insieme di principi e linee guida che stabiliscono come usare gli standard del Web nelle API. Si basa su **HTTP** e sugli **URI**, e offre un'interfaccia comune e coerente basata su un uso «corretto» di HTTP.

È lo stile prevalente per le API web, con un tasso di adozione del **93,4%**.

Un ripasso di HTTP

HTTP (*HyperText Transfer Protocol*) ha due tipi di messaggi, la **richiesta** e la **risposta**. La prima riga della richiesta contiene il **verbo**, l'**URI** della risorsa e la versione di HTTP; quella della risposta la versione e lo **stato**. Seguono gli **header** e, se serve, il **corpo**.



Esempio 14.1 — Una richiesta e una risposta, così come viaggiano

richiesta

```
GET /todos/1 HTTP/1.1
Host: api.example.com
Accept: application/json
```

risposta

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 46

{"id": 1, "todo": "Learn REST", "done": false}
```

La richiesta non ha corpo: chiede soltanto. La risposta porta nel corpo la rappresentazione della risorsa, in JSON, e nell'header `Content-Type` dice in che formato è.

I verbi e i codici di stato

HTTP può chiedere tipi diversi di operazioni, i **verbi**; la risorsa su cui operare è identificata dall'URI. I verbi principali corrispondono alle quattro operazioni **CRUD** (*Create, Retrieve, Update, Delete*).

Verbo	Operazione	Significato
`POST`	Create	«ecco dei dati nuovi: salvali e **crea** una nuova risorsa»
`GET`	Retrieve	«**dammi** le informazioni sulla risorsa»
`PUT`	Update	«ecco le informazioni **aggiornate** sulla risorsa»
`DELETE`	Delete	«**elimina** questa risorsa»

I **codici di stato** della risposta informano sull'esito della richiesta.

Codice	Significato
200	la richiesta è stata gestita con successo
201	una risorsa è stata creata con successo
400	la richiesta non si capisce (*Bad Request*)
401	autenticazione fallita, o non autorizzato
404	risorsa non trovata
405	metodo non supportato dalla risorsa
500	errore dell'applicazione

La prima cifra dice già molto: 2 è un successo, 4 un errore **del client** (ha chiesto qualcosa di sbagliato), 5 un errore **del server** (la richiesta era giusta, ma il server non è riuscito a soddisfarla).

I FONDAMENTI DI REST



- Un'API REST permette di interagire con delle **risorse** usando HTTP.
- Ogni risorsa è associata a un **URI unico**.
- *«Una risorsa è qualunque cosa sia abbastanza importante da essere citata come una cosa a sé»* (Richardson e Ruby, *RESTful Web Services*).
- Le risorse corrispondono tipicamente, ma non necessariamente, agli **oggetti persistenti del dominio**: le classi del modello di dominio del Capitolo 7.
- Per leggere o modificare le risorse si usano i **verbi HTTP**.
- Un'API REST dovrebbe essere **stateless**: per esempio, non usare sessioni lato server. Ogni richiesta porta con sé tutto ciò che serve per servirla.

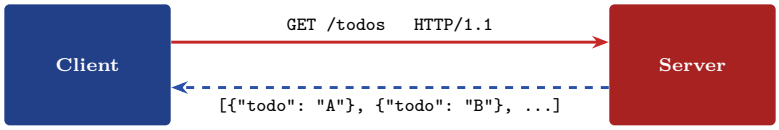
Perché stateless vuol dire scalabile

Se il server non conserva nulla fra una richiesta e l'altra, **qualunque** replica può servire **qualunque** richiesta. Il load balancer del Capitolo 12 può allora mandare ogni richiesta alla replica meno carica, si possono aggiungere repliche quando il carico cresce (l'elasticità del Capitolo 13) e se una replica cade nessun utente perde la propria «sessione». Con le sessioni lato server, invece, ogni utente resta legato alla replica che conosce la sua sessione, o le sessioni vanno condivise fra tutte le repliche.

Lo stato e le sue rappresentazioni

Il nome stesso dice come funziona REST: si **trasferisce** una **rappresentazione** dello **stato**.

- Lo **stato dell'applicazione** sta sul **client** (che cosa sta facendo l'utente, che pagina vede).
- Lo **stato delle risorse** sta sul **server** (quali to-do esistono, e se sono fatti).
- Lo stato delle risorse viaggia usando **rappresentazioni** appropriate: JSON, XML…



I formati più usati per lo scambio dei dati sono **JSON** e **XML**. Ecco la stessa lista di to-do nei due formati.

todos.json

```json
{
  "todos": [{
    "todo": "Learn REST",
    "done": false
  }, {
    "todo": "Learn JavaScript",
    "done": true
  }]
}
```

todos.xml

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<root>
  <todos>
    <todo>
      <text>Learn REST</text>
      <done>false</done>
    </todo>
    <todo>
      <text>Learn JavaScript</text>
      <done>true</done>
    </todo>
  </todos>
</root>
```

JSON è più compatto e si legge direttamente come un oggetto JavaScript; per questo è diventato il formato prevalente. Il client sceglie il formato che preferisce con l'header `Accept`, il server dichiara quello che usa con `Content-Type`.

PROGETTARE UN'API REST

Le risorse e i loro URI

L'esempio classico è un'API per una lista di cose da fare. La risorsa è il **to-do**; la collezione dei to-do sta all'URI `/todos`, e il to-do con identificativo `ID` all'URI `/todos/ID`.

Verbo e URI	Significato
`GET /todos`	legge la lista di tutti i to-do salvati
`GET /todos/<ID>`	legge solo il to-do con identificativo `<ID>`
`POST /todos`	salva un nuovo to-do; i suoi dati sono nel corpo della richiesta
`PUT /todos/<ID>`	sostituisce (o crea) il to-do con identificativo `<ID>`, con i dati del corpo
`DELETE /todos/<ID>`	elimina il to-do con identificativo `<ID>`

Gli URI contengono solo **nomi** (le risorse); i **verbi** sono quelli di HTTP. Un URI come `/getTodos` o `/deleteTodo?id=3` mette l'operazione nel posto sbagliato.

Esempio 14.2 — L'API dei to-do, una richiesta alla volta

Con `curl` si possono fare a mano le richieste che farebbe un frontend. L'opzione `-i` mostra anche la riga di stato e gli header della risposta.

```
>_ Terminale
$ curl -i -X POST https://api.example.com/todos \
       -H "Content-Type: application/json" \
       -d '{"todo": "Learn REST", "done": false}'
HTTP/1.1 201 Created
Location: /todos/3
Content-Type: application/json

{"id": 3, "todo": "Learn REST", "done": false}

$ curl -i https://api.example.com/todos/3
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 3, "todo": "Learn REST", "done": false}

$ curl -i -X PUT https://api.example.com/todos/3 \
       -H "Content-Type: application/json" \
       -d '{"todo": "Learn REST", "done": true}'
HTTP/1.1 200 OK

$ curl -i -X DELETE https://api.example.com/todos/3
HTTP/1.1 204 No Content

$ curl -i https://api.example.com/todos/3
HTTP/1.1 404 Not Found
```

La creazione risponde **201** e dice nell'header `Location` l'URI della nuova risorsa; dopo la cancellazione la stessa richiesta `GET` risponde **404**. Il codice 204 è un altro successo: «fatto, e non c'è nulla da restituire».

Esempio 14.3 — Lo stesso contratto, lato server

Nel backend ogni riga della tabella diventa un metodo. Con Spring Boot, il framework più diffuso per i servizi REST in Java, il verbo e l'URI si dichiarano con delle annotazioni, e il framework converte da solo gli oggetti Java in JSON e viceversa.

TodoController.java

```java
@RestController
@RequestMapping("/todos")
public class TodoController {
  private final TodoRepository repository;

  public TodoController(TodoRepository repository) {
    this.repository = repository;
  }

  @GetMapping                                   // GET /todos
  public List<Todo> list() {
    return repository.findAll();
  }

  @GetMapping("/{id}")                          // GET /todos/3
  public ResponseEntity<Todo> get(@PathVariable long id) {
    return repository.findById(id)
        .map(ResponseEntity::ok)                // 200, con il to-do nel corpo
        .orElse(ResponseEntity.notFound().build());   // 404
  }

  @PostMapping                                  // POST /todos
  public ResponseEntity<Todo> create(@RequestBody Todo nuovo) {
    Todo salvato = repository.save(nuovo);
    return ResponseEntity.created(URI.create("/todos/" + salvato.id()))   // 201 + Location
        .body(salvato);
  }

  @PutMapping("/{id}")                          // PUT /todos/3
  public Todo replace(@PathVariable long id, @RequestBody Todo todo) {
    return repository.save(new Todo(id, todo.todo(), todo.done()));
  }

  @DeleteMapping("/{id}")                       // DELETE /todos/3
  public ResponseEntity<Void> delete(@PathVariable long id) {
    repository.deleteById(id);
    return ResponseEntity.noContent().build();   // 204
  }
}
```

Todo.java

```java
public record Todo(Long id, String todo, boolean done) { }
```

Il controller sta nel livello di presentazione dell'architettura a livelli (Capitolo 11): riceve le richieste, delega i dati al `TodoRepository` e traduce i risultati in codici di stato. Non contiene logica di business.

Le risorse annidate

A volte fra le risorse c'è una **gerarchia**: un'azienda ha da zero a molti dipartimenti, ciascuno con da zero a molti dipendenti. Progettando un'API si possono definire **risorse annidate**: `GET /listings/{id}/reviews` elenca tutte le recensioni di un dato annuncio. Va però tenuto presente che **ogni risorsa dovrebbe avere un solo URI**: se una recensione si raggiunge come `/listings/7/reviews/12`, non dovrebbe essere raggiungibile anche come `/reviews/12`.

ReviewController.java

```java
@RestController
public class ReviewController {
  // ...
  @GetMapping("/listings/{id}/reviews")          // le recensioni dell'annuncio {id}
  public List<Review> reviewsOf(@PathVariable long id) {
    return reviews.findByListing(id);
  }
}
```

Filtrare e ordinare

Spesso i client devono leggere una lista di risorse con un certo filtro o un certo ordinamento (per esempio gli annunci ordinati per prezzo). Si è tentati di aggiungere nuovi percorsi:

Verbo e URI	Significato
`GET /todos`	legge la lista di tutti i to-do
`GET /todosSortedAsc`	legge i to-do in ordine alfabetico crescente
`GET /todos/filter/key`	legge i to-do che contengono la parola «key»

Non è REST

Le risorse vanno raggiunte con **un solo percorso**: `/todosSortedAsc` e `/todos/filter/key` sono altri URI per la stessa collezione. E l'API diventa presto complicata e difficile da usare: si pensi a tutte le combinazioni di ordinamenti e filtri che potrebbero servire, ciascuna con il suo percorso.

La soluzione è non aggiungere percorsi ma usare i **parametri di query**:

Verbo e URI	Significato
`GET /todos`	legge la lista di tutti i to-do
`GET /todos?sort=title&mode=asc`	legge i to-do ordinati per titolo, in ordine crescente
`GET /todos?q=key`	legge i to-do che contengono la parola «key»

TodoController.java

```java
@GetMapping                                    // GET /todos?q=...&sort=...&mode=...
public List<Todo> list(@RequestParam(required = false) String q,
                       @RequestParam(defaultValue = "id") String sort,
                       @RequestParam(defaultValue = "asc") String mode) {
  return repository.search(q, sort, mode);     // un solo URI, infinite combinazioni
}
```

I parametri sono facoltativi: senza parametri la richiesta resta la `GET /todos` di prima.

Esempio 14.4 — Un client Java

Dal lato client, Java ha un client HTTP nella libreria standard. Il programma crea un to-do e poi cerca quelli che contengono «REST», presentando a ogni richiesta il token di autenticazione (la prossima sezione spiega da dove viene).

TodoClient.java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class TodoClient {
  private static final String API = "http://localhost:8080/todos";

  public static void main(String[] args) throws Exception {
    HttpClient client = HttpClient.newHttpClient();
    String token = args[0];                       // ottenuto con POST /auth

    // POST /todos: crea un nuovo to-do, descritto in JSON nel corpo
    HttpRequest crea = HttpRequest.newBuilder(URI.create(API))
        .header("Content-Type", "application/json")
        .header("Authorization", "Bearer " + token)
        .POST(HttpRequest.BodyPublishers.ofString("{\"todo\": \"Studiare REST\", \"done\": false}"))
        .build();
    HttpResponse<String> creato = client.send(crea, HttpResponse.BodyHandlers.ofString());
    System.out.println(creato.statusCode() + " " + creato.headers().firstValue("Location").orElse("?"));

    // GET /todos?q=REST: la ricerca e' un parametro, non un nuovo percorso
    HttpRequest cerca = HttpRequest.newBuilder(URI.create(API + "?q=REST"))
        .header("Authorization", "Bearer " + token)
        .GET()
        .build();
    HttpResponse<String> trovati = client.send(cerca, HttpResponse.BodyHandlers.ofString());
    System.out.println(trovati.statusCode() + " " + trovati.body());
  }
}
```

Terminale

```
$ java TodoClient.java eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
201 /todos/3
200 [{"id":3,"todo":"Studiare REST","done":false}]
```

Proteggere un'API REST

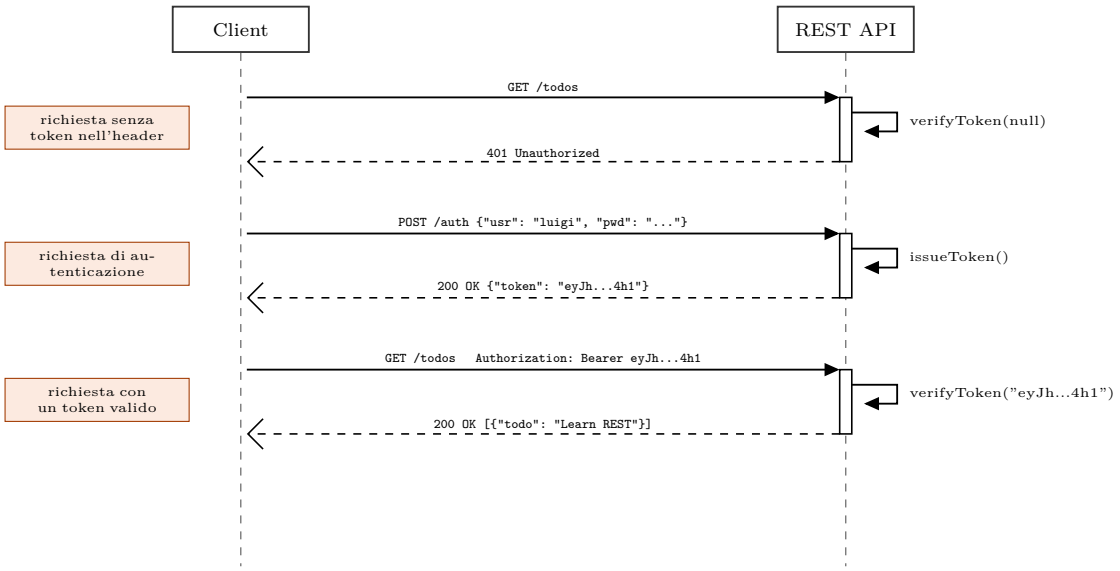
Un'API REST permette di **manipolare** le risorse, e nella maggior parte dei casi non vogliamo che chiunque possa farlo.

- **Autenticazione**: vogliamo che solo gli utenti legittimi possano accedere alle risorse (*chi sei?*).
- **Autorizzazione**: possiamo anche volere che certi utenti accedano solo a certe risorse (*che cosa puoi fare?*): un dipendente, per esempio, non dovrebbe poter aggiornare il proprio stipendio.

L'autenticazione basata su token

Uno schema di autenticazione molto diffuso si basa sui **token**.

1. Il client invia all'API una richiesta con nome utente e password.
2. L'API verifica nome utente e password e genera un **token**.
3. Il token, una stringa, viene restituito al client.
4. A ogni richiesta successiva che richiede autenticazione, il client deve ripresentare il token all'API, nell'header `Authorization`.
5. L'API verifica il token prima di rispondere.



Il token rende l'autenticazione **stateless**: il server non deve ricordare chi ha fatto login, perché ogni richiesta porta con sé la prova di chi è il mittente. Resta da capire come il server faccia a fidarsi di un token che gli arriva dal client.

JSON Web Token (JWT)

JSON Web Token

Il **JWT** (*JSON Web Token*) è uno standard aperto molto diffuso (RFC 7519) che permette di condividere in modo sicuro delle **asserzioni** (*claim*) fra due parti. Un JWT è una stringa fatta di tre parti separate da un punto: **Header.Payload.Signature**.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJuYW1lIjoibHVpZ2kiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE2NzAzOTg0MzJ9.
fopBYrax8wcB7rnPjCcOMc62IT2lJdvyOdyixMWMZAQ
```

- L'**header** contiene, in formato JSON, il tipo di token e la funzione di hash usata per firmarlo. È codificato con la **codifica Base64Url**.
- Il **payload** è un oggetto JSON che contiene le **asserzioni**. Alcune asserzioni **registrate** sono raccomandate (per esempio `exp`, la data di scadenza del token); le altre sono libere: nel payload si può scrivere qualunque asserzione. Anche il payload è in Base64Url.
- La **firma** garantisce che il token non sia stato manomesso o falsificato.

Codificato non vuol dire cifrato

Base64Url è soltanto una **codifica**, non una cifratura: si inverte facilmente, e chiunque abbia il token può leggerne header e payload. In un JWT non si mettono mai password o altri segreti.

```
>_ Terminale ×
$ echo eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 | base64 -d
{"alg":"HS256","typ":"JWT"}
$ echo eyJuYW1lIjoibHVpZ2kiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE2NzAzOTg0MzJ9 | base64 -d
{"name":"luigi","role":"admin","exp":1670398432}
```

Il valore di `exp` è un istante in secondi dal 1° gennaio 1970: 1670398432 è il 7 dicembre 2022 alle 7:33 UTC.

Funzione di hash crittografica

Una **funzione di hash crittografica** associa a una stringa binaria arbitraria (l'input) una stringa binaria di lunghezza fissa (il **digest** o **hash**). Le funzioni sicure hanno proprietà interessanti:

- sono praticamente **prive di collisioni**: è di fatto impossibile trovare due input diversi con lo stesso digest;
- si calcolano facilmente ma **non si invertono**: dato un input è facile calcolarne il digest, ma dato un digest è impraticabile ricavare l'input che l'ha generato.

Basta un carattere in più per cambiare completamente il digest:

Terminale

```
$ echo -n "Web Technologies" | sha256sum
72bdf1226df1010dfb47cffaed85f5b35ae15ec9b482388691fa7a662a114113  -
$ echo -n "Web Technologies!" | sha256sum
1a6791a2eab52c7c71d3e1225632d866b48a3abad91df15b2d44ab6004877993  -
```

La firma di un JWT si ottiene applicando una funzione di hash crittografica alla stringa ottenuta concatenando l'header del token, il payload e una **chiave segreta** nota solo al server che emette il token:

```
HMACSHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), chiave_segreta )
```

In realtà le firme dei JWT sono un po' più complesse di una concatenazione: **HS256** usa un HMAC, mentre **RS256** usa la crittografia asimmetrica (firma con una chiave privata, verifica con quella pubblica). L'idea però è quella: senza la chiave nessuno può calcolare la firma giusta per un payload, quindi nessuno può cambiare il payload (trasformarsi da `user` in `admin`, o spostare la scadenza) senza che il server se ne accorga.

Esempio 14.5 — Emettere e verificare un JWT

La classe crea e verifica token HS256 usando solo la libreria standard di Java. Nei progetti veri si usa una libreria (come jjwt o Nimbus), ma il meccanismo è esattamente questo.

Jwt.java

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.time.Instant;
import java.util.Base64;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class Jwt {
  private static final Base64.Encoder B64 = Base64.getUrlEncoder().withoutPadding();
  private final byte[] chiave;                       // nota solo al server

  public Jwt(String chiaveSegreta) {
    this.chiave = chiaveSegreta.getBytes(StandardCharsets.UTF_8);
  }

  /** Crea un token header.payload.firma che scade fra durataSecondi. */
  public String issueToken(String nome, String ruolo, long durataSecondi) {
    long exp = Instant.now().getEpochSecond() + durataSecondi;
    String header  = encode("{\"alg\":\"HS256\",\"typ\":\"JWT\"}");
    String payload = encode("{\"name\":\"" + nome + "\",\"role\":\"" + ruolo + "\",\"exp\":" + exp + "}");
    return header + "." + payload + "." + firma(header + "." + payload);
  }

  /** Il token è valido se la firma torna e non è scaduto. */
  public boolean verifyToken(String token) {
    String[] parti = token.split("\\.");
    if (parti.length != 3) return false;
    byte[] attesa   = firma(parti[0] + "." + parti[1]).getBytes(StandardCharsets.US_ASCII);
    byte[] ricevuta = parti[2].getBytes(StandardCharsets.US_ASCII);
    if (!MessageDigest.isEqual(attesa, ricevuta)) return false;   // manomesso o falso
    String json = new String(Base64.getUrlDecoder().decode(parti[1]), StandardCharsets.UTF_8);
    Matcher m = Pattern.compile("\"exp\":(\\d+)").matcher(json);
    return m.find() && Long.parseLong(m.group(1)) > Instant.now().getEpochSecond();
```

```
  }

  private static String encode(String json) {
    return B64.encodeToString(json.getBytes(StandardCharsets.UTF_8));
  }

  private String firma(String dati) {
    try {
      Mac mac = Mac.getInstance("HmacSHA256");
      mac.init(new SecretKeySpec(chiave, "HmacSHA256"));
      return B64.encodeToString(mac.doFinal(dati.getBytes(StandardCharsets.UTF_8)));
    } catch (Exception e) {
      throw new IllegalStateException(e);
    }
  }
}
```

Una prova: si emette un token per l'utente `luigi` con ruolo `user`, poi lo si manomette cambiando il ruolo in `admin` senza toccare la firma, lo si verifica con un'altra chiave e se ne emette uno già scaduto.

```
>_ Terminale  ×
$ javac Jwt.java ProvaJwt.java && java ProvaJwt
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoibHVpZ2kiLCJyb2xlIjoidXNlciIsImV4cCI6MTc5MjY2MTUyOH0.1
   o9UHIA4gyXYcdzhMFUG5FhNAgRgYD2PRSTcEKv-6hg
valido: true
manomesso: false
altra chiave: false
scaduto: false
```

L'header del token è identico a quello dell'esempio sopra: stesso JSON, stessa codifica. Il confronto della firma usa `MessageDigest.isEqual`, che impiega lo stesso tempo qualunque sia il punto in cui le stringhe differiscono, per non dare a un attaccante indizi sulla firma giusta.

Documentare un'API: OpenAPI

Un'API vale quanto la sua documentazione: chi scrive un client deve sapere quali risorse esistono, con quali verbi, quali parametri e quali risposte. **OpenAPI** è lo standard per **descrivere** un'API REST in un file (YAML o JSON) leggibile sia dalle persone sia dai programmi. È nato per non affidare la descrizione di un'API a documenti scritti a mano, che invecchiano appena il codice cambia; dallo stesso file si generano la documentazione interattiva (come Swagger UI), il codice dei client e dei server e i test. Non serve saper scrivere a memoria una specifica OpenAPI, ma conviene saperne leggere una.

openapi.yaml

```yaml
openapi: 3.1.0
info:
  title: To-do API
  version: 1.0.0
paths:
  /todos/{id}:
    get:
      summary: Legge un to-do
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        "200":
          description: Il to-do richiesto
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Todo" }
        "404":
          description: Nessun to-do con quell'identificativo
```

In sintesi

- Il software comunica via Internet attraverso le **API**; invece di protocolli ad hoc (socket, CORBA, RMI, SOAP) si usa HTTP.
- **REST** è uno **stile architetturale**, non un protocollo: le **risorse** hanno un URI unico e si manipolano con i verbi HTTP (`POST`, `GET`, `PUT`, `DELETE`: le operazioni CRUD); l'esito è nel **codice di stato** (200, 201, 400, 401, 404, 405, 500).
- Un'API REST è **stateless**: ogni richiesta è autosufficiente, quindi qualunque replica può servirla.
- Lo stato delle risorse viaggia come **rappresentazione**, di solito JSON (o XML).
- Ogni risorsa ha un solo URI; le gerarchie si esprimono con le **risorse annidate**, filtri e ordinamenti con i **parametri di query**, non con nuovi percorsi.
- **Autenticazione** (chi sei) e **autorizzazione** (che cosa puoi fare) si realizzano spesso con un **token** da presentare nell'header `Authorization`.
- Un **JWT** è `header.payload.firma`: header e payload sono solo codificati in Base64Url (leggibili da tutti), la firma, calcolata con una chiave segreta del server, impedisce di manometterli.
- **OpenAPI** descrive un'API in un file da cui si generano documentazione, client e test.

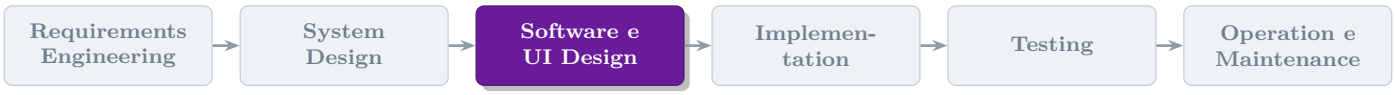
Parte VII

Object Design

I DESIGN PATTERN

Nei capitoli sulle architetture abbiamo deciso la struttura **ad alto livello** di un sistema: quali sottosistemi, quali livelli, come comunicano. Ogni sottosistema, però, è fatto di molte classi, e anche a questo livello più basso gli stessi problemi si ripresentano da un progetto all'altro: creare un oggetto complicato, far collaborare due classi nate separatamente, avvisare automaticamente chi è interessato a un cambiamento. Questo capitolo raccoglie la lezione *Software Design: Design Patterns*: che cos'è un design pattern, da dove viene, e i pattern più importanti, ognuno con il suo codice Java (compilato ed eseguito: gli output riportati sono quelli veri) e, dove aiuta, con un **disegno della memoria** che mostra che cosa succede davvero agli oggetti.

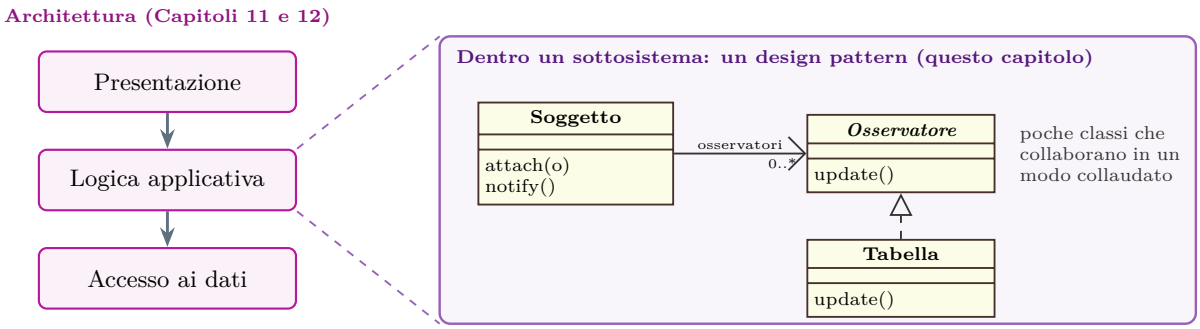
DOVE SIAMO NEL PROCESSO



I design pattern appartengono alla terza fase della cascata, *Software e UI Design*, presentata nel Capitolo 3. Il System Design ha definito l'architettura e assegnato i requisiti ai sottosistemi; ora, per ciascun sottosistema, si definiscono gli **oggetti** necessari a realizzarlo. Gli strumenti della fase sono i **design pattern** (questo capitolo), l'*usability engineering* e i *wireframe* ad alta fedeltà dell'interfaccia utente.

Dalle architetture ai design pattern

- Un'architettura definisce la struttura di un'applicazione **ad alto livello**: in ogni modulo si raggruppano molte classi.
- Grazie alle architetture già note (client/server, a livelli, a microservizi…) abbiamo una vasta scelta di soluzioni **ampiamente collaudate** per una grande classe di problemi.
- Esiste qualcosa di simile a un livello più basso? Esistono modi «standard» di combinare le classi fra loro per svolgere funzioni tipiche? Sì: i **design pattern**.



Che cos'è un design pattern

Design pattern

Un **design pattern** è una **soluzione comune a un problema ricorrente** di progettazione. Astrae una struttura di progetto che si ripresenta in molti sistemi e la descrive in termini di classi e oggetti: le loro **dipendenze**, le loro **strutture**, le loro **interazioni** e le **convenzioni** da rispettare. Dà un **nome** alla struttura, la specifica in modo esplicito e distilla l'esperienza di chi l'ha già usata.

Un po' di storia

- L'idea viene dall'architettura, quella degli edifici. L'architetto **Christopher Alexander** raccolse in *A Pattern Language: Towns, Buildings, Construction* (1977) 253 pattern per progettare città e case, come «luce da due lati in ogni stanza»: le stanze illuminate da un lato solo sono sgradevoli, e la soluzione si può riusare in mille case diverse. Ogni pattern descrive un problema che si ripete e il nucleo della sua soluzione, da adattare ogni volta.
- Nel 1995 la «**Gang of Four**» (GoF: Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides) portò l'idea nel software con il libro *Design Patterns: Elements of Reusable Object-Oriented Software*, che descrive 23 pattern per la programmazione a oggetti. Per Java si può vedere anche S. J. Metsker, *Design Patterns in Java*, Pearson, 2003.
- Da allora ne sono stati proposti molti altri, in conferenze, simposi e libri. Non è raro che un singolo sviluppatore proponga un nuovo pattern e lo diffonda in modo informale, con blog e riviste.

Le quattro parti di un pattern

Parte	Che cosa contiene
1. Nome	Una o due parole che identificano il pattern (Singleton, Observer…) e diventano parte del vocabolario dei progettisti.
2. Problema	Quando applicarlo: il problema da risolvere e il contesto in cui si presenta.
3. Soluzione	Gli elementi del progetto (classi e oggetti), le loro relazioni, responsabilità e collaborazioni. Non è un progetto concreto ma uno **schema**, da riempire ogni volta.
4. Conseguenze	I risultati e i compromessi (*trade-off*) dell'applicazione: che cosa si guadagna e che cosa si paga.

Tabella 15.1: Le quattro parti di un design pattern.

Un design pattern è inoltre:

- **indipendente dal linguaggio e dall'implementazione**: descrive un'idea, che si può realizzare in Java, in C# o in Python;
- una **micro-architettura**: poche classi che collaborano, non un sistema intero;
- da applicare **con giudizio**, mai meccanicamente: la soluzione va tradotta in termini concreti nel contesto dell'applicazione, ed è compito dello sviluppatore farlo.

A che cosa servono

- **Codificare la buona progettazione**: distillano e diffondono l'esperienza, aiutano allo stesso modo principianti ed esperti, insegnano un modo di ragionare sul progetto.
- **Dare nomi espliciti alle strutture**: un vocabolario comune riduce la complessità e aumenta l'espressività. «Qui ci vuole un Observer» comunica in quattro parole una struttura di quattro classi.
- **Catturare e conservare le informazioni di progetto**: le decisioni si descrivono in modo sintetico e la documentazione migliora.
- **Facilitare la ristrutturazione** (*refactoring*): i pattern sono collegati fra loro e danno al codice una flessibilità in più.

Come leggere i disegni della memoria

Molti pattern si capiscono davvero solo guardando gli **oggetti** mentre il programma gira: quanti ne esistono, chi punta a chi, che cosa resta in memoria. Per questo, nel capitolo, accanto al codice compaiono dei **disegni della memoria** di un programma Java, divisi in tre zone.

- **Stack**. Ogni chiamata di metodo in corso ha un suo *frame*, che contiene i parametri e le variabili locali. Il frame nasce quando il metodo viene chiamato e sparisce quando il metodo termina; il metodo in esecuzione è quello disegnato più in alto.
- **Heap**. Qui vivono gli **oggetti**, creati con `new`. Un oggetto resta in memoria finché è **raggiungibile**, cioè finché una catena di riferimenti, partendo dallo stack o dai campi statici, arriva fino a lui; quando non lo è più, il *garbage collector* lo elimina. Nei disegni gli oggetti irraggiungibili hanno il bordo tratteggiato.
- **Campi statici**. I campi `static` appartengono alla **classe**, non a un oggetto: ne esiste una sola copia, creata quando la classe viene caricata. (Nella JVM più diffusa, HotSpot, anche questi valori stanno tecnicamente nello heap, dentro l'oggetto che rappresenta la classe; li disegniamo a parte perché ne esiste una copia sola.)

Una variabile di tipo primitivo (`int`, `double`, `boolean`) contiene direttamente il suo valore. Una variabile di tipo oggetto contiene invece un **riferimento**, disegnato come un pallino con una freccia verso l'oggetto; `null` è il riferimento che non punta a nulla. Anche le stringhe sono oggetti, ma per non affollare i disegni le scriviamo direttamente nel campo, fra virgolette.

Esempio 15.1 — Due riferimenti, un solo oggetto

Punto.java

```java
public class Punto {
    int x, y;

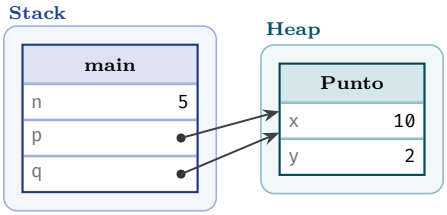
    Punto(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

Alias.java

```java
public class Alias {
    public static void main(String[] args) {
        int n = 5;
        Punto p = new Punto(1, 2);
        Punto q = p;      // copia il riferimento, non l'oggetto
        q.x = 10;
        System.out.println(p.x + " " + q.x + " " + n);
    }
}
```

Terminale

```
$ javac Punto.java Alias.java
$ java Alias
10 10 5
```



La memoria alla fine di `main`.

`Punto q = p` non crea un secondo punto: copia nella variabile `q` lo stesso riferimento contenuto in `p`. Le due variabili puntano allo stesso oggetto, quindi `q.x = 10` cambia anche `p.x`. Il valore di `n`, primitivo, sta invece direttamente nel frame di `main`.

I pattern della Gang of Four

I pattern della GoF sono divisi in tre famiglie, secondo il problema che risolvono:

- **creazionali**: come si **creano** gli oggetti, nascondendo e controllando i dettagli della costruzione;
- **strutturali**: come si **compongono** classi e oggetti per formare strutture più grandi;
- **comportamentali**: come si distribuiscono **algoritmi e responsabilità** fra gli oggetti, e come questi interagiscono.

Pattern	Che cosa fa
Creazionali	
Builder	Separa la costruzione di un oggetto complesso dalla sua rappresentazione.
Abstract Factory	Fornisce un'interfaccia per creare famiglie di oggetti correlati senza specificarne le classi concrete.
Factory Method	Definisce un'interfaccia per creare un oggetto, ma lascia alle sottoclassi la scelta della classe da istanziare.
Prototype	Indica il tipo di oggetto da creare con un'istanza prototipo, e crea i nuovi oggetti copiando il prototipo.
Singleton	Garantisce che la classe abbia una sola istanza e fornisce un modo per accedervi.
Strutturali	
Adapter	Converte l'interfaccia di una classe in un'altra, così che due classi con interfacce diverse possano lavorare insieme.
Bridge	Disaccoppia un'astrazione dalla sua implementazione, così che le due possano variare indipendentemente.
Composite	Compone gli oggetti in strutture ad albero per realizzare composizioni ricorsive.
Decorator	Aggiunge dinamicamente nuove responsabilità a un oggetto: è un'alternativa alle sottoclassi per estenderne le funzionalità.
Facade	Fornisce un'interfaccia unificata a un sottosistema, per renderlo più facile da usare.
Proxy	Fornisce un surrogato di un oggetto per controllarne l'accesso.
Flyweight	Usa la condivisione per gestire in modo efficiente un gran numero di oggetti piccoli.
Comportamentali	
Observer	Definisce una dipendenza 1:N fra oggetti: quando uno cambia stato, gli altri vengono aggiornati automaticamente.
State	Permette a un oggetto di cambiare comportamento a seconda del suo stato interno.
Strategy	Definisce una famiglia di algoritmi, li incapsula uno per uno e li rende intercambiabili, così che possano variare indipendentemente da chi li usa.
Template Method	Definisce lo scheletro di un algoritmo, lasciando alle sottoclassi la definizione di alcuni passi.
Visitor	Rappresenta un'operazione da eseguire sugli elementi di una struttura di oggetti: si possono aggiungere operazioni senza modificare le classi degli elementi.

Tabella 15.2: I pattern GoF presentati nelle slide. In rosso, come nelle slide, quelli che il corso approfondisce e che vediamo in questo capitolo.

Il catalogo originale ne conta 23: le slide riportano tutti i creazionali e tutti gli strutturali, e 5 degli 11 comportamentali (mancano, fra gli altri, *Iterator*, quello degli iteratori delle collezioni Java, e *Command*).

PATTERN CREAZIONALI

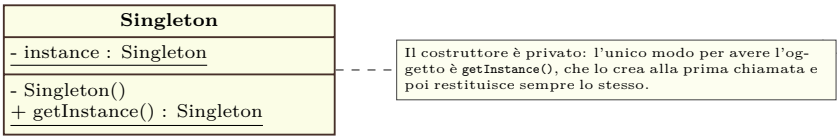
In Java un oggetto si crea con `new` e un costruttore. È un meccanismo semplice ma rigido: il costruttore crea **sempre** un oggetto nuovo, deve ricevere tutti i dati in una volta sola e lega chi lo chiama alla classe concreta. I pattern creazionali **nascondono e controllano** la creazione degli oggetti; ne vediamo due, Singleton e Builder.

Singleton

Singleton (creazionale)

Il pattern **Singleton** costruisce classi che ammettono **una e una sola istanza** e forniscono un **punto di accesso globale** a quell'istanza.

Serve quando nel sistema deve esistere un solo oggetto di un certo tipo, e tutti devono poterlo raggiungere: la connessione al database, la configurazione dell'applicazione, il registro dei log, la coda di stampa. Nel diagramma delle classi i membri statici sono sottolineati.



Ecco l'implementazione delle slide.

SingletonClass.java

```java
public class SingletonClass {
    private static SingletonClass singletonObject = null;   // l'unica istanza

    private SingletonClass() {
        // codice di inizializzazione (facoltativo)
    }

    public static SingletonClass getSingletonObject() {
        if (singletonObject == null) {
            singletonObject = new SingletonClass();
        }
        return singletonObject;
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException();
    }
}
```

- Il riferimento `singletonObject` è **static**: appartiene alla classe, non a un oggetto, quindi è uno solo e sopravvive mentre gli oggetti nascono e muoiono.
- È **private**: non può essere cambiato dall'esterno della classe (nessuno può scriverci `null` o un altro oggetto).
- Il **costruttore è privato**: lo può invocare solo il codice della classe stessa, cioè `getSingletonObject()`.
- Chiunque può ottenere il riferimento all'istanza chiamando il metodo **statico e pubblico** `getSingletonObject()`, che crea l'oggetto se non esiste ancora (*lazy initialization*, inizializzazione pigra).
- La possibilità di **copiare** l'oggetto è disabilitata: `clone()` è ridefinito in modo da lanciare sempre un'eccezione.

Esempio 15.2 — Quanti oggetti esistono davvero?

Il programma chiede l'istanza due volte e prova a copiarla.

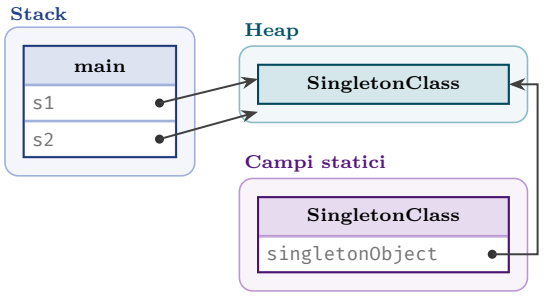
ProvaSingleton.java

```java
public class ProvaSingleton {
    public static void main(String[] args) {
        SingletonClass s1 = SingletonClass.getSingletonObject();
        SingletonClass s2 = SingletonClass.getSingletonObject();
        System.out.println("s1 == s2? " + (s1 == s2));   // stesso oggetto?

        try {
            Object copia = s1.clone();
        } catch (CloneNotSupportedException e) {
            System.out.println("Copia rifiutata: " + e);
        }
    }
}
```

Terminale

```
$ javac SingletonClass.java ProvaSingleton.java
$ java ProvaSingleton
s1 == s2? true
Copia rifiutata: java.lang.CloneNotSupportedException
```



La prima chiamata trova `singletonObject` a `null` e crea l'oggetto; la seconda lo trova già impostato e restituisce lo stesso riferimento. Nel disegno ci sono tre riferimenti ma **un solo oggetto**, e `s1 == s2`, che confronta i riferimenti, vale `true`.

Se un'altra classe prova a creare un secondo esemplare con `new`, a fermarla è già il **compilatore**:

Intruso.java

```java
public class Intruso {
    public static void main(String[] args) {
        SingletonClass altro = new SingletonClass();
    }
}
```

Terminale

```
$ javac Intruso.java
Intruso.java:3: error: SingletonClass() has private access in SingletonClass
        SingletonClass altro = new SingletonClass();
                               ^
1 error
```

L'esempio del corso: un nome che non cambia

L'esempio distribuito con il corso (`EsempioSingleton`) chiama il metodo `getInstance` e gli passa un parametro, che diventa una variabile di stato, `nome`. Il programma chiede l'istanza due volte, con due nomi diversi.

SingletonClass.java

```java
package singleton;

public class SingletonClass {
    private static SingletonClass instance = null;   // all'inizio non ci sono istanze
    private static String nome;                      // variabile di stato

    private SingletonClass(String nome) {            // costruttore privato
        setNome(nome);
    }

    public static SingletonClass getInstance(String nome) {
        if (instance == null) {
            instance = new SingletonClass(nome);
        }
        return instance;
    }

    public String getNome() {
        return nome;
    }

    private static void setNome(String nome) {
        SingletonClass.nome = nome;
    }
}
```

Esecutore.java

```java
package singleton;

public class Esecutore {
    public static void main(String[] args) {
        SingletonClass sc1;
        SingletonClass sc2;

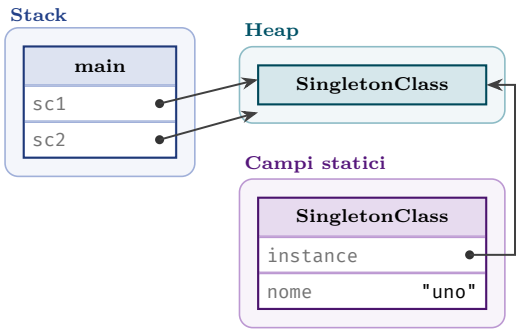
        sc1 = SingletonClass.getInstance("uno");
        sc2 = SingletonClass.getInstance("due");

        System.out.println(sc1.getNome()
                           + " --- " + sc2.getNome());
    }
}
```

Terminale

```
$ javac singleton/*.java
$ java singleton.Esecutore
uno --- uno
```

Stampa `uno --- uno`, non `uno --- due`. La seconda chiamata trova `instance` già impostato: non entra nell'`if`, non chiama il costruttore, e il parametro `"due"` viene semplicemente **ignorato**. `sc1` e `sc2` sono lo stesso oggetto, e il nome è quello dato alla prima chiamata.
Nel disegno `nome` sta fra i campi statici, perché è dichiarato `static`; l'oggetto nello heap non ha campi propri. Con un'istanza sola le due scelte si equivalgono, ma di solito lo stato si mette in un campo d'istanza.



Un parametro che a volte non serve a nulla

Un `getInstance` con parametri è ingannevole: chi scrive `getInstance("due")` si aspetta un oggetto di nome `"due"` e riceve, senza alcun avviso, quello creato da qualcun altro con un altro nome. Se il Singleton va configurato, è più chiaro farlo una volta sola, all'avvio, con un metodo dedicato (che segnala un errore se la configurazione è già stata fatta), e lasciare `getInstance()` senza parametri.

Le conseguenze

- Della classe si può istanziare **al massimo un oggetto**. In alcune varianti si ammette invece un numero **fisso** di istanze (per esempio al massimo 3), tenute in un array di riferimenti che `getInstance` gestisce di conseguenza.
- Il Singleton è **migliore di un oggetto statico** (cioè di una classe con soli campi e metodi `static`):
  - la creazione dell'oggetto si può **personalizzare**, nel costruttore e in `getInstance`;
  - la **copia** si può disabilitare;
  - **non occupa memoria** finché l'oggetto non viene chiesto la prima volta.
- È la soluzione che nel corso abbiamo già usato per l'oggetto `DatabaseConnection`.

Esempio 15.3 — Al massimo tre istanze

Una variante con un numero fisso di oggetti: tre stampanti, assegnate a turno a chi le chiede.

Stampante.java

```java
public class Stampante {
    private static final int MAX = 3;
    private static final Stampante[] istanze = new Stampante[MAX];   // al massimo 3 oggetti
    private static int prossima = 0;
    private final int numero;

    private Stampante(int numero) {
        this.numero = numero;
    }

    public static Stampante getStampante() {
        int i = prossima;
        prossima = (prossima + 1) % MAX;          // a turno: 0, 1, 2, 0, 1, ...
        if (istanze[i] == null) {
            istanze[i] = new Stampante(i + 1);    // creata solo alla prima richiesta
        }
        return istanze[i];
    }

    public int getNumero() {
        return numero;
    }
}
```

ProvaStampanti.java

```java
public class ProvaStampanti {
    public static void main(String[] args) {
        for (int k = 1; k <= 5; k++) {
            Stampante s = Stampante.getStampante();
            System.out.println("Richiesta " + k
                    + " -> stampante " + s.getNumero());
        }
    }
}
```

Terminale

```
$ javac Stampante.java ProvaStampanti.java
$ java ProvaStampanti
Richiesta 1 -> stampante 1
Richiesta 2 -> stampante 2
Richiesta 3 -> stampante 3
Richiesta 4 -> stampante 1
Richiesta 5 -> stampante 2
```

Esempio 15.4 — La connessione al database

Una sola connessione condivisa da tutta l'applicazione, aperta solo quando serve la prima volta. È uno schema possibile per `DatabaseConnection`; indirizzo e credenziali sono di esempio.

DatabaseConnection.java

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    private static DatabaseConnection istanza = null;
    private final Connection connessione;

    private DatabaseConnection() throws SQLException {
        connessione = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/negozio", "app", "segreta");
    }

    public static DatabaseConnection getInstance() throws SQLException {
        if (istanza == null) {
            istanza = new DatabaseConnection();      // la connessione si apre qui, una volta sola
        }
        return istanza;
    }

    public Connection getConnection() {
        return connessione;
    }
}
```

Ovunque serva il database si scrive `DatabaseConnection.getInstance().getConnection()`, senza passare la connessione di classe in classe.

Singleton e thread

L'inizializzazione pigra delle slide non è sicura se più *thread* chiamano `getSingletonObject()` nello stesso momento: entrambi possono trovare il riferimento a `null` ed entrambi creano un oggetto. Due rimedi comuni sono dichiarare il metodo `synchronized`, così che lo esegua un thread alla volta, oppure creare l'istanza subito, quando la classe viene caricata (cosa che la JVM fa una volta sola): `private static final SingletonClass ISTANZA = new SingletonClass();`. In quel caso si perde l'inizializzazione pigra.

Builder

Il Builder appartiene a un'intera famiglia di pattern creazionali (i *Builder* e le *Factory*) che affrontano lo stesso problema: le classi **complesse**, per le quali il meccanismo di base dei costruttori non garantisce abbastanza flessibilità ed efficienza.

Builder (creazionale)

Il pattern **Builder** rende la costruzione di un oggetto **flessibile e scomponibile** in passi, separandola dal costruttore vero e proprio dell'oggetto.

Il problema: costruttori con troppi parametri

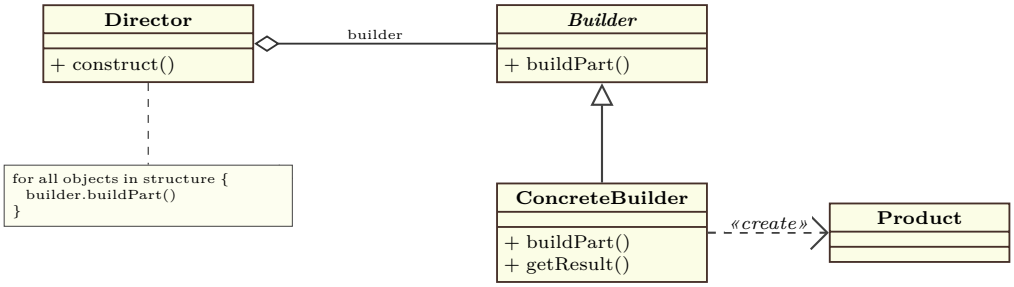
Un computer ha molti componenti: alcuni obbligatori (processore, RAM), molti facoltativi (scheda video, disco, sistema operativo), e in futuro se ne aggiungeranno altri. Con un solo costruttore, ogni creazione diventa un elenco di valori senza nome:

```
1 Computer pc = new Computer("AMD Ryzen 7", 32, "AMD Radeon RX 7800", "SSD", 2000, true);
```

Che cosa sono `2000` e `true`? E se un componente manca, si passa `null`? Le alternative non sono migliori: un costruttore per ogni combinazione di parametri facoltativi (costruttori che si moltiplicano a ogni nuova opzione), oppure un costruttore vuoto seguito da tanti metodi `set`, che lascia esistere per un po' un computer montato a metà e lo rende modificabile per sempre.

La soluzione

- Il singolo oggetto **Product** è costruito dal **ConcreteBuilder**, un passo alla volta (`buildPart()`), e consegnato alla fine (`getResult()`).
- La classe **Director** può gestire la costruzione di un insieme di oggetti `Product`: conosce la sequenza dei passi e la ripete chiamando il Builder.



Il Builder si usa quando:

- la creazione di un prodotto complesso deve essere **indipendente dalle parti** che lo compongono; in particolare, il processo di creazione non deve conoscere il processo di **assemblaggio**, cioè come le parti vengono messe insieme per formare il prodotto;
- il processo di creazione deve permettere **rappresentazioni diverse** dell'oggetto costruito.

Gli esempi delle slide sono edifici: una casa con un piano, 3 stanze, 2 corridoi, 1 garage e tre porte; un grattacielo con 50 piani e, su ogni piano, 15 uffici e 5 corridoi. Gli stessi passi (*aggiungi un piano*, *aggiungi una stanza*…) producono edifici diversissimi.

Un esempio: costruire un computer

Vogliamo costruire oggetti della classe `Computer` per gestirne la pubblicità, la vendita e così via. Un computer ha molti parametri, e in futuro potrebbe servirne qualcuno in più. Separiamo allora l'oggetto `Computer` da un oggetto `ComputerBuilder`, che si occupa soltanto di gestire, in modo flessibile, tutte le varianti possibili della costruzione. L'esempio, distribuito con il corso (`EsempioBuilder`), usa un ConcreteBuilder ma non il Director, che aggiungiamo più avanti.

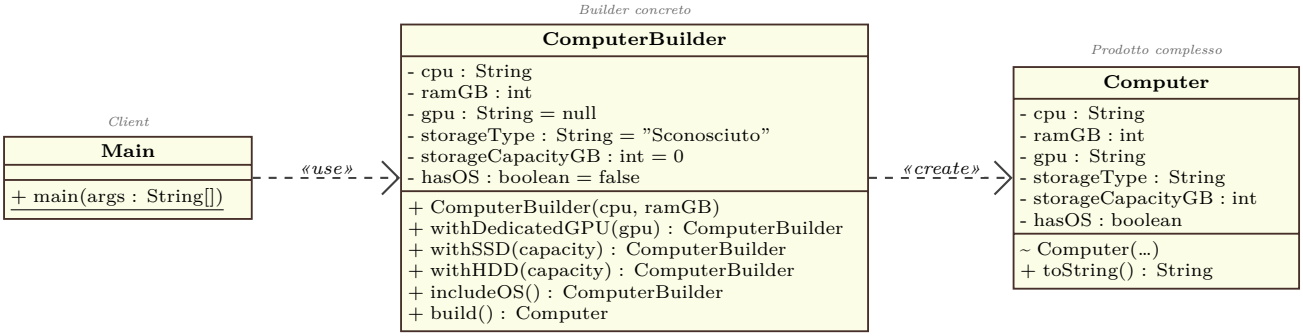


Figura 15.1: Il diagramma delle classi dell'esempio. Il simbolo ~ indica la visibilità di package.

Il **prodotto** ha tutti i campi `final`: una volta costruito, un computer non cambia più. Il suo costruttore non ha modificatore di accesso (visibilità di *package*): lo può chiamare solo il codice dello stesso package, cioè il builder.

Computer.java

```java
public class Computer {
    // Componenti obbligatori
    private final String cpu;
    private final int ramGB;

    // Componenti opzionali
    private final String gpu;
    private final String storageType;      // SSD o HDD
    private final int storageCapacityGB;
    private final boolean hasOS;           // sistema operativo

    // Costruttore package-private: lo usa solo ComputerBuilder (stesso package)
    Computer(String cpu, int ramGB, String gpu, String storageType,
             int storageCapacityGB, boolean hasOS) {
        this.cpu = cpu;
        this.ramGB = ramGB;
        this.gpu = gpu;
        this.storageType = storageType;
        this.storageCapacityGB = storageCapacityGB;
        this.hasOS = hasOS;
    }

    @Override
    public String toString() {
        return "Configurazione Computer:\n" +
               "  - CPU: " + cpu + "\n" +
               "  - RAM: " + ramGB + "GB\n" +
               "  - GPU: " + (gpu != null ? gpu : "Integrata") + "\n" +
               "  - Storage: " + (storageCapacityGB > 0
                   ? storageCapacityGB + "GB " + storageType : "Nessuno Storage") + "\n" +
               "  - OS Installato: " + hasOS;
    }
}
```

Il **builder** riceve nel costruttore i parametri obbligatori e tiene quelli facoltativi, con i loro valori di default, in campi modificabili. Ogni metodo `with…` imposta un'opzione e restituisce `this`, cioè il builder stesso: per questo le chiamate si possono **concatenare** (*fluent interface*). Il metodo `build()` controlla i dati e solo alla fine crea il prodotto.

ComputerBuilder.java

```java
public class ComputerBuilder {
    // Parametri obbligatori: final, fissati dal costruttore del builder
    private final String cpu;
    private final int ramGB;

    // Parametri opzionali, con i valori di default
    private String gpu = null;
    private String storageType = "Sconosciuto";
    private int storageCapacityGB = 0;
    private boolean hasOS = false;

    public ComputerBuilder(String cpu, int ramGB) {
        this.cpu = cpu;
        this.ramGB = ramGB;
    }

    // Metodi "with": impostano un'opzione e restituiscono il builder stesso
    public ComputerBuilder withDedicatedGPU(String gpu) {
        this.gpu = gpu;
        return this;
    }

    public ComputerBuilder withSSD(int capacity) {
        this.storageType = "SSD";
        this.storageCapacityGB = capacity;
        return this;
    }

    public ComputerBuilder withHDD(int capacity) {
        this.storageType = "HDD";
        this.storageCapacityGB = capacity;
        return this;
    }

    public ComputerBuilder includeOS() {
        this.hasOS = true;
        return this;
    }

    // Metodo finale: controlla i dati e crea il Computer
    public Computer build() {
        if (ramGB < 4) {
            throw new IllegalStateException("RAM insufficiente. Minimo richiesto: 4GB.");
        }
        return new Computer(cpu, ramGB, gpu, storageType, storageCapacityGB, hasOS);
    }
}
```

Il **client** ordina tre computer. Ogni ordine si legge come una frase: «un PC con un Ryzen 7 e 32 GB, con una GPU dedicata, un SSD da 2000 GB e il sistema operativo».

Main.java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("--- Ordine 1: PC Gaming ---");
        Computer gamingPC = new ComputerBuilder("AMD Ryzen 7", 32)
            .withDedicatedGPU("AMD Radeon RX 7800")
            .withSSD(2000)
            .includeOS()
            .build();
        System.out.println(gamingPC);

        System.out.println("\n--- Ordine 2: PC Ufficio Base ---");
        Computer officePC = new ComputerBuilder("Intel Core i3", 8)
            .withHDD(500)
            .build();                       // nessuna GPU dedicata, niente OS
        System.out.println(officePC);

        System.out.println("\n--- Ordine 3: PC obsoleto ---");
        Computer obsoletePC = new ComputerBuilder("Intel Core i3", 2)
            .withHDD(250)
            .build();
        System.out.println(obsoletePC);
    }
}
```

Terminale

```
$ javac *.java
$ java Main
--- Ordine 1: PC Gaming ---
Configurazione Computer:
  - CPU: AMD Ryzen 7
  - RAM: 32GB
  - GPU: AMD Radeon RX 7800
  - Storage: 2000GB SSD
  - OS Installato: true

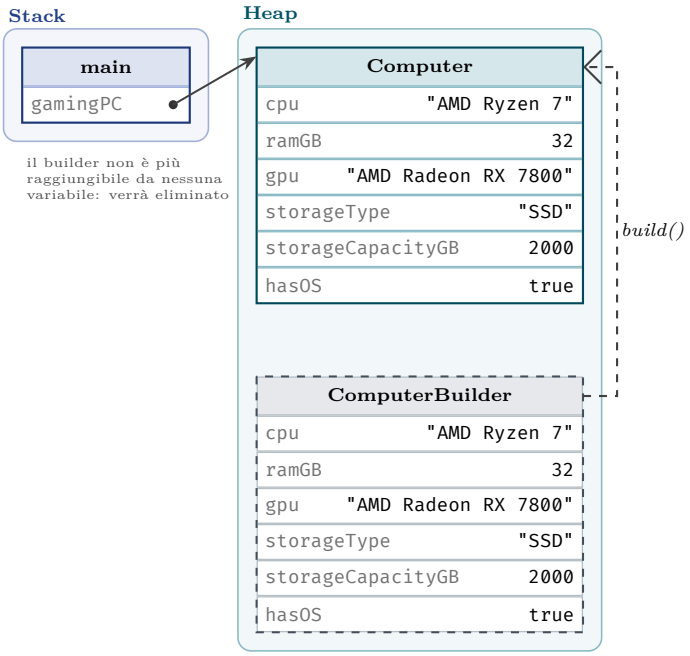
--- Ordine 2: PC Ufficio Base ---
Configurazione Computer:
  - CPU: Intel Core i3
  - RAM: 8GB
  - GPU: Integrata
  - Storage: 500GB HDD
  - OS Installato: false

--- Ordine 3: PC obsoleto ---
Exception in thread "main" java.lang.IllegalStateException: RAM insufficiente. Minimo richiesto: 4GB.
        at ComputerBuilder.build(ComputerBuilder.java:43)
        at Main.main(Main.java:20)
```

Il terzo PC **non viene costruito**: `build()` lancia l'eccezione alla riga 43 di `ComputerBuilder.java`, e il programma si ferma. La regola sui computer obsoleti non sta nella classe `Computer`, ma nel builder: il prodotto resta semplice, e la logica di costruzione e di validazione è tutta in un posto.

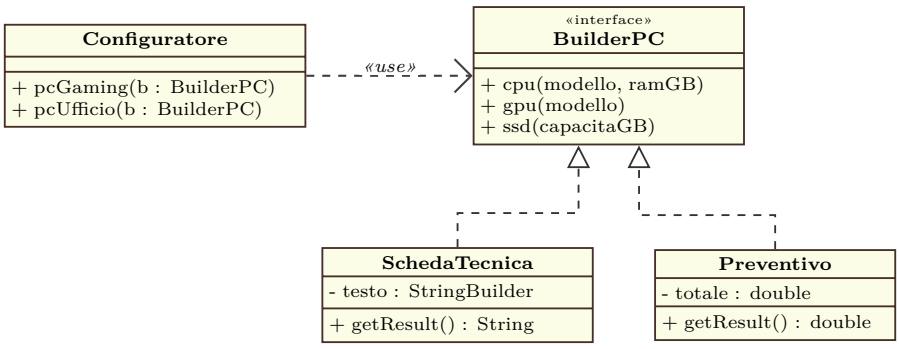
Che cosa succede in memoria

`new ComputerBuilder(…)` crea un oggetto nello heap. Ogni `with…` lo modifica e ne restituisce il riferimento (`return this`): la chiamata successiva della catena lavora quindi **sullo stesso oggetto**. `build()` copia i valori in un oggetto `Computer` nuovo, ed è il suo riferimento che finisce in `gamingPC`. Del builder, a quel punto, non resta alcun riferimento: è irraggiungibile, e il garbage collector lo eliminerà. Il `Computer`, con i campi `final`, non si può più modificare.



Il Director: stessi passi, rappresentazioni diverse

L'esempio del computer non usa il Director. Per vederlo all'opera servono un builder **astratto**, cioè un'interfaccia con i passi di costruzione, e almeno due builder concreti. Il `Configuratore` (il Director) conosce le «ricette» delle configurazioni ma non sa che cosa verrà prodotto: la stessa ricetta dà una scheda tecnica con un builder, un preventivo con l'altro. Il metodo `getResult()` non sta nell'interfaccia, perché i due prodotti hanno tipi diversi (`String` e `double`): è proprio come nel diagramma generale, dove `getResult()` compare solo nel ConcreteBuilder.



BuilderPC.java

```java
// Builder astratto: solo i passi di costruzione
public interface BuilderPC {
    void cpu(String modello, int ramGB);
    void gpu(String modello);
    void ssd(int capacitaGB);
}
```

Configuratore.java

```java
// Director: conosce le ricette, non il prodotto
public class Configuratore {
    public void pcGaming(BuilderPC b) {
        b.cpu("AMD Ryzen 7", 32);
        b.gpu("AMD Radeon RX 7800");
        b.ssd(2000);
    }

    public void pcUfficio(BuilderPC b) {
        b.cpu("Intel Core i3", 8);
        b.ssd(256);
    }
}
```

SchedaTecnica.java

```java
// Primo ConcreteBuilder: un testo
public class SchedaTecnica implements BuilderPC {
    private final StringBuilder testo =
            new StringBuilder();

    public void cpu(String modello, int ramGB) {
        testo.append("CPU " + modello
                + ", RAM " + ramGB + " GB\n");
    }
    public void gpu(String modello) {
        testo.append("GPU " + modello + "\n");
    }
    public void ssd(int capacitaGB) {
        testo.append("SSD " + capacitaGB + " GB\n");
    }
    public String getResult() {
        return testo.toString();
    }
}
```

Preventivo.java

```java
// Secondo ConcreteBuilder: un prezzo
public class Preventivo implements BuilderPC {
    private double totale = 0;

    public void cpu(String modello, int ramGB) {
        totale += 250 + 4 * ramGB;
    }
    public void gpu(String modello) {
        totale += 550;
    }
    public void ssd(int capacitaGB) {
        totale += 0.08 * capacitaGB;
    }
    public double getResult() {
        return totale;
    }
}
```

Negozio.java

```java
public class Negozio {
    public static void main(String[] args) {
        Configuratore director = new Configuratore();

        SchedaTecnica scheda = new SchedaTecnica();
        director.pcGaming(scheda);         // stessa ricetta...
        System.out.print(scheda.getResult());

        Preventivo preventivo = new Preventivo();
        director.pcGaming(preventivo);     // ...prodotto diverso
        System.out.println("Prezzo: " + preventivo.getResult()
                + " euro");
    }
}
```

Terminale

```
$ javac *.java
$ java Negozio
CPU AMD Ryzen 7, RAM 32 GB
GPU AMD Radeon RX 7800
SSD 2000 GB
Prezzo: 1088.0 euro
```

I prezzi sono di fantasia: $250 + 4 \cdot 32 = 378$ per processore e RAM, 550 per la scheda video, $0{,}08 \cdot 2000 = 160$ per il disco, in tutto 1088 euro. Per aggiungere una nuova rappresentazione (un elenco dei pezzi da ordinare al magazzino, per esempio) basta un nuovo ConcreteBuilder: il Director non cambia.

Il Builder nelle librerie di Java

Il pattern è molto usato nelle classi della libreria standard di Java:

- `java.lang.StringBuilder` costruisce stringhe un pezzo alla volta (le `String` sono immutabili: senza un builder ogni concatenazione creerebbe una stringa nuova);
- `java.util.Locale.Builder` costruisce oggetti `Locale` complessi, definendo lingua, paese, sistema di scrittura e varianti;
- `java.time.format.DateTimeFormatterBuilder` costruisce formattatori di date e orari.

BuilderDiLibreria.java

```java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeFormatterBuilder;
import java.util.Locale;

public class BuilderDiLibreria {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Design");
        sb.append(' ').append("Patterns").append(", GoF ").append(1995);
        System.out.println(sb.toString());

        Locale locale = new Locale.Builder().setLanguage("it").setRegion("IT").build();
        System.out.println(locale + " = " + locale.getDisplayName(locale));

        DateTimeFormatter formato = new DateTimeFormatterBuilder()
                .appendPattern("dd/MM/yyyy")
                .appendLiteral(" alle ")
                .appendPattern("HH:mm")
                .toFormatter();
        System.out.println(LocalDateTime.of(2026, 10, 25, 9, 30).format(formato));
    }
}
```

Terminale

```
$ java BuilderDiLibreria.java
Design Patterns, GoF 1995
it_IT = italiano (Italia)
25/10/2026 alle 09:30
```

Un builder l'avevamo già usato nel Capitolo 14, senza chiamarlo per nome: `HttpRequest.newBuilder(…)` seguito da `.header(…)`, `.GET()` e infine `.build()`. E nel Capitolo 10 avevamo osservato che catene come `builder.append("a").append("b")` non violano la Legge di Demetra, perché ogni chiamata restituisce lo stesso oggetto.

PATTERN STRUTTURALI

I pattern strutturali si occupano della **composizione** di classi e oggetti per creare strutture più grandi. Alcuni usano l'ereditarietà (una classe può ereditare le proprietà di più superclassi indipendenti: in Java, estendendo una classe e implementando delle interfacce), altri la composizione di oggetti. Permettono per esempio a librerie nate in modo indipendente di lavorare insieme. Ne vediamo tre: Adapter, Composite e Decorator.

Adapter

Adapter (strutturale)

Il pattern **Adapter** (sinonimo: *Wrapper*) converte l'interfaccia di una classe in un'altra interfaccia, quella che il client si aspetta. Permette così di far cooperare classi che altrimenti avrebbero interfacce incompatibili.

Motivazione: si vuole usare una classe già esistente e collaudata, ma la sua interfaccia non corrisponde a quella che serve. È la situazione dell'adattatore da viaggio: la spina italiana funziona benissimo, ma la presa americana ha un'altra forma; non si cambia né la spina né la presa, si mette in mezzo un adattatore.

I partecipanti sono quattro:

- **Client**: la classe che vuole usare il servizio da adattare;
- **Target**: l'interfaccia che vorremmo dal servizio;
- **Adaptee**: la classe esistente che fornisce il servizio;
- **Adapter**: la classe che adatta l'interfaccia fornita dall'Adaptee all'interfaccia Target.

L'adattamento si può realizzare in due modi: con l'**ereditarietà** (*adapter di classe*: l'Adapter eredita sia da Target sia da Adaptee) oppure con la **composizione** (*adapter di oggetto*: l'Adapter eredita da Target e contiene un riferimento a un Adaptee, a cui delega).

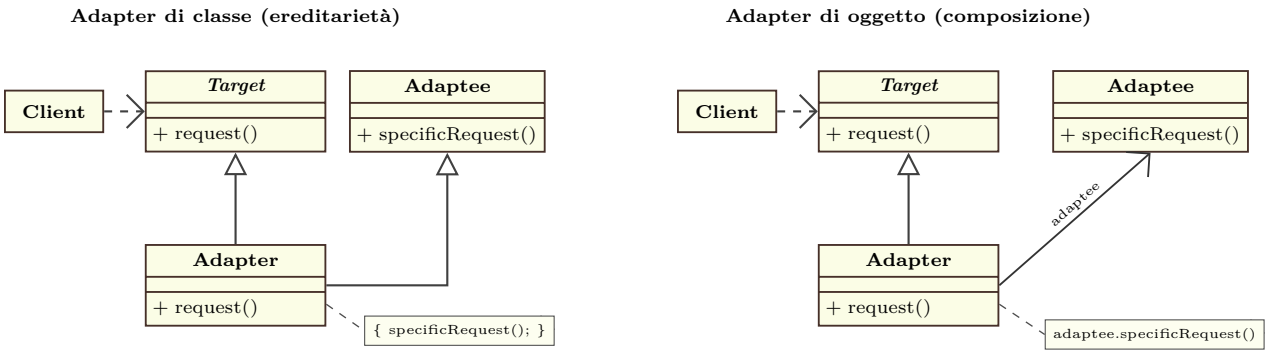


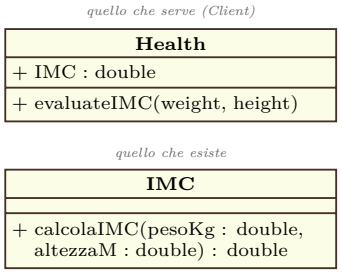
Figura 15.2: Le due forme dell'Adapter. In Java una classe estende una sola classe: l'adapter di classe si scrive `class Adapter extends Adaptee implements Target`.

L'Adapter si usa:

1. se si vuole usare una classe esistente (l'Adaptee) la cui interfaccia è incompatibile con quella richiesta;
2. se si vuole creare una classe (riusabile) che dovrà collaborare con classi non prevedibili al momento della sua creazione;
3. se si vuole riusare un insieme di sottoclassi esistenti, ma è scomodo adattarne l'interfaccia creando una sottoclasse per ciascuna: meglio creare un adapter per la classe padre. Un adapter di oggetto può contenere un'istanza di qualunque sottoclasse dell'Adaptee, e le adatta tutte in una volta.

Un esempio: l'indice di massa corporea

Stiamo realizzando un software a oggetti per il mercato statunitense, e serve una classe `Health` (il Client) che fornisca, fra le altre cose, l'**indice di massa corporea** (IMC) di una persona, con l'interfaccia qui a destra. Non conosciamo l'algoritmo per calcolare l'indice, e decidiamo di riusare la classe `IMC` di un software scritto per il mercato italiano. Negli Stati Uniti però il peso si misura in **libbre** e l'altezza in **pollici**: sappiamo che 1 libbra vale 0,453 chilogrammi e 1 pollice 0,0254 metri. Vogliamo liberare chi programma `Health` dal dover conoscere l'interfaccia e l'implementazione della classe `IMC`.



L'indice è il peso in chilogrammi diviso per il quadrato dell'altezza in metri: IMC = peso/altezza^2. Ecco la soluzione con un adapter di oggetto, nella stessa forma delle slide; `Adaptee` è la classe che avevamo già a disposizione, che a sua volta usa il calcolo originale della classe `IMC`.

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Health health = new Health();
        // 165 libbre, 70 pollici
        health.evaluateIMC(165, 70);
        System.out.println(health.IMC);
    }
}
```

Health.java

```java
// Classe da realizzare (misure americane)
public class Health {
    public double IMC;

    public double evaluateIMC(double weight,
                              double height) {
        Target adapter = new Adapter();
        IMC = adapter.evaluate(weight, height);
        return IMC;
    }
}
```

Target.java

```java
// Interfaccia che richiediamo
public interface Target {
    double evaluate(double weight, double height);
}
```

Adapter.java

```java
// Classe che adatta la soluzione esistente
public class Adapter implements Target {
    private Adaptee adaptee = new Adaptee();

    @Override
    public double evaluate(double weight,
                           double height) {
        // libbre -> kg, pollici -> metri
        return adaptee.calcolaIMC(weight * 0.453,
                                  height * 0.0254);
    }
}
```

Adaptee.java

```java
// Classe che avevamo gia' a disposizione
public class Adaptee {
    public double calcolaIMC(double pesoKg,
                             double altezzaM) {
        IMC imc = new IMC();
        return imc.calcolaIMC(pesoKg, altezzaM);
    }
}
```

IMC.java

```java
// Calcolo originale (misure italiane)
public class IMC {
    public double calcolaIMC(double pesoKg,
                             double altezzaM) {
        return pesoKg / (altezzaM * altezzaM);
    }
}
```

Terminale

```
$ javac *.java
$ java Main
23.643873818359882
```

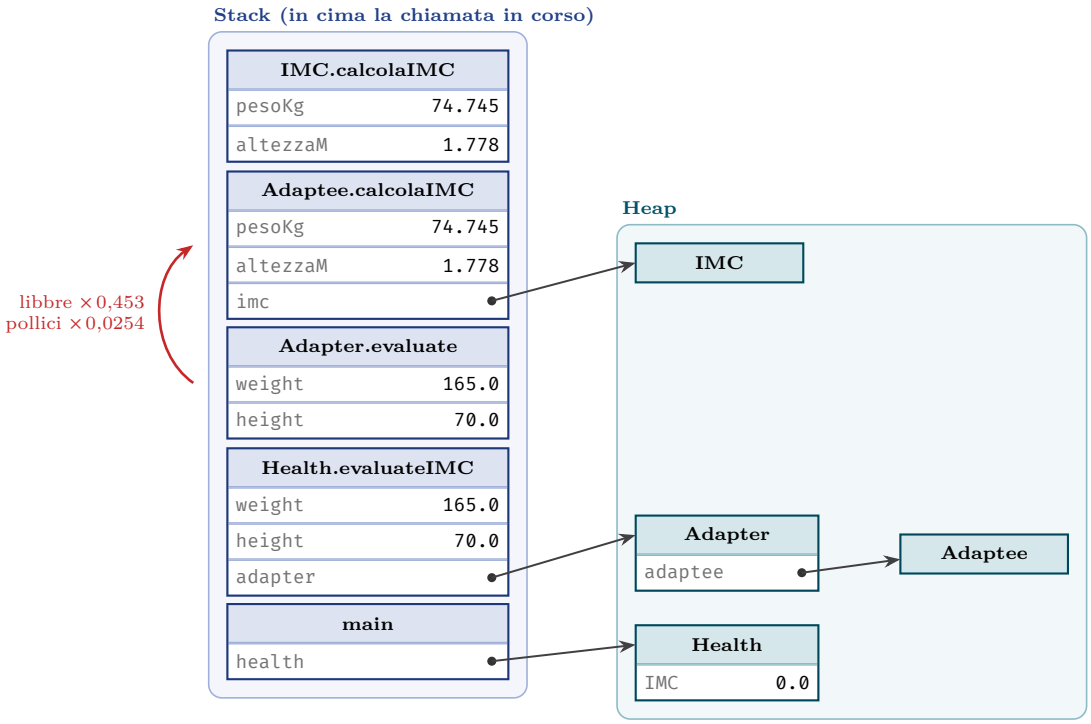
`Health` conosce soltanto l'interfaccia `Target`: non sa che esistono `Adaptee` e `IMC`, né che il calcolo originale lavora in chilogrammi e metri. Tutta la conoscenza della «traduzione» è nell'`Adapter`, in una riga.

Nelle slide le conversioni sono al contrario

Nella slide della soluzione l'adapter **divide**: `adaptee.calcolaIMC(weight/0.453, height/0.0254)`. Ma se 1 libbra vale 0,453 kg, per passare dalle libbre ai chilogrammi bisogna **moltiplicare**: 165 libbre sono $165 \cdot 0{,}453 = 74{,}745$ kg, mentre $165/0{,}453 \simeq 364$ (sarebbero chili trasformati in libbre). Con le divisioni il programma stampa `4.795756183268009E-5`, cioè 0,000048: un indice senza senso. Con le moltiplicazioni stampa 23,64, il valore corretto per una persona di 75 kg alta 1,78 m. Qui il codice è corretto.

Che cosa succede in memoria

Il disegno fotografa lo stack nel momento più profondo, mentre la classe `IMC` esegue il calcolo. Ogni chiamata ha il suo frame, e si vede dove avviene l'adattamento: nel frame di `Adapter.evaluate` i valori sono ancora in libbre e pollici, in quello di `Adaptee.calcolaIMC` sono già in chilogrammi e metri. (Nei metodi d'istanza ogni frame contiene anche il riferimento `this`, che qui non disegniamo.)



`Health.IMC` vale ancora `0.0`: il valore verrà scritto solo quando le chiamate termineranno e i frame, uno dopo l'altro dall'alto, spariranno restituendo il risultato.

L'adapter di classe

La stessa conversione si può scrivere con l'ereditarietà: l'adapter **è** un `Adaptee`, e chiama il metodo ereditato. Il risultato è lo stesso.

AdapterDiClasse.java

```java
// Adapter di classe: eredita dall'Adaptee invece di contenerlo
public class AdapterDiClasse extends Adaptee implements Target {
    @Override
    public double evaluate(double weight, double height) {
        return calcolaIMC(weight * 0.453, height * 0.0254);   // metodo ereditato
    }
}
```

ProvaDiClasse.java

```java
public class ProvaDiClasse {
    public static void main(String[] args) {
        Target t = new AdapterDiClasse();
        System.out.println(t.evaluate(165, 70));
    }
}
```

Terminale

```
$ javac *.java
$ java ProvaDiClasse
23.643873818359882
```

L'adapter di classe è più compatto, ma è legato a **una** classe Adaptee e la espone tutta (chi ha un `AdapterDiClasse` può chiamarne anche `calcolaIMC` direttamente). L'adapter di oggetto può adattare anche le sottoclassi dell'Adaptee e nasconde completamente l'oggetto adattato: per questo, in Java, è la forma più comune.

L'uso tipico: servizi e microservizi

L'Adapter interviene di continuo quando si lavora con servizi o microservizi:

- il progetto del nostro software ci porta a definire un'**interfaccia** (il Target, che può essere proprio un'interfaccia Java, perché chiede solo un servizio);
- un microservizio esistente fornisce il servizio che ci serve, ma con la **sua** interfaccia;
- scriviamo una classe Adapter che interagisce con la classe che fornisce il servizio (l'Adaptee), converte i parametri da lui forniti in quelli che interessano a noi, eventualmente riorganizzandoli ed elaborandoli, e li fornisce attraverso l'interfaccia Target.

Perché non modificare direttamente il servizio? Perché non l'abbiamo scritto noi e ne abbiamo solo l'interfaccia, oppure perché è già usato da altri, che si aspettano l'interfaccia di oggi.

Esempio 15.5 — Un convertitore di valuta sopra un servizio esterno

Il nostro progetto vuole convertire importi in euro con una sola chiamata; la libreria del servizio dei cambi, che non possiamo modificare, offre invece il tasso fra due valute qualsiasi. I tassi sono di esempio.

ConvertitoreValuta.java

```java
// Target: l'interfaccia decisa dal nostro progetto
public interface ConvertitoreValuta {
    double inEuro(double importo, String valuta);
}
```

ExchangeRateClient.java

```java
// Adaptee: il client di un servizio esterno, che non possiamo modificare
public class ExchangeRateClient {
    // quante unita' di "to" vale una unita' di "from"
    public double getRate(String from, String to) {
        // in realta' chiamerebbe il microservizio: GET /rates?from=USD&to=EUR
        return switch (from + "->" + to) {
            case "USD->EUR" -> 0.92;
            case "GBP->EUR" -> 1.17;
            default -> throw new IllegalArgumentException("Cambio non disponibile: " + from + "->" + to);
        };
    }
}
```

AdapterCambi.java

```java
// Adapter: traduce la nostra domanda nella lingua del servizio
public class AdapterCambi implements ConvertitoreValuta {
    private final ExchangeRateClient client = new ExchangeRateClient();

    @Override
    public double inEuro(double importo, String valuta) {
        return importo * client.getRate(valuta, "EUR");
    }
}
```

Cassa.java

```java
public class Cassa {
    public static void main(String[] args) {
        // il codice conosce solo il Target
        ConvertitoreValuta conv = new AdapterCambi();
        System.out.println("100 USD = "
                + conv.inEuro(100, "USD") + " EUR");
        System.out.println("100 GBP = "
                + conv.inEuro(100, "GBP") + " EUR");
    }
}
```

Terminale

```
$ javac *.java
$ java Cassa
100 USD = 92.0 EUR
100 GBP = 117.0 EUR
```

Se domani il servizio dei cambi cambia (o lo sostituiamo con un altro), si riscrive solo `AdapterCambi`: il resto del programma continua a parlare con `ConvertitoreValuta`.

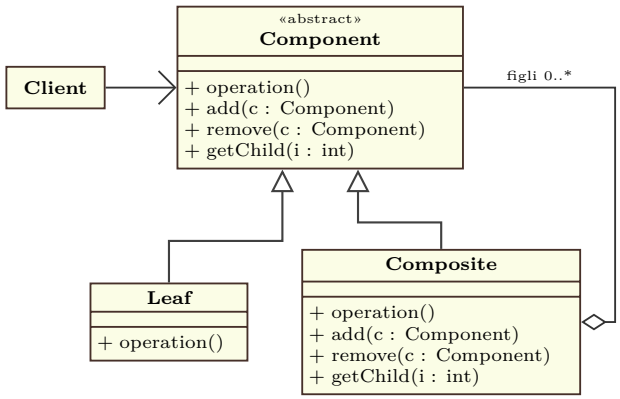
Composite

Composite (strutturale)

Il pattern **Composite** compone gli oggetti in **strutture ad albero** che rappresentano gerarchie tutto-parte, e permette ai client di trattare **allo stesso modo** gli oggetti composti e gli oggetti singoli.

I partecipanti sono:

- **Component**: dichiara l'interfaccia degli oggetti della composizione e il loro comportamento di default; dichiara anche un'interfaccia per accedere ai componenti figli e gestirli;
- **Leaf** (foglia): implementa gli oggetti senza figli e il loro comportamento;
- **Composite**: definisce il comportamento dei componenti con figli, memorizza i figli e implementa le operazioni sui figli definite dall'interfaccia Component;
- **Client**: usa gli oggetti della composizione attraverso l'interfaccia Component.



La relazione decisiva è l'aggregazione da `Composite` a `Component`: un composto contiene dei **componenti**, e quindi può contenere sia foglie sia altri composti. È questo che rende la struttura ricorsiva, e permette alberi di qualunque profondità.

Un esempio: il file system

Vogliamo realizzare un albero di directory e file, con una funzione `ls` che li elenca rispettando il **rientro**: ogni livello dell'albero si sposta un po' più a destra. Una `Directory` può contenere file e altre directory, eventualmente vuote; `AbstractFile` rappresenta tutto ciò che può stare in una directory. L'esempio è quello distribuito con il corso (`EsempioComposite`).

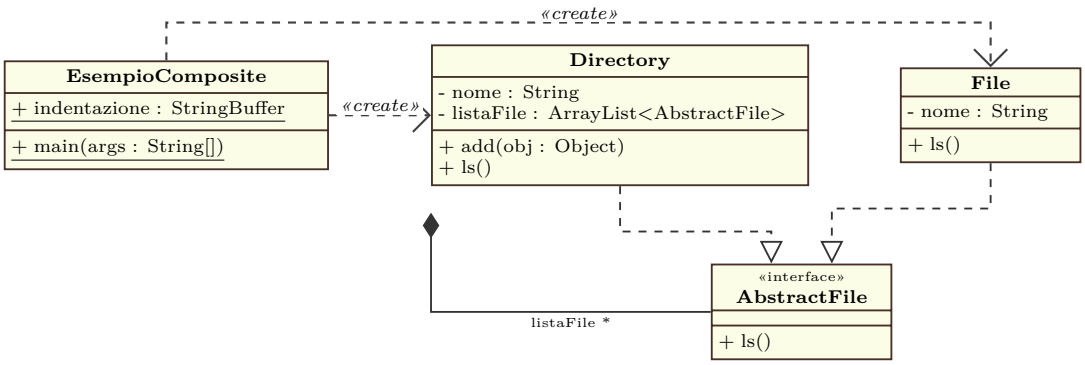


Figura 15.3: Il diagramma delle classi dell'esempio: `Directory` è il Composite, `File` la Leaf, `AbstractFile` il Component.

AbstractFile.java

```java
interface AbstractFile {
    public void ls();
}
```

File.java

```java
class File implements AbstractFile {
    private String nome;

    public File(String n) {
        nome = n;
    }

    public void ls() {
        System.out.println(EsempioComposite.indentazione + nome);
    }
}
```

La `Directory` tiene i figli in una lista di `AbstractFile`. Il suo `ls()` stampa il proprio nome, aggiunge tre spazi al rientro, chiama `ls()` su ogni figlio (senza sapere se è un file o una directory: è il **polimorfismo**), e alla fine toglie i tre spazi.

Directory.java

```java
import java.util.ArrayList;

class Directory implements AbstractFile {
    private String nome;
    private ArrayList<AbstractFile> listaFile = new ArrayList<AbstractFile>();

    public Directory(String name) {
        nome = name;
    }

    public void add(Object obj) {
        listaFile.add((AbstractFile) obj);
    }

    public void ls() {
        System.out.println(EsempioComposite.indentazione + nome);
        EsempioComposite.indentazione.append("   ");
        for (int i = 0; i < listaFile.size(); ++i) {
            AbstractFile obj = listaFile.get(i);
            obj.ls();                                   // polimorfismo
        }
        EsempioComposite.indentazione.setLength(EsempioComposite.indentazione.length() - 3);
    }
}
```

EsempioComposite.java

```java
public class EsempioComposite {
    public static StringBuffer indentazione = new StringBuffer();

    public static void main(String[] args) {
        Directory one = new Directory("dir111"),
                  two = new Directory("dir222"),
                  thr = new Directory("dir333");
        File a = new File("a"), b = new File("b"), c = new File("c"),
             d = new File("d"), e = new File("e");
        one.add(a);
        one.add(two);
        one.add(b);
        two.add(c);
        two.add(d);
        two.add(thr);
        thr.add(e);
        one.ls();
    }
}
```

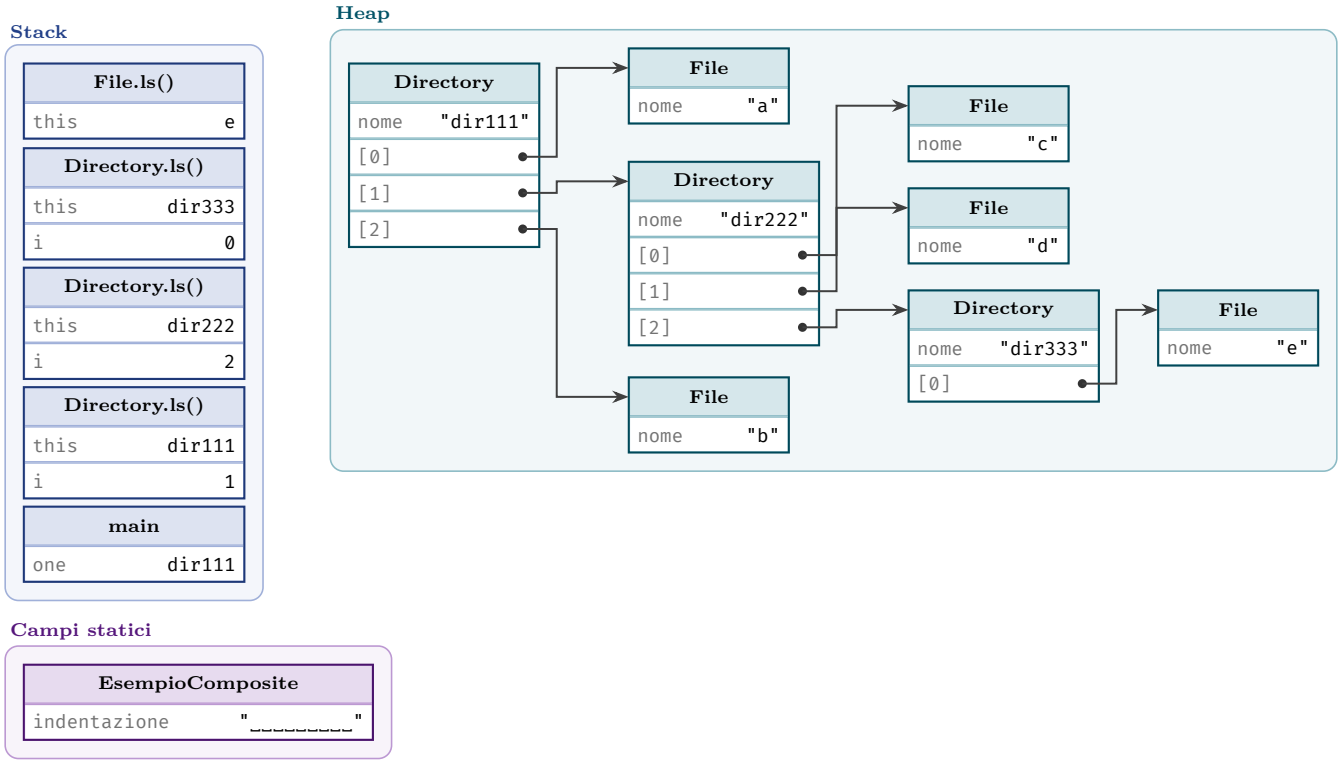
Terminale

```
$ javac *.java
$ java EsempioComposite
dir111
   a
   dir222
      c
      d
      dir333
         e
   b
```

Il `main` chiama `ls()` **una sola volta**, sulla radice: l'albero intero viene visitato dalla ricorsione. Rispetto alle slide, il codice del corso usa una lista generica (`ArrayList<AbstractFile>`), così che `get(i)` restituisce già un `AbstractFile` senza conversioni.

Che cosa succede in memoria

A destra c'è l'albero degli oggetti costruito dalle `add`; a sinistra lo stack nel momento in cui viene stampata la riga di `e`, la più profonda. Per non affollare il disegno, nei frame il riferimento `this` è indicato con il nome dell'oggetto puntato, e le caselle della lista `listaFile` (che è un oggetto `ArrayList` a parte) sono disegnate dentro la directory.



Ogni chiamata di `ls()` su una directory ha il suo frame, con il **suo** indice `i`: `dir111` è fermo al figlio 1 (`dir222`), `dir222` al figlio 2 (`dir333`), `dir333` al figlio 0 (`e`). Il rientro, condiviso da tutti perché statico, in quel momento vale nove spazi: tre per ogni directory aperta. Quando `e.ls()` termina, il suo frame sparisce e riprende il ciclo di `dir333`, che toglie tre spazi e termina a sua volta; poi riprende `dir222`, che ha finito i figli; infine `dir111` passa al figlio 2 e stampa `b` con tre spazi soltanto.

Trasparenza o sicurezza?

Nel diagramma generale le operazioni sui figli (`add`, `remove`, `getChild`) sono dichiarate nel Component: il client può trattare foglie e composti nello stesso identico modo (**trasparenza**), ma può anche chiedere a una foglia di aggiungere un figlio, cosa che non ha senso e va gestita, per esempio con un'eccezione. Nell'esempio del file system `add` esiste solo in `Directory`: il compilatore impedisce di aggiungere un figlio a un `File` (**sicurezza**), ma il client deve sapere quando ha in mano una directory. È un compromesso, e nessuna delle due scelte è sbagliata. Con `add(AbstractFile f)` invece di `add(Object obj)`, infine, il compilatore rifiuterebbe anche gli oggetti che non sono né file né directory, che oggi passano la compilazione e falliscono solo a tempo di esecuzione, sulla conversione `(AbstractFile) obj`.

Esempio 15.6 — Il rientro come parametro, invece che come variabile statica

La `StringBuffer` statica è uno **stato globale**: tutte le chiamate la modificano, e ogni `ls()` deve ricordarsi di rimettere le cose a posto alla fine. Si può fare a meno, passando il rientro come parametro: ogni chiamata riceve il rientro del suo livello, e non c'è nulla da ripristinare.

Directory.java

```java
import java.util.ArrayList;
import java.util.List;

class Directory implements AbstractFile {
    private final String nome;
    private final List<AbstractFile> figli = new ArrayList<>();
    public Directory(String nome) { this.nome = nome; }

    public void add(AbstractFile f) {       // solo file e directory
        figli.add(f);
    }

    public void ls(String rientro) {
        System.out.println(rientro + nome);
        for (AbstractFile f : figli) {
            f.ls(rientro + "   ");          // tre spazi in piu'
        }
    }
}
```

AbstractFile.java

```java
interface AbstractFile {
    void ls(String rientro);
}
```

File.java

```java
class File implements AbstractFile {
    private final String nome;
    public File(String nome) {
        this.nome = nome;
    }

    public void ls(String rientro) {
        System.out.println(rientro + nome);
    }
}
```

La radice si stampa con `one.ls("")`, e l'output è identico a quello di prima.

Le conseguenze

- Il Composite permette di definire gerarchie di classi fatte di oggetti **primitivi** e **composti**: gli oggetti primitivi si compongono in oggetti più complessi, che a loro volta si compongono ricorsivamente. Ovunque il client si aspetta un oggetto primitivo, si può usare indifferentemente un oggetto composto.
- **Semplifica il client**: i client usano in modo uniforme le strutture composte e gli oggetti singoli, e di solito non sanno (né dovrebbero preoccuparsi di sapere) se stanno lavorando con una foglia o con un composto.
- È **flessibile**: rende facile aggiungere nuovi tipi di componenti; nuove sottoclassi Leaf o Composite funzionano automaticamente con le strutture e con il codice dei client esistenti.
- È spesso usato per la gerarchia dei **widget** di un'interfaccia grafica. In Java AWT e Swing, per esempio, un `Container` (una finestra, un pannello) è a sua volta un `Component` e contiene altri `Component`: pulsanti, etichette e altri pannelli.

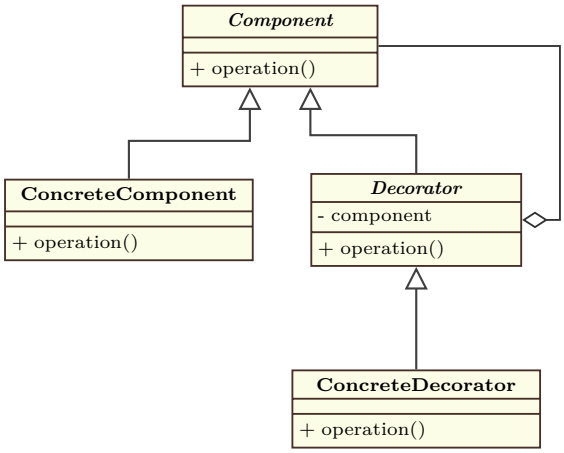
Il pattern Decorator permette di creare oggetti complessi **aggiungendo funzionalità** a oggetti più semplici.

- L'oggetto complesso **include** l'oggetto semplice e lo «decora», aggiungendo funzionalità: l'oggetto originale viene passato come parametro al costruttore del decoratore, e si ottiene l'oggetto «decorato».
- Più decoratori si possono anche **concatenare**, aggiungendo funzionalità un passo alla volta: l'oggetto complesso è l'ultimo anello della catena.
- La concatenazione può avvenire secondo una composizione **arbitraria**: il numero dei comportamenti possibili cresce in modo combinatorio rispetto al numero dei decoratori disponibili.

È un'**alternativa all'ereditarietà**, singola o multipla. Con l'ereditarietà le funzionalità si aggiungono in modo **statico**, secondo la gerarchia delle classi scritta nel codice: non si può ottenere a tempo di esecuzione una combinazione arbitraria di funzionalità, né aggiungerne o toglierne.

Decorator (strutturale)

Il pattern **Decorator** aggiunge **dinamicamente** nuove responsabilità a un oggetto, avvolgendolo in un altro oggetto che ha la stessa interfaccia: è un'alternativa flessibile alle sottoclassi per estenderne le funzionalità.



- **Component** è una classe astratta o un'interfaccia.
- **ConcreteComponent** è l'implementazione di base del Component.
- **Decorator** è una classe astratta che **include** un Component (lo ha come attributo) e ne **eredita** l'operazione: è un Component che contiene un Component.
- **ConcreteDecorator** migliora il Component: può assumerne tutte le caratteristiche, perché ha un attributo `component` a cui delegare, e può modificarne i metodi ridefinendoli (*override*).

Un esempio: la pizza

Una pizza base (olio e pomodoro) costa 3 euro; ogni ingrediente aggiunto la rende più ricca e più cara. Invece di scrivere una classe per ogni combinazione, scriviamo una classe per ogni **ingrediente**, che decora una pizza qualsiasi. L'esempio è quello distribuito con il corso (`EsempioDecorator`).

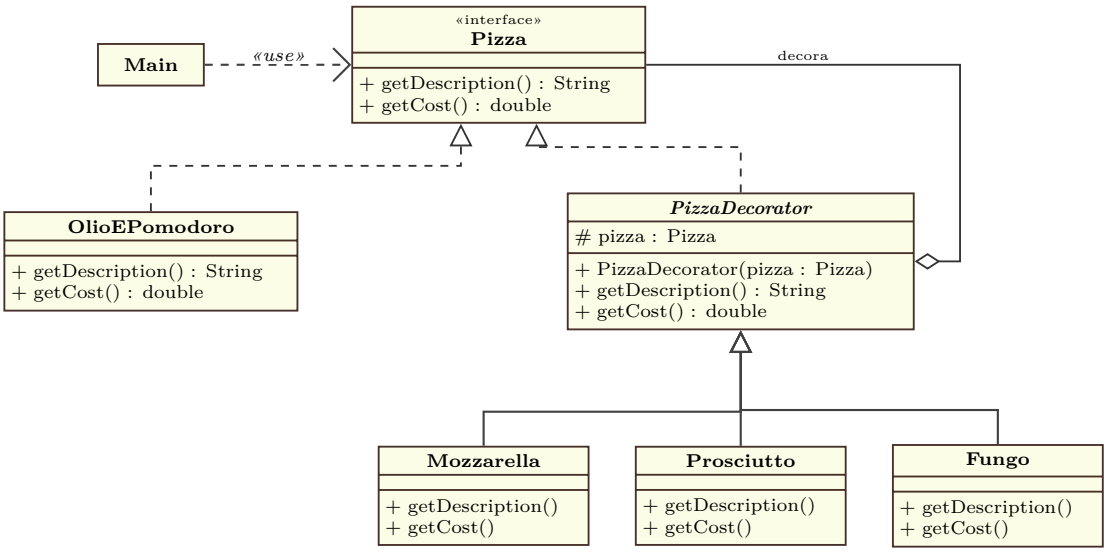


Figura 15.4: Il diagramma delle classi della pizza: `Pizza` è il Component, `OlioEPomodoro` il ConcreteComponent, `PizzaDecorator` il Decorator, gli ingredienti i ConcreteDecorator.

Pizza.java

```java
// Component: l'interfaccia comune
public interface Pizza {
    String getDescription();
    double getCost();
}
```

OlioEPomodoro.java

```java
// ConcreteComponent: la pizza base
public class OlioEPomodoro implements Pizza {
    @Override
    public String getDescription() {
        return "Pizza Olio e Pomodoro";
    }

    @Override
    public double getCost() {
        return 3.00;
    }
}
```

PizzaDecorator.java

```java
// Decorator: e' una Pizza e contiene una Pizza
public abstract class PizzaDecorator
        implements Pizza {
    protected Pizza pizza;       // la pizza avvolta

    public PizzaDecorator(Pizza pizza) {
        this.pizza = pizza;
    }

    @Override
    public String getDescription() {
        return pizza.getDescription();   // delega
    }

    @Override
    public double getCost() {
        return pizza.getCost();          // delega
    }
}
```

Ogni ingrediente è un decoratore concreto: aggiunge il proprio nome alla descrizione e il proprio prezzo al costo della pizza che avvolge. `Prosciutto` (1 euro) e `Fungo` (2 euro, descrizione «Funghi») sono identici a `Mozzarella`, tranne che per il testo e il prezzo.

Mozzarella.java

```java
// ConcreteDecorator: aggiunge la mozzarella
public class Mozzarella extends PizzaDecorator {
    public Mozzarella(Pizza pizza) {
        super(pizza);
    }

    @Override
    public String getDescription() {
        return pizza.getDescription()
                + ", Mozzarella";
    }

    @Override
    public double getCost() {
        return pizza.getCost() + 1.50;
    }
}
```

Fungo.java

```java
// ConcreteDecorator: aggiunge i funghi
public class Fungo extends PizzaDecorator {
    public Fungo(Pizza pizza) {
        super(pizza);
    }

    @Override
    public String getDescription() {
        return pizza.getDescription()
                + ", Funghi";
    }

    @Override
    public double getCost() {
        return pizza.getCost() + 2.00;
    }
}
```

Il `main` ordina una pizza base, poi la stessa con la mozzarella, poi una pizza «personalizzata» con quattro ingredienti, fra cui la mozzarella due volte.

Main.java

```java
public class Main {
    public static void main(String[] args) {
        // Ordiniamo una pizza base
        Pizza myPizza = new OlioEPomodoro();
        System.out.println("Pizza ordinata: " + myPizza.getDescription());
        System.out.println("Costo: €" + myPizza.getCost());
        System.out.println("--------------------");

        // Aggiungiamo la mozzarella alla pizza
        myPizza = new Mozzarella(myPizza);
        System.out.println("Pizza ordinata: " + myPizza.getDescription());
        System.out.println("Costo: €" + myPizza.getCost());
        System.out.println("--------------------");

        // Una pizza piu' ricca, con doppia mozzarella
        Pizza customPizza = new OlioEPomodoro();
        customPizza = new Prosciutto(customPizza);
        customPizza = new Mozzarella(customPizza);
        customPizza = new Fungo(customPizza);
        customPizza = new Mozzarella(customPizza);
        System.out.println("\n*** La tua pizza personalizzata ***");
        System.out.println("Descrizione: " + customPizza.getDescription());
        System.out.println("Costo totale: €" + customPizza.getCost());
    }
}
```

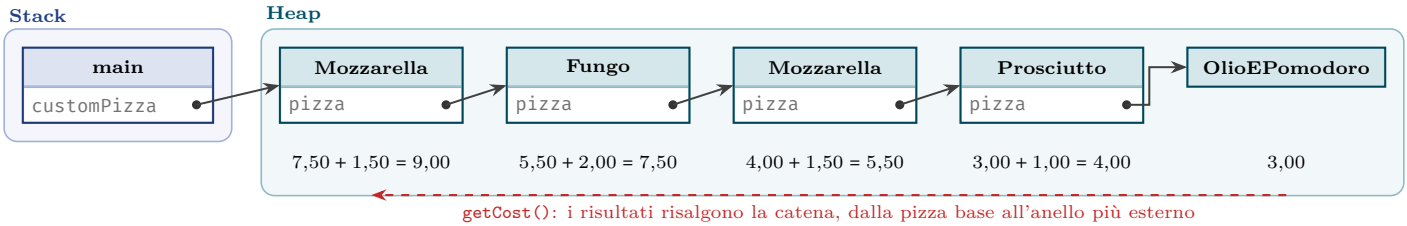
Terminale

```
$ javac *.java
$ java Main
Pizza ordinata: Pizza Olio e Pomodoro
Costo: €3.0
--------------------
Pizza ordinata: Pizza Olio e Pomodoro, Mozzarella
Costo: €4.5
--------------------

*** La tua pizza personalizzata ***
Descrizione: Pizza Olio e Pomodoro, Prosciutto, Mozzarella, Funghi, Mozzarella
Costo totale: €9.0
```

Che cosa succede in memoria

Ogni `new` aggiunge un anello alla catena: il nuovo decoratore riceve nel costruttore la pizza costruita fin lì e la conserva nel campo `pizza`; il riferimento al nuovo anello rimpiazza quello vecchio in `customPizza`. Alla fine `customPizza` punta all'anello più esterno, e da lì si arriva, di riferimento in riferimento, fino alla pizza base.



La chiamata `customPizza.getCost()` **scende** lungo la catena: ogni decoratore, prima di aggiungere il suo prezzo, chiede il costo alla pizza che contiene. Solo `OlioEPomodoro` risponde subito, con 3,00; poi i risultati **risalgono**, e ogni anello aggiunge il suo pezzo: 4,00, 5,50, 7,50, 9,00. La descrizione si costruisce allo stesso modo, ed è per questo che gli ingredienti compaiono nell'ordine in cui sono stati aggiunti.

Quante classi servirebbero con l'ereditarietà?

Con tre ingredienti, una sottoclasse per ogni combinazione richiederebbe $2^3 = 8$ classi (*base*, *Mozzarella*, *Prosciutto*, *Mozzarella e Prosciutto*, …), e non permetterebbe comunque la doppia mozzarella; con dieci ingredienti sarebbero $2^{10} = 1024$. Con i decoratori servono tre classi, una per ingrediente, e le combinazioni, ripetizioni comprese, si scelgono a tempo di esecuzione.

Il Decorator nelle librerie di Java

Il pattern è usato spesso nelle librerie di Java, in particolare nell'input/output: `Reader`, `BufferedReader(Reader in)`, `InputStream`, `InputStreamReader(InputStream in)`, `BufferedInputStream(InputStream in)`. Ogni classe riceve nel costruttore un oggetto del suo stesso tipo (o del tipo da cui legge) e gli aggiunge una capacità:

- `System.in` è un `InputStream`: legge **byte**;
- `InputStreamReader` lo avvolge e trasforma i byte in **caratteri**, secondo una codifica;
- `BufferedReader` avvolge il lettore di caratteri e aggiunge un **buffer** e il metodo `readLine()`, che legge una riga intera.

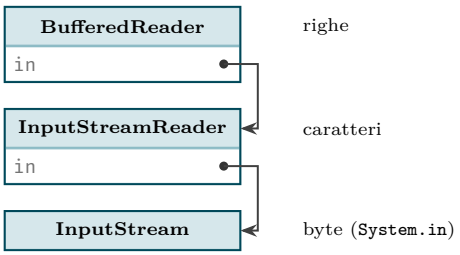
LeggiRighe.java

```java
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class LeggiRighe {
    public static void main(String[] args) throws IOException {
        BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
        String riga;
        while ((riga = in.readLine()) != null) {      // una riga intera alla volta
            System.out.println("Hai scritto: " + riga);
        }
    }
}
```

Terminale

```
$ printf 'ciao\nDesign Patterns\n' | java LeggiRighe.java
Hai scritto: ciao
Hai scritto: Design Patterns
```



Nel disegno la catena di oggetti creata dalla prima riga di `main`; i nomi dei campi interni delle classi della libreria sono semplificati.

Vantaggi e svantaggi

- **Vantaggio**: combinando i decoratori si ottiene un numero illimitato di combinazioni di chiamate ai costruttori. Non serve dichiarare le classi «prosciutto e funghi», «margherita», «margherita prosciutto e funghi», «margherita doppia mozzarella»…
- **Svantaggio**: costruire un oggetto richiede di istanziare più oggetti e di chiamare più costruttori, e ogni chiamata di metodo attraversa tutta la catena. Il pattern è quindi poco efficiente dal punto di vista computazionale (e, di conseguenza, anche da quello energetico).

PATTERN COMPORTAMENTALI

I pattern comportamentali riguardano gli **algoritmi** e l'**assegnazione delle responsabilità** fra gli oggetti: come gli oggetti interagiscono e si dividono il lavoro senza legarsi troppo strettamente a un algoritmo particolare. Ne vediamo tre: Observer e State, a cui le slide dedicano un esempio, e Strategy, che la tabella delle slide evidenzia.

Observer

Observer (comportamentale)

Il pattern **Observer** definisce una dipendenza 1:N fra oggetti: quando un oggetto (il **soggetto**) cambia stato, tutti gli oggetti che dipendono da lui (gli **osservatori**) vengono avvisati e aggiornati automaticamente.

- **Contesto**: uno o più oggetti sono interessati a osservare i cambiamenti di stato di un soggetto.
- **Problema**: il soggetto può avere osservatori diversi, anche di tipi diversi, e deve essere possibile aggiungere nuovi osservatori mentre l'applicazione è in esecuzione.
- **Soluzione**: il soggetto offre delle operazioni con cui un osservatore dichiara il proprio interesse per i cambiamenti di stato (`attach`) o lo ritira (`detach`). Quando lo stato cambia, il soggetto chiama `notify`, che a sua volta chiama `update` su ogni osservatore registrato.

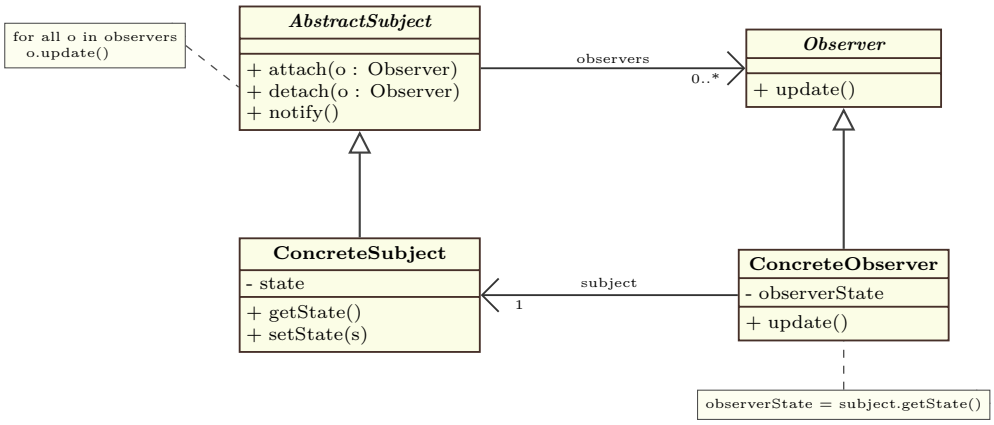
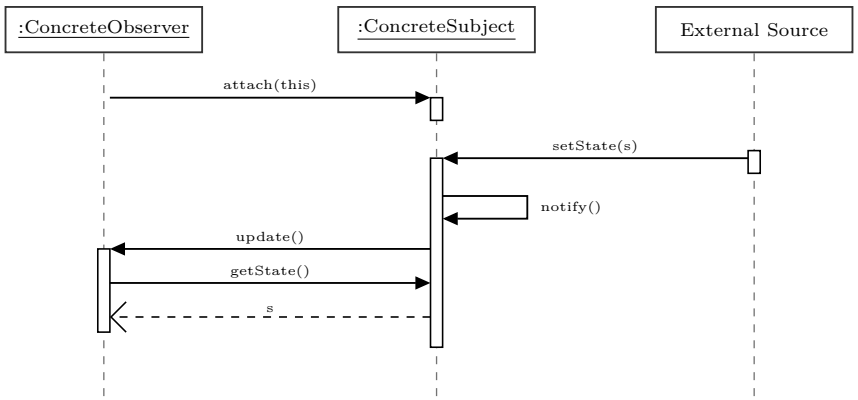


Figura 15.5: La struttura dell'Observer. Il soggetto conosce i suoi osservatori solo attraverso l'interfaccia `Observer`.

Il diagramma di sequenza mostra il comportamento: l'osservatore si registra; più tardi una sorgente esterna cambia lo stato del soggetto, che notifica tutti gli osservatori; ognuno, ricevuto l'`update()`, chiede al soggetto il nuovo stato.



L'esempio delle slide

L'osservatore è un'interfaccia e il soggetto una classe astratta: entrambi sono **invarianti**, cioè restano uguali in ogni applicazione, e potrebbero essere un'interfaccia e una classe di libreria. Cambiano solo le classi concrete. I nomi del codice corrispondono a quelli del diagramma così:

Nel diagramma	Nel codice dell'esempio
`AbstractSubject`, `ConcreteSubject`	`Soggetto`, `SoggettoConcreto`
`attach()`, `detach()`	`registraOsservatore()`, `rimuoviOsservatore()`
`notify()`	`update()` di `Soggetto` (protetto)
`Observer`, `ConcreteObserver`	`Osservatore`, `OsservatoreConcreto`
`update()` dell'osservatore	`aggiorna()`
`observerState`	il campo `stato` dell'osservatore

Tabella 15.3: Corrispondenza fra il diagramma e il codice. Il metodo `rimuoviOsservatore()` non è nelle slide: l'abbiamo aggiunto per completare l'interfaccia del soggetto.

Osservatore.java

```java
// osservatore (interfaccia): invariante,
// potrebbe essere un'interfaccia di libreria
public interface Osservatore {
    public void aggiorna();
}
```

OsservatoreConcreto.java

```java
// osservatore (concreto): e' legato al soggetto
// di cui vuole conoscere le modifiche
public class OsservatoreConcreto
        implements Osservatore {
    private int stato;           // copia dello stato
    private Soggetto osservato;  // il soggetto

    public OsservatoreConcreto(Soggetto soggetto) {
        this.osservato = soggetto;
    }

    @Override
    public void aggiorna() {
        stato = osservato.getStato();
        System.out.println(
                "Notificato; Nuovo Stato:");
        System.out.println(stato);
    }
}
```

Soggetto.java

```java
import java.util.ArrayList;

// soggetto (astratto): invariante,
// potrebbe essere una classe di libreria
public abstract class Soggetto {
    protected int stato;  // stato del soggetto
    protected ArrayList<Osservatore> osservatori;

    public Soggetto() {
        osservatori = new ArrayList<Osservatore>();
        stato = 0;
    }

    // attach
    public void registraOsservatore(Osservatore o) {
        osservatori.add(o);
    }

    // detach
    public void rimuoviOsservatore(Osservatore o) {
        osservatori.remove(o);
    }

    // notify: avvisa tutti gli osservatori
    protected void update() {
        for (int i = 0; i < osservatori.size(); i++) {
            Osservatore o = osservatori.get(i);
            o.aggiorna();
        }
    }

    public abstract void setStato(int s);
    public abstract int getStato();
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Soggetto soggetto = new SoggettoConcreto();
        Osservatore o1 =
                new OsservatoreConcreto(soggetto);
        soggetto.registraOsservatore(o1);

        Osservatore o2 =
                new OsservatoreConcreto(soggetto);
        soggetto.registraOsservatore(o2);

        soggetto.setStato(20);
    }
}
```

SoggettoConcreto.java

```java
// soggetto (concreto)
public class SoggettoConcreto extends Soggetto {
    public SoggettoConcreto() {
        super();
    }

    @Override
    public void setStato(int s) {
        stato = s;
        update();    // la chiamata in piu' rispetto
                     // a una classe normale
    }

    @Override
    public int getStato() {
        return stato;
    }
}
```

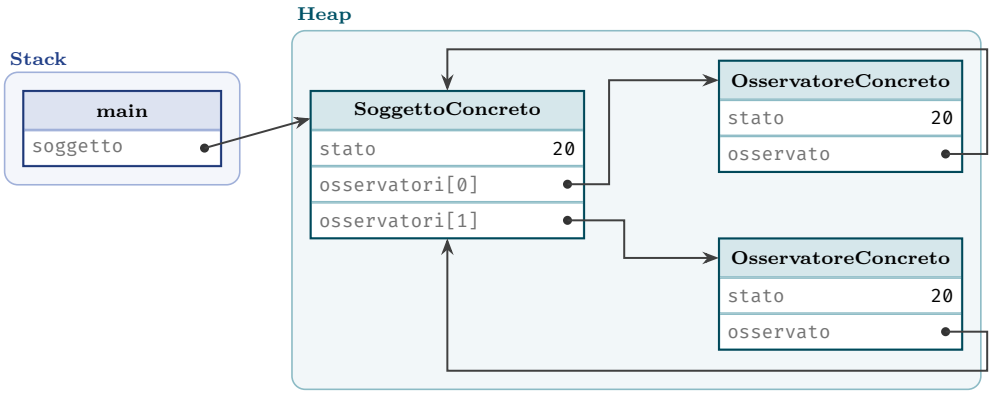
Terminale

```
$ javac *.java
$ java Main
Notificato; Nuovo Stato:
20
Notificato; Nuovo Stato:
20
```

Rispetto a una classe qualsiasi, `SoggettoConcreto` ha una sola chiamata in più, `update()` dentro `setStato`: è quella che fa partire le notifiche. Una sola chiamata a `setStato(20)` produce due stampe, una per osservatore. (Nella slide il testo stampato inizia con una virgoletta «, che in Java non chiude la stringa: qui è corretta.)

Che cosa succede in memoria

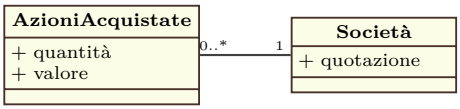
Dopo `setStato(20)` la memoria è questa. I riferimenti vanno **nei due sensi**: il soggetto conosce i suoi osservatori attraverso la lista (disegnata, come nel Composite, dentro l'oggetto), e ogni osservatore conosce il soggetto attraverso il campo `osservato`, da cui legge il nuovo stato. Ogni osservatore ne tiene una **copia** nel proprio campo `stato`. Le variabili `o1` e `o2` di `main`, che puntano agli stessi due osservatori, non sono disegnate.



Un problema: il valore delle azioni

Si vuole gestire un attributo `valore` nella classe `AzioniAcquistate`, che misuri il valore istantaneo di un insieme di azioni acquistate di una società. È calcolato come

AzioniAcquistate.valore = Società.quotazione × AzioniAcquistate.quantità.



Quando il sistema aggiorna la quotazione delle azioni di una società, si vuole che gli attributi `valore` di tutte le istanze di `AzioniAcquistate` di quella società siano aggiornati **automaticamente**.

La soluzione delle slide applica l'Observer: la società è il soggetto, le azioni acquistate sono gli osservatori. Ogni azione, appena creata, si registra presso la sua società (`attach`); `setQuotazione` cambia la quotazione e chiama `notify`; l'`update` di ogni azione ricalcola `valore = societa.getQuotazione() * quantita`.

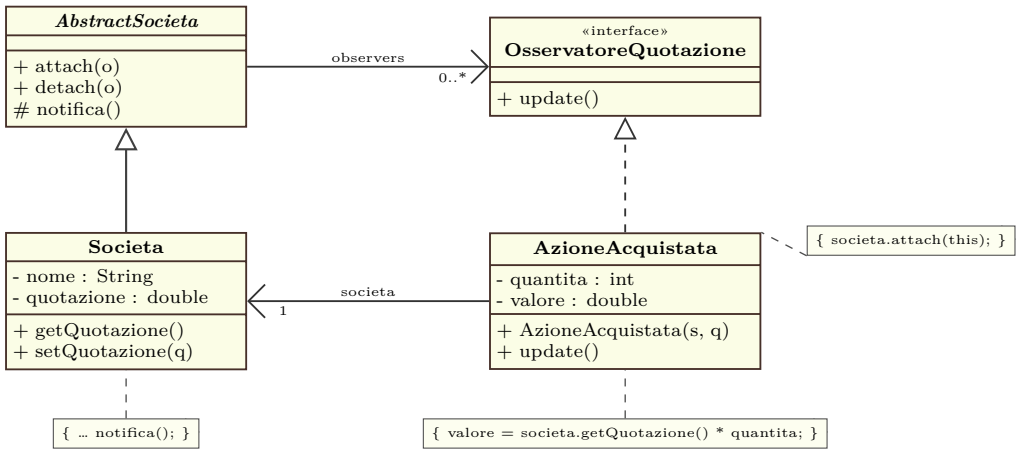


Figura 15.6: La soluzione con l'Observer. Nelle slide la società concreta si chiama `ConcreteSocieta`; qui `Societa`.

Le slide si fermano al diagramma; ecco il codice.

OsservatoreQuotazione.java

```java
public interface OsservatoreQuotazione {
    void update();
}
```

AbstractSocieta.java

```java
import java.util.ArrayList;
import java.util.List;

public abstract class AbstractSocieta {
    private final List<OsservatoreQuotazione>
            observers = new ArrayList<>();

    public void attach(OsservatoreQuotazione o) {
        observers.add(o);
    }
    public void detach(OsservatoreQuotazione o) {
        observers.remove(o);
    }

    protected void notifica() {  // il notify()
        for (OsservatoreQuotazione o : observers) {
            o.update();
        }
    }
}
```

Societa.java

```java
public class Societa extends AbstractSocieta {
    private final String nome;
    private double quotazione;

    public Societa(String nome, double quotazione) {
        this.nome = nome;
        this.quotazione = quotazione;
    }

    public String getNome() { return nome; }
    public double getQuotazione() {
        return quotazione;
    }

    public void setQuotazione(double quotazione) {
        this.quotazione = quotazione;
        notifica();    // le azioni si ricalcolano
    }
}
```

AzioneAcquistata.java

```java
public class AzioneAcquistata
        implements OsservatoreQuotazione {
    private final Societa societa;
    private final int quantita;
    private double valore;

    public AzioneAcquistata(Societa societa,
                            int quantita) {
        this.societa = societa;
        this.quantita = quantita;
        societa.attach(this);  // si iscrive
        update();              // valore iniziale
    }

    @Override
    public void update() {
        valore = societa.getQuotazione() * quantita;
    }

    @Override
    public String toString() {
        return quantita + " azioni "
                + societa.getNome() + " = "
                + valore + " euro";
    }
}
```

Portafoglio.java

```java
public class Portafoglio {
    public static void main(String[] args) {
        Societa acme = new Societa("ACME", 10.0);
        AzioneAcquistata mie =
                new AzioneAcquistata(acme, 100);
        AzioneAcquistata tue =
                new AzioneAcquistata(acme, 40);
        System.out.println(mie);
        System.out.println(tue);

        // la borsa aggiorna la quotazione...
        acme.setQuotazione(12.5);
        // ...e i valori sono gia' aggiornati
        System.out.println(mie);
        System.out.println(tue);
    }
}
```

Terminale

```
$ javac *.java
$ java Portafoglio
100 azioni ACME = 1000.0 euro
40 azioni ACME = 400.0 euro
100 azioni ACME = 1250.0 euro
40 azioni ACME = 500.0 euro
```

Il `main` non chiede mai alle azioni di ricalcolarsi: basta cambiare la quotazione, e le due azioni lo vengono a sapere da sole.

Perché notifica() e non notify()

Il diagramma GoF chiama `notify()` il metodo che avvisa gli osservatori, ma in Java non si può: `notify()` esiste già in `java.lang.Object`, la superclasse di tutte le classi, serve alla sincronizzazione fra thread ed è `final`, quindi non si può ridefinire. Con un metodo `public void notify() { }` in `AbstractSocieta` il compilatore rifiuta la classe:

Terminale

```
$ javac AbstractSocieta.java
AbstractSocieta.java:2: error: notify() in AbstractSocieta cannot override notify() in Object
    public void notify() { }
                ^
  overridden method is final
1 error
```

Per questo il codice delle slide usa nomi diversi (`update()` nel soggetto, `aggiorna()` nell'osservatore), e qui usiamo `notifica()`.

Quando usarlo

- Quando i cambiamenti di un soggetto devono essere «trasmessi» (*broadcast*) a molti oggetti.
- Quando oggetti indipendenti dall'applicazione (i soggetti) devono invocare oggetti specifici dell'applicazione per notificare un cambiamento del proprio stato interno. Il soggetto non ha bisogno di sapere nulla dei suoi osservatori né di come funzionano: per esempio quando un sensore o un servizio fornisce un dato aggiornato e l'interfaccia utente deve essere avvisata.
- L'Observer è il cuore del pattern **MVC** (Model-View-Controller), in cui gli oggetti dell'interfaccia utente (*View*) possono essere osservatori diretti degli oggetti del modello (*Model*): gli oggetti del Model hanno le funzioni di notifica, gli oggetti della GUI le funzioni di aggiornamento.

Esempio 15.7 — L'Observer nelle interfacce grafiche di Java

Nelle librerie grafiche di Java (AWT e Swing) ogni pulsante è un soggetto: gli osservatori si chiamano *listener* e si registrano con i metodi `add…Listener`. Quando l'utente fa clic, il pulsante notifica tutti i listener registrati.

Finestra.java

```java
import javax.swing.JButton;
import javax.swing.JFrame;

public class Finestra {
    public static void main(String[] args) {
        JFrame finestra = new JFrame("Editor");
        JButton salva = new JButton("Salva");
        // attach: il pulsante (soggetto) registra un osservatore
        salva.addActionListener(evento -> System.out.println("Salvataggio..."));
        finestra.add(salva);
        finestra.pack();
        finestra.setVisible(true);
    }
}
```

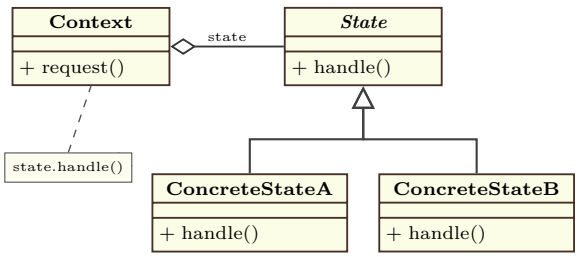
La libreria standard aveva anche una coppia `java.util.Observable` / `java.util.Observer` pronta all'uso, ma è deprecata da Java 9: oggi si scrivono interfacce proprie, come negli esempi di questo capitolo, o si usano i *listener*.

State

State (comportamentale)

Il pattern **State** permette a un oggetto di **cambiare comportamento a seconda del suo stato interno**. È una soluzione generale e flessibile per realizzare le macchine a stati.

La classe `State` contiene implementazioni **vuote** di tutti i metodi che corrispondono alle transizioni (i *trigger*, o *handler*). Il polimorfismo permette al `Context` di avere un attributo di tipo `State` e di interagire con qualunque stato possibile: `request()` esegue `state.handle()`, e a seconda dell'oggetto che c'è in `state` viene eseguito il codice di uno stato o dell'altro.



Il pattern l'abbiamo già incontrato nel Capitolo 9, dove traduceva in codice il diagramma di stato del **distributore automatico**: là si trovano il diagramma di stato, la classe contenitore `Distributore`, la classe generale `State` e lo stato `Attesa`. Il diagramma di stato si traduce così:

- una **classe generale** che dichiara i metodi corrispondenti a tutte le azioni, ma li implementa senza fare nulla;
- una serie di **classi derivate**, una per ogni stato del diagramma, che implementano (con un *override*) solo i metodi corrispondenti agli eventi che possono scattare in quello stato.

Qui completiamo l'esempio. Il diagramma delle classi delle slide mostra quali eventi gestisce ciascuno stato.

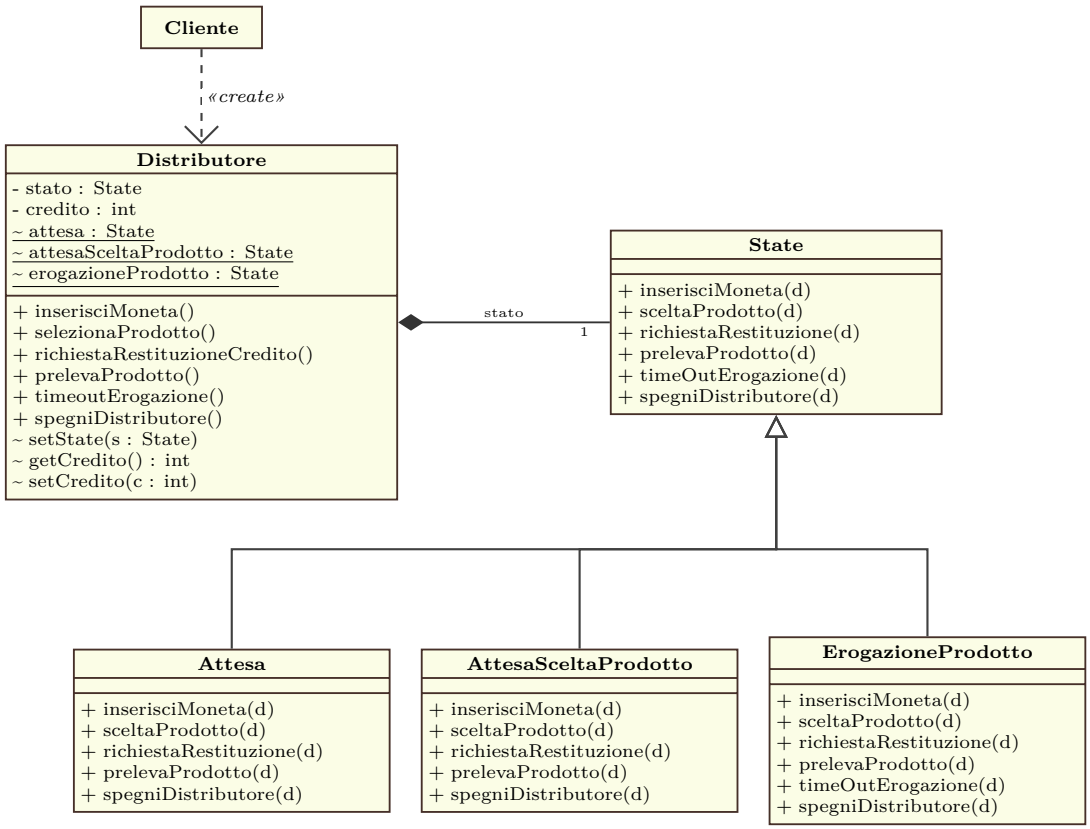


Figura 15.7: Il distributore con il pattern State, dalle slide (`d` è il `Distributore` passato a ogni metodo). Il simbolo ~ indica la visibilità di package; i tre stati sono attributi statici, quindi sottolineati.

Le slide mostrano il codice di `Distributore`, di `State` e di `Attesa` (quello del Capitolo 9) e dicono che la soluzione completa è su Teams. Qui ricostruiamo gli altri due stati dal diagramma di stato del Capitolo 9: una moneta in più in **Attesa scelta prodotto** incrementa il credito; la scelta del prodotto avvia l'**erogazione**; l'erogazione avvenuta (il prelievo) scala 2 euro e porta in Attesa scelta prodotto se prima il credito era di almeno 4 euro, altrimenti in Attesa; il *time out* torna alla scelta senza scalare nulla. I messaggi stampati sono nostri.

AttesaSceltaProdotto.java

```java
public class AttesaSceltaProdotto extends State {
    @Override
    public void inserisciMoneta(Distributore distributore) {      // autotransizione
        distributore.setCredito(distributore.getCredito() + 1);
        System.out.println("Credito attuale: " + distributore.getCredito());
    }
    @Override
    public void richiestaRestituzione(Distributore distributore) {
        System.out.println("Ritira il tuo credito: " + distributore.getCredito());
        distributore.setCredito(0);
        distributore.setState(Distributore.attesa);
    }
    @Override
    public void sceltaProdotto(Distributore distributore) {
        System.out.println("Erogazione in corso...");
        distributore.setState(Distributore.erogazioneProdotto);
    }
    @Override
    public void spegniDistributore(Distributore distributore) {
        System.out.println("Ritira il tuo credito: " + distributore.getCredito());
        distributore.setCredito(0);
        System.out.println("Spengo il distributore");
        System.exit(0);
    }
    @Override
    public void prelevaProdotto(Distributore distributore) {
        System.out.println("Il prodotto non e' pronto");
    }
}
```

ErogazioneProdotto.java

```java
public class ErogazioneProdotto extends State {
    @Override
    public void prelevaProdotto(Distributore distributore) {     // erogazione avvenuta
        int credito = distributore.getCredito();
        distributore.setCredito(credito - 2);
        System.out.println("Ecco il prodotto. Credito residuo: " + distributore.getCredito());
        if (credito >= 4) {
            distributore.setState(Distributore.attesaSceltaProdotto);
        } else {
            distributore.setState(Distributore.attesa);
        }
    }
    @Override
    public void timeOutErogazione(Distributore distributore) {   // erogazione fallita
        System.out.println("Erogazione fallita: scegli di nuovo");
        distributore.setState(Distributore.attesaSceltaProdotto);
    }
    @Override
    public void inserisciMoneta(Distributore distributore) {
        System.out.println("Attendi: erogazione in corso");
    }
    @Override
    public void sceltaProdotto(Distributore distributore) {
        System.out.println("Attendi: erogazione in corso");
    }
    @Override
    public void richiestaRestituzione(Distributore distributore) {
        System.out.println("Attendi: erogazione in corso");
    }
    @Override
    public void spegniDistributore(Distributore distributore) {
        System.out.println("Ritira il tuo credito: " + distributore.getCredito());
        distributore.setCredito(0);
        System.out.println("Spengo il distributore");
        System.exit(0);
    }
}
```

Il `main` di prova delle slide simula un cliente. Nella stessa sequenza di comandi la stessa chiamata, `dis.selezionaProdotto()`, produce effetti diversi a seconda dello stato:

Cliente.java

```java
public class Cliente {
    public static void main(String[] args) {
        Distributore dis = new Distributore();

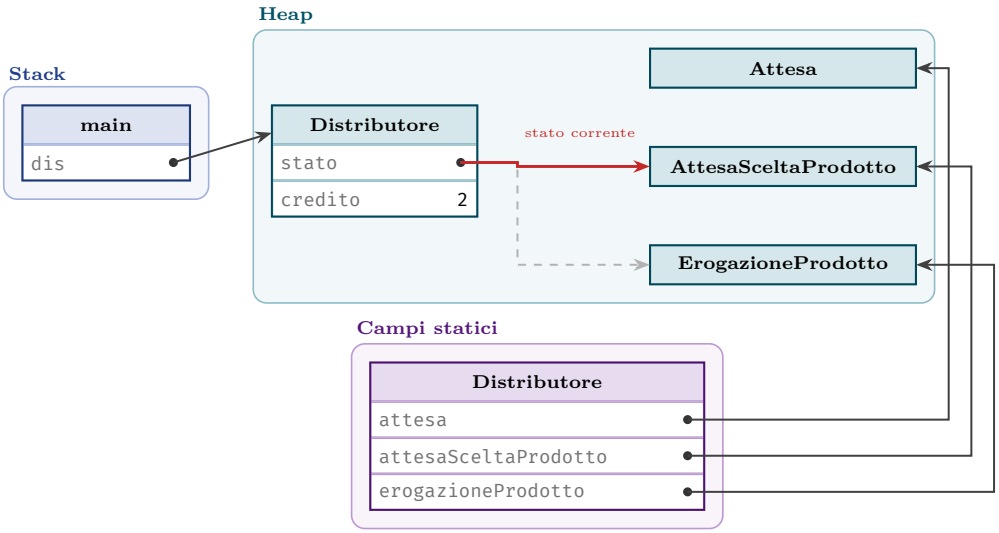
        // scelgo un prodotto senza aver inserito monete
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
        System.out.print("Prelevo il prodotto. "); dis.prelevaProdotto();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
        System.out.print("Richiedo la restituzione del credito. ");
        dis.richiestaRestituzioneCredito();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
        System.out.print("Prelevo il prodotto. "); dis.prelevaProdotto();
        System.out.print("Spengo il distributore. "); dis.spegniDistributore();
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    }
}
```

```
>_ Terminale ×
$ javac *.java
$ java Cliente
Scelgo un prodotto. Non hai credito sufficiente
Inserisco un euro. Credito attuale: 1
Scelgo un prodotto. Non hai credito sufficiente
Inserisco un euro. Credito attuale: 2
Scelgo un prodotto. Erogazione in corso...
Prelevo il prodotto. Ecco il prodotto. Credito residuo: 0
Inserisco un euro. Credito attuale: 1
Inserisco un euro. Credito attuale: 2
Inserisco un euro. Credito attuale: 3
Inserisco un euro. Credito attuale: 4
Scelgo un prodotto. Erogazione in corso...
Scelgo un prodotto. Attendi: erogazione in corso
Richiedo la restituzione del credito. Attendi: erogazione in corso
Inserisco un euro. Attendi: erogazione in corso
Prelevo il prodotto. Ecco il prodotto. Credito residuo: 2
Spengo il distributore. Ritira il tuo credito: 2
Spengo il distributore
```

La prima scelta trova il distributore in Attesa (*non hai credito sufficiente*), la terza in Attesa scelta prodotto (parte l'erogazione), la seconda delle due scelte consecutive in Erogazione prodotto (*attendi*). L'ultima riga del `main`, «Inserisco un euro», non viene mai eseguita: `spegniDistributore()` termina il programma con `System.exit(0)`.

Che cosa succede in memoria

Il disegno mostra la memoria prima dello spegnimento. Gli stati sono tre oggetti creati **una volta sola**, nei campi statici di `Distributore`; il campo `stato` del distributore punta a uno di loro. Una **transizione** non crea nulla: `setState(…)` sposta il riferimento. L'ultimo prelievo, con 4 euro di credito, l'ha spostato da ErogazioneProdotto ad AttesaSceltaProdotto (la freccia grigia è quella di prima).



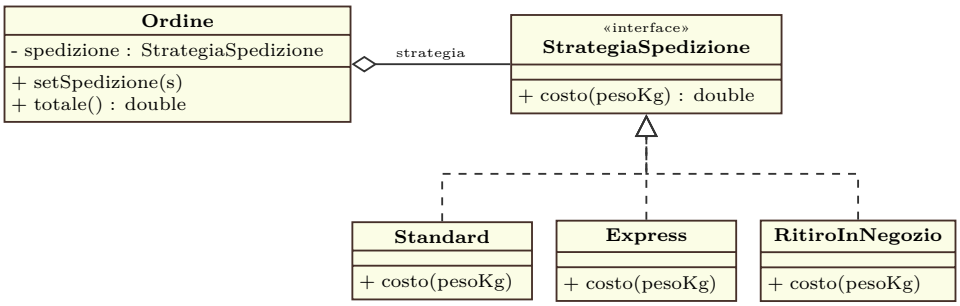
Perché gli stati possono essere condivisi

Gli oggetti stato **non hanno campi**: tutti i dati (il credito) stanno nel contesto, che ogni metodo riceve come parametro. Per questo bastano tre oggetti per tutti i distributori del programma, creati una volta sola nei campi statici: due distributori diversi possono trovarsi nello stesso stato condividendo lo stesso oggetto, ciascuno con il proprio credito. Se uno stato dovesse ricordare qualcosa di suo, servirebbe un oggetto stato per ogni distributore.

Strategy (comportamentale)

Il pattern **Strategy** definisce una famiglia di algoritmi, incapsula ciascuno in una classe e li rende **intercambiabili**: l'algoritmo può variare indipendentemente da chi lo usa.

Le slide lo evidenziano nella tabella dei pattern comportamentali senza dedicargli un esempio. Eccone uno breve, anche perché la sua struttura è identica a quella dello State, e conviene vedere in che cosa i due differiscono. Un ordine deve calcolare il costo della spedizione, e i modi di spedire sono diversi: ciascuno diventa una classe che implementa la stessa interfaccia.



StrategiaSpedizione.java

```java
// Strategy: la famiglia di algoritmi,
// dietro un'unica interfaccia
public interface StrategiaSpedizione {
    double costo(double pesoKg);
}
```

Standard.java

```java
public class Standard implements StrategiaSpedizione {
    public double costo(double pesoKg) {
        return 5.0 + 0.5 * pesoKg;
    }
}
```

Express.java

```java
public class Express implements StrategiaSpedizione {
    public double costo(double pesoKg) {
        return 12.0 + 1.0 * pesoKg;
    }
}
```

RitiroInNegozio.java

```java
public class RitiroInNegozio
        implements StrategiaSpedizione {
    public double costo(double pesoKg) {
        return 0.0;
    }
}
```

Ordine.java

```java
// Context: usa una strategia senza sapere quale
public class Ordine {
    private final double merce;
    private final double pesoKg;
    private StrategiaSpedizione spedizione =
            new Standard();

    public Ordine(double merce, double pesoKg) {
        this.merce = merce;
        this.pesoKg = pesoKg;
    }

    // la sceglie il client
    public void setSpedizione(StrategiaSpedizione s) {
        this.spedizione = s;
    }

    public double totale() {
        return merce + spedizione.costo(pesoKg);
    }
}
```

Negozio.java

```java
public class Negozio {
    public static void main(String[] args) {
        // 40 euro di merce, 6 kg
        Ordine ordine = new Ordine(40.0, 6.0);
        System.out.println("Standard:   " + ordine.totale());

        // si cambia algoritmo
        ordine.setSpedizione(new Express());
        System.out.println("Express:    " + ordine.totale());

        ordine.setSpedizione(new RitiroInNegozio());
        System.out.println("In negozio: " + ordine.totale());
    }
}
```

Terminale

```
$ javac *.java
$ java Negozio
Standard:   48.0
Express:    58.0
In negozio: 40.0
```

	State	**Strategy**
Chi cambia l'oggetto	Gli stati stessi, con `setState`, quando scatta una transizione	Il client, con `setSpedizione`, quando vuole
Gli oggetti si conoscono?	Sì: ogni stato sa in quali altri stati si può passare	No: ogni strategia ignora le altre
Che cosa rappresenta	Una fase della vita dell'oggetto	Un modo diverso di fare la stessa cosa

Tabella 15.4: Stessa struttura, intenzioni diverse.

Anche questo pattern è nella libreria di Java: un `Comparator` è una strategia di confronto, e la stessa chiamata `lista.sort(c)` ordina la lista per prezzo, per nome o per data a seconda del `Comparator c` che le si passa.

Pattern moderni: la Dependency Injection

Non tutti i pattern usati oggi fanno parte del catalogo originale della GoF. Uno dei più diffusi, e già incontrato in questa dispensa, è la Dependency Injection.

Dependency Injection

Si ha **Dependency Injection** (iniezione delle dipendenze) quando le risorse da cui un componente dipende gli vengono **passate (iniettate) dall'esterno**, invece di essere istanziate dal componente stesso durante la sua esecuzione. Il componente dichiara, per esempio nel costruttore, il tipo (eventualmente generico) e la quantità di tutte le risorse da cui dipende; i riferimenti alle risorse gli vengono poi passati, per esempio attraverso il costruttore.

L'avevamo vista all'opera nel Capitolo 4, dove la `Cassa` riceveva nel costruttore il `GatewayPagamenti` (così da poterla provare con un gateway finto), e nel Capitolo 10, dove il `Traveler` riceveva dall'esterno il mezzo di trasporto.

Un esempio semplice

MyClass.java

```java
import java.util.logging.Logger;

public class MyClass {
    private Logger logger;

    public MyClass(Logger logger) {
        this.logger = logger;
        // scrive un messaggio informativo nel log
        logger.info("This is a log message.");
    }
}
```

Avvio.java

```java
import java.util.logging.Logger;

public class Avvio {
    public static void main(String[] args) {
        // prima si crea la dipendenza...
        Logger log = Logger.getLogger("ordini");
        // ...poi la si inietta
        MyClass oggetto = new MyClass(log);
    }
}
```

Terminale

```
$ javac *.java
$ java Avvio
Oct 25, 2026 10:15:39 PM MyClass <init>
INFO: This is a log message.
```

`MyClass` può scrivere in un log, e il log su cui scrive è l'oggetto di tipo `Logger` che riceve come parametro: non sa (e non deve sapere) dove finiscono i messaggi. Qui usiamo il `Logger` della libreria standard, `java.util.logging`, che scrive su standard error la data, la classe e il metodo che hanno prodotto il messaggio (`<init>` è il costruttore). Nella slide alla chiamata di `logger.info` manca il punto e virgola. L'esempio viene dal tutorial di Vogella sulla Dependency Injection (`https://www.vogella.com/tutorials/DependencyInjection/article.html`).

Le conseguenze

- Rende la classe **più generica**, perché ne separa il comportamento dall'oggetto specifico che le viene passato. Resta però una dipendenza fra l'oggetto e una classe (o un'interfaccia): il **tipo** della risorsa.
- Non è sempre ottimale dal punto di vista dell'**accoppiamento**: chi crea l'oggetto deve conoscere e creare anche tutte le sue dipendenze.
- Si chiama anche **inversione del controllo** (*inversion of control*), perché richiede di creare **prima** l'oggetto da cui si dipende e **poi** quello che lo userà: la direzione è opposta, per esempio, a quella del modello BCE (Boundary-Control-Entity), in cui l'interfaccia (Boundary) crea il controllo che le serve e il controllo crea e usa le entità.

Dependency Injection e test

La Dependency Injection è fondamentale per il **test in isolamento** e per il **test di integrazione**: permette di provare un oggetto passandogli un oggetto **fittizio** su cui agire, al posto dell'oggetto concreto, che potrebbe non essere disponibile o non essere ancora stato testato. Il caso tipico delle slide: vogliamo testare una classe che chiama un servizio, ma il servizio non è ancora disponibile.

- Se si segue il pattern, la classe sotto test riceve attraverso il costruttore un riferimento all'oggetto servizio: nel test le si passa un servizio fittizio, e la classe si può eseguire anche senza quello originale.
- Se non lo si segue, il riferimento al servizio è scritto dentro la classe (oppure il servizio viene istanziato al suo interno) e al momento del test non si può cambiare.

ServizioMeteo.java

```java
public interface ServizioMeteo {
    double temperatura(String citta);   // gradi C
}
```

AllarmeCaldo.java

```java
public class AllarmeCaldo {
    private final ServizioMeteo meteo;   // la risorsa

    public AllarmeCaldo(ServizioMeteo meteo) {
        this.meteo = meteo;              // iniettata
    }

    public String controlla(String citta) {
        double t = meteo.temperatura(citta);
        if (t >= 35) {
            return "ALLARME: a " + citta
                    + " ci sono " + t + " gradi";
        }
        return "Nessun allarme a " + citta;
    }
}
```

MeteoFinto.java

```java
// Oggetto fittizio per i test: restituisce
// sempre la temperatura scelta dal test
public class MeteoFinto implements ServizioMeteo {
    private final double temperatura;

    public MeteoFinto(double temperatura) {
        this.temperatura = temperatura;
    }

    @Override
    public double temperatura(String citta) {
        return temperatura;
    }
}
```

App.java

```java
public class App {
    public static void main(String[] args) {
        // prima la dipendenza...
        ServizioMeteo meteo = new MeteoHttp("https://api.meteo.example");
        // ...poi chi la usa
        AllarmeCaldo allarme = new AllarmeCaldo(meteo);
        System.out.println(allarme.controlla("Napoli"));
    }
}
```

In produzione `AllarmeCaldo` riceve `MeteoHttp`, che interroga un servizio REST con l'`HttpClient` visto nel Capitolo 14; nel test riceve `MeteoFinto`, che risponde subito e sempre allo stesso modo. Il test non ha bisogno né della rete né del servizio, e può scegliere la temperatura che gli serve per provare entrambi i casi.

MeteoHttp.java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

// Implementazione vera: interroga un servizio REST esterno
public class MeteoHttp implements ServizioMeteo {
    private final HttpClient client = HttpClient.newHttpClient();
    private final String baseUrl;

    public MeteoHttp(String baseUrl) {
        this.baseUrl = baseUrl;
    }

    @Override
    public double temperatura(String citta) {
        try {
            URI uri = URI.create(baseUrl + "/temperatura?citta=" + citta);
            HttpRequest richiesta = HttpRequest.newBuilder(uri).build();
            String corpo = client.send(richiesta, HttpResponse.BodyHandlers.ofString()).body();
            return Double.parseDouble(corpo);
        } catch (Exception e) {
            throw new IllegalStateException("Servizio meteo non raggiungibile", e);
        }
    }
}
```

AllarmeCaldoTest.java

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class AllarmeCaldoTest {
    @Test
    void segnalaIlCaldoDa35GradiInSu() {
        AllarmeCaldo allarme = new AllarmeCaldo(new MeteoFinto(38.0));
        assertEquals("ALLARME: a Napoli ci sono 38.0 gradi", allarme.controlla("Napoli"));
    }

    @Test
    void taceSottoI35Gradi() {
        AllarmeCaldo allarme = new AllarmeCaldo(new MeteoFinto(22.5));
        assertEquals("Nessun allarme a Napoli", allarme.controlla("Napoli"));
    }
}
```

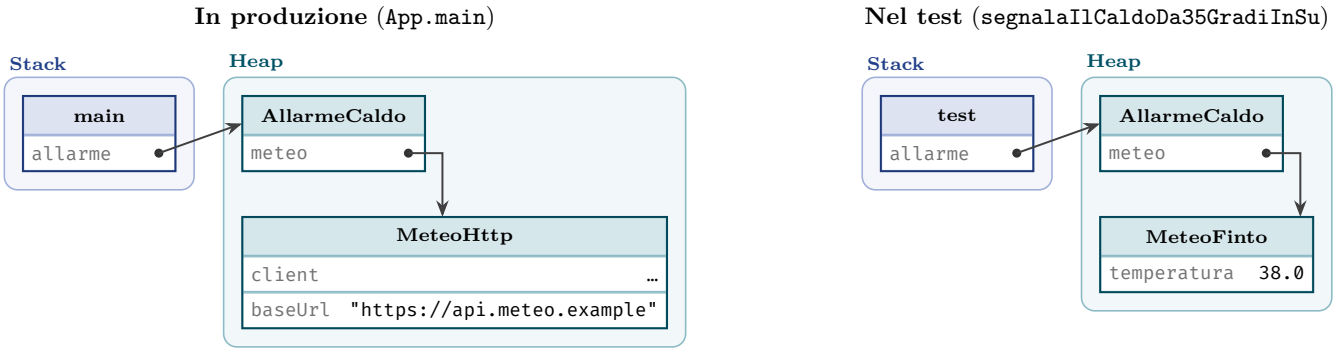
Terminale

```
$ java -jar junit-platform-console-standalone.jar execute --class-path out \
       --select-class AllarmeCaldoTest --details=tree --details-theme=ascii
+-- JUnit Jupiter [OK]
| '-- AllarmeCaldoTest [OK]
|   +-- taceSottoI35Gradi() [OK]
|   '-- segnalaIlCaldoDa35GradiInSu() [OK]
...
[         2 tests successful      ]
[         0 tests failed          ]
```

(L'uscita di JUnit è accorciata: le righe omesse riassumono i contenitori di test trovati ed eseguiti.)

Che cosa succede in memoria

La classe `AllarmeCaldo` è la stessa nei due casi; cambia solo l'oggetto a cui punta il suo campo `meteo`, e lo decide chi crea l'allarme.



Esempio 15.8 — Il controesempio: la dipendenza scritta dentro la classe

AllarmeCaldoRigido.java

```java
public class AllarmeCaldoRigido {
    // la dipendenza e' creata qui dentro: nessun test potra' sostituirla
    private final ServizioMeteo meteo = new MeteoHttp("https://api.meteo.example");

    public String controlla(String citta) {
        double t = meteo.temperatura(citta);      // chiama sempre il servizio vero
        return t >= 35 ? "ALLARME: a " + citta + " ci sono " + t + " gradi"
                       : "Nessun allarme a " + citta;
    }
}
```

Per provare `AllarmeCaldoRigido` serve il servizio vero, raggiungibile e acceso; e non c'è modo di fargli restituire 38 gradi a comando per verificare l'allarme.

Chi crea gli oggetti? I framework

In un'applicazione grande, creare a mano tutte le dipendenze nell'ordine giusto diventa faticoso. Framework come **Spring** o **Jakarta EE** lo fanno da soli: si dichiara nel costruttore di che cosa ha bisogno una classe (in Jakarta EE con l'annotazione `@Inject`; in Spring basta il costruttore, o `@Autowired`), e il framework crea gli oggetti e li collega. Il codice delle classi resta quello visto qui.

Conclusioni

Famiglia	Su che cosa si concentra	Quali problemi risolve
Strutturali	Come gli oggetti si compongono per formare strutture più grandi	Realizzare funzionalità nuove a partire da quelle esistenti; dare flessibilità ed estensibilità
Comportamentali	Gli algoritmi e l'assegnazione delle responsabilità agli oggetti	L'accoppiamento troppo stretto con un algoritmo particolare
Creazionali	La creazione di oggetti complessi	Nascondere come gli oggetti complessi vengono creati e assemblati

Tabella 15.5: Le tre famiglie di pattern in sintesi.

Alcune osservazioni finali:

- I pattern permettono di progettare a un **livello più astratto**: trattano molte interazioni fra classi e oggetti come un'unità; spesso sono utili dopo un primo progetto, come obiettivi di ristrutturazione delle classi (*refactoring*).
- Una progettazione **orientata alla variazione**: si considera quali aspetti del progetto possono variare, si individuano i pattern applicabili, li si varia per valutarne i compromessi, e si ripete.
- I design pattern forniscono soluzioni a problemi comuni, portano a modelli e codice estensibili, si possono usare così come sono oppure come esempi di **ereditarietà di interfaccia** e di **delega**, e applicano gli stessi principi alla struttura e al comportamento.

I design pattern non risolvono tutti i problemi

Le slide chiudono con una provocazione: «i design pattern risolvono tutti i vostri problemi di ingegneria del software». Non è così. Si potrebbe continuare a lungo a presentare pattern, ma alla fine bisogna **prima definire il problema** e poi cercare una soluzione; e poiché i pattern sono soluzioni a problemi, **non si guardano i pattern finché il problema non è stato definito**. Un pattern applicato dove non serve aggiunge solo classi e complessità.

Per approfondire

- C. Alexander, *The Timeless Way of Building*, Oxford, 1979; *A Pattern Language*, Oxford, 1977.
- E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns*, Addison-Wesley, 1995.
- F. Buschmann et al., *Pattern-Oriented Software Architecture*, Wiley, 1996.
- M. Fowler, *Analysis Patterns*, Addison-Wesley, 1996.
- K. Beck, *Smalltalk Best Practice Patterns*, Prentice Hall, 1997.
- S. Alpert et al., *The Design Patterns Smalltalk Companion*, Addison-Wesley, 1998.
- W. Brown et al., *AntiPatterns*, Wiley, 1998.

In sintesi

- Un **design pattern** è una soluzione comune a un problema ricorrente di progettazione, descritta da nome, problema, soluzione e conseguenze; nasce dall'architettura di Alexander e dal catalogo della **GoF** (1995).
- **Singleton**: costruttore privato, istanza in un campo statico, accesso con `getInstance()`; un solo oggetto, creato alla prima richiesta.
- **Builder**: la costruzione si fa a passi, su un oggetto builder (`with…` restituisce `this`); il prodotto si crea, già validato, con `build()`. Il Director ripete la stessa ricetta con builder diversi.
- **Adapter**: una classe che implementa l'interfaccia attesa (Target) e traduce le chiamate verso la classe esistente (Adaptee), per ereditarietà o, più spesso, per composizione.
- **Composite**: foglie e composti hanno la stessa interfaccia; il composto contiene componenti, e le operazioni si propagano ricorsivamente sull'albero.
- **Decorator**: decoratori con la stessa interfaccia dell'oggetto decorato, concatenati a tempo di esecuzione; ogni chiamata attraversa la catena.
- **Observer**: il soggetto tiene la lista degli osservatori (`attach`, `detach`) e a ogni cambiamento li notifica; è il cuore dell'MVC e dei *listener*.
- **State**: uno stato è un oggetto, una transizione è il cambio di un riferimento; **Strategy**: stessa struttura, ma è il client a scegliere l'algoritmo.
- **Dependency Injection**: le dipendenze si passano dall'esterno, di solito nel costruttore (inversione del controllo); così nei test si possono sostituire con oggetti fittizi.